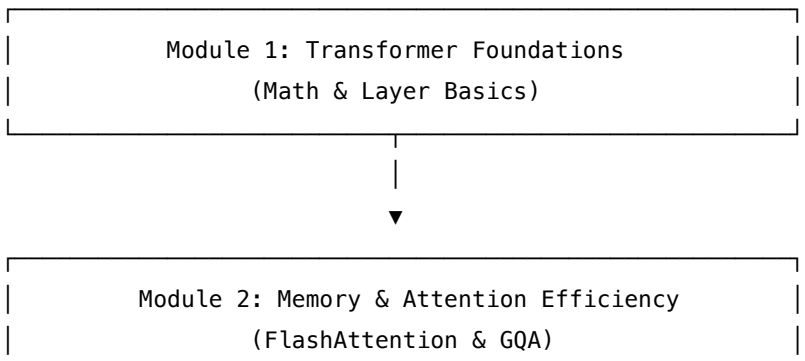
Contents

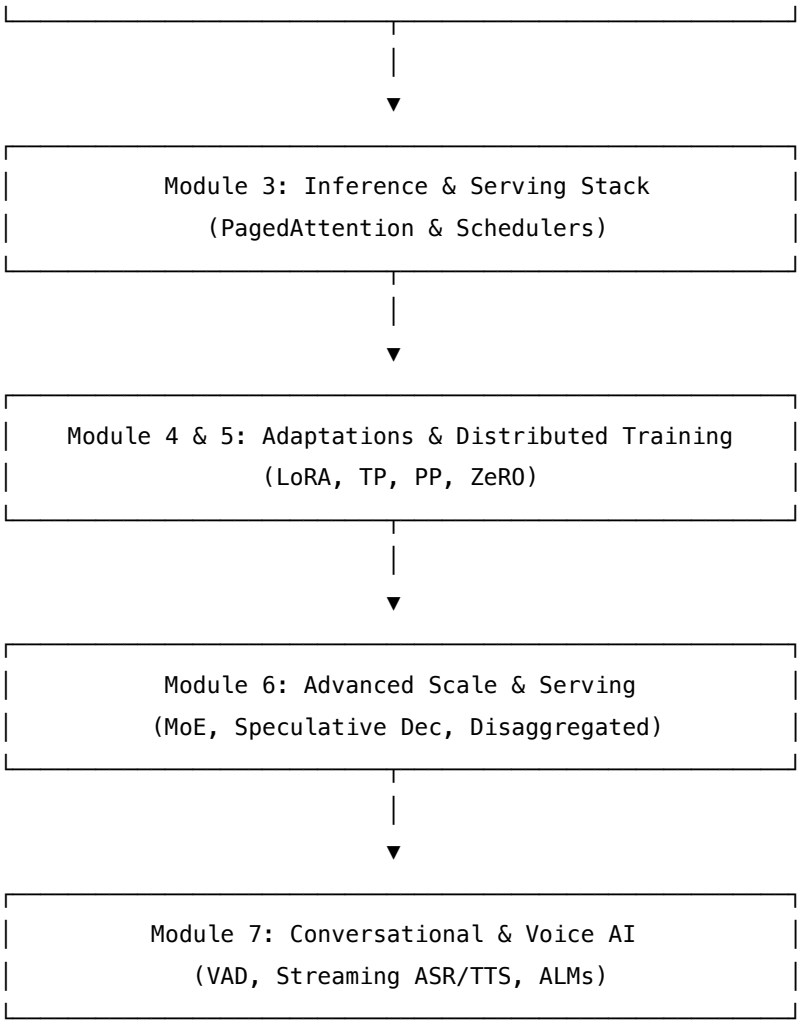
1 LLM Systems — Interview Preparation Roadmap

Welcome to the **LLM Systems study guide**. This directory contains 34 deep-dive interview-preparation guides covering Transformer fundamentals, serving optimization, distributed training, and voice/speech pipelines.

1.1 Progression Roadmap

To build solid LLM systems intuition, progress through the modules in sequence:





1.2 Learning Modules & File Index

1.2.1 Module 1: Transformer Foundations

Goal: Understand layer-by-layer mathematics, normalization stability, and subword tokenization mechanics.

Topic	Guide Link	Key Focus Area
Layer Normalization	Normalization	LayerNorm vs. RMSNorm, BF16 underflow, and FP32 upcast
Tokenization	Tokenization	BPE rank-greedy merges, WordPiece, and SentencePiece CJK rules
Position Encoding	Rotary Position Embedding (RoPE)	Absolute PE vs. RoPE, norm-preservation, and context scaling
MLP Activation	SwiGLU Activation	Swish gating, parameter footprint, and validation benefits

Topic	Guide Link	Key Focus Area
Causal Masking	Causal Mask	Prefill vs. decode attention, lower-triangular masking, and QK-Norm

1.2.2 Module 2: Memory & Attention Efficiency

Goal: Overcome memory bandwidth bottlenecks and minimize KV Cache footprints.

Topic	Guide Link	Key Focus Area
KV Cache Basics	KV Cache (Dense vs Paged)	Autoregressive prefill/decode phases, and sequence waste math
FlashAttention	FlashAttention	SRAM tiling, online softmax normalization, and HBM memory walls
Grouped Attention	Grouped-Query Attention (GQA)	MHA vs. MQA vs. GQA, sharding layouts, and tensor contiguous steps
Quantization	Quantization (INT8, FP8, AWQ, GPTQ)	Outliers, weight-only vs. activation quantization, SmoothQuant, and AWQ

1.2.3 Module 3: Inference & Serving Stack

Goal: Design high-throughput serving systems and optimize kernels for GPU execution.

Topic	Guide Link	Key Focus Area
Sampling	Sampling Strategies	Temperature scaling, top-k/top-p nucleus cutoff, and entropy
PagedAttention	PagedAttention Kernel	Logical-to-physical block tables, address gather math, and page sizes
Block Manager	Block Manager	Dynamic allocation, free list tracking, reference count sharing, and preemption
LLM Scheduler	Scheduler	Continuous batching, prefill/decode slot allocation, and FCFS scheduling
Prefix Caching	Prefix Cache	Radix-tree prefix sharing, hit rate numbers, and LRU eviction

Topic	Guide Link	Key Focus Area
CUDA Graphs	CUDA Graphs	Decode launch overhead, eager mode vs. captured graph execution, and dynamic padding

1.2.4 Module 4: Fine-Tuning & Adaptation

Goal: Scale down model updates and optimize memory overhead during training.

Topic	Guide Link	Key Focus Area
LoRA	Low-Rank Adaptation (LoRA)	Frozen parameters, $\Delta W = B \cdot A$ rank-reduction, and S-LoRA multi-adapter serving
Recomputation	Gradient Checkpointing	Activation storing vs. selective recomputation, and $O(\sqrt{L})$ memory scaling

1.2.5 Module 5: Distributed Training & Large-Scale Systems

Goal: Train giant models across multi-node GPU clusters, partitioning memory and compute.

Topic	Guide Link	Key Focus Area
Collectives	NCCL Collectives	Broadcast, Reduce, AllReduce, Ring-AllReduce step mechanics, and network bounds
Data Parallelism	DDP	Weight replication, Ring-AllReduce gradient sync, and Amp mixed precision
Tensor Parallelism	Tensor Parallelism	Megatron Column/Row projections, activation cancellation, and NVLink limits
Pipeline Parallelism	Pipeline Parallelism	GPipe bubbles, 1F1B scheduling, and virtual stages
ZeRO Optimizer	ZeRO Optimizer	Partitioning optimizer states, gradients, and parameters; communication overhead
Distributed Scale	Distributed GPU Training	1-Trillion parameter model cluster layout, GPUDirect RDMA, and fault tolerance

1.2.6 Module 6: Advanced Serving & Scale

Goal: Deploy massive expert models and coordinate high-speed GPU-to-CPU offloading.

Topic	Guide Link	Key Focus Area
MoE Routing	MoE Routing	Sparse gating, top-k routing, load balancing, and DeepSeek aux-free routing
Speculative Decoding	Speculative Decoding	Draft-target synchronization, and parallel target validation
Global Prefix Cache	LMCache	GPU-CPU-NVMe-S3 storage hierarchy, and RDMA block transfers
Prefill-Decode Split	Disaggregated Serving	Dedicated prefill and decode nodes, and KV cache transfer latencies
CPU Offloading	KTransformers Offload	CPU expert offloading, activation-only transfers, and AVX/AMX GEMMs
TPU Compilation	JAX / XLA / TPU	Jaxpr tracing, XLA kernel fusion, and Pallas low-level TPU code

1.2.7 Module 7: Conversational & Voice AI Systems

Goal: Build sub-500ms voice agents with streaming audio pipelines and robust guardrails.

Topic	Guide Link	Key Focus Area
NVIDIA Riva Stack	NVIDIA Conversational AI	FastConformer downsampling, Riva NIM Triton deployments, and guardrails
Voice Agent Design	Voice Agent Architecture	Decoupled queues, streaming VAD, state synchronization, and barge-in
Voice Profiling	Voice Latency Optimization	Real-time factor (RTF), memory-bound vocoders, and batch latency tradeoffs
Audio ALMs	Multimodal Audio-Language Models	Codec audio tokenizers, Residual Vector Quantization (RVQ), and vocoders
Voice Evaluation	Voice Agent Evaluation	PESQ, POLQA, ViSQOL, and simulated acoustic noise testing

---

1.3 LLM Systems Interview Strategy

1. **Be Exact on Roofline Analysis**: When discussing inference latency, distinguish between the **compute-bound prefill phase** (high batch size, parallel context processing) and the **memory-**

bound decode phase (batch size 1, reading model weights from High-Bandwidth Memory (HBM) for every single token).

2. **Explain Paged Memory and DSU Analogy**: Connect PagedAttention back to operating system virtual memory concepts. Explain how the block table acts as page-table mapping, and how references are updated during preemption or swapping.
3. **Know the Quantization outlier math**: Explain that activations have vertical outliers (specific channels with 100× larger magnitudes). Explain how SmoothQuant and AWQ preserve these channels while quantizing the rest.

2 Layer Normalization: LayerNorm → RMSNorm

- **Category**: LLM Systems
- **Difficulty**: Medium
- **Target Role**: LLM Inference Engineer / ML Systems Engineer
- **Source**: NORMALIZATION.md + normalization_output.txt (Zhang & Sennrich 2019, Ba et al. 2016)
- **Flashcards**: LLM Systems deck

2.1 Concept Overview

Think of normalization as a **volume knob** for each token inside a Transformer block. After multiplication-heavy steps like attention and matrix projections, the numbers in each token's feature vector can drift wildly — exploding toward `NaN` or vanishing toward zero. Normalization quietly re-pins every token to a calm, unit-ish scale so the next layer always receives well-conditioned inputs.

LayerNorm (2016) does this in two moves: first it *centers* the vector at zero (subtracts the mean), then *rescales* it to unit spread (divides by the standard deviation). **RMSNorm (2019)** discovered that centering is unnecessary — just dividing by the vector's "loudness" (its root-mean-square) is enough to stabilize training, and it does so **7–64% faster**. This is why every modern open-source LLM (Llama, Mistral, Qwen, Gemma) uses RMSNorm, while GPT-2 still uses LayerNorm.

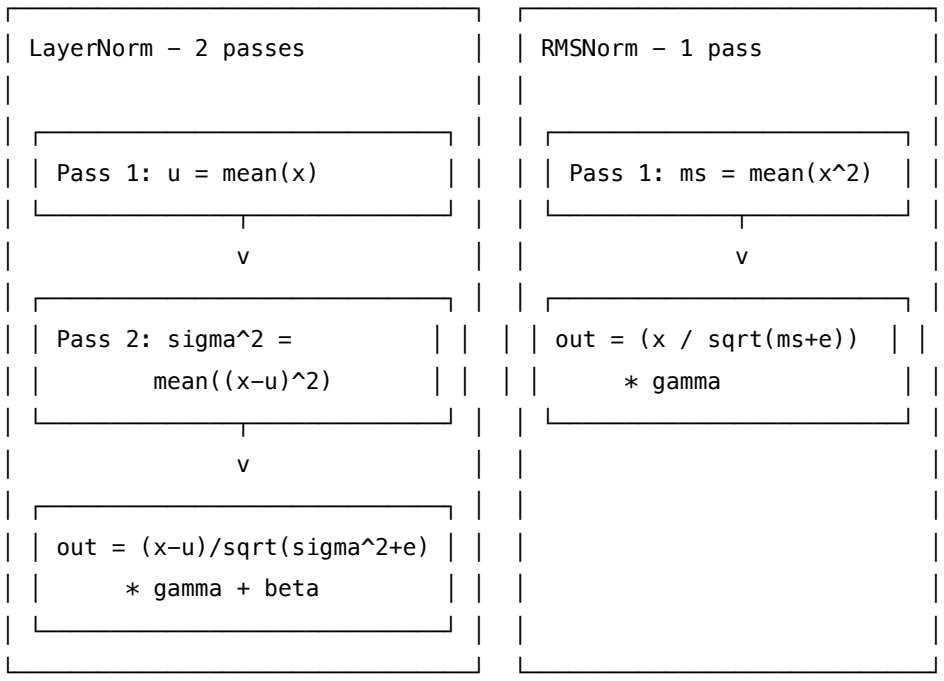
Both normalize over the **last axis** (the feature/embedding dimension `E`), per token, independently — so a batch `[B, L, E]` generates `[B, L, 1]` worth of statistics that are broadcast back over `E`.

2.1.1 The Problem It Solves

Without normalization, a 100-layer Transformer is explosive: - Each block multiplying by 1.05 → after 100 layers, **×131** (explode → `NaN`) - Each block multiplying by 0.95 → after 100 layers, **×0.006** (vanish → dead gradients)

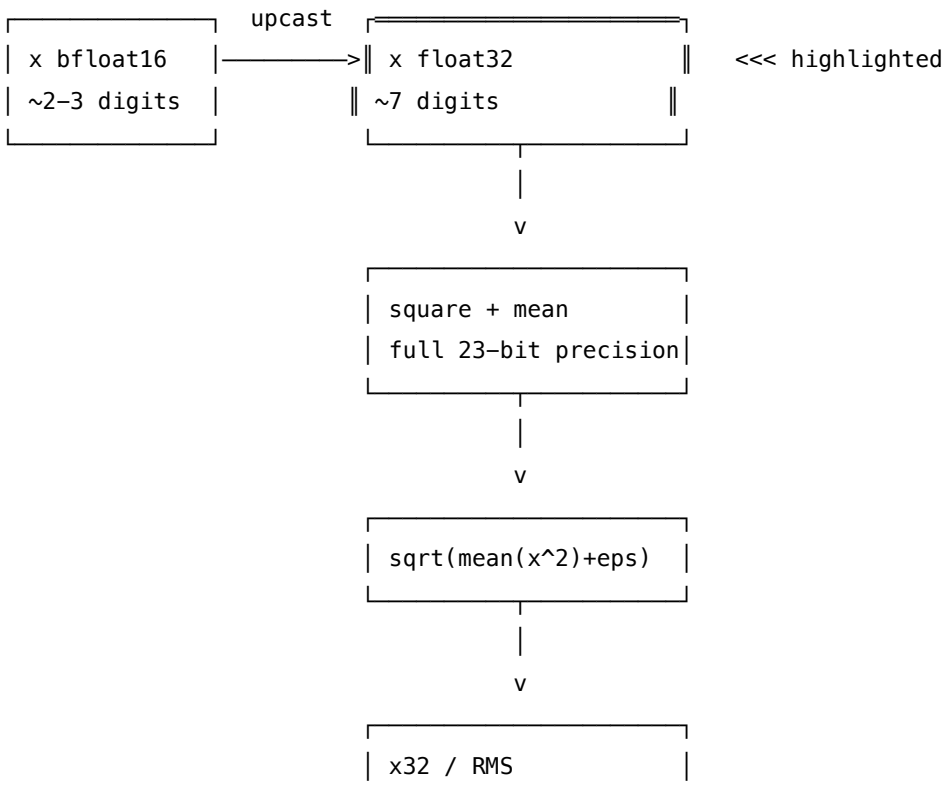
The `normalization_output.txt` confirms this: tiny activations (`x ≈ 1e-22`) squared in `bfloat16` underflow to **exactly 0**, producing `NaN` without the float32 upcast fix.

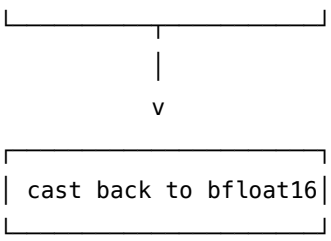
2.1.2 How It Works



RMSNorm step by step (on $x = [1, 2, 3, 4, 5, 6, 7, 8]$, $\gamma = \mathbf{1}$, $\epsilon = 10^{-5}$): 1. **Square each value:** $[1, 4, 9, 16, 25, 36, 49, 64]$ 2. **Average the squares:** $204/8 = 25.5$ 3. **Square root:** $\sqrt{25.5 + 10^{-5}} = 5.049753$ — this is the **RMS** (the loudness) 4. **Divide each value by RMS:** $1/5.049753 = 0.198029$, $2/5.049753 = 0.396059$, and so on. 5. **Apply learned scale:** Multiply by $\gamma = 1$ (no change here).

bfloat16 upcast — MANDATORY:





2.2 Worked Example

2.2.1 LayerNorm on `x = [1,2,3,4,5,6,7,8]`

From `normalization_output.txt` **Section A**:

```
pass 1  mean(x)              = 4.500000
pass 2  var = mean((x-mu)^2) = 5.250000   (BIASED: divides by E not E-1)
        sqrt(var + eps)      = 2.291290
```

field	x	$x - \text{mean}$	$(x - \text{mean})/\text{std}$	$\times\gamma + \beta$
d0	+1.000000	−3.500000	**−1.527524**	−1.527524
d1	+2.000000	−2.500000	**−1.091088**	−1.091088
d2	+3.000000	−1.500000	**−0.654653**	−0.654653
d3	+4.000000	−0.500000	**−0.218218**	−0.218218
d4	+5.000000	+0.500000	**+0.218218**	+0.218218
d5	+6.000000	+1.500000	**+0.654653**	+0.654653
d6	+7.000000	+2.500000	**+1.091088**	+1.091088
d7	+8.000000	+3.500000	**+1.527524**	+1.527524

2.2.2 RMSNorm on the same `x = [1,2,3,4,5,6,7,8]`

From `normalization_output.txt` **Section B**:

```
mean(x^2) = (1/E) * sum(x_i^2) = 25.500000
RMS = sqrt(mean(x^2) + eps)    = 5.049753
NOTE: mean(x) was 4.500000 but RMSNorm IGNORES it.
```

field	x	x^2	x/RMS	$\times\gamma$
d0	+1.000000	+1.000000	**+0.198029**	+0.198029
d1	+2.000000	+4.000000	**+0.396059**	+0.396059
d2	+3.000000	+9.000000	**+0.594088**	+0.594088
d3	+4.000000	+16.000000	**+0.792118**	+0.792118
d4	+5.000000	+25.000000	**+0.990147**	+0.990147
d5	+6.000000	+36.000000	**+1.188177**	+1.188177
d6	+7.000000	+49.000000	**+1.386206**	+1.386206
d7	+8.000000	+64.000000	**+1.584236**	+1.584236

2.2.3 Side-by-Side Comparison (Section C)

From `normalization_output.txt` — same input, both layers:

field	x	LayerNorm	RMSNorm	diff (LN−RN)
d0	+1.0	**−1.527524**	+0.198029	−1.725553
d1	+2.0	**−1.091088**	+0.396059	−1.487147
d3	+4.0	−0.218218	+0.792118	−1.010336
d7	+8.0	+1.527524	**+1.584236**	−0.056712

Key: LayerNorm output mean ≈ 0.000000 (centered). RMSNorm output mean $\approx +0.891133$ (NOT centered). $\max|\text{LN} - \text{RN}| = 1.725553$ — a real, non-trivial difference.

2.2.4 Full Batch: `B=1, L=4, E=8` (Section F)

From `normalization_output.txt` — RMS per token before vs after RMSNorm:

```
[check] RMS per token BEFORE: [0.1468, 0.2461, 0.3458, 0.4456]
[check] RMS per token AFTER : [0.9998, 0.9999, 1.0, 1.0]
```

Token 0 (tiny inputs) and token 3 (larger inputs) both pinned to RMS 1.0 — that is the whole point.

2.2.5 Float32 Upcast Demo (Section D)

Case	mean(x^2) bfloat16	mean(x^2) float32	Result
Small activations ($x \approx 0.001$), $\epsilon = 10^{-5}$	4.202127×10^{-6}	4.189518×10^{-6}	1.003× bias
Tiny activations ($x \approx 10^{-22}$), $\epsilon = 0$	0.000×10^{00} (underflow!)	2.550×10^{-43}	BF16 → **NaN/inf**

2.3 Complexity & Trade-offs

Metric	LayerNorm	RMSNorm	Notes
Reduction passes	2	**1**	Mean then variance vs mean-of-squares
Affine parameters	+ (2× E)	only (1× E)	is the centering knob
Speed	baseline	**7–64% faster**	Zhang & Sennrich NeurIPS 2019
Invariance	re-centering AND re-scaling	re-scaling only	Enough for stable training
Output mean guarantee	0	0 (0.891 on [1..8])	NOT interchangeable
Used by	GPT-2, nanoGPT	Llama, Mistral, Qwen, Gemma	Read checkpoint config!

Metric	LayerNorm	RMSNorm	Notes
Float32 upcast	Mandatory in bf16/fp16	Mandatory in bf16/fp16	Both square activations
eps placement	INSIDE the sqrt	INSIDE the sqrt	$\sqrt{\cdot + \epsilon}$, not $\sqrt{\cdot} + \epsilon$

2.4 Common Interview Questions & How to Answer

2.4.1 Q1: What is the single key difference between LayerNorm and RMSNorm?

- **Answer**: RMSNorm drops the mean-subtraction step. LayerNorm does two passes — compute mean, then compute variance around that mean, then normalize. RMSNorm does one pass — compute mean of squares, take square root (the RMS), divide. The hypothesis from Zhang & Sennrich 2019 (verbatim from the abstract): *"re-centering invariance is dispensable; re-scaling invariance is what actually helps."* This one deletion removes a full reduction pass and the `β` parameter, making RMSNorm 7–64% faster with comparable accuracy.

2.4.2 Q2: Why is the float32 upcast mandatory when running in bfloat16?

- **Answer**: Both LayerNorm and RMSNorm **square** the activations. `bfloat16` has only ~8 mantissa bits (float32 has 23). For small activation values, squaring makes them even smaller, and `bfloat16` rounds the sum of squares to **exactly 0**. Then we divide by $\sqrt{0 + \epsilon} = \sqrt{10^{-5}} \approx 0.003$ — a wildly wrong scale. With $\epsilon = 0$ it's even worse: division by zero → `NaN`. The fix: `x = x.to(torch.float32)` before squaring, then cast back. From the output: bf16 underflows to `0.000e+00` while float32 correctly gives 2.550×10^{-43}.

2.4.3 Q3: LayerNorm guarantees mean=0; does RMSNorm guarantee anything?

- **Answer**: RMSNorm guarantees $\text{mean}(x^2) \approx 1$ (unit mean-square, not unit variance). It does NOT center the output — from the output file, RMSNorm's output has mean $\approx +0.891133$ on the $[1, 2, \ldots, 8]$ test vector, while LayerNorm's output has mean ≈ 0.000000. They are **not interchangeable**: swapping them silently changes every activation. Always check the checkpoint config — Qwen/Llama use RMSNorm; GPT-2/nanoGPT use LayerNorm.

2.4.4 Q4: What axis does normalization run over, and why does that matter?

- **Answer**: Both norms run over the **last axis** (the feature/embedding dimension E), independently per token. A $[B, L, E]$ tensor generates $[B, L, 1]$ statistics broadcast over E. Each **token** is normalized independently — B and L are untouched. If you accidentally normalize over L, you mix information across token positions. Always `dim=-1` or `normalized_shape=[E]`.

2.4.5 Q5: Does LayerNorm use biased or unbiased variance?

- **Answer**: **Biased** — divides by E, not $E - 1$ (Bessel's correction). From the output: var = $\text{mean}((x - \mu)^2) = 5.250000$, computed dividing by 8 not 7. `torch.nn.LayerNorm` uses biased variance. Implementing with `torch.var(x, unbiased=True)` won't match PyTorch bit-for-bit — a classic implementation trap.

2.4.6 Q6: Where does normalization live in the Transformer residual stream?

- **Answer**: Modern LLMs (Llama/Qwen) use **pre-norm**: $x \to$ RMSNorm $\to$ Attention $\to$ add residual $\to$ RMSNorm $\to$ MLP $\to$ add residual. The original Transformer used post-norm (normalize after the add). Additionally, Qwen3/OLMo2/Gemma2 apply a separate **QK-Norm** (per-head RMSNorm on Q and K inside attention, before RoPE) to cap logit magnitudes from ± 60 down to ± 1.7, preventing softmax saturation during training.

2.5 Pro-Tip: How to Impress the Interviewer

- **Quote the paper numbers**: Zhang & Sennrich 2019 (arXiv:1910.07467) report **7–64% speedup** at comparable accuracy. Knowing the paper name and the range shows depth.
- **Cite production code**: "Every production RMSNorm — Llama's `modeling_llama.py`, Qwen's — has `hidden_states = hidden_states.to(torch.float32)` as literally the first line of the forward pass. Miss it in an inference port and you'll get NaN only on extremely small activations — very hard to reproduce."
- **Know the config fields**: Check `rms_norm_eps` (typically `1e-5` or `1e-6`) and whether the architecture uses `RMSNorm` or `LayerNorm`. Swapping is a silent, no-crash failure.
- **The non-interchangeability gotcha**: Mention that LN and RMSNorm differ by $\max|\text{LN} - \text{RN}| = 1.725553$ on a simple [1..8] vector. The outputs are structurally different (one centered, one not), so you cannot swap them in a loaded checkpoint without retraining.
- **Gold values to cite verbatim**: $\text{RMSNorm}([1,2,3,4,5,6,7,8], \epsilon = 10^{-5}, \gamma = 1) = [0.1980, 0.3961, 0.5941, 0.7921$ Having memorized a concrete gold value signals you have actually run the code.

3 Tokenization Pipelines: BPE, WordPiece, SentencePiece

- **Category**: LLM Systems
- **Difficulty**: Medium
- **Target Role**: LLM Inference Engineer / ML Systems Engineer
- **Source**: TOKENIZATION.md + tokenization_output.txt (Sennrich et al. 2016, Kudo & Richardson 2018)
- **Flashcards**: LLM Systems deck

3.1 Concept Overview

A Transformer cannot read strings — it reads **integer token IDs**, each pointing at a row of an embedding table. Tokenization is the deterministic assembly line that turns raw text into those IDs. Think of it as chopping text into **reusable puzzle pieces**: the model has memorized one integer ID for every piece it knows. Short, common pieces (`low`, `est`, `ing`, `_the`) are reused across thousands of words, so a modest vocabulary covers a whole language. Rare or unseen words are assembled from smaller pieces — `lowest` = `low` + `est`.

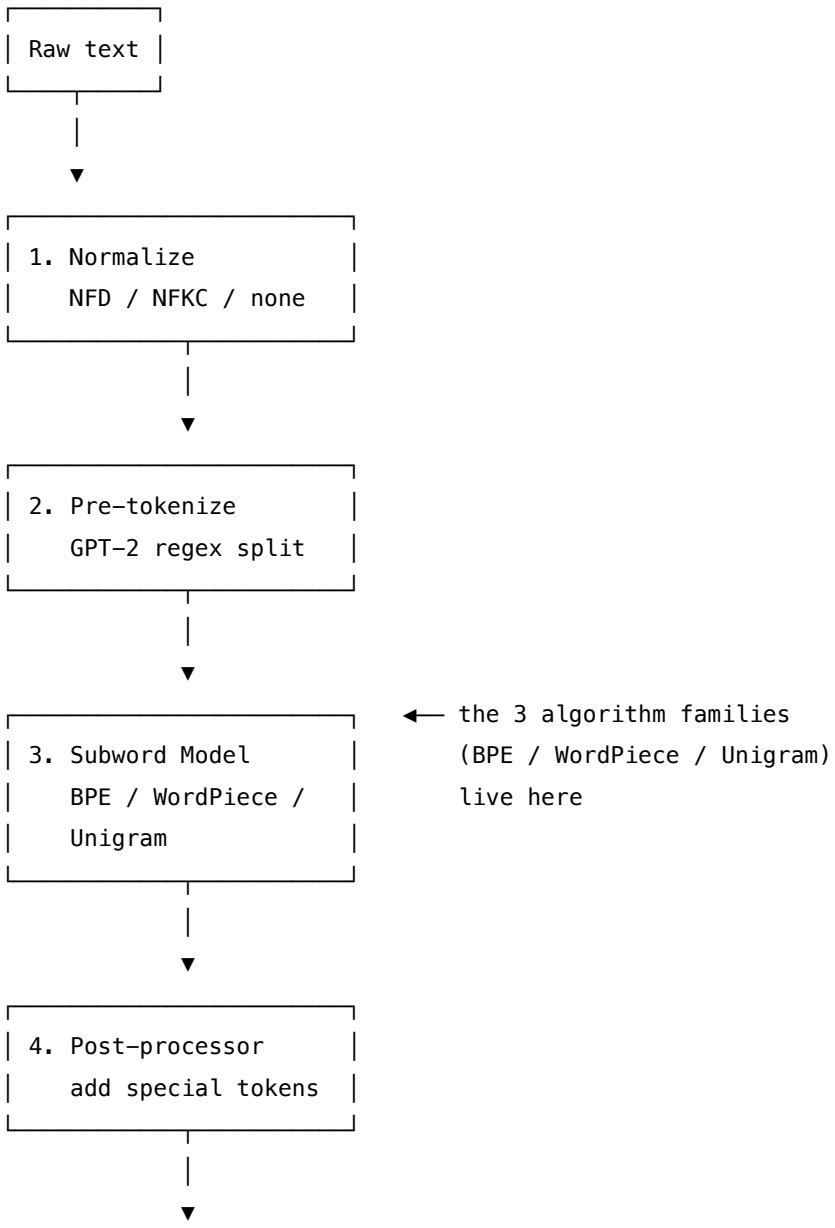
The assembly line has exactly **4 stages**: normalize $\to$ pre-tokenize $\to$ subword model $\to$ post-process.

The **subword model** (stage 3) is where the three algorithm families live: **BPE** (rank-greedy merging, used by GPT-2/Llama/Qwen), **WordPiece** (greedy longest-match, used by BERT), and **SentencePiece** (raw-stream, no pre-split, used for CJK/multilingual). Every LLM in production uses one of these three families — knowing how they differ, when they produce `[UNK]`, and what bugs arise from mixing them is core inference engineering knowledge.

3.1.1 The Problem It Solves

A character-level vocabulary (26 letters) makes sequences impossibly long. A word-level vocabulary cannot handle rare words and requires a massive vocab table. Subword tokenization hits the sweet spot: a vocabulary of ~50k–150k pieces covers any language with manageable sequence lengths. GPT-4's tiktoken vocabulary is 100,277 tokens. Qwen3's BPE vocabulary is 151,936 tokens.

3.1.2 How It Works



```
┌──────────────────────────┐
│ Token IDs                │
│   -> embedding table     │
└──────────────────────────┘
```

BPE Training algorithm (Sennrich et al. 2016): 1. Init vocabulary = every base char/byte in the corpus 2. Count every adjacent symbol pair, weighted by pre-token frequency 3. Merge the most frequent pair → new token; replace everywhere in corpus 4. Repeat `V` times (V = desired vocab size)

BPE Encoding: split to chars, then repeatedly apply the **lowest-rank** (earliest-learned) merge until no learned pair remains.

WordPiece Encoding (BERT): greedy **longest-prefix** match; first piece is bare, continuations get `##` prefix; unknown char → `[UNK]`.

SentencePiece: no pre-tokenization; spaces → `▁` (U+2581); trains BPE or Unigram on the raw stream; works for CJK because it never needs spaces to split words.

3.2 Worked Example

3.2.1 BPE Training on the Gold Corpus

From `tokenization_output.txt` **Section B** — corpus:

```
"low low low low low"
"lower lower newest newest newest"
"newest widest widest"
```

Word frequencies: `low:5, lower:2, newest:4, widest:2`. Initial vocab = 10 sorted unique chars.

Complete merge sequence (12 merges → vocab size 22):

rank	pair	→ new token	count
0	`('l','o')`	`lo`	7
1	`('lo','w')`	`low`	7
2	`('e','s')`	`es`	6
3	`('es','t')`	`est`	6
4	`('n','e')`	`ne`	4
5	`('ne','w')`	`new`	4
6	`('new','est')`	`newest`	4
7	`('low','e')`	`lowe`	2
8	`('lowe','r')`	`lower`	2
9	`('w','i')`	`wi`	2
10	`('wi','d')`	`wid`	2
11	`('wid','est')`	`widest`	2

Final vocabulary (size 22): `[d,e,i,l,n,o,r,s,t,w, lo,low,es,est,ne,new,newest,lowe,lower,wi,wid,widest]`

3.2.2 BPE Encoding: Trace of `lowest → [11, 13]`

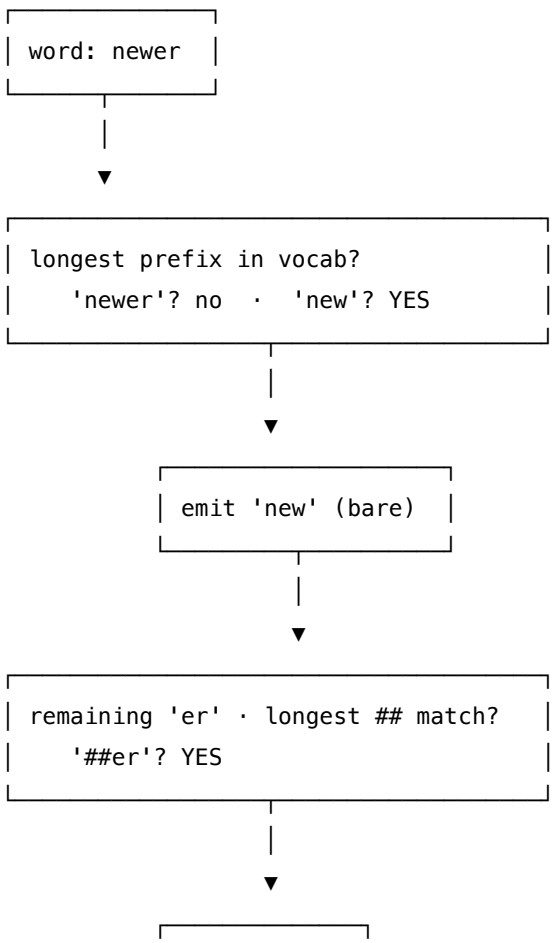
From `tokenization_output.txt` **Section C** — the gold used by the HTML check:

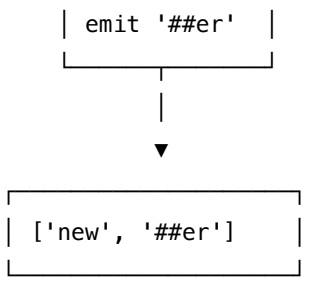
```
start      [l, o, w, e, s, t]
r=0 (l,o)  → [lo, w, e, s, t]        ← lowest-rank pair is ('l','o')
r=1 (lo,w) → [low, e, s, t]
r=2 (e,s)  → [low, es, t]
r=3 (es,t) → [low, est]              ← no learned pair left → stop
IDs: low=11, est=13  =>  [11, 13]
```

word	char split	BPE pieces	token IDs
`lowest`	`l o w e s t`	`['low','est']`	**`[11, 13]`**
`newer`	`n e w e r`	`['new','e','r']`	**`[15, 1, 6]`**
`newest`	`n e w e s t`	`['newest']`	`[16]`
`xyz`	`x y z`	`['x','y','z']`	`[UNK]` (char-level)

`newer` → `[new, e, r]`: no `('e','r')` merge was ever learned, so rank-greedy stops after `new`. This is the key difference from WordPiece.

3.2.3 WordPiece vs BPE (Section D)





word	BPE pieces	WordPiece pieces	key difference
`lowest`	`['low','est']`	`['low','##est']`	Same split, `##` prefix on continuation
`newer`	`['new','e','r']`	`['new','##er']`	WordPiece grabs `##er` directly (longest match)
`xyz`	`['x','y','z']`	`['[UNK]']`	BPE never `[UNK]` if byte-level; WP can

3.2.4 SentencePiece (Section E)

`"low low new new"` → stream: `'low_low_new_new'`

Because there is no pre-tokenization, merges can glue a word-final character to the following space:

rank	pair	→ token	note
0	`('w','_')`	`w_`	space-trailing token
2	`('lo','w_')`	`low_`	space-trailing token
4	`('low_','low_')`	`low_low_`	**inside token — crosses word boundary**

CJK: "□ □ □ □ " = 4 chars = 12 UTF-8 bytes. No spaces → a whitespace pre-tokenizer sees the whole sentence as ONE word. SentencePiece streams it char-by-char and byte-fallback keeps it tokenizable. **This is why Llama/Qwen/T5/ALBERT ship SentencePiece-style tokenizers.**

3.3 Complexity & Trade-offs

Property	BPE	WordPiece	SentencePiece
Train criterion	Most frequent adjacent pair	Max likelihood of training data	BPE or Unigram LM
Encode rule	**Rank-greedy** (lowest rank first)	**Greedy longest-match**	Rank-greedy or Viterbi
Unit trained on	Words (pre-split) or bytes	Words (pre-split)	**Raw stream** (no pre-split)

Property	BPE	WordPiece	SentencePiece
Space handling	`Ġ` via byte map (GPT-2)	Stripped (lost)	Escaped to `▁` U+2581
`[UNK]` possible?	Byte-level: **never**	**Yes**	Byte fallback: **never**
CJK-friendly?	Needs pre-tok	Poor	**Yes**
Used by	GPT-2, GPT-4, Llama, Qwen	BERT	Llama, Qwen, T5, ALBERT
Normalization (stage 1)	None (raw bytes)	NFD + lowercase	NFKC

3.4 Common Interview Questions & How to Answer

3.4.1 Q1: BPE and WordPiece both produce subword tokens — what is the fundamental algorithmic difference?

- **Answer**: The training criterion and the encoding algorithm differ. **BPE training** greedily merges the **most frequent** adjacent pair. **WordPiece training** merges the pair that maximizes training-data **likelihood** (a subtle but important difference). At **encoding time**, BPE applies merges in **rank order** (lowest rank = earliest learned first); WordPiece uses **greedy longest-prefix match** against a fixed dictionary. This is why `newer` → BPE `[new, e, r]` but WordPiece `[new, ##er]` — WordPiece finds `##er` in the dictionary directly, while BPE stops where learned merges ran out.

3.4.2 Q2: What makes byte-level BPE (GPT-2/GPT-4) special compared to character-level BPE?

- **Answer**: The base vocabulary is all 256 byte values instead of Unicode characters. GPT-2 maps each byte through `bytes_to_unicode()` to a printable character, so the base vocab is always 256 and `[UNK]` is **impossible** — any input string is representable. Character-level BPE (like our demo) produces `[UNK]` for `xyz` because those chars were never in the training corpus. In production, byte-level BPE also means you never need special handling for unusual Unicode — every code point decomposes into 1–4 bytes, all of which are in the base vocab.

3.4.3 Q3: Why does SentencePiece work for Chinese/Japanese but word-boundary BPE does not?

- **Answer**: Word-boundary BPE relies on a pre-tokenization step (e.g., the GPT-2 regex) that splits on whitespace and punctuation. Chinese and Japanese have no word boundaries — the whole sentence looks like a single "word" to a whitespace splitter. SentencePiece skips pre-tokenization entirely; it treats the raw character stream as input and escapes spaces to `▁`. So □ □ □ □ is processed char-by-char, and the BPE algorithm can learn useful character n-grams directly. UTF-8 byte fallback ensures every character is representable even if it never appeared in training.

3.4.4 Q4: The GPT-2 pre-tokenization regex uses `\p{L}` — what is the production gotcha?

- **Answer**: Python's standard `re` module does **not** support `\p{L}` (Unicode letter property) — that requires the third-party `regex` module. This is why GPT-2's `encoder.py` depends on `regex`, and

why a naive port that uses the stdlib `re` will silently mis-tokenize strings containing accented letters or non-ASCII characters. The full GPT-2 regex (verbatim): `'s|'t|'re|'ve|'m|'ll|'d| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+`. Each alternation handles a specific linguistic category — contractions, letter runs, digit runs, punctuation runs.

3.4.5 Q5: How do you debug a tokenization mismatch between training and serving?

- **Answer**: The four-stage pipeline must be **identical** at train and serve time: (1) same Unicode normalization (`NFKC` vs `NFD` vs none), (2) same pre-tokenization regex or `▁` escape, (3) same merge ranks from the same vocabulary file, (4) same post-processing template (e.g., `<|im_start|>system...`). The most common mismatches are: wrong normalization form (NFKC at train, none at serve), mixing `Ġ` (GPT-2 byte map) with `▁` (SentencePiece), and non-deterministic merge order on ties. From the output: `[check] merge sequence matches pinned gold (12 merges): OK` — pin the merge sequence to a deterministic order (here: earliest-first on ties).

3.4.6 Q6: What is token healing and when does it matter?

- **Answer**: Token healing arises during **inference serving** when the prompt ends in the middle of a token. Example: the prompt ends with `"import nump"` — `nump` is the start of `numpy` but it tokenizes differently as a standalone word than as a prefix of `numpy`. Without token healing, the model's first generated token is conditioned on a weird boundary. The fix: when the prompt ends mid-token, the server re-tokenizes the boundary (backs up one token, re-tokenizes from there with the next generated token). This is implemented in vLLM and other high-performance inference engines.

3.5 Pro-Tip: How to Impress the Interviewer

- **Know the encoding algorithm type**: "BPE encodes by **rank-greedy** (lowest merge rank first, not longest match). WordPiece encodes by **greedy longest-prefix match**. The difference is testable: `newer` → BPE `[new,e,r]` but WordPiece `[new,##er]`." Most candidates blur these.
- **Qwen3 tokenizer specifics**: Qwen3's vocabulary is 151,936 tokens with special tokens `<|im_start|>`, `<|im_end|>`, `<|endoftext|>`. The training algorithm is byte-level BPE (not SentencePiece), though they may apply SentencePiece-style NFKC normalization first.
- **The 1 word ≈ 1–3 tokens rule**: English text averages ~ 1.3 tokens/word. A CJK character is typically 3 UTF-8 bytes → up to 3 tokens. Never equate tokens with words in latency/cost estimates.
- **The tie-break matters for reproducibility**: BPE tie-breaking (equal pair frequencies) must be deterministic. The gold in this guide uses earliest-first-seen order. A different tie-break produces a different vocabulary — two "identical" trainers that differ only in tie-break will diverge silently.
- **Gold to cite**: `lowest` → `[low, est]` → IDs `[11, 13]`; `newer` → `[new, e, r]` → IDs `[15, 1, 6]`. 12 merges, vocab size 22 on the standard BPE demo corpus.

4 Positional Encoding: Absolute PE vs RoPE

- **Category**: LLM Systems
- **Difficulty**: Hard

- **Target Role**: LLM Inference Engineer / ML Systems Engineer
- **Source**: ROPE.md + ABSOLUTE_PE.md + rope_output.txt (Su et al. 2021, Vaswani et al. 2017, Radford 2019)
- **Flashcards**: LLM Systems deck

4.1 Concept Overview

The Transformer's attention mechanism is a dot product $Q \cdot K$ that is completely **blind to token order** — shuffle the tokens and every score stays the same. Position information must be injected from outside. There are two fundamentally different families:

Absolute PE (the "stamp-a-barcode" family) adds a position-specific vector to the token's embedding once at the input, before any Transformer block. Each token carries its seat number permanently baked in. GPT-2 uses a **learned** version (an `nn.Embedding` table `wpe`); the original Transformer uses a **sinusoidal** version (fixed sin/cos formula).

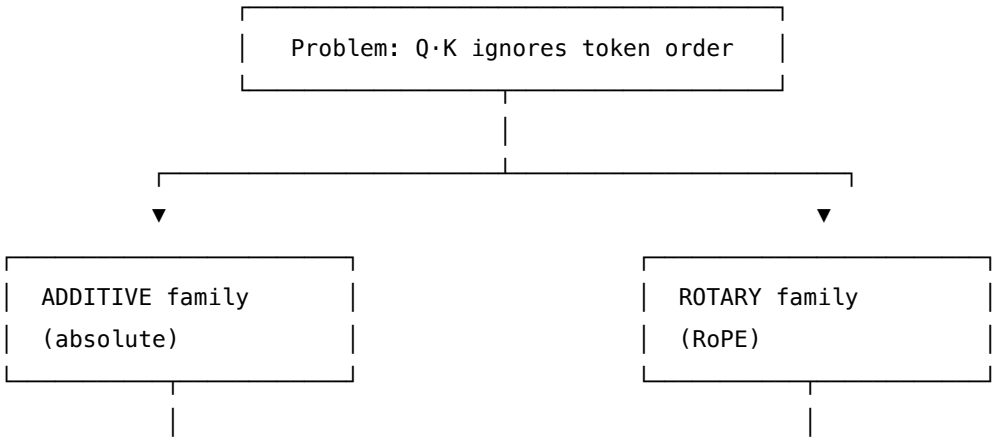
RoPE (Rotary Position Embedding, the "spin-a-compass-needle" family) works differently: instead of adding a barcode, it *rotates* the Query and Key vectors by an angle proportional to their position, inside **every** attention layer. The key insight: when two rotated vectors are dot-producted, the absolute angles cancel and only the *relative angle difference* (i.e., the distance between tokens) survives. Relative position emerges for free from a rotation — no explicit relative-position table needed.

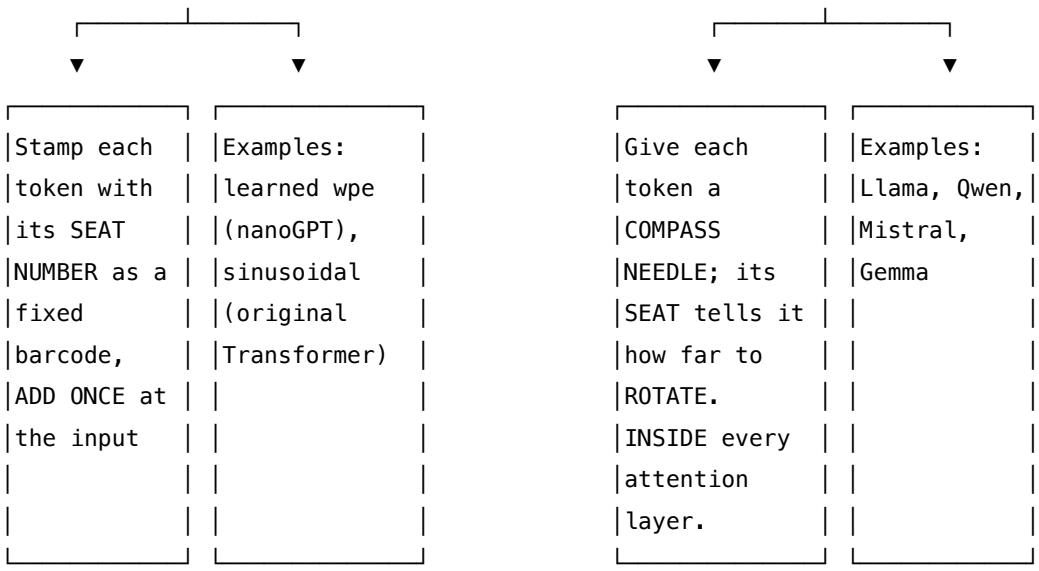
This guide covers both families side by side. RoPE is used by Llama, Qwen, Mistral, Gemma. Absolute PE (learned) is used by GPT-2/nanoGPT.

4.1.1 The Problem It Solves

Without position encoding, `"The cat sat"` and `"sat cat The"` produce identical attention scores and identical model outputs. From the output file: with RoPE, $Q \cdot K$ score for gap $= 1$ is always **+0.514498** regardless of absolute seat (positions $(2,1)$, $(5,4)$, $(10,9)$ all give the same score). With absolute PE, the same gap $= 1$ gives **+7.0556, +4.9014, +7.2618** — completely different for different seats. RoPE encodes relative distance; absolute PE encodes absolute position.

4.1.2 How It Works

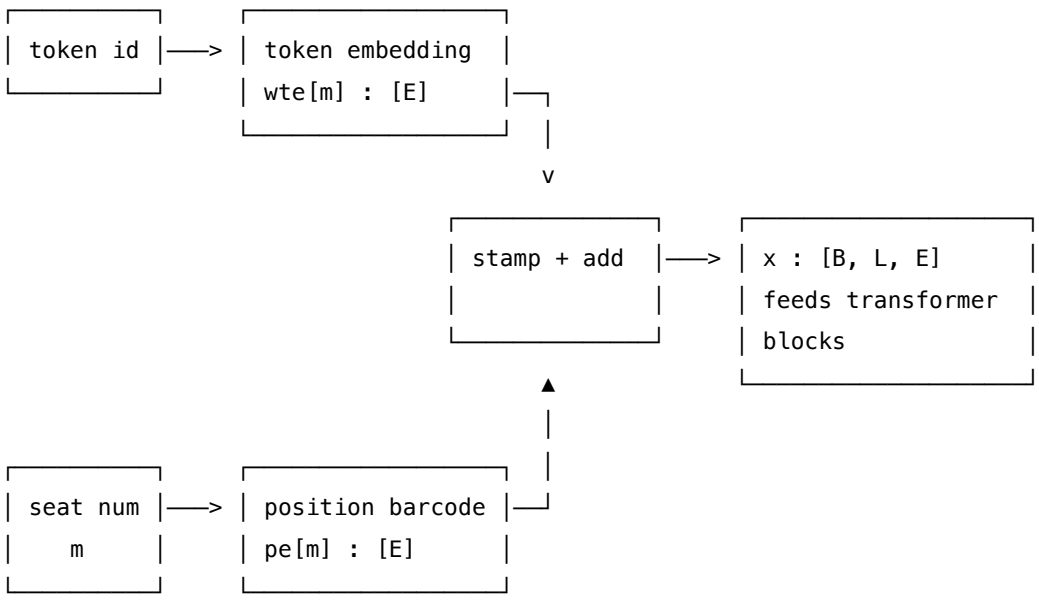




4.2 Family 1: Absolute Positional Encoding

4.2.1 Where It Lives

Applied **once** at the input embedding, to the **full model dimension `E`**:



This is the defining property: absolute PE operates on `[B, L, E]`, added **once** before any block. Contrast with RoPE, which operates on `[B, L, H, D]` inside every block.

4.2.2 The Shared Frequency Ladder

Both families use the same inverse-exponential frequency structure:

$$\theta_i = \text{base}^{-\frac{2i}{\text{dim}}}$$

(same formula, different dim: E for absolute, D for RoPE)

From `rope_output.txt` **Section A** (`D=8, base=10000`):

j	_j	meaning
0	**1.000000**	FAST rotation — tracks local position
1	**0.100000**	FAST
2	**0.010000**	SLOW — tracks long-range position
3	**0.001000**	SLOW

The same four frequencies appear in the sinusoidal PE table (from `absolute_pe.py` Section A) — because the formula is identical. The **operation** differs: RoPE uses them as rotation angles; absolute PE uses them as values to add.

4.2.3 Sinusoidal PE Table (Section B from absolute_pe output)

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{\text{base}^{2i/E}}\right), \quad \text{PE}(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{\text{base}^{2i/E}}\right)$$

m	d0 (sin)	d1 (cos)	d2 (sin)	d3 (cos)	d6 (sin)	d7 (cos)
0	0.0000	1.0000	0.0000	1.0000	0.0000	1.0000
1	0.8415	0.5403	0.0998	0.9950	0.0010	1.0000
2	0.9093	−0.4161	0.1987	0.9801	0.0020	1.0000
3	0.1411	−0.9900	0.2955	0.9553	0.0030	1.0000

The high-frequency pair (d0/d1) swings wildly; the low-frequency pair (d6/d7) barely moves. Seat 0 gets the neutral barcode `[0,1,0,1,0,1,0,1]` (sin(0)=0, cos(0)=1 everywhere).

4.2.4 Learned PE (nanoGPT `wpe`)

No formula — just a table `nn.Embedding(block_size, n_embd)` trained by SGD:

```
self.wpe = nn.Embedding(block_size, n_embd)   # [max_len, E], trained
x = self.wte[idx] + self.wpe[pos]             # token emb + learned pos emb
```

Initial values are small random noise (~`*0.02`). The model learns what each seat's barcode should look like during training.

4.2.5 Why Absolute PE Cannot Encode Relative Position

From `absolute_pe.py` **Section E** — same fixed Q and K, different absolute seats:

m_q	m_k	gap = m_q−m_k	Q · K score
2	1	**1**	**+7.055621**
5	4	**1**	**+4.901406**

m_q	m_k	gap = m_q−m_k	Q · K score
10	9	**1**	**+7.261821**
2	0	**2**	**+5.189337**
5	3	**2**	**+4.768941**
10	8	**2**	**+6.810799**

Same gap, completely different scores. Adding the barcode bakes m_q into Q and m_k into K separately. The dot product then has cross terms $pe[m_q] \cdot pe[m_k]$, $q \cdot pe[m_k]$, $pe[m_q] \cdot k$ — no trig identity cancels them to a function of $m_q - m_k$ alone.

4.3 Family 2: RoPE (Rotary Position Embedding)

4.3.1 The Compass Analogy

Each token holds $D/2$ compass needles (called "clock dials"). The token's seat number m tells each needle how far to rotate. When two rotated tokens are later compared (dot product $Q \cdot K$), what survives is only the **difference** in rotation angles — i.e., only $m_q - m_k$ survives. Absolute positions cancel; relative distance appears for free.

4.3.2 The Frequency Table (Section A)

From `rope_output.txt` **Section A** ($D = 8, \text{base} = 10000$):

j	inner = $j/(D/2)$	$\theta_j = \text{base}^{-\text{inner}}$	meaning
0	0.0000	**1.000000**	FAST rotation — tracks local position
1	0.2500	**0.100000**	FAST
2	0.5000	**0.010000**	SLOW — tracks long-range position
3	0.7500	**0.001000**	SLOW — barely moves over 1000 seats

Pair $j = 0$ is a twitchy stopwatch; pair $j = 3$ is a slow hour-hand. Together they record both fine-grained and coarse position simultaneously — a Fourier-like decomposition.

4.3.3 Precomputed Lookup Tables (Section B+C)

Angle table $\text{angle}[m, j] = m \cdot \theta_j$ (radians):

mJ	j=0	j=1	j=2	j=3
m=0	0.0000	0.0000	0.0000	0.0000
m=1	1.0000	0.1000	0.0100	0.0010

m\j	j=0	j=1	j=2	j=3
m=2	2.0000	0.2000	0.0200	0.0020
m=3	3.0000	0.3000	0.0300	0.0030

COS table (what gets looked up at seat $m = 2$):

m\j	j=0	j=1	j=2	j=3
m=2	**−0.4161**	0.9801	0.9998	1.0000

SIN table at $m = 2$:

m\j	j=0	j=1	j=2	j=3
m=2	**0.9093**	0.1987	0.0200	0.0020

Seat 0's row is all $\cos = 1, \sin = 0 \rightarrow$ identity rotation (no spin). Seat 3's $j = 0$ clock has $\cos = -0.99$ (swung almost 180°); $j = 3$ clock is still at $\cos \approx 1.000$ (barely moved).

4.3.4 Rotating One Token (Section D)

Input token x at position $m = 2$, head dim $D = 8$: - x_1 (first half) = `[1.0, 0.5, -0.3, 0.8]` - x_2 (second half) = `[0.2, -0.1, 0.4, 0.6]`

Per-pair rotation (complex multiply $(x_1 + i \cdot x_2) \cdot (\cos + i \cdot \sin)$):

pair j	x1	x2	cos	sin	real = x1 · cos − x2 · sin	imag = x2 · cos + x1 · sin
0	+1.0	+0.20	−0.4161	+0.9093	**−0.5980**	**+0.8261**
1	+0.5	−0.10	+0.9801	+0.1987	**+0.5099**	**+0.0013**
2	−0.3	+0.40	+0.9998	+0.0200	**−0.3079**	**+0.3939**
3	+0.8	+0.60	+1.0000	+0.0020	**+0.7988**	**+0.6016**

Reassemble (concat real then imag):

```
RoPE(x, m=2) = [-0.598, 0.5099, -0.3079, 0.7988, 0.8261, 0.0013, 0.3939, 0.6016]
original x   = [ 1.0,   0.5,   -0.3,    0.8,   0.2,  -0.1,   0.4,   0.6  ]
```

Norm preserved: $\max \left| \|\text{out}\| - \|\text{in}\| \right| = 2.98 \times 10^{-8}$ — rotation never stretches a vector.

4.3.5 Full Batch Output (Section E)

From `rope_output.txt` — head h=0, before/after:

m	d0	d1	d4	d7
0	+0.001	+0.002	+0.005	+0.008
1	**−0.034**	+0.091	**+0.142**	+0.108
2	**−0.270**	+0.157	**+0.097**	+0.208
3	**−0.341**	+0.198	**−0.259**	+0.309

Row m=0 is unchanged (identity rotation). Rows get progressively more churned as `m` grows. The slow clocks (j=2,3, affecting d6/d7) barely deviate from input even at m=3.

4.3.6 The Relative Position Proof (Section H)

The math: rotating Q by $m_q \cdot \theta$ and K by $m_k \cdot \theta$, then taking the dot product:

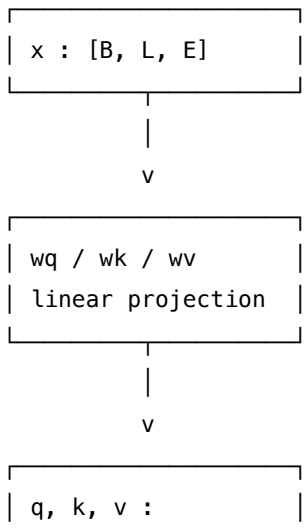
$$\text{clock contribution} = (q_1 k_1 + q_2 k_2)\cos((m_q - m_k)\theta) + (q_1 k_2 - q_2 k_1)\sin((m_q - m_k)\theta)$$

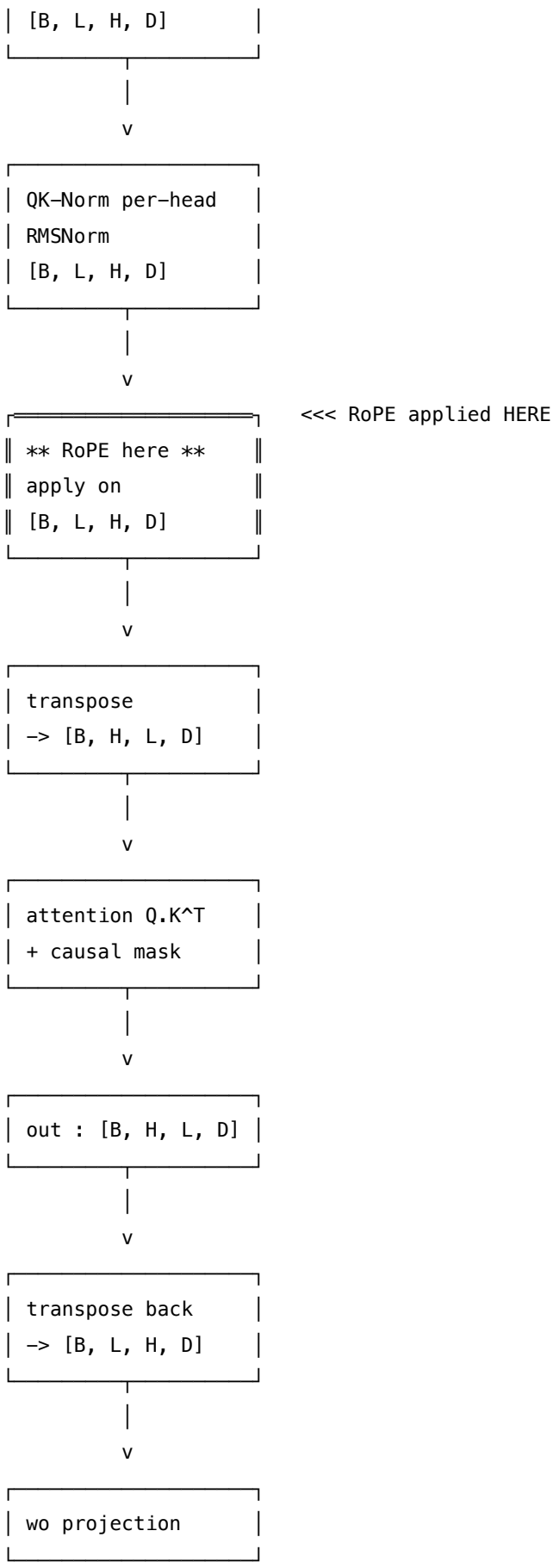
Only $(m_q - m_k)$ survives. From `rope_output.txt` **Section H**:

m_q	m_k	relative = m_q−m_k	Q · K score
2	1	**1**	**+0.514498**
5	4	**1**	**+0.514498**
10	9	**1**	**+0.514499**
2	0	**2**	**+0.285792**
5	3	**2**	**+0.285792**
10	8	**2**	**+0.285792**

Same gap → identical score, regardless of absolute seat. This is the property absolute PE structurally cannot provide.

4.3.7 Where RoPE Operates: The Tensor Shape Dance




[B, L, H, D]
QK-Norm per-head
RMSNorm
[B, L, H, D]
** RoPE here **
apply on
[B, L, H, D]
<<< RoPE applied HERE
transpose
-> [B, H, L, D]
attention Q.K^T
+ causal mask
out : [B, H, L, D]
transpose back
-> [B, L, H, D]
wo projection

Critical: RoPE must be applied **while still in `[B, L, H, D]`**, BEFORE the transpose to `[B, H, L, D]`. The position axis is `L`, and cos/sin tables index by `L`. After transposing, the L axis is in the wrong slot for a clean per-position lookup.

4.3.8 Split vs Traditional Layout (Section F)

Two ways to pair coordinates into clock dials:

Layout	Pairs	Key
Split (Llama/Qwen, `traditional=False`)	(dim0, dim4), (dim1, dim5), ...	`x1=x[..., :D/2]`, `x2=x[..., D/2:]`
Traditional (GPT-NeoX, `traditional=True`)	(dim0, dim1), (dim2, dim3), ...	reshape `x[..., D/2, 2]`

From `rope_output.txt` **Section F** — same input `x=[1..8]`, rotated at m=2:

```
Split:       [-4.9626, 0.7681, 2.8594, 3.984, -1.1714, 6.2777, 7.0586, 8.008]
Traditional: [-2.2347, 0.077, 2.1455, 4.5163, 4.879, 6.0988, 6.984, 8.014]
```

Different outputs. No error. Silent corruption if you use the wrong one. Always read the checkpoint config. Qwen3/Llama = `traditional=False`.

4.3.9 The Offset Parameter: KV Cache Prefill vs Decode (Section G)

During decode, each new token has a **true seat number** equal to the current sequence length, not position 0:

```
CORRECT offset=slice(3,4) → row m=3: [-0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.003]
WRONG   offset=slice(0,1) → row m=0: [ 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0  ]

[check] decode(offset=3) == prefill_all[3]? True
```

With the wrong offset (`slice(0,1)`), the new token's needle doesn't spin at all — it looks like it's at position 0, destroying all relative-position information. Output turns to nonsense after prefill. Fix: `offset = slice(current_len, current_len + 1)`.

4.4 Complexity & Trade-offs

Property	Absolute PE (sinusoidal/learned)	RoPE
Mental model	Stamp a seat-number barcode	Spin a compass needle
Operation	`x + pe[m]` (additive)	rotate `(x1+i·x2)·e^(imθ)`
Where applied	Input, **once**, `[B, L, E]`	Every layer, Q&K only, `[B, L, H, D]`
Frequency ladder	Same `[1, 0.1, 0.01, 0.001]`	Same `[1, 0.1, 0.01, 0.001]`
Norm preservation	**NO** (addition changes length)	**YES** (`max drift ≈ 3e-8`)

Property	Absolute PE (sinusoidal/learned)	RoPE
Encodes	Absolute seat number	**Relative** gap
Q · K depends on	`m_q` AND `m_k` separately	Only `m_q - m_k`
Length generalization	Weak (learned: hard cap at max_len)	Strong (YaRN/NTK scaling)
Used by	GPT-2, nanoGPT	Llama, Qwen, Mistral, Gemma
Base (theta)	N/A	10000 (Llama-classic); **1,000,000** (Qwen3)

4.5 Common Interview Questions & How to Answer

4.5.1 Q1: Why does RoPE encode relative position but absolute PE does not?

- **Answer**: It's a consequence of the math. With absolute PE, adding a position vector `pe[m]` to the token embedding bakes `m` permanently into the vector. When you dot-product two such vectors, you get cross-terms like `pe[m_q]·pe[m_k]`, `q·pe[m_k]`, `pe[m_q]·k` — these are functions of both `m_q` and `m_k` **separately**, not just their difference. No trig identity collapses them. With RoPE, rotating Q by `m_q·θ` and K by `m_k·θ` and then multiplying uses the identity that `cos(A)cos(B) + sin(A)sin(B) = cos(A−B)`, so the dot product becomes a function of `(m_q - m_k)·θ` only — absolute positions cancel exactly.

4.5.2 Q2: What is the RoPE frequency table and why does it use a range of speeds?

- **Answer**: The frequency table has `D/2` entries where `θ_j = base^(−j/(D/2))`. With `D=8, base=10000`, that gives `[1.0, 0.1, 0.01, 0.001]` — a geometric sequence spanning 3 orders of magnitude. The fast pairs (j=0: =1.0) rotate by 1 radian per seat, so they can distinguish nearby tokens (they're fully differentiated by position 3). The slow pairs (j=3: =0.001) barely move across 1000 seats, so they encode coarse/global position. Together, it's a Fourier-like decomposition of position — fine-grained and coarse at the same time, with a single compact table.

4.5.3 Q3: What is the split vs traditional layout in RoPE, and what breaks if you mix them?

- **Answer**: Both layouts apply the same rotation math but pair different coordinate dimensions. **Split** (Llama/Qwen, `traditional=False`): clock j pairs dim `j` with dim `j + D/2` — i.e., `x1 = x[..., :D/2]`, `x2 = x[..., D/2:]`. **Traditional** (GPT-NeoX, `traditional=True`): clock j pairs dim `2j` with dim `2j+1` — adjacent coordinates. From the output: the same vector at m=2 gives `[−4.963, 0.768, 2.859, ...]` under split but `[−2.235, 0.077, 2.146, ...]` under traditional. If you mix them, you get wrong outputs with no error message — the shapes match, the computation runs, but every attention score is wrong. Always read `config.json` for the `rope_scaling` and traditional flag.

4.5.4 Q4: Why is the `offset` parameter critical during KV cache decode, and what is the failure mode?

- **Answer**: During decode, only one new token is processed per step, but it must be rotated at its **true** position `m = current_len`, not position 0. The `offset` parameter tells RoPE which row of the precomputed cos/sin table to look up: `offset = slice(current_len, current_len + 1)`. If you forget the offset and use `slice(0, 1)`, you look up row 0 (the identity rotation — cos=1, sin=0), meaning the new token's needle never spins. Its Q is unrotated, while the cached K vectors were correctly rotated at positions 0,1,2,... — the relative geometry is destroyed and the model emits nonsense. No crash, no error. From the output: correct gives `[..., 0.003]` in the last dimension; wrong gives `[..., 0.0]`.

4.5.5 Q5: What is the difference between `base=10000` (Llama) and `base=1000000` (Qwen3)?

- **Answer**: A larger base means **slower** overall rotation — at position `m`, the angle for pair `j` is `m · base^(−j/(D/2))`. With `base=10000`, position 1000 has angle 1000 for the fastest pair — already past multiple full rotations, starting to alias. With `base=1000000`, position 1000 has angle $1000 \cdot (10^{6})0 = 1000$ for j=0 but the slower pairs are 10× slower, effectively stretching the model's "distance scale" by 100×. This is a form of context length extrapolation — a larger base shifts the aliasing problem to much longer sequences. Qwen3's choice of `base=1_000_000` is why it handles 128k+ context better than Llama-classic with `base=10_000`.

4.5.6 Q6: What is the tensor shape order for applying RoPE, and why does it matter?

- **Answer**: RoPE must be applied while the tensor is in `[B, L, H, D]` layout, **before** the transpose to `[B, H, L, D]`. Reason: position is the `L` axis, and the cos/sin lookup tables are indexed by `L`. In `[B, L, H, D]`, the L axis is at position 1 and is directly accessible for per-position rotation; broadcasting over H works cleanly. After transposing to `[B, H, L, D]`, L is at position 2 — the code still works but the convention in all reference implementations (Qwen, Llama) is to rotate first. If you apply RoPE after the transpose using the wrong axis, you'll rotate over heads instead of positions — silent garbage with no shape error.

4.6 Pro-Tip: How to Impress the Interviewer

- **The relative-position proof in one breath**: "RoPE's dot product collapses to `cos((m_q−m_k)·θ)` terms via the trig identity `cos(A−B) = cosA·cosB + sinA·sinB`. The absolute positions `m_q` and `m_k` cancel — only their difference survives. Absolute PE has no such cancellation because addition bakes both positions in separately."
- **Know the Qwen3 specifics**: `rope_theta = 1_000_000` (not 10,000), `traditional = False` (split layout), per-head RMSNorm (QK-Norm) applied **before** RoPE. Getting these three details right in a code review is what separates inference engineers from ML practitioners.
- **Norm preservation is a useful invariant**: "RoPE preserves L2 norm exactly (max drift 2.98e−8 from the output file). This means ‖ `Q_rotated` ‖ = ‖ `Q_raw` ‖, so the attention score magnitude `Q·K/√D` is controlled purely by the query/key content, not position. Absolute PE doesn't have this property — adding a barcode changes vector length."

- **Gold values to cite**: `RoPE(x=[1..8], m=2) = [-0.598, 0.510, -0.308, 0.799, 0.826, 0.001, 0.394, 0.602]` (split layout). Gap-1 score = `+0.514498` regardless of absolute seat.
- **YaRN/NTK connection**: For context extension beyond training length, mention that RoPE's base can be interpolated (NTK-aware scaling, YaRN) to extend context 2–32× with minimal fine-tuning. Absolute PE (especially learned `wpe`) hard-crashes at `max_len` with an index error — no such extension is possible without retraining the position table.

5 MLP Activations: ReLU → GELU → SiLU & SwiGLU

- **Category**: LLM Systems
- **Difficulty**: Medium
- **Target Role**: LLM Inference Engineer / ML Systems Engineer
- **Source**: MLP_ACTIVATION.md + mlp_activation_output.txt (Hendrycks & Gimpel 2016, Shazeer 2020)
- **Flashcards**: LLM Systems deck

5.1 Concept Overview

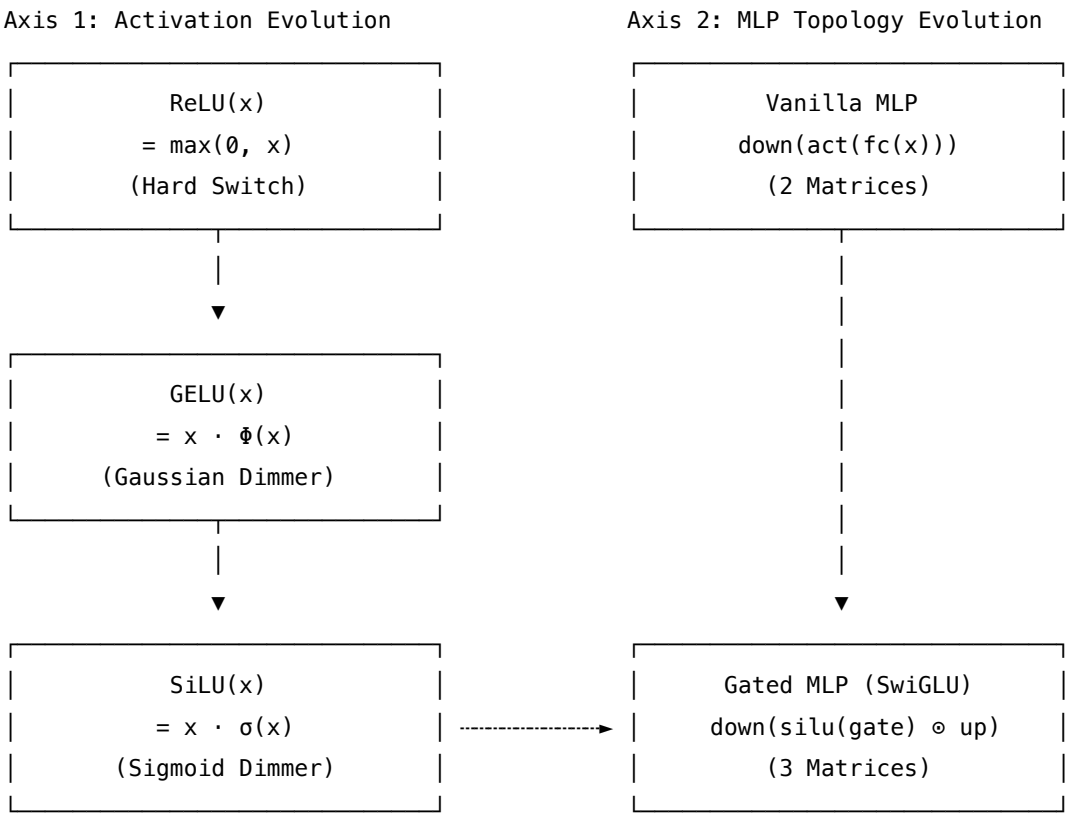
Think of a neural network's activation function as a **decision gate** or **dimmer switch** that controls how much signal flows through a neuron, while the MLP (Multi-Layer Perceptron) block is a **tiny brain** that mixes and extracts features after the attention layer. Traditional networks used **ReLU** (a hard wall switch that cuts off everything below zero), but modern LLMs use smooth, probabilistic curves like **GELU** (in GPT-2) and **SiLU** (in LLaMA/Qwen). The ultimate evolution is **SwiGLU**, which introduces a **gated faucet**: one path acts as the 'handle' that regulates flow (`silu(gate)`), another path is the 'water' (`up`), and their element-wise product is passed downstream. By dropping the mean-centering and employing multiplicative gating, SwiGLU achieves significantly better learning capacity.

5.1.1 The Problem It Solves

Without proper activation functions and MLP architectures, deep networks fail to learn or run efficiently: - **Dying ReLUs**: In traditional networks using ReLU ($\max(0, x)$), any neuron with a negative activation gets a gradient of exactly 0 during backward passes. It becomes stuck "off" forever, leading to underutilized capacity. - **Optimization Bottlenecks**: Discontinuous boundaries (like the kink in ReLU at $x = 0$) introduce optimization noise. Smooth activation functions permit more stable gradient propagation. - **Representational Capacity**: Vanilla MLPs apply a non-linearity to a single projection and then map it back. SwiGLU replaces this with a gated structure, resolving the representation bottleneck by letting features conditionally gate each other. - **Silent Implementation Corruption**: In SwiGLU, if the activation function is incorrectly applied to the wrong branch (e.g., `silu(up) * gate` instead of `silu(gate) * up`), the tensor shapes remain identical, and code runs with no errors. However, the outputs are corrupted, leading to an absolute difference of up to **0.0035** on a small test tensor and causing silent model degradation.

5.1.2 How It Works

Modern MLP blocks evolved along two independent axes: activation functions and network topology.



5.1.2.1 Step-by-Step Gated MLP (SwiGLU) Pipeline:

1. **Parallel Projections**: Project the input tensor x (shape `[B, L, E]`) into two parallel branches of intermediate dimension `FFN` (typically tuned to $\approx 5.4 \times E$ in Qwen3-0.5B):
   - $\text{gate}(x) = xW_{\text{gate}}$
   - $\text{up}(x) = xW_{\text{up}}$
2. **Gating Nonlinearity**: Apply the SiLU activation function only to the gate path:
   - $\text{silu\_gate} = \text{silu}(\text{gate}(x)) = \text{gate}(x) \cdot \sigma(\text{gate}(x))$
3. **Element-wise Multiplication (Gated Fusion)**: Multiply the activated gate and raw up-projection to let the gate regulate the features:
   - $\text{gated\_features} = \text{silu\_gate} \odot \text{up}(x)$
4. **Output Projection**: Mix and project the gated features back to the model dimension E:
   - $\text{output} = \text{gated\_features}W_{\text{down}}$

5.2 Worked Example

5.2.1 Activation Functions Comparison

For the input grid $x = [-2.0, -1.0, -0.5, 0.0, 0.5, 1.0, 2.0, 3.0]$, the table below demonstrates how the non-linearities behave (sourced from `mlp_activation_output.txt`):

x	$\mathrm{ReLU}(x)$	$\mathrm{GELU}_{\mathrm{tanh}}(x)$	$\mathrm{GELU}_{\mathrm{exact}}(x)$	$\mathrm{SiLU}(x)$
-2.0	$+0.0000$	-0.0454	-0.0455	$\mathbf{-0.2384}$
-1.0	$+0.0000$	-0.1588	-0.1587	$\mathbf{-0.2689}$
-0.5	$+0.0000$	-0.1543	-0.1543	-0.1888
$+0.0$	$+0.0000$	$+0.0000$	$+0.0000$	$+0.0000$
$+0.5$	$+0.5000$	$+0.3457$	$+0.3457$	$+0.3112$
$+1.0$	$+1.0000$	$\mathbf{+0.8412}$	$+0.8413$	$\mathbf{+0.7311}$
$+2.0$	$+2.0000$	$+1.9546$	$+1.9545$	$+1.7616$
$+3.0$	$+3.0000$	$+2.9964$	$+2.9960$	$+2.8577$

Key takeaway: ReLU zeros out negative values completely, GELU allows a small negative bleed (minimum at ≈ -0.16), and SiLU yields a deeper negative lobe (floor at ≈ -0.278 near $x = -1.278$).

5.2.2 SwiGLU Step-by-Step Execution

For input token $m = 0$: `[0.9635, 0.7436, 0.4504, -1.0528, 0.3392, -0.6173, -0.0215, -0.8023]` (dimensions: $E = 8, FFN = 16$), the first 6 intermediate values are:

FFN Index	$\mathrm{gate}(x)$	$\mathrm{silu}(\mathrm{gate}(x))$	$\mathrm{up}(x)$	$\mathrm{silu}(\mathrm{gate}) \odot \mathrm{up}$
0	$+0.1026$	$+0.0539$	-0.0122	$\mathbf{-0.0007}$
1	-0.2043	-0.0917	-0.1630	$\mathbf{+0.0150}$
2	-0.1920	-0.0868	-0.0385	$\mathbf{+0.0033}$
3	$+0.1967$	$+0.1080$	-0.3284	$\mathbf{-0.0355}$
4	$+0.3060$	$+0.1762$	-0.0111	$\mathbf{-0.0020}$
5	$+0.1981$	$+0.1088$	$+0.4402$	$\mathbf{+0.0479}$

Applying the final linear transformation W_{down} results in the output vector for token $m = 0$: `[0.0120, 0.0047, 0.0058, -0.0141, 0.0046, 0.0084, -0.0110, 0.0173]`. The gold pin value at index `[0, 0, 0]` is exactly **0.012014**.

5.2.3 The Operand-Order Pitfall

Swapping the branches to compute `silu(up) * gate` instead of `silu(gate) * up` yields incorrect output logits (dim $d = 0$ comparison):

Token index (m)	y_{correct} ($\mathrm{silu}(\mathrm{gate}) \odot \mathrm{up}$)	y_{buggy} ($\mathrm{silu}(\mathrm{up}) \odot \mathrm{gate}$)	Absolute Difference
0	$+0.0120$	$+0.0133$	0.0013
1	$+0.0056$	$+0.0048$	0.0008
2	$+0.0052$	$+0.0077$	0.0025
3	-0.0042	-0.0043	0.0000

Maximum absolute difference over the whole tensor: ***0.0035****.*

5.3 Complexity & Trade-offs

Metric	Value	Notes
Weight Matrices	3 (`gate_proj`, `up_proj`, `down_proj`)	Vanilla MLPs only require 2 matrices (`fc`, `proj`)
Parameter Size	$3 \cdot E \cdot FFN$	In Qwen3-0.5B, this accounts for ~88% of all hidden layer parameters
FFN Dimension Ratio	$\approx 5.43\times$ (e.g., $FFN = 4864$ for $E = 896$)	GPT-2 hardcoded 4×. SwiGLU models empirically tune this ratio to match baseline params
FLOPs per Token	$6 \cdot E \cdot FFN$	Gated structures require ~1.5x more FLOPs than a vanilla MLP of the same FFN dimension
Activation Memory	High	Storing intermediate activations for `gate` and `up` paths increases backward-pass memory requirements
Non-linearity Locality	Only on the gate path	Up-projection remains linear, simplifying element-wise kernel fusion opportunities

---

5.4 Common Interview Questions & How to Answer

5.4.1 Q1: What is the difference between a vanilla MLP and a SwiGLU MLP, and why does SwiGLU perform better?

- **Answer**: A vanilla MLP projects inputs to a hidden space, applies an activation, and projects back: $y = \text{down}(\text{act}(\text{fc}(x)))$. SwiGLU uses three matrices and a multiplicative gating mechanism: $y = \text{down}(\text{silu}(\text{gate}(x)) \odot \text{up}(x))$. The activation function is applied only to the gate path. SwiGLU performs better because the gate operates as a soft, continuous feature mask. It scales and filters the linear features on the `up` path dynamically based on context, providing much higher representational capacity and regularization than a static per-element threshold.

5.4.2 Q2: What is the "operand-order pitfall" in SwiGLU, and how do you prevent it?

- **Answer**: The operand-order pitfall occurs when the activation function is accidentally applied to the `up` branch rather than the `gate` branch, i.e., computing $\text{silu}(\text{up}(x)) \odot \text{gate}(x)$ instead of $\text{silu}(\text{gate}(x)) \odot \text{up}(x)$. Because PyTorch or Triton will compile this without errors (shapes are identical), it becomes a silent bug. Swapping the operands yields different results (with a maximum absolute difference of **0.0035** on a tiny test tensor), corrupting model weights and performance. To prevent this, always refer to the trained model's config and reference code to ensure the activation is strictly applied to the first projection (`W_gate`).

5.4.3 Q3: Why is the MLP intermediate dimension in LLaMA or Qwen not exactly $4 \times E$?

- **Answer**: In older models like GPT-2, the intermediate size was hardcoded to $4 \times E$. However, in SwiGLU-based models, using three matrices instead of two increases parameters by 50% for the same hidden size. To keep the parameter count and computational footprint equivalent to a vanilla MLP, Shazeer (2020) suggested setting the FFN size to $\approx \frac{8}{3}E$. In practice, models tune the FFN size empirically to align with hardware-friendly boundaries (e.g., multiples of 128 or 256 for optimal Tensor Core utilization). For Qwen3-0.5B, the hidden size $E = 896$ and intermediate size $FFN = 4864$, yielding a ratio of **5.4286x**.

5.4.4 Q4: From a GPU optimization perspective, why is naive SwiGLU slow, and how do we optimize it?

- **Answer**: Naive SwiGLU executes three separate GEMMs (`gate_proj`, `up_proj`, and `down_proj`) and two element-wise kernels (SiLU activation, and element-wise multiplication). This is memory-bandwidth bound, especially during the auto-regressive decode phase. We optimize this by:
  1. **Fused Projections**: Combining `w_gate` and `w_up` into a single fused projection matrix of shape $[2 \cdot FFN, E]$, reducing kernel launches from three to two.
  2. **Fused Element-wise Kernel**: Fusing the SiLU activation and element-wise multiplication into a single Custom CUDA/Triton kernel. This keeps the intermediate outputs in GPU registers/shared memory, avoiding round-trips to High Bandwidth Memory (HBM).

5.5 Pro-Tip: How to Impress the Interviewer

- **Demonstrate Kernel-Level Awareness**: Explain that in production serving stacks (like vLLM or Hugging Face's TGI), the gate and up projections are always concatenated. Cite the parameter mapping: `gate_up_proj = nn.Linear(hidden_size, 2 * intermediate_size)`. Mention that you know that in Triton, you slice the output of this linear layer along the last dimension, apply SiLU to the first half, multiply by the second half, and write only the fused result back to global memory.
- **Trace the Activation Lineage Accurately**: Clarify that **SiLU** was actually introduced and named in the original **GELU paper** (Hendrycks & Gimpel 2016, Section 2) as the "Sigmoid Linear Unit", before being popularized as **Swish** with $\beta = 1$ by Google (Ramachandran et al. 2017). Showing you know this historical detail demonstrates deep literature reading.
- **Reference Gold Values**: Memorize and quote specific values to show you have worked with the code: e.g., SiLU(1.0) = **0.7311** and GELU_tanh(1.0) = **0.8412**.

6 Causal Masking & QK-Norm

- **Category**: LLM Systems
- **Difficulty**: Easy
- **Target Role**: LLM Inference Engineer / ML Systems Engineer
- **Source**: CAUSAL_MASK.md + causal_mask_output.txt (Vaswani et al. 2017, Dehghani et al. 2023)
- **Flashcards**: LLM Systems deck

6.1 Concept Overview

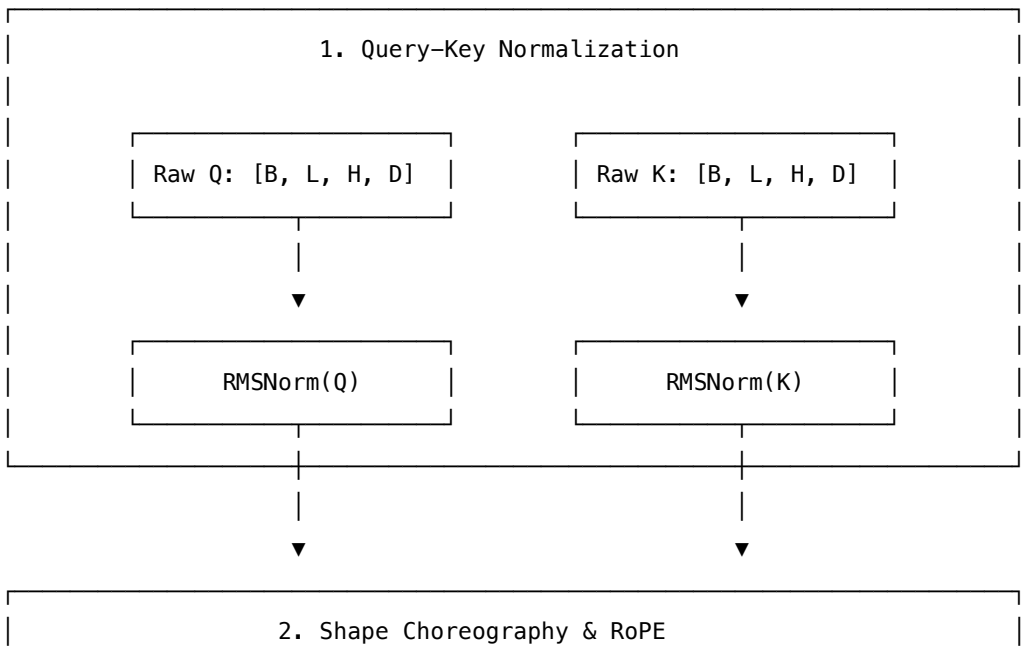
Think of causal masking as a **bouncer** at a VIP club who ensures that during language model training, each token can only read messages written **before** it, preventing it from peeking into the future. It accomplishes this by adding a value of $-\infty$ to future attention logits before the softmax step, which converts those probabilities to exactly 0%. In multi-token prefilling and single-token decoding phases, the attention mask boundary shifts using a diagonal offset index $k = S - L$ to keep the past-future dividing line aligned with the actual generated history. To prevent the attention logits from growing too large and saturating the softmax function, modern architectures introduce **QK-Norm**, which normalizes the query and key vectors using RMSNorm prior to the dot product.

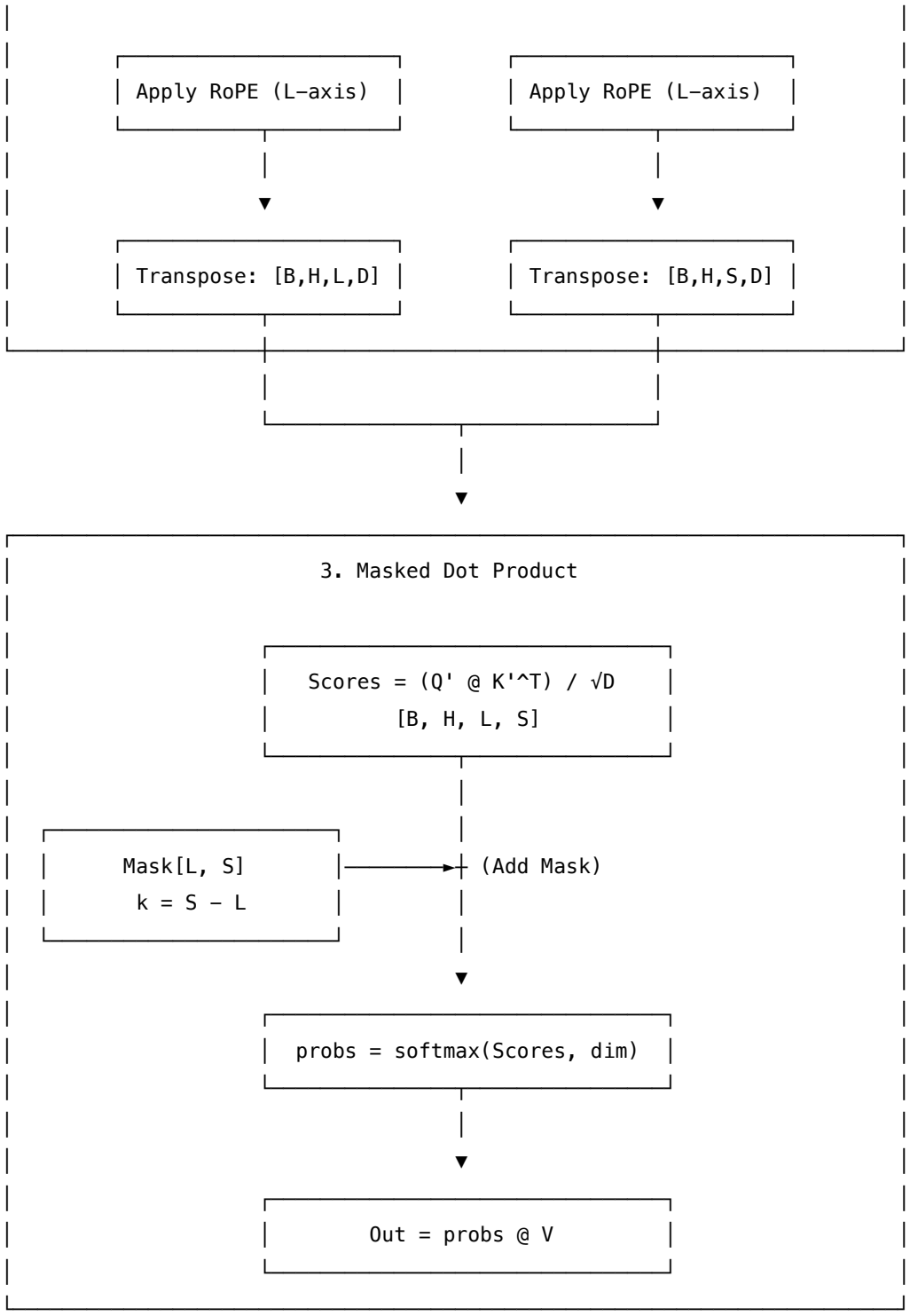
6.1.1 The Problem It Solves

Without these attention stabilization and causal guarding mechanics: - **Information Leakage**: An autoregressive language model must not see tokens to the right of the current position during training. Without a causal mask, the model would simply memorize the future tokens, failing to generalize at inference time. - **The Decode Off-by-One Bug**: During generation (decode phase), we compute attention for one new query token ($L = 1$) against all cached keys ($S = N$). Without the diagonal offset $k = S - L$, the mask blocks the token from seeing its own history. As shown in `causal_mask_output.txt`, it forces the new token to attend **100%** to the very first token ($s = 0$) and **0%** to the rest, resulting in immediate silent generation failure. - **Softmax Saturation**: In deep models, the query-key dot products can grow extremely large (e.g., reaching scores of **61.7954**). If the scores spread widely, softmax saturates, producing a one-hot probability distribution with near-zero gradients, stalling training.

6.1.2 How It Works

Causal self-attention operates using three primary mechanisms working together.





6.1.2.1 Step-by-Step Execution:

1. **QK-Norm**: Apply RMSNorm to the projected query and key states along the head-dim axis D.
   - $Q' = \text{RMSNorm}(Q) \odot W_q$
   - $K' = \text{RMSNorm}(K) \odot W_k$
2. **Positional Encoding & Transpose**: Apply RoPE to Q' and K' along the position axis L, then transpose the states to shape `[B, H, L, D]`.

3. **Scaled Dot Product**: Compute raw scores: scores $= \frac{Q' K'^T}{\sqrt{D}}$.
4. **Causal Mask Application**: Shift the mask diagonal based on the difference between key length (S) and query length (L):
   - $k = S - L$
   - $\text{mask}_{l,s} = \begin{cases} 0 & s \leq l + k \\ -\infty & \text{otherwise} \end{cases}$
   - Add the mask to the scores: scores $=$ scores $+$ mask.
5. **Softmax & Value Mixing**: Run softmax to get probabilities (masked values become 0.0) and multiply by V:
   - probs $=$ softmax(scores, dim $= -1$)
   - output $=$ probs$V$

6.2 Worked Example

6.2.1 Prefill vs. Decode Causal Mask (L vs. S)

Here is how the mask matrix changes depending on the execution phase (sourced from `causal_mask_output.txt`):

6.2.1.1 Prefill Phase ($L = 4, S = 4, k = S - L = 0$): Every query token l attends to past and current keys $s \leq l$:

$l \backslash s$	s=0	s=1	s=2	s=3
l=0	0.0	$-\infty$	$-\infty$	$-\infty$
l=1	0.0	0.0	$-\infty$	$-\infty$
l=2	0.0	0.0	0.0	$-\infty$
l=3	0.0	0.0	0.0	0.0

After softmax, the resulting probabilities show a strictly lower-triangular profile (upper triangle is exactly `0.0000`):

$l \backslash s$	s=0	s=1	s=2	s=3
l=0	**1.0000**	0.0000	0.0000	0.0000
l=1	0.5624	**0.4376**	0.0000	0.0000
l=2	0.3139	0.3309	**0.3552**	0.0000
l=3	0.2429	0.2788	0.2336	**0.2446**

The gold pin output vector for $l = 3$ is exactly `[0.2429, 0.2788, 0.2336, 0.2446, 0.0, 0.0, 0.0, 0.0]`.

6.2.1.2 Decode Phase ($L = 1, S = 4, k = S - L = 3$): The single query token represents the last position and attends to the entire history:

$l \backslash s$	s=0	s=1	s=2	s=3
l=0	0.0	0.0	0.0	0.0

6.2.1.3 The Decode Mask Bug ($k = 0$ instead of $k = 3$): If the offset is neglected, the mask incorrectly defaults to:

$l \backslash s$	s=0	s=1	s=2	s=3
l=0	0.0	$-\infty$	$-\infty$	$-\infty$

Below is the resulting probability distribution comparison:

Mask used	probs[s=0]	probs[s=1]	probs[s=2]	probs[s=3]
Correct ($k = 3$)	0.4195	0.1553	0.3751	0.0501
Wrong ($k = 0$)	**1.0000**	**0.0000**	**0.0000**	**0.0000**

The wrong mask forces the model to ignore tokens 1..3 entirely, attending only to the very first token.

6.2.2 QK-Norm Performance (Seeded x5 Inputs)

Comparing attention stats with and without QK-Norm (from `causal_mask_output.txt`):

Metric	Without QK-Norm	With QK-Norm
Max $\|q\|$	20.0820	**2.8284** ($\sqrt{D}$)
Max $\|k\|$	20.8228	**2.8284** ($\sqrt{D}$)
Max RMS(q)	7.1001	**1.0000**
Max Abs Score ($Q \cdot K/\sqrt{D}$)	61.7954	**1.6563**
Std of Scores	23.9671	**0.8721**

QK-Norm bounds the dot products to $\approx \pm 1.6$, preventing logits from saturating softmax and stabilizing gradient flow.

6.2.3 Shape Choreography Step-by-Step

For tensor sizes $B = 1, L = 4, H = 2, D = 8$, the dimensions flow as follows:

Step	Operation	Result Shape
1. Project	`q = linear(x, wq).reshape(B, L, H, D)`	`(1, 4, 2, 8)`
2. QK-Norm	`q = rms_norm(q)` (on D axis, before RoPE)	`(1, 4, 2, 8)`

Step	Operation	Result Shape
3. RoPE	`q = apply_rope(q)` (over sequence axis L)	`(1, 4, 2, 8)`
4. Transpose	`q = q.transpose(1, 2)`	`(1, 2, 4, 8)`
5. Attention	`scores = q @ k.T * scale + mask`	`(1, 2, 4, 4)`
6. Transpose back	`out = out.transpose(1, 2)`	`(1, 4, 2, 8)`
7. Reassemble	`out = out.reshape(B, L, H * D)`	`(1, 4, 16)`

6.3 Complexity & Trade-offs

Metric	Value	Notes
Attention Mask Space	$O(L \cdot S)$ per head	Typically generated on the fly to save memory bandwidth
Attention Flops	$O(B \cdot H \cdot L \cdot S \cdot D)$	Dominates computation at long contexts
QK-Norm Parameter Count	$2 \cdot H \cdot D$ parameters	One learned scale parameter per head per dimension for Q and K
QK-Norm Computation	$O(B \cdot L \cdot H \cdot D)$ FLOPs	Extremely cheap; requires only one division pass per head vector
Memory Bandwidth	High	Mask additions and softmax require element-wise GPU memory operations

---

6.4 Common Interview Questions & How to Answer

6.4.1 Q1: Why do we add $-\infty$ to the future tokens instead of setting their values to 0 before softmax?

- **Answer**: Softmax exponentiates its input: $\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$. If we set a masked score to 0, $e^0 = 1$, which still assigns a positive probability mass to that token. By adding $-\infty$, we ensure that $e^{-\infty} = 0$, giving the masked position exactly 0% probability after softmax and guaranteeing that future tokens cannot leak information to the past.

6.4.2 Q2: What is the $k = S - L$ offset in causal masking, and what happens if you forget it during decode?

- **Answer**: The offset $k = S - L$ shifts the causal diagonal boundary of the mask. In the prefill phase, sequence lengths are equal ($L = S$), so $k = 0$, yielding a standard lower-triangular mask. In the decode phase, we process one new query token ($L = 1$) against all cached keys ($S = N$). The new token is the last token in the sequence, so it must attend to all keys ($0..N-1$). The offset shifts the

triangle boundary right by $N-1$, making the mask row all zeros (fully allowed). If you forget the offset ($k=0$), the mask row becomes `[0, -inf, -inf, -inf]`, forcing the new token to attend **100%** to the first token ($s=0$) and ignore the rest of the sequence, corrupting outputs.

6.4.3 Q3: What training issue does QK-Norm solve, and how does it work?

- **Answer**: As models grow deeper, query and key activations can grow in scale, making their dot products blow up (e.g., above 60). This spreads the logits widely, saturating the softmax function, which drives gradients to zero and halts learning. QK-Norm applies an RMSNorm or LayerNorm to the Q and K states independently along the head dimension D before computing the dot product. Because it pins the RMS of each head vector to 1.0 (constraining the vector's L_2 norm to $\sqrt{D}$), the maximum dot product is mathematically bounded (reducing scores from $\approx \pm 60$ to $\approx \pm 1.6$), preventing softmax saturation and keeping training stable.

6.4.4 Q4: Why must QK-Norm and RoPE be applied before the transpose to `[B, H, L, D]`?

- **Answer**: In attention projections, the sequence position axis is the second dimension (L). Both QK-Norm and RoPE are position-aware: RoPE rotates queries and keys based on their sequence index, and QK-Norm scales vectors per token position. Thus, they must be applied while the tensor is still in `[B, L, H, D]`. If you transpose to `[B, H, L, D]` first, the sequence dimension shifts to the third axis, meaning any naive position lookup or head-dimension reduction will index along the wrong axis, silently scrambling position embeddings and scaling factors.

6.5 Pro-Tip: How to Impress the Interviewer

- **Coordinate RoPE and Causal Mask Offsets**: Point out that the causal mask offset $k = S - L$ and RoPE's position offset are mathematically linked. During decode, the new query has sequence position $M = S - 1$. Explain that both the causal mask offset ($k = S - 1$) and the RoPE offset (M) represent the same physical property: the current logical position of the token in the sequence.
- **QK-Norm as Cheap Insurance**: Highlight that QK-Norm is "cheap training insurance". It introduces a tiny parameter overhead ($2 \cdot H \cdot D$ params) and negligible FLOPs ($O(B \cdot L \cdot H \cdot D)$) but prevents catastrophic loss spikes in large models. Mention its adoption in production models like Qwen3, Gemma 2, and OLMo 2.
- **Triton FlashAttention Mask Optimization**: Mention that in highly optimized serving systems (like vLLM using FlashAttention), the causal mask is almost never instantiated as a physical tensor in HBM. Instead, the boundary checks $s \leq l + (S - L)$ are compiled directly into the Triton GPU kernel's loop conditions, completely eliminating the $O(L \cdot S)$ memory allocation and bandwidth overhead.
- **Reference Gold Verification Values**: Cite exact values to show hands-on experience: for instance, with QK-Norm using weight=1, the L2 norm of the query vector is pinned exactly to $\sqrt{D}$ (e.g., $\sqrt{8} = \mathbf{2.8284}$), and the final token attention output row under prefill is `out[0,0,3,:] = [0.2429, 0.2788, 0.2336, 0.2446, 0, 0, 0, 0]`.

7 KV Cache — Dense vs Paged

- **Category**: LLM Systems
- **Difficulty**: Hard
- **Target Role**: LLM Inference Engineer / ML Systems Engineer
- **Source**: PagedAttention (Kwon et al., SOSP 2023) + vLLM
- **Flashcards**: LLM Systems deck

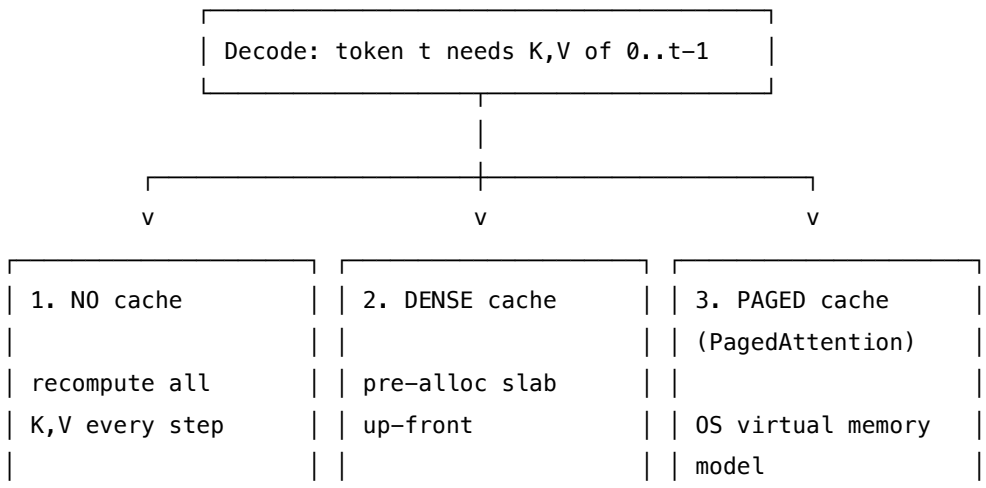
7.1 Concept Overview

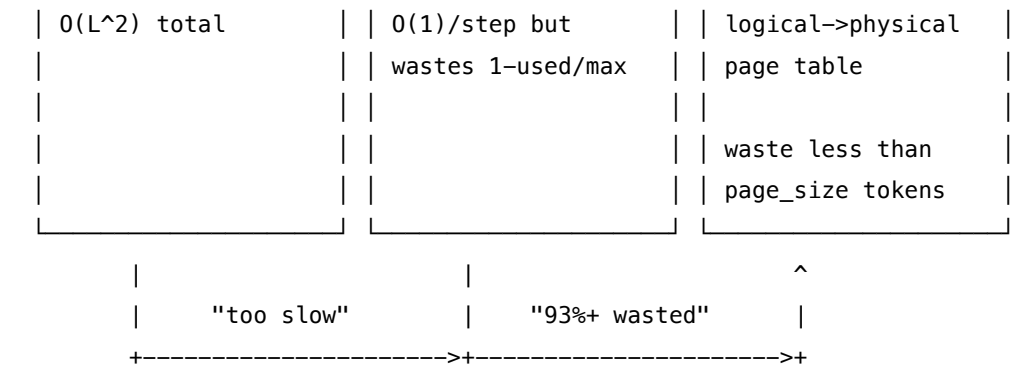
Think of autoregressive decoding like a student writing an essay one word at a time, who must re-read the *entire* essay from scratch before writing each new word — that's the **no-cache** nightmare. A **KV cache** is the student's notebook: jot down the key ideas as you read once, then only look up your notes for each new word. The notebook is fast. The problem is that a *dense* notebook reserves the entire maximum-length shelf the moment a request arrives — if the essay ends at page 10, pages 11–8192 sit blank forever (up to 93.75% wasted). **PagedAttention** solves this by borrowing the OS virtual memory trick: shared "pages" of a fixed size, handed out on demand, with each request holding only an index card (block table) pointing to where its notes physically live.

7.1.1 The Problem It Solves

- **No cache**: every decode step recomputes K,V for the *entire* prefix. Generating 5 tokens costs $1+2+3+4+5=15$ K,V projections — $O(L^2)$ total. Token-0's K,V is recomputed **5×** without a cache vs **1×** with one.
- **Dense cache**: reduces decode to $O(1)$ per step, but pre-allocates `[B, H_kv, max_seq_len, D]` up front. With `max_seq_len=8192` and a 512-token response, **93.75%** of the slab is dead memory ($1-512/8192 = 0.9375$). Across 100 concurrent requests: reserved = **400 GiB**, used = **25 GiB**, wasted = **375 GiB**.
- **Paged cache**: wastes only the last partial page. vLLM measured **< 4% system-wide waste** vs **60–80%** for prior dense/over-reserving systems.

7.1.2 How It Works

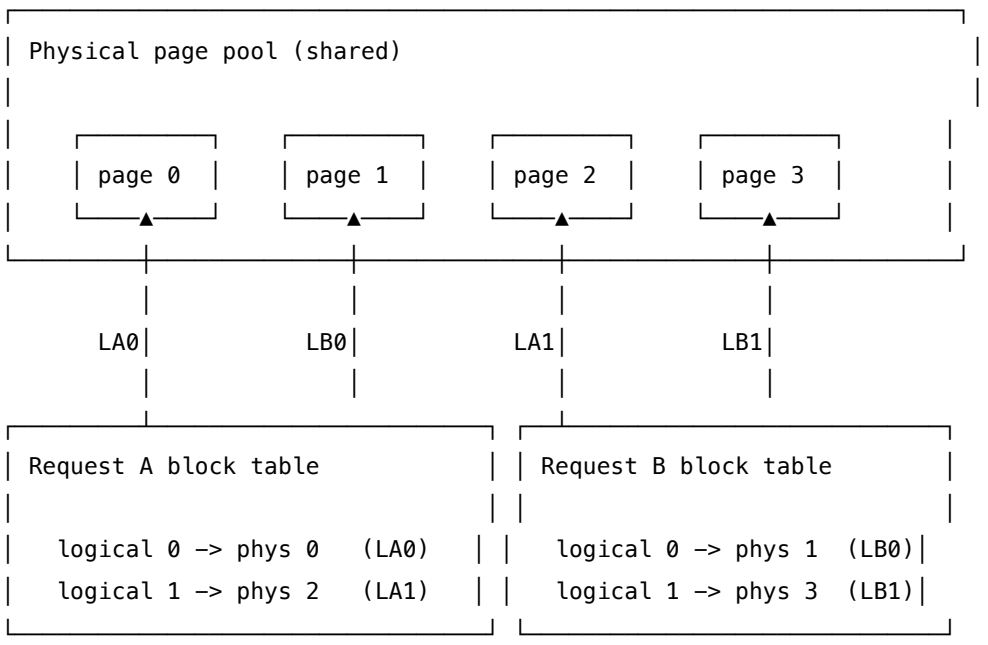




Dense cache decode loop (step by step):

1. New token arrives → project only *it* → $Q[1,d]$, $K[1,d]$, $V[1,d]$
2. Rotate Q and K at `offset = slice(current_len, current_len+1)` via RoPE (critical — wrong offset → gibberish)
3. Append $K[1]$, $V[1]$ to the slab at `axis=2` (sequence length axis)
4. Attention: $Q[1,d]K[S,d]^T$ — $O(S)$ per step, not $O(S^2)$

Paged cache (PagedAttention):



Key mechanics: - A **page** (block) holds `page_size` token slots (vLLM default: 16) - Each request holds a **block table**: logical page index → physical page id - A **free list** is the OS-style frame allocator; freed pages are immediately reusable - Physical pages are **non-contiguous** — the attention kernel gathers K,V via the block table

---

7.2 Worked Example

7.2.1 No Cache vs Dense Cache (5 tokens)

From `kv_cache.py` Section A — generating 5 tokens:

step t	seq len processed	K,V tokens computed	token-0 K,V recomputed?
1	1	1	yes (1)
2	2	2	yes (2)
3	3	3	yes (3)
4	4	4	yes (4)
5	5	5	yes (5)

```
no-cache total projections : 1+2+3+4+5 = 15  (= L(L+1)/2 = O(L²))
with-cache total           : 5               (1 per step = O(L))
speedup (projections)      : 3.0x
```

7.2.2 Dense Cache — Shape Evolution

From `kv_cache.py` Section B (`B=1, H_kv=2, D=8, max_seq_len=8`):

```
Pre-allocated slab shape: (1, 2, 8, 8) = [B=1, H_kv=2, max_seq_len=8, D=8]
Reserved bytes (K+V): 1024

PREFILL 3 tokens:  cache.k[:,:,:offset,:].shape = (1, 2, 3, 8)  (offset=3)
DECODE  1 token:   cache.k[:,:,:offset,:].shape = (1, 2, 4, 8)  (offset=4)
                   Q shape = [1, H_q, 1, D]  <- ONLY the new token
                   attention: Q[1,d] @ K[4,d]^T = O(S=4), not O(S^2)

Note: offset=4 but max_seq_len=8 -> 50% of the slab is unused right now.
```

7.2.3 Fragmentation Math

From `kv_cache.py` Section C (fp16 = 2 bytes):

Per-request slab (LLaMA-7B: 32 layers, 32 KV heads, d=128):

```
2(KV) x 32 x 32 x 8192 x 128 x 2 B = 4,294,967,296 bytes = 4.000 GiB
```

scenario	reserved	used	wasted
1 request, max=8192, actual=512	—	512 tokens	**93.75%**
100 concurrent (used=512 each)	400.0 GiB	25.0 GiB	**375 GiB (93.8%)**
PagedAttention (vLLM default block=16)	on-demand	~exact	**< 4%**

Paged waste bound — only the last page is partial:

page_size	worst-case internal waste
4	25.00%
16	6.25% <- vLLM default
128	0.78%

7.2.4 Paged Block Table (page_size=4)

From `kv_cache.py` Section D — interleaving 2 requests:

request	logical pages	physical pages	contiguous?
A	['L0', 'L1']	['P0', 'P2']	NO (scattered)
B	['L0', 'L1']	['P1', 'P3']	NO (scattered)

```
A.page_ids  = [0, 2]    A.page_lens = [4, 4]
B.page_ids  = [1, 3]    B.page_lens = [4, 1]
```

7.2.5 Correctness Proof — All Three Paths Match

From `kv_cache.py` Section E (prefill 3 tokens, then decode token 3):

path	K[0,0,0,:4] (token 0)	K[0,0,3,:4] (token 3)
all-at-once	[0.101, 0.102, 0.103, 0.104]	[-0.4541, 0.2641, 0.3906, 0.4028]
dense	[0.101, 0.102, 0.103, 0.104]	[-0.4541, 0.2641, 0.3906, 0.4028]
paged	[0.101, 0.102, 0.103, 0.104]	[-0.4541, 0.2641, 0.3906, 0.4028]

```
[check] dense full-K  == all-at-once K? True
[check] paged full-K  == all-at-once K? True
[check] WRONG-offset decode differs?    True  <- proves offset is mandatory
[check] ALL THREE PATHS MATCH: OK
```

7.2.6 Rewind(n) — The Ceil Trap (Speculative Decoding)

From `kv_cache.py` Section F (page_size=4):

Start: `page_ids=[0,1], page_lens=[4,1], offset=5, free_list=[2,3,4,5,6,7]`

After `rewind(1)`: - `page_ids = [0]` ← physical page 1 RETURNED to free list - `page_lens = [4]` ← page 0 back to full - `offset = 4` - `free_list = [1,2,3,4,5,6,7]`

Off-by-one trap:

offset	rewind(n)	new_offset	ceil(new/4) kept	floor would keep	correct?
4	1	3	1	**0 — too few!**	yes (ceil)

offset	rewind(n)	new_offset	ceil(new/4) kept	floor would keep	correct?
8	1	7	2	**1 — silent data loss!**	yes (ceil)
5	5	0	0	0	yes (ceil)

```
rewind algorithm (always use ceil):
  new_offset   = offset - n
  target_pages = ceil(new_offset / page_size)  # <-- NEVER floor
  pop pages > target_pages -> free_list
  page_lens[-1] = new_offset - page_size*(target_pages - 1)
```

7.3 Complexity & Trade-offs

Metric	No Cache	Dense Cache	Paged Cache
K,V compute per step	O(L) — recompute all	O(1) — append 1	O(1) — append 1
Total over L steps	**$O(L^2)$**	O(L)	O(L)
Memory reserved	none	`[B,H_kv,max_len,d]` up-front	on-demand pages
Worst-case waste	—	up to **93.75%**	< page_size tokens (< 4%)
Rewind complexity	free (no-op)	`offset -= n`	ceil-based page pop
LLaMA-7B per seq (8192 tok)	—	**4.0 GiB**	~exact usage
Production use	toy code	rarely used	**vLLM, TGI, SGLang**

7.4 Common Interview Questions & How to Answer

7.4.1 Q1: Why does the KV cache exist? What does it actually cache?

- **Answer**: Without a cache, every new token triggers a full forward pass over the growing prefix — token 0's K,V is recomputed at every step, making decode $O(L^2)$ total (for 5 tokens: `1+2+3+4+5=15` projections vs 5 with a cache, 3× speedup). The cache stores the *already-computed, already-RoPE-rotated* Keys and Values for every past token. At each decode step, we project only the **one new token** (O(1)), append its K,V, and attend the single new query against the full cached K,V (O(S) per step, O(L) total). The cache changes *when* K,V are computed, not *what* they are.

7.4.2 Q2: What is the 93.75% waste number and how does it arise?

- **Answer**: Dense KV caches pre-allocate the full `[B, H_kv, max_seq_len, D]` slab the moment a request arrives, regardless of actual generation length. For `max_seq_len=8192` and an actual response of 512 tokens: `1 - 512/8192 = 0.9375 = 93.75%` dead. Across 100 concurrent requests: 400 GiB reserved vs

25 GiB used = 375 GiB wasted. This caps batch size, caps GPU utilization, caps throughput. The fix is paged allocation.

7.4.3 Q3: How does PagedAttention eliminate that waste?

- **Answer**: PagedAttention borrows OS virtual memory. The KV store is a shared pool of fixed-size pages (vLLM default: 16 tokens/page). Each request holds only a block table (logical→physical mapping). Pages are allocated from a free list on-demand and returned when the request finishes. Waste is bounded to the last partial page per sequence (< `page_size` tokens). vLLM reduced system-wide waste from 60–80% to < 4%, enabling 2–4× throughput improvement. Non-contiguous physical storage is handled by the kernel walking the block table — exactly like an OS page walker.

7.4.4 Q4: Why is the RoPE offset critical, and what breaks without it?

- **Answer**: The cache stores K,V *after* RoPE rotation, encoding absolute position. When decoding at step `t`, the new token must be rotated at `offset = slice(t, t+1)` — its true position. If we use `offset = slice(0,1)` (treating every decoded token as position 0), the cached path and the all-at-once path diverge — proven empirically: correct offset → identical outputs; wrong offset → different K,V (shown in the output: `[check] WRONG-offset decode differs: True`).

7.4.5 Q5: Why does `rewind()` use `ceil` and not `floor`?

- **Answer**: When speculative decoding rejects `n` tokens, `new_offset = offset - n`. The pages to *keep* = `ceil(new_offset / page_size)`. At `offset=8, rewind(1)`: `new=7`, `ceil(7/4)=2` — correct (4+3 tokens). `floor(7/4)=1` silently drops page 2 which still holds 3 valid tokens. The next decode step reads stale garbage from a freed page — a completely silent bug (no error raised, no shape mismatch).

7.4.6 Q6: What is the difference between the 93.75% and 60–80% numbers?

- **Answer**: These measure different things. **93.75%** is clean per-request arithmetic: `1 - 512/8192` for one request's reserved-vs-used slab. **60–80%** is vLLM's measured *system-wide* aggregate waste in prior serving systems, combining internal fragmentation + external fragmentation + over-reservation across many heterogeneous concurrent requests. **< 4%** is PagedAttention's measured system-wide result. All three are correct — they describe different scopes.

7.5 Pro-Tip: How to Impress the Interviewer

- **Name the paper and the OS analogy**: "PagedAttention (Kwon et al., SOSP 2023) explicitly models KV pages as OS virtual memory pages — 'blocks as pages, tokens as bytes, sequences as processes.'"
- **Cite all three waste numbers** and distinguish them: 93.75% (per-request arithmetic), 60–80% (system-wide before PagedAttention), < 4% (system-wide after). Interviewers often conflate these.
- **Mention Copy-on-Write**: paged storage enables beam search and parallel sampling to *share* KV pages via reference counting — a win dense layout cannot replicate without copying the entire slab.

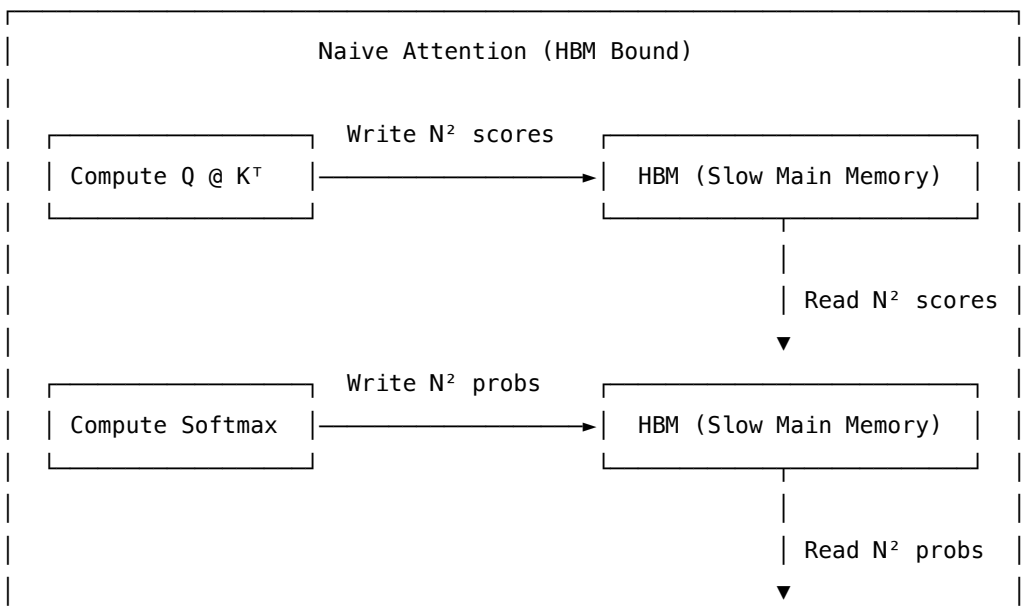
- **Mention the `offset`/RoPE coupling**: the cache is correct *only* because each new token is rotated at its true position. Bringing up this cross-cutting constraint signals deep understanding of the full decode stack.
- **Trade-off awareness**: paged adds kernel complexity (block-table gather scatter vs sequential reads) and a small per-request overhead. The tradeoff is worthwhile at scale but adds latency for single-request benchmarks.

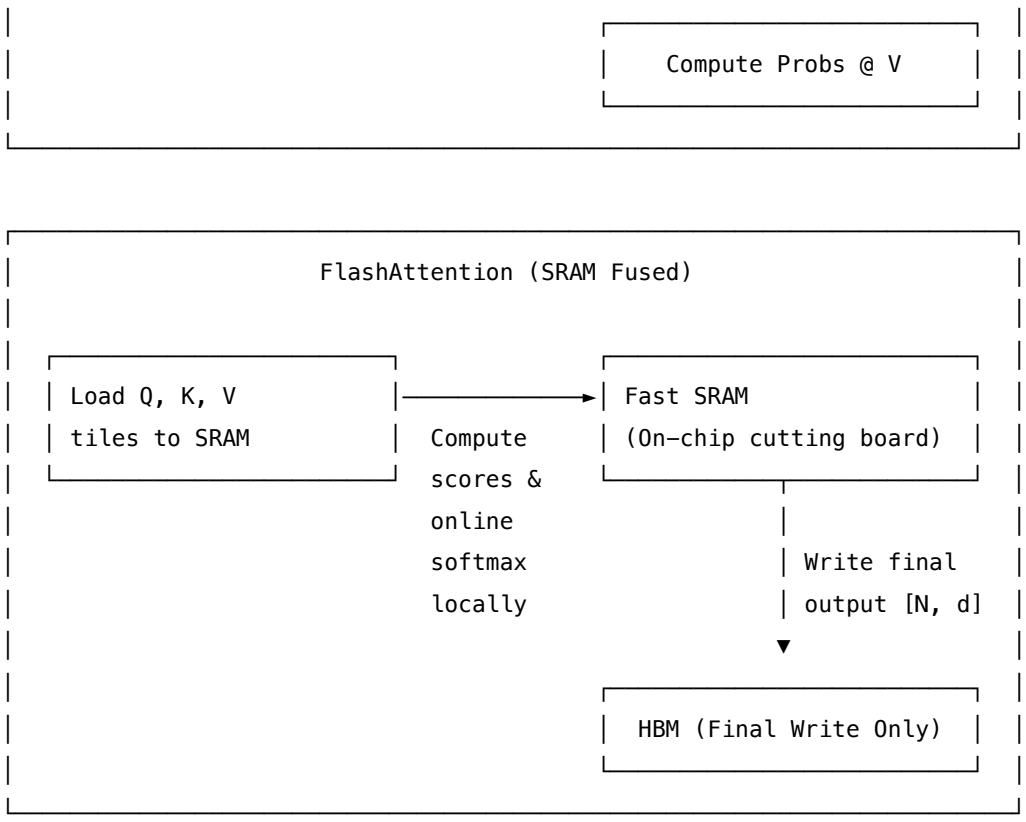
8 FlashAttention: IO-Aware Exact Attention

- **Category**: LLM Systems
- **Difficulty**: Expert
- **Target Role**: LLM Inference Engineer / ML Systems Engineer
- **Source**: FlashAttention (Dao et al. 2022) + Online Softmax (Milakov & Gimelshein 2018)
- **Flashcards**: LLM Systems deck

8.1 Concept Overview

Think of the GPU as a chef working in a kitchen: the GPU's fast on-chip **SRAM** is the cutting board right in front of the chef, while the slow main memory **HBM (High-Bandwidth Memory)** is a pantry at the far end of the kitchen. Standard attention forces the chef to chop a tiny bit of ingredients, walk all the way to the pantry to store them, walk back, retrieve them, and repeat — resulting in the chef spending 95% of their time walking back and forth rather than cooking. **FlashAttention** reorganizes the work so that the chef brings small, bite-sized "tiles" of ingredients to the cutting board, completes all necessary steps (including the tricky softmax normalization) on that board, and only walks to the pantry once to deliver the final dish. By processing the attention matrix in small, local blocks instead of materializing the entire quadratic table in HBM, it eliminates memory round-trips without changing the mathematical output.





8.1.1 The Problem It Solves

- **Memory Bandwidth Bottleneck**: Modern GPUs have massive compute capacity (hundreds of TFLOPs) but relatively slow memory access speeds. Standard attention projects queries and keys to build a giant score table $S = QK^T$ of shape $[N, N]$.
- **Quadratic Scaling**: For a sequence length $N = 8192$ and head dimension $d = 64$, the score table contains $N^2 \approx 67$ million cells. Storing this table in 4-byte FP32 floats takes **256 MiB** per head, per layer ($8192 \times 8192 \times 4$ bytes). Multiplying this across 32 heads and 80 layers yields a massive footprint.
- **Excessive Memory Traffic**: Standard attention performs $\approx 4N^2$ memory transactions (writing scores, reading scores for softmax, writing softmax probabilities, reading probabilities for value aggregation). Because LLM inference is **memory-bandwidth bound**, the GPU cores spend most of their cycles idle, waiting for these memory transactions to finish.

8.1.2 How It Works

FlashAttention avoids materializing the $[N, N]$ score matrix by partitioning the inputs into blocks (tiles), loading them into SRAM, and computing attention incrementally using a technique called **online softmax**.

Standard softmax requires the maximum value of the entire row (m) to prevent numerical overflow during exponentiation:

$$\text{softmax}(x)_i = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}} \quad \text{where} \quad m = \max_j(x_j)$$

This requirement makes tiling non-obvious, as the maximum value is not known until the entire row is

read.

To overcome this, FlashAttention uses the online softmax recurrence. For each row, it maintains three running metrics in SRAM across K/V tiles: 1. m: The running maximum score seen so far (initialized to $-\infty$). 2. l: The running normalizer sum (initialized to 0). 3. o: The running output vector (initialized to all zeros).

When moving from a tile with running max m_{old} and normalizer l_{old} to a new tile with local max $m_{\text{new}} = \max(m_{\text{old}}, \text{rowmax}(s))$, we apply a **correction factor** $\text{corr} = e^{m_{\text{old}} - m_{\text{new}}}$ to our running aggregates before adding the new tile's contributions:

$$l_{\text{new}} = e^{m_{\text{old}} - m_{\text{new}}} \cdot l_{\text{old}} + \sum e^{s - m_{\text{new}}}$$

$$o_{\text{new}} = e^{m_{\text{old}} - m_{\text{new}}} \cdot o_{\text{old}} + e^{s - m_{\text{new}}} V_{\text{tile}}$$

Because $m_{\text{new}} \geq m_{\text{old}}$, the exponent is always ≤ 0, yielding a correction factor ≤ 1 that shrinks the past contributions to align with the new, larger denominator. At the end of the row, the true normalized output is simply o/l.

8.2 Worked Example

Below are the exact values from a simulated attention forward pass with sequence length $N = 8$, head dimension $d = 8$, scale factor $1/\sqrt{d} = 0.3536$, and tile size $B_r = B_c = 4$ (2×2 tiles).

8.2.1 The Materialized Score Matrix S (Naive Attention)

Standard attention writes this entire $[8, 8]$ table to slow HBM:

$q \backslash k$	$k=0$	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	$k=6$	$k=7$	**Row Max**
$q=0$	−0.1160	+0.0909	−0.0998	−0.4400	−0.3969	+0.0736	+0.4049	−0.1240	+0.4049
$q=1$	+0.1595	+0.0673	−0.0434	−0.4481	−0.0200	+0.0276	+0.2197	+0.2710	+0.2710
$q=2$	−0.2861	+0.1418	−0.0041	−0.2182	+0.2089	−0.0232	+0.4081	+0.2560	+0.4081
$q=3$	+0.0296	−0.1254	+0.0781	−0.4158	−0.4133	+0.0830	+0.2035	+0.0956	+0.2035
$q=4$	−0.0327	+0.0913	+0.1463	+0.0832	−0.2202	−0.0010	−0.0073	+0.0617	+0.1463
$q=5$	+0.5159	+0.0707	−0.1574	−0.1376	−0.5931	−0.1283	+0.0715	−0.1468	+0.5159
$q=6$	+0.1365	−0.0496	+0.0134	+0.0490	−0.0689	+0.1214	−0.2589	+0.1266	+0.1365
$q=7$	+0.4462	−0.2285	+0.0145	+0.1127	+0.5438	+0.3833	−0.4868	+0.0673	+0.5438

8.2.2 Step-by-Step Online Softmax for Row $q = 0$

Row $q = 0$ is processed over $T_c = 2$ tiles of keys:

8.2.2.1 Tile $j = 0$ (keys $0, 1, 2, 3$)

- **Local Scores**: $s = [-0.1160, +0.0909, -0.0998, -0.4400]$

- **Local Max**: $\text{rowmax}(s) = +0.0909$
- **Accumulators**: $m_{\text{old}} = -\infty,\ l_{\text{old}} = 0,\ o_{\text{old}} = \mathbf{0}$
- **New Max**: $m_{\text{new}} = \max(-\infty, +0.0909) = +0.0909$
- **Correction Factor**: $\text{corr} = e^{-\infty - 0.0909} = 0.0$ (wipes out the empty history)
- **Exponentiated Shifted Scores**: $p = e^{s - m_{\text{new}}} = [0.8131, 1.0000, 0.8264, 0.5881]$
- **Update Running Normalizer**: $l_{\text{new}} = 0.0 \cdot 0 + \sum(p) = 3.2277$
- **Update Running Output**: $o_{\text{new}} = 0.0 \cdot \mathbf{0} + pV_{\text{tile},0} = [-0.4464, -0.2905, -1.0848, +0.1282, +0.1119, -0.0819, +0$
- **State after** $j = 0$: $m = +0.0909,\ l = 3.2277$

8.2.2.2 Tile $j = 1$ (keys $4, 5, 6, 7$)

- **Local Scores**: $s = [-0.3969, +0.0736, +0.4049, -0.1240]$
- **Local Max**: $\text{rowmax}(s) = +0.4049$
- **New Max**: $m_{\text{new}} = \max(+0.0909, +0.4049) = +0.4049$ (the running maximum increases!)
- **Correction Factor**: $\text{corr} = e^{0.0909 - 0.4049} = \mathbf{0.7305}$
- **Exponentiated Shifted Scores**: $p = e^{s - m_{\text{new}}} = [0.4485, 0.7180, 1.0000, 0.5893]$
- **Update Running Normalizer**: $l_{\text{new}} = \mathbf{0.7305} \cdot 3.2277 + \sum(p) = 2.3578 + 2.7558 = 5.1135$
- **Update Running Output**: $o_{\text{new}} = \mathbf{0.7305} \cdot o_{\text{old}} + pV_{\text{tile},1} = [-0.3441, -0.7496, -1.1122, +0.1029, +0.9256, -0.78$
- **State after** $j = 1$: $m = +0.4049,\ l = 5.1135$

8.2.2.3 Final Row Normalization

$$\text{out} = \frac{o}{l} = [-0.0673, -0.1466, -0.2175, +0.0201, +0.1810, -0.1534, +0.2226, -0.0995]$$

8.2.3 Running Metrics Trace (First 4 rows of Q-tile $i = 0$)

The table below tracks the state transition of $(m\ /\ l\ /\ |o|_{\max})$ after each K/V tile:

Step	$q = 0$ $(m\ /\ l\ /\ \lvert o\rvert_{\max})$	$q = 1$ $(m\ /\ l\ /\ \lvert o\rvert_{\max})$	$q = 2$ $(m\ /\ l\ /\ \lvert o\rvert_{\max})$	$q = 3$ $(m\ /\ l\ /\ \lvert o\rvert_{\max})$
$j = 0$	+0.091 / 3.228 / 1.085	+0.159 / 3.273 / 1.093	+0.142 / 3.214 / 1.120	+0.078 / 3.379 / 1.086
$j = 1$	+0.405 / 5.113 / 1.138	+0.271 / 6.409 / 1.260	+0.408 / 5.790 / 1.151	+0.204 / 6.304 / 1.235

8.2.4 Verification of Exact Equivalence

Comparing the output from our tiled implementation against standard attention: - **Max Absolute Difference**: 2.98×10^{-8} (pure floating-point rearrangement noise). - **Broken Variant (No Rescaling)**: If we omit the $e^{m_{\text{old}} - m_{\text{new}}}$ correction, the output deviates from the baseline by 2.18×10^{-2} (over $700,000\times$ larger error), resulting in garbled text.

8.3 Complexity & Trade-offs

Metric	Value	Notes
HBM Read/Write Complexity	$\mathcal{O}(Nd + N^2d^2/M)$	Standard attention requires $\mathcal{O}(N^2)$ reads/writes. Here, M is the size of on-chip SRAM, reducing traffic significantly.
FLOPs (Forward Pass)	$\mathcal{O}(N^2d)$	Identical to standard attention. FlashAttention is an I/O optimization, not a mathematical approximation.
FLOPs (Backward Pass)	$\mathcal{O}(N^2d)$	Slightly higher than standard because we recompute the attention scores on the fly to avoid storing them.
Activation Memory (Training)	$\mathcal{O}(N)$	Standard attention stores the full $N \times N$ matrix ($\mathcal{O}(N^2)$). FlashAttention only caches m and l of size N.
Numerical Accuracy	Exact ($\approx 3 \times 10^{-8}$ diff)	Not an approximation. Differs only due to floating-point addition order.

8.4 Common Interview Questions & How to Answer

8.4.1 Q1: FlashAttention claims to speed up LLM attention, yet it performs the same (or even slightly more) FLOPs than standard attention. How does this work?

- **Answer**: GPUs have two types of memory: slow, large main memory (HBM) and fast, small local memory (SRAM). In standard attention, the GPU's compute cores calculate the $N \times N$ score matrix, write it to HBM, read it back to compute the softmax, write the probabilities back to HBM, and read them again to multiply by the values. Since LLM inference is bandwidth-bound, the GPU spends most of their time waiting for these HBM transfers to complete. FlashAttention fuses these steps by keeping local tiles in SRAM and doing the calculations in place. Since the intermediate $N \times N$ matrix never leaves SRAM, HBM traffic drops from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$, letting the GPU compute at its maximum potential.

8.4.2 Q2: What is the online softmax recurrence, and why is the scaling factor $\exp(m_{\textbf{old}} - m_{\textbf{new}})$ critical? What happens if you omit it?

- **Answer**: Standard softmax requires the global maximum value of a row to remain numerically stable. In a tiled setting, we do not know the global maximum beforehand. Online softmax solves this by carrying a running maximum m, a running normalizer sum l, and a running output o. When a new tile has a local maximum higher than m_{old}, the scaling factor $\exp(m_{\text{old}} - m_{\text{new}})$ is used to rescale the previously accumulated l and o to the new, larger scale before adding the new tile's contributions.

Without this correction, the values inside the accumulator would represent mixed scales, causing silent numerical corruption and producing incorrect outputs.

8.4.3 Q3: How does FlashAttention reduce the memory footprint during training?

- **Answer**: Standard attention caches the entire $N \times N$ attention matrix in HBM during the forward pass to use during backpropagation. This is the primary memory bottleneck when training long-context LLMs. FlashAttention does not store the $N \times N$ matrix. Instead, it only stores the running row maximum m and the normalizer sum l (which are $\mathcal{O}(N)$). During the backward pass, it loads the tiles of Q, K, V back into SRAM and recomputes the $N \times N$ blocks on the fly. This increases FLOPs by about 30%, but drastically reduces memory consumption, enabling training with much larger batch sizes and sequence lengths.

8.5 Pro-Tip: How to Impress the Interviewer

- **Warp-level Scheduling & Asynchronous Copies**: In interviews, stand out by discussing how FlashAttention partitions work across CUDA thread blocks and warps. SRAM tiles (like $B_r = B_c = 128$) are loaded using hardware asynchronous copy commands (e.g., `cp.async` on NVIDIA Ampere/Hopper) to bypass registers and fetch data directly from HBM to SRAM. While one warp group computes matmuls on the current tile, another warp group prefetches the next tile in the background, overlapping compute and memory access.
- **SRAM Budget Calculation**: Demonstrate system design maturity by showing how to select tile sizes based on head dimension d. On an NVIDIA A100 with 96 KB of shared memory per SM:

$$(\text{Tile}_q \times d + \text{Tile}_k \times d) \times \text{bytes\_per\_element} \leq \text{shared\_memory\_budget}$$

If $d = 128$ and we use FP16 (2 bytes), a tile size of $B_r = 128, B_c = 64$ consumes $(128 \times 128 + 64 \times 128) \times 2 = 48$ KB, leaving plenty of space for accumulators and double-buffering. If d increases, the tile size must be scaled down accordingly.

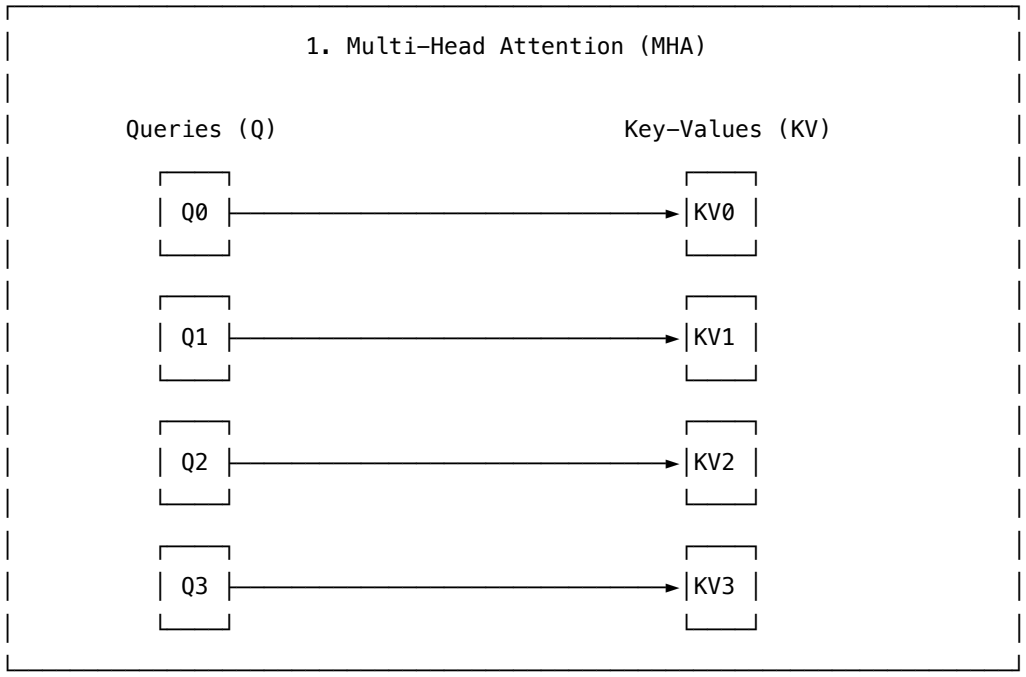
9 Grouped-Query Attention (GQA)

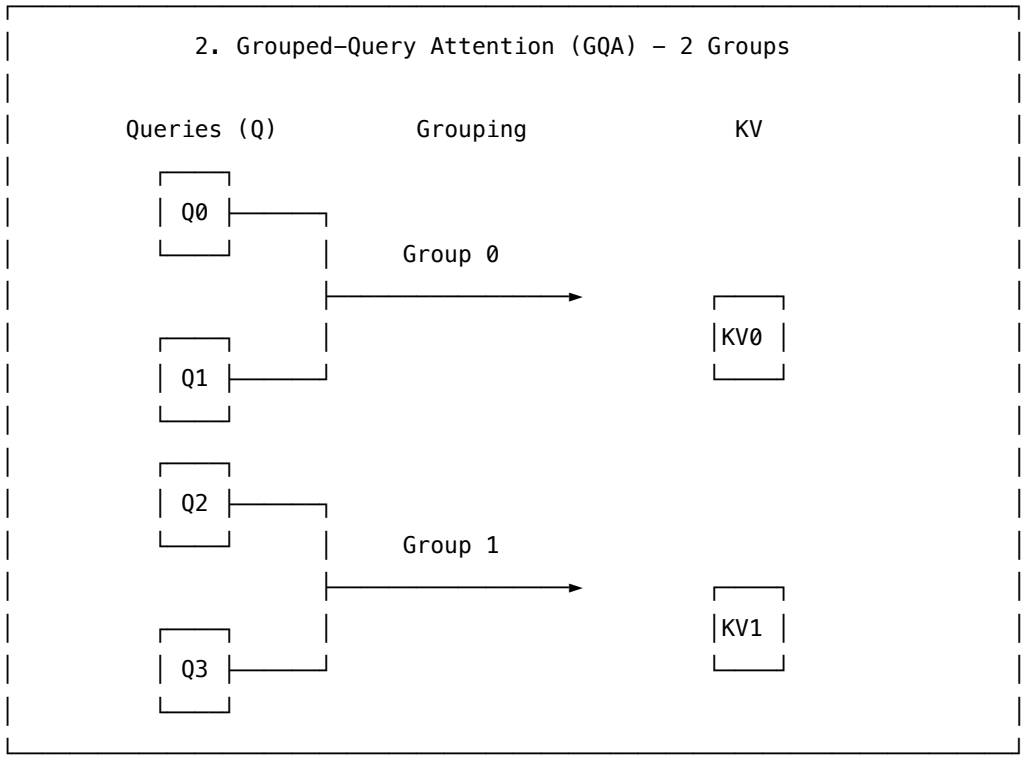
- **Category**: LLM Systems
- **Difficulty**: Hard
- **Target Role**: LLM Inference Engineer / ML Systems Engineer
- **Source**: GQA (Ainslie et al. 2023) + MQA (Shazeer 2019)
- **Flashcards**: LLM Systems deck

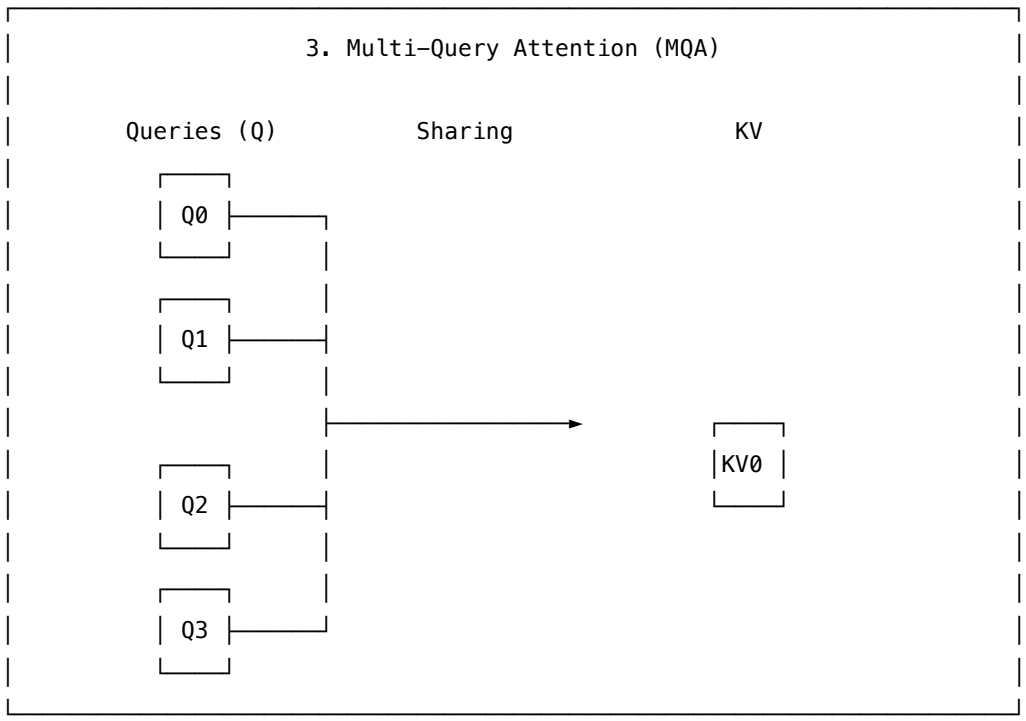
9.1 Concept Overview

Imagine a corporate office with 14 researchers (**Query heads**) who need to compile a report. In **Multi-Head Attention (MHA)**, each researcher has their own personal filing assistant (**KV head**) managing a separate filing cabinet of past data. This offers maximum accuracy, but maintaining 14 cabinets wastes

a massive amount of office space and makes retrieval slow. In **Multi-Query Attention (MQA)**, the office downsizes to just 1 filing cabinet shared by all 14 researchers. This saves massive space, but because everyone is queuing up for the same cabinet, the researchers' notes get cluttered and quality drops. **Grouped-Query Attention (GQA)** is the sweet spot: the researchers are divided into groups (e.g., 2 groups of 7 researchers), and each group shares 1 filing cabinet (2 cabinets total). This recovers almost all the accuracy of having individual cabinets while running nearly as fast as the single-cabinet setup.







9.1.1 The Problem It Solves

- **The KV Cache Bottleneck**: During the autoregressive generation (decode) phase of LLM serving, the model produces text one token at a time. To generate each new token, the GPU must reload the key-value matrices (the **KV Cache**) of *all previous tokens* from High-Bandwidth Memory (HBM) into SRAM.
- **Memory Bandwidth Bottleneck**: In MHA, the KV cache size is proportional to the number of query heads H_q. The memory bandwidth required per layer at sequence length S with head dimension D is:

  $$\text{KV Cache Elements} = 2 \times H_{kv} \times S \times D \quad (\text{where } H_{kv} = H_q)$$

  For long context lengths (e.g., $S = 4096$), fetching this cache for every decode step saturates the memory bus, slowing down generation speed (rendering the chip bandwidth-bound).
- **Quality-Memory Trade-off**: MQA ($H_{kv} = 1$) reduces this cache footprint by $H_q\times$, but causes accuracy degradation. GQA groups query heads to share KV heads, cutting the KV cache size by a factor of H_q/H_{kv} while matching the performance of MHA.

9.1.2 How It Works

GQA interpolates between MHA and MQA by grouping H_q query heads into H_{kv} groups. Each group contains $n_{\text{repeats}} = H_q/H_{kv}$ query heads that share a single KV head.

To implement this efficiently in hardware *without* physically duplicating the key and value vectors in memory (which would erase the memory savings), we use the **broadcast reshape trick**:

1. **Expose Group Structure**: Reshape the query tensor q of shape $[B, H_q, L, D]$ into q_g of shape $[B, H_{kv}, n_{\text{repeats}}, L, D]$.

2. **Inject Singleton Dimension**: Reshape key k and value v tensors of shape $[B, H_{kv}, S, D]$ into shape $[B, H_{kv}, 1, S, D]$.
3. **Implicit Broadcasting**: Perform attention score calculation. The 1 in the key tensor's shape is implicitly broadcast across the n_{repeats} dimension of the query tensor during matrix multiplication:

$$\text{scores} = \frac{q_g k_g^T}{\sqrt{D}} \quad \text{of shape} \quad [B, H_{\text{kv}}, n_{\text{repeats}}, L, S]$$

4. **Aggregate Output**: Compute softmax over the sequence length axis S and multiply by the value tensor v_g (which also broadcasts over n_{repeats}), producing an output shape of $[B, H_{kv}, n_{\text{repeats}}, L, D]$.
5. **Flatten Back**: Reshape the output back to $[B, H_q, L, D]$.

9.2 Worked Example

Consider a model with batch size $B = 1$, query heads $H_q = 4$, KV heads $H_{kv} = 2$, sequence length $L = S = 4$, head dimension $D = 8$, and $n_{\text{repeats}} = 2$.

9.2.1 KV Cache Memory Footprint Comparison

Let's look at the memory savings for a real-world scale model based on the Qwen2.5-0.5B config ($H_q = 14$, $H_{kv} = 2$, $D = 64$, 28 layers) at context length $S = 4096$ using FP16 (2 bytes per element):

Config	H_{kv}	Elements per Layer ($2 \cdot H_{kv} \cdot S \cdot D$)	Bytes per Layer (FP16)	Total Cache (28 Layers)	Cache Savings vs MHA
MHA	14	$7,340,032$	14.00 MiB	392.00 MiB	1.0× (Baseline)
GQA	2	$1,048,576$	2.00 **MiB**	56.00 **MiB**	7.0× **smaller**
MQA	1	$524,288$	1.00 MiB	28.00 MiB	14.0× smaller

9.2.2 Exposing the Reshape Broadcast Shapes

During the forward pass of GQA, the tensors are reshaped as follows: - **Input Query** q: $[1, 4, 4, 8]$ → **Reshaped** q_g: $[1, 2, 2, 4, 8]$ (where the dimensions represent $[B, H_{kv}, n_{\text{repeats}}, L, D]$) - **Input Key** k: $[1, 2, 4, 8]$ → **Reshaped** k_g: $[1, 2, 1, 4, 8]$ (where the dimensions represent $[B, H_{kv}, 1, S, D]$) - **Input Value** v: $[1, 2, 4, 8]$ → **Reshaped** v_g: $[1, 2, 1, 4, 8]$

During $\text{scores} = q_g k_g^T$, the $n_{\text{repeats}} = 2$ queries in group 0 both multiply the *same* KV head 0, while the queries in group 1 multiply KV head 1.

9.2.3 Verified Golden Output (Group 0, Head 0)

The exact attention output for the first query head ($h = 0$) across all sequence positions is:

Position	$d=0$	$d=1$	$d=2$	$d=3$	$d=4$	$d=5$	$d=6$	$d=7$
Pos 0	+0.1477	+0.1235	+0.1470	+0.2909	−0.0165	+0.1233	−0.1583	−0.0593
Pos 1	+0.1701	+0.1540	+0.1353	+0.2926	−0.0287	+0.1402	−0.1822	−0.0644
Pos 2	+0.1984	+0.0860	+0.1432	+0.3035	−0.0228	+0.0652	−0.1475	−0.1401
Pos 3	+0.1580	+0.1518	+0.1443	+0.2910	−0.0318	+0.1529	−0.1785	−0.0530

- **Golden Scalar**: $\text{out}[h=0, pos=0, d=0] = +\mathbf{0.147747}$

9.2.4 The Reshape Order Pitfall (The #1 GQA Implementation Bug)

Since query heads are contiguous in memory, they are grouped as $[Q_0, Q_1 \mid Q_2, Q_3]$. - **Correct Reshape**: `[H_kv, n_repeats]` → row-major partition splits contiguous blocks: $[[Q_0, Q_1], [Q_2, Q_3]]$. Heads Q_0, Q_1 pair with KV_0 (Correct). - **Wrong Reshape**: `[n_repeats, H_kv]` → row-major partition splits into striped blocks: $[[Q_0, Q_2], [Q_1, Q_3]]$. This maps Q_1 to KV_1 and Q_2 to KV_0.

Because the shapes still align (2×2 in both cases), **no code crash occurs**. However, the output is silently corrupted:

Variant	Value of $\text{out}[h=1, pos=0, d=0]$	Result
Correct Reshape (`H_kv, n_repeats`)	$+\mathbf{0.1724}$ (Pairs $Q_1 \to KV_0$)	**Correct**
Wrong Reshape (`n_repeats, H_kv`)	$-\mathbf{0.2905}$ (Pairs $Q_1 \to KV_1$)	**Scrambled (Error = 0.6535)**

9.3 Complexity & Trade-offs

Metric	Multi-Head Attention (MHA)	Grouped-Query Attention (GQA)	Multi-Query Attention (MQA)
KV Cache Bytes	$\mathcal{O}(H_q \cdot S \cdot D)$	$\mathcal{O}(H_{kv} \cdot S \cdot D)$	$\mathcal{O}(1 \cdot S \cdot D)$
HBM Read Bandwidth	High (Slowest Decode)	**Low (Fast Decode)**	Lowest (Fastest Decode)
Forward FLOPs	$\mathcal{O}(H_q \cdot S \cdot D)$	$\mathcal{O}(H_q \cdot S \cdot D)$	$\mathcal{O}(H_q \cdot S \cdot D)$
Downstream Accuracy	Upper Bound	**Near-MHA quality**	Noticeable accuracy drop
Key Advantage	Maximum expressivity	**Optimal throughput/quality**	Minimum memory usage

---

9.4 Common Interview Questions & How to Answer

9.4.1 Q1: Why is GQA considered a crucial optimization for serving long-context LLMs, and why does MHA fail at scale?

- **Answer**: LLM generation (decoding) is memory-bandwidth bound because the GPU must load the entire history of key-value vectors (the KV Cache) from HBM for every token generated. In MHA, the KV Cache grows quadratically with sequence length and linearly with the number of attention heads. For long context lengths (e.g., $S \geq 32\text{k}$), the cache becomes too large to fit in fast memory, causing severe latency bottlenecks or Out-of-Memory (OOM) errors. GQA addresses this by allowing groups of query heads to share KV heads. This reduces the KV Cache footprint and HBM read traffic by a factor of H_q/H_{kv} (e.g., $8\times$ in Llama 3), directly accelerating decoding speed while maintaining near-MHA-level quality.

9.4.2 Q2: What is the broadcast reshape trick, and why is it used instead of copying key-value tensors?

- **Answer**: The broadcast reshape trick is a tensor manipulation technique that implements GQA without duplicating KV cache memory. Instead of copying the KV heads n_{repeats} times to match the shape of the queries (which would waste memory and CPU/GPU cache space), we reshape the query tensor to $[B, H_{kv}, n_{\text{repeats}}, L, D]$ and insert a singleton dimension of 1 into the KV tensors: $[B, H_{kv}, 1, S, D]$. During the subsequent matrix multiplication, the hardware-level matrix engine automatically broadcasts the key/value data across the n_{repeats} axis in place. This avoids physical memory duplication and keeps the KV cache footprint small.

9.4.3 Q3: Why does changing the query reshape order from `[H_kv, n_repeats]` to `[n_repeats, H_kv]` corrupt the output, and why is it hard to debug?

- **Answer**: In the query tensor, the attention heads are arranged contiguously in memory. For example, with 4 heads and 2 groups, the layout is $[Q_0, Q_1 \mid Q_2, Q_3]$. Reshaping to `[H_kv, n_repeats]` (2×2) splits the heads into contiguous groups: group 0 gets $[Q_0, Q_1]$ and group 1 gets $[Q_2, Q_3]$. Reshaping to `[n_repeats, H_kv]` (2×2) splits them row-major into striped groups: group 0 gets $[Q_0, Q_2]$ and group 1 gets $[Q_1, Q_3]$. This routes the queries of Q_1 and Q_2 to the wrong KV cabinets. It is difficult to debug because the tensor shapes still match perfectly, meaning the code runs without error but outputs silent, garbled garbage.

9.5 Pro-Tip: How to Impress the Interviewer

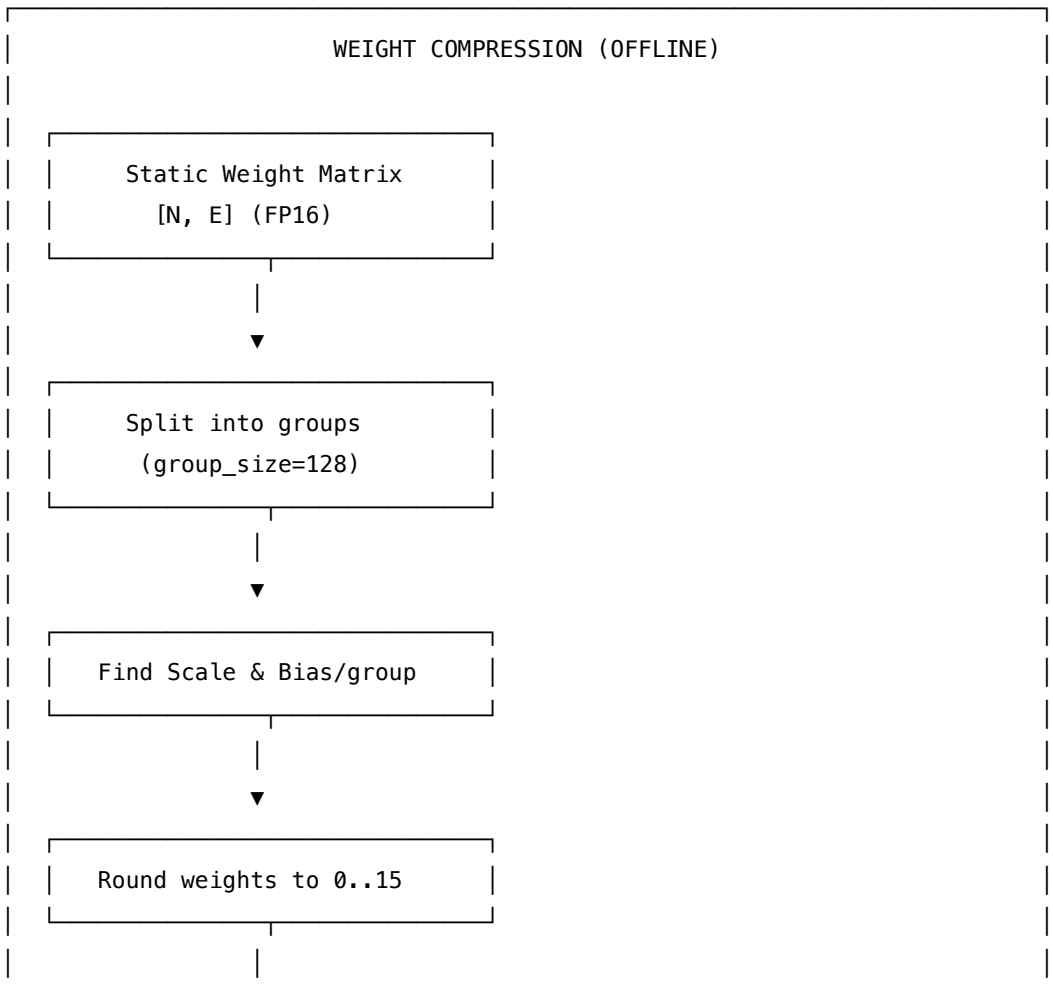
- **The GQA Split Limit (Tensor Parallelism)**: In large-scale deployments, models are split across multiple GPUs using Tensor Parallelism (TP). The query, key, and value heads are divided evenly among the GPUs. For example, if a model has $H_{kv} = 8$ KV heads and is split across TP = 8 GPUs, each GPU holds exactly 1 KV head. If you attempt to scale this setup to TP = 16 GPUs, the KV heads cannot be split further because you cannot have a fraction of a head. To scale past this limit, you must either replicate KV heads across GPUs (which increases memory usage) or transition to Pipeline Parallelism (PP), which introduces latency bubble overheads.

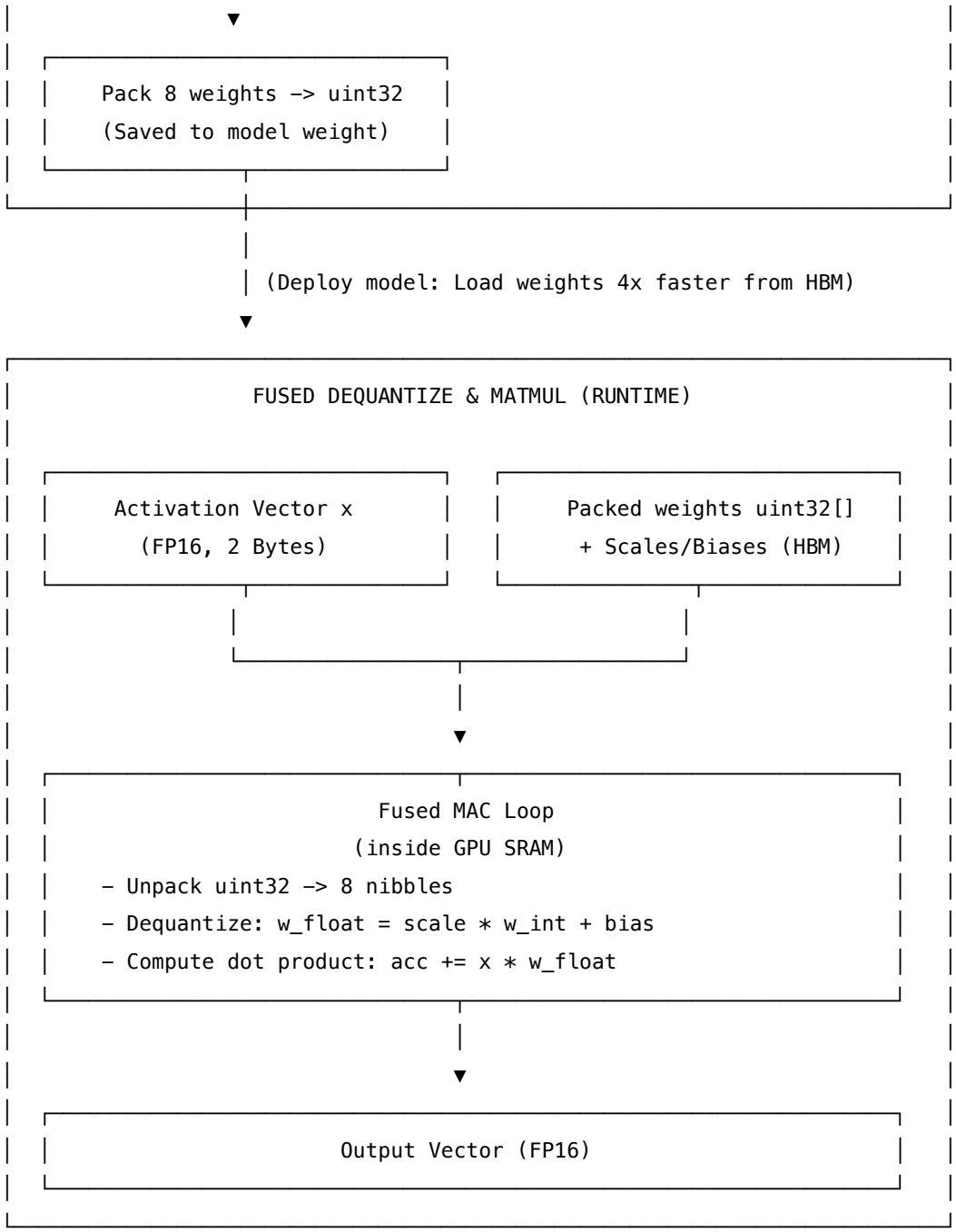
10 W4A16 Group Quantization

- **Category**: LLM Systems
- **Difficulty**: Hard
- **Target Role**: LLM Inference Engineer / ML Systems Engineer
- **Source**: LLM.int8() (Dettmers et al. 2022) + AWQ (Lin et al. 2024) + MLX affine quantization
- **Flashcards**: LLM Systems deck

10.1 Concept Overview

Think of quantization like a digital camera's color depth. A raw image stores millions of precise colors (float16/FP32), which takes huge storage. If you reduce the photo to a 16-color palette (4-bit), the file size shrinks dramatically. You might notice a slight "color banding" in the gradients, but the main features of the photo are still perfectly recognizable. In **W4A16 Group Quantization**, we do exactly this: we round the model's static **weights** to a 16-color palette (4 bits, or 16 discrete values) per group of weights, while keeping the dynamic **activations** (the actual data passing through the model) at full 16-bit resolution. Since LLM decoding is limited by how fast we can load weights from memory, compressing the weights to 4-bit makes the model load $\approx 4\times$ faster, while keeping activations in high-precision protects the model's reasoning quality.





10.1.1 The Problem It Solves

- **Memory Bandwidth Bottleneck**: In the autoregressive decode phase, the GPU cores compute the output for one token at a time. The computation is extremely simple, but the GPU must load the *entire* model weight matrix from slow High-Bandwidth Memory (HBM) to fast on-chip SRAM for every single token generated.
- **The Speed Limit**: Because HBM reads are slow, the GPU cores spend most of their time waiting for weights to arrive. This makes LLM serving **memory-bandwidth bound**.
- **Outlier Activation Sensitivity**: Standard methods that quantize both weights and activations to 8-bit (W8A8) or 4-bit (W4A4) suffer from severe accuracy drops. This is because activations

contain high-magnitude "outlier" channels (as shown in the LLM.int8() paper) that carry critical representation capacity. Rounding them to a coarse grid destroys the model's accuracy.

10.1.2 How It Works

W4A16 addresses this by compressing only the weights (saving bandwidth) and keeping the activations in FP16 (preserving precision). To minimize accuracy loss, it applies **Group Quantization**:

1. **Partitioning**: The weight matrix is split into contiguous groups along the input dimension (typically `group_size = 128`).
2. **Finding the Scale & Bias**: For each group, we compute a floating-point `scale` and `bias` to map the weight range $[w_{\min}, w_{\max}]$ to the 4-bit integer range $[0, 15]$.
3. **Quantizing**:
$$w_{\text{int}} = \text{clip}\left(\text{round}\left(\frac{w - \text{bias}}{\text{scale}}\right), 0, 15\right)$$
4. **Packing**: Eight 4-bit integers (each occupying a 4-bit *nibble*) are squeezed into a single 32-bit unsigned integer (`uint32`).
5. **Runtime Dequantization**: During inference, the weights are loaded from HBM in their packed `uint32` format. Inside the multiply-accumulate (MAC) loop, the GPU cores unpack the nibbles using bitwise shifts and masks, dequantize them back to FP16:
$$w_{\text{float}} = \text{scale} \times w_{\text{int}} + \text{bias}$$
and immediately multiply by the activation. The full-precision weight matrix is never materialized in HBM.

10.2 Worked Example

Consider a single group of 8 weights (group size = 8, 4-bit resolution) being quantized using the MLX affine convention.

10.2.1 Step 1: Deriving the Group Parameters

Let the input weight group be:

$$w = [-1.80,\ 0.55,\ -0.35,\ 1.20,\ -0.85,\ 0.25,\ -1.15,\ 1.05]$$

- **Extremes**: $w_{\min} = -1.80$, $w_{\max} = 1.20$.
- **Heavier Extreme**: $|w_{\min}| > |w_{\max}| \rightarrow 1.80 > 1.20$. The negative end is heavier, so we set the anchor `edge = w_min = -1.80`.
- **Target Quantization Level**: For 4 bits, the negative extreme maps to index -9 (since the levels range from -8 to $+7$, scaled appropriately).
- **Scale**: scale $= \text{edge}/q_0 = -1.80/-9 = \mathbf{0.200000}$
- **Bias**: bias $= \text{edge} = \mathbf{-1.800000}$ (in float units)

10.2.2 Step 2: Mapping Weights to 4-bit Integers

For each weight, we compute $w_{\text{int}} = \text{clip}(\text{round}((w - \text{bias})/\text{scale}), 0, 15)$:

Element (k)	Original $w[k]$	$\frac{w-\text{bias}}{\text{scale}}$	Round	Stored w_{int}	Dequantized Value	Rounding Error
0	-1.80	$+0.00$	0	0	-1.80	$+0.00$ (Zero error anchor)
1	$+0.55$	$+11.75$	12	12	$+0.60$	-0.05
2	-0.35	$+7.25$	7	7	-0.40	$+0.05$
3	$+1.20$	$+15.00$	15	15	$+1.20$	$+0.00$ (Other extreme anchor)
4	-0.85	$+4.75$	5	5	-0.80	-0.05
5	$+0.25$	$+10.25$	10	10	$+0.20$	$+0.05$
6	-1.15	$+3.25$	3	3	-1.20	$+0.05$
7	$+1.05$	$+14.25$	14	14	$+1.00$	$+0.05$

The maximum rounding error is 0.05 (exactly $\leq \text{scale}/2 = 0.1$).

10.2.3 Step 3: Packing into `uint32`

The 8 integers $w_{\text{int}} = [0, 12, 7, 15, 5, 10, 3, 14]$ are packed LSB-first: - $w_{\text{int}}[0] = 0 \rightarrow$ bits $[0 : 4]$ - $w_{\text{int}}[1] = 12$ (0xC) $\rightarrow$ bits $[4 : 8]$ - ... - $w_{\text{int}}[7] = 14$ (0xE) $\rightarrow$ bits $[28 : 32]$

Packed Decimal = **3819304896** Hex = **0xE3A5F7C0**

Note: Reading the hex representation right-to-left recovers the original integers: $0xE3A5F7C0 \rightarrow 0, C\ (12), 7, F\ (15), 5, A\ (10), 3, E\ (14)$.

10.2.4 Step 4: Quantized Matrix Multiplication

Let an activation vector be $x = [0.5, -0.3, 0.8, -0.1, 0.4, -0.6, 0.2, 0.7]$. We perform $x@W^T$ against a 4×8 weight matrix (where row 0 is our quantized weight above):

Output Index	Full-Precision Matmul ($x@W^T$)	Quantized Matmul (Dequant in MAC)	Absolute Error
Row 0	-1.4500	-1.5000	0.0500
Row 1	$+2.0800$	$+2.0800$	0.0000
Row 2	-1.9500	-2.0267	0.0767
Row 3	$+2.0600$	$+1.9987$	0.0612

- **Mean Relative Error**: 2.59% (extremely minor deviation, which is why accuracy is preserved at scale).

10.3 Memory Footprint Analysis (Qwen2.5-0.5B scale)

For one linear layer weight matrix of shape $[896, 896]$:

Format	Weight Memory Footprint	Compression vs FP16
FP16 / BF16 Baseline	$896 \times 896 \times 2$ bytes = **1.606** MB	1.0×
W4A16 (group_size = 8)	401,408 B (weights) + 401,408 B (scale/bias) = **0.803** MB	2.0×
W4A16 (group_size = 128)	401,408 B (weights) + 25,088 B (scale/bias) = **0.426** MB	3.76×

Note: Group size = 128 is the real-world standard. Group size = 8 wastes too much memory on scale/bias overhead (1 scale and 1 bias for every 8 weights).

10.4 The Sign-Convention Pitfall (The #1 Quantization Bug)

There are two major dequantization standards in the LLM ecosystem: 1. **MLX Affine**: $w = \text{scale} \times w_{\text{int}} + \text{bias}$ (where `bias` is stored in **float** units). 2. **Textbook Zero-Point (GPTQ/AWQ)**: $w = \text{scale} \times (w_{\text{int}} - \text{zero\_point})$ (where `zero_point` is stored in **integer** units).

They are equivalent *only* if:

$$\text{zero\_point} = -\frac{\text{bias}}{\text{scale}}$$

If a developer plugs an MLX float `bias` (-1.8) into a textbook-style implementation (e.g. `(w_int + bias) * scale`), it scrambles the weights:

Variant	Equation	Result for $w_{\text{int}} = 12$	Verdict
MLX Convention	$0.2 \times 12 + (-1.8)$	**+0.6000**	**Correct**
Textbook Convention	$0.2 \times (12 - 9.0)$	**+0.6000**	**Correct**
Incorrect Prose Cross	$(12 + (-1.8)) \times 0.2$	**+2.0400**	**Garbage (Error = 1.44)**

10.5 Complexity & Trade-offs

Metric	Value	Notes
HBM Read Bandwidth	$\approx 3.76\times$ reduction	Speeds up decode phase since we load far fewer bytes from main memory.
GPU Compute (FLOPs)	Slightly higher	We perform additional bitwise shifts, masks, and FMA operations to dequantize.
SRAM Memory Usage	Bounded	Activations and unpacked registers live in SRAM; no full weight dequantization in SRAM.
Model Size	$\approx 70-75\%$ reduction	Allows larger models (e.g., Llama 3 70B) to run on a single consumer GPU.
Outlier Handling	Excellent	Leaving activations in FP16 ensures outlier channels are not clipped.

10.6 Common Interview Questions & How to Answer

10.6.1 Q1: Why do we choose W4A16 (weights in 4-bit, activations in 16-bit) rather than quantizing activations to 4-bit as well (W4A4)?

- **Answer**: Activation vectors contain dynamic "outlier channels" — specific dimensions that have exceptionally large values (up to $100\times$ higher than other channels) across almost all tokens. If we quantize activations to 4-bit, these outlier values force the quantization step size to be very large, which destroys the resolution of the non-outlier values, leading to a catastrophic drop in model accuracy. Weights do not exhibit this extreme, token-dependent outlier behavior. Since LLM decoding is memory-bandwidth bound (limited by weight loading, not activation loading), keeping activations in 16-bit preserves representation capacity while quantizing weights to 4-bit achieves the desired memory throughput.

10.6.2 Q2: How does dequantization on-the-fly work inside the matrix multiplication kernel, and why is this faster than dequantizing the weights beforehand?

- **Answer**: If we dequantized the 4-bit weights back to FP16 in memory *before* running the matrix multiplication, we would have to write the FP16 weights back to HBM and read them again. This would defeat the entire purpose of quantization, as we would still be bottlenecked by FP16 HBM traffic. Instead, we perform "fused dequantization" inside the Multiply-Accumulate (MAC) loop. The thread blocks load the packed `uint32` weights from HBM directly into the GPU registers/SRAM. Inside the execution pipeline, the cores unpack the 4-bit integers using bitwise shifts and masks, apply the scale and bias, and immediately multiply by the activations. The full-precision weights are never written to main memory, keeping the memory bandwidth requirement to a minimum.

10.7 Pro-Tip: How to Impress the Interviewer

- **Avoiding Dequantization Latency (Bit-Unpacking Bottlenecks)**: Explain that while W4A16 saves memory bandwidth, the extra instructions required to unpack the `uint32` values (shifts, bitwise ANDs, casting to floats) can introduce computation overhead, making the kernel instruction-bound. On modern NVIDIA GPUs, we can optimize this by using specialized SIMD byte-permutation instructions (like `prmt` or warp-level shuffle assembly) to unpack multiple nibbles simultaneously. Additionally, storing the scale and bias in half-precision (FP16) allows us to perform the dequantization using fast Half-Precision Tensor Core instructions, overlapping the unpacking latency with the main matrix operations.

11 LLM Sampling Strategies

- **Category**: LLM Systems
- **Difficulty**: Medium
- **Target Role**: LLM Inference Engineer / LLM Serving Engineer
- **Source**: sampling.py / SAMPLING.md (tiny-llm)
- **Flashcards**: LLM Systems deck

11.1 Concept Overview

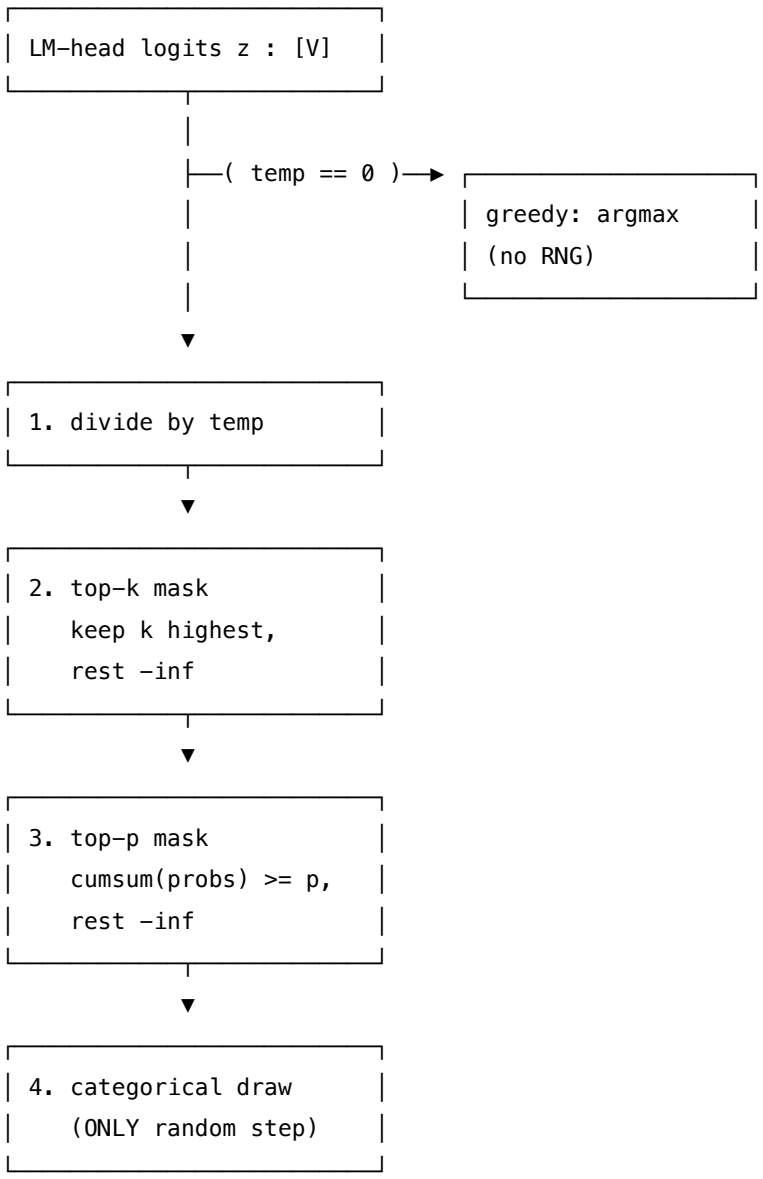
Think of sampling as rolling a **loaded die with one face per vocabulary word**. The model doesn't hand you the next token directly — it gives you a *preference score* (logit) for every possible next word, and sampling is how you convert those scores into a single choice. Four strategies give you four different ways to *shape the die* before you roll it: greedy always picks the heaviest face, temperature reshapes all the weights, top-k blanks out all but the highest k faces, and top-p (nucleus) blanks everything past a cumulative probability threshold — adapting the kept set to the distribution's actual shape.

The full token-generation pipeline: each step, the model produces a logit vector of shape `[vocab_size]`, you apply your strategy to filter/reshape it, then draw one token from the resulting distribution. That token is appended to the input and the loop repeats.

11.1.1 The Problem It Solves

Without a strategy you face two failure modes: - **Always argmax (greedy)**: the model loops and degenerates. The single most-likely next word after a repeated phrase is the phrase again ("neural text degeneration", Holtzman et al. 2019). - **Pure sampling ($T = 1$, no filtering)**: the low-probability tail gets drawn too often → incoherent outputs. On the worked example, the nonsense token "qqq" would appear 2.6% of the time.

11.1.2 How It Works



Production pipeline order (from `make_sampler`): 1. $T == 0 \rightarrow$ return `argmax` (greedy, skip everything else) 2. Divide logits by T 3. Apply top-k mask (mask outside top-k to $-\infty$) 4. Apply top-p mask (mask outside nucleus to $-\infty$) 5. `categorical draw` — the **only** random step

---

11.2 Worked Example

All numbers from `sampling_output.txt` (run `uv run python sampling.py`).

11.2.1 Base Distribution (Section A)

Fixed logits (vocab size = 8):

```
LOGITS = [2.3, 2.0, 0.4, 1.5, 0.1, 2.5, 0.7, 1.2]
```

```
TOKENS = ["the", "cat", "xyz", "sat", "qqq", "on", "a", "mat"]
```

idx	token	logit	prob (softmax)	logprob
0	the	+2.3000	**0.2377**	−1.4367
1	cat	+2.0000	**0.1761**	−1.7367
2	xyz	+0.4000	0.0356	−3.3367
3	sat	+1.5000	0.1068	−2.2367
4	qqq	+0.1000	0.0263	−3.6367
5	**on**	+2.5000	**0.2903**	−1.2367
6	a	+0.7000	0.0480	−3.0367
7	mat	+1.2000	0.0791	−2.5367

$\sum$ probs = 1.000000 · argmax = idx 5 ("on")

11.2.2 Temperature Scaling (Section B)

$$p_i(T) = \text{softmax}\left(\frac{z_i}{T}\right)$$

High T = flatter (more random); low T = sharper.

idx	token	T=0.5	T=1.0	T=2.0
0	the	0.2917	0.2377	0.1858
1	cat	0.1601	0.1761	0.1599
2	xyz	0.0065	0.0356	0.0719
3	sat	0.0589	0.1068	0.1245
4	qqq	0.0036	0.0263	0.0618
5	**on**	**0.4351**	**0.2903**	**0.2053**
6	a	0.0119	0.0480	0.0835
7	mat	0.0323	0.0791	0.1072

- entropy($T = 0.5$) = 1.3981 nats (sharp)
- entropy($T = 1.0$) = 1.8062 nats
- entropy($T = 2.0$) = 1.9984 nats (flat)

Rule: $T \to 0 \Rightarrow$ greedy; $T \to \infty \Rightarrow$ uniform. $T < 1$ = confident, $T > 1$ = creative.

11.2.3 Greedy Decoding (Section C)

```
greedy(LOGITS) = argmax = idx 5 ("on")
```

$T = 0$ is a **special case**: skip softmax entirely, return argmax directly. Deterministic, zero RNG, same token every call. Use for evaluation / factual QA — never for creative generation.

11.2.4 Top-$k = 3$ (Section D)

Blank all but the 3 highest logits to $-\infty$. Renormalize over survivors.

- **KEPT** indices: `[0, 1, 5]` → tokens `['the', 'cat', 'on']`
- **MASKED** indices: `[2, 3, 4, 6, 7]`

idx	token	logit	prob (renorm)
0	the	+2.3000	0.3376
1	cat	+2.0000	0.2501
5	on	+2.5000	**0.4123**
others	—	-inf	0.0000

The flaw of top-k: k is *fixed*. On a peaked distribution (one token at 95%), $k = 50$ lets 49 junk tokens back in. On a flat distribution, $k = 10$ cuts good options. Top-k is blind to distribution shape — that's what top-p fixes.

11.2.5 Top-$p = 0.6$ / Nucleus Sampling (Section E)

Keep the **smallest set** whose cumulative probability $\geq p$. Adaptive size.

Algorithm: 1. Sort tokens DESC by probability 2. cumsum(probs) — **on probs, NOT logprobs** (see pitfall below) 3. Keep where cumsum $< p$; **always keep top-1** (nucleus never empty) 4. Mask the rest to $-\infty$

rank	idx	token	prob	cumsum	cumsum<0.6?	kept?
0	5	on	0.2903	0.2903	yes	**KEEP**
1	0	the	0.2377	**0.5281**	yes	**KEEP**
2	1	cat	0.1761	0.7042	no	mask
3	3	sat	0.1068	0.8110	no	mask
4	7	mat	0.0791	0.8901	no	mask
5	6	a	0.0480	0.9381	no	mask
6	2	xyz	0.0356	0.9737	no	mask
7	4	qqq	0.0263	1.0000	no	mask

NUCLEUS (top-$p = 0.6$) = indices `[0, 5]` (`['the', 'on']`), prob mass `0.5281`.

Top-k vs top-p on this distribution:

	top-$k = 3$	top-$p = 0.6$
Kept indices	`[0, 1, 5]`	`[0, 5]`

	top-$k = 3$	top-$p = 0.6$
Size	3 (fixed)	2 (adaptive)
Why	always 3	top-2 already cover $52.8\% > p$; 3rd token excluded

---

11.2.6 The #1 Bug: cumsum on logprobs instead of probs (Section F)

Logprobs are always ≤ 0. Their cumsum stays negative forever — always less than any positive p. Result: **nothing gets masked**; the nucleus silently becomes the entire vocabulary.

	Correct: cumsum on **probs**	Wrong: cumsum on **logprobs**
Nucleus	**2 tokens** `[0, 5]`	**8 tokens** (entire vocab!)
Filtered?	YES	NO — silent bug

Concrete numbers ($p = 0.6$): - cumsum(probs) at cat = **0.7042** $\rightarrow > 0.6 \rightarrow$ masked - cumsum($\log$ probs) at cat = **−4.4100** $\rightarrow < 0.6 \rightarrow$ NOT masked (bug!)

Fix: `cumsum(exp(logprobs))` — the `exp` is mandatory.

11.2.7 Combined Pipeline (Section G)

Config: $T = 0.7$, top-$k = 3$, top-$p = 0.6$

After top-$k = 3$ then top-$p = 0.6$, only `idx 5 ("on")` survives both filters. After $/T = 0.7$ and renormalize: `idx 5 ("on")` has prob = 1.0000.

```
torch.manual_seed(0); multinomial → idx 5 ("on")
```

> **Key insight**: composing filters is NOT set-intersection. Top-p runs on the **renormalized post-top-k distribution**, so the nucleus boundary shifts. The order and renormalization both matter.

11.3 Complexity & Trade-offs

Metric	Value	Notes
Greedy complexity	$\mathcal{O}(V)$	argmax scan
Temperature overhead	$\mathcal{O}(V)$	scalar division + softmax
Top-k overhead	$\mathcal{O}(V \log k)$	partial sort
Top-p overhead	$\mathcal{O}(V \log V)$	full sort + cumsum
Top-k: fixed or adaptive?	Fixed	Always keeps exactly k tokens
Top-p: fixed or adaptive?	**Adaptive**	Set size varies per distribution

Metric	Value	Notes
RNG steps per token	1	Only the final multinomial draw
Seed usage	Once per generation	Makes entire decode reproducible
Production default	$T = 0.7$, top-$p = 0.95$	Nucleus with mild temperature

---

11.4 Common Interview Questions & How to Answer

11.4.1 Q1: What is the difference between top-k and top-p sampling?

- **Answer**: Top-k keeps a **fixed number** of tokens (always k). Top-p (nucleus) keeps the **smallest set** of tokens whose cumulative probability reaches p — so the set size *adapts* to the distribution shape. On a confident distribution, top-p may keep just 1 token; on a flat distribution it may keep 20+. Concrete example: top-k=3 keeps `[0,1,5]` (always 3), top-p=0.6 keeps `[0,5]` (just 2, because the top-2 already cover 52.8%). Top-p is adaptive; top-k is blind to distribution shape.

11.4.2 Q2: Why must you cumsum on probabilities and NOT log-probabilities in top-p?

- **Answer**: Logprobs are always ≤ 0, so their cumsum stays negative forever, making `cumsum(logprobs) < p` always true for any positive p. This means *nothing* ever gets masked — the nucleus silently becomes the whole vocabulary with no error. The fix is `cumsum(exp(logprobs))`. This is the single most common top-p implementation bug.

11.4.3 Q3: What does temperature do mathematically, and what is its effect on entropy?

- **Answer**: Temperature `T` divides the logits *before* softmax: `probs_T = softmax(z / T)`. Dividing by `T<1` widens the logit gaps (sharpens toward argmax); `T>1` narrows gaps (flattens toward uniform). Real numbers: `entropy(T=0.5) = 1.3981 nats`, `entropy(T=1.0) = 1.8062`, `entropy(T=2.0) = 1.9984`. `T=0` collapses to a one-hot $\rightarrow$ greedy.

11.4.4 Q4: What is the correct sampling pipeline order and why does it matter?

- **Answer**: The order is: `/temp → top-k → top-p → multinomial`. Temperature must come first because it reshapes the distribution that top-p then cuts. Applying temperature *after* top-p means top-p computed on the wrong distribution. Top-k is order-insensitive to temperature (positive scaling preserves rank), but top-p is sensitive (temperature reshapes cumulative probabilities). Filter steps are deterministic; only the final `multinomial` uses the RNG.

11.4.5 Q5: When would you use greedy vs nucleus sampling?

- **Answer**: Use **greedy** (`temp=0`) for factual QA, benchmarks, or any task needing reproducibility and the single-best answer. Use **nucleus** (`top_p=0.9–0.95, temp=0.7`) for creative generation, dialogue, and summarization — where diversity and avoiding repetition loops matter. Greedy is prone to degenerate looping on creative tasks.

11.4.6 Q6: What is "neural text degeneration" and which sampling strategy addresses it?

- **Answer**: Neural text degeneration (Holtzman et al., ICLR 2020) is the failure mode where greedy or beam search produces repetitive, loopy text — because the most likely next token after a repeated phrase is the phrase again. Top-p (nucleus) sampling addresses it by cutting the low-probability tail while adapting the shortlist to the distribution shape, preventing the model from getting stuck in high-probability repetition loops.

11.5 Pro-Tip: How to Impress the Interviewer

- **Cite real numbers**: "On 8-token vocab with logits `[2.3,2.0,0.4,1.5,0.1,2.5,0.7,1.2]`, top-k=3 keeps `[0,1,5]` but top-p=0.6 keeps `[0,5]` because the top-2 already hold 52.8% — and adding a third would push past 0.6."
- **Know the silent failure mode**: logprob cumsum produces no error, no crash — just a nucleus equaling the entire vocabulary. "I always `assert nucleus_size < vocab_size` in integration tests."
- **Know all three components**: `temp` (reshapes), `top-k` (hard cutoff), `top-p` (adaptive cutoff). Production systems use all three in sequence.
- **Mention beam search as a contrast**: keeps top-B *sequences* (not tokens) across steps. Deterministic but expensive `O(B × V)` per step and prone to generic outputs. Nucleus remains the production default.
- **Temperature entropy intuition**: at T=0.5, entropy drops to 1.3981 nats (−23% from T=1); at T=2.0, it rises to 1.9984 nats (+11%). The "creativity dial" has concrete, measurable effects.

12 PagedAttention: Virtual Memory for KV Cache

- **Category**: LLM Systems
- **Difficulty**: Hard
- **Target Role**: LLM Inference Engineer / LLM Serving Engineer
- **Source**: paged_attention.py / PAGED_ATTENTION.md (vLLM, SOSP 2023)
- **Flashcards**: LLM Systems deck

12.1 Concept Overview

Imagine a **library with shared shelves carved into fixed-size pages**. Before PagedAttention, every reader (request) got their own giant private shelf reserved the moment they walked in — even if they only used 500 of 8192 available slots, the other 7692 stayed empty and reserved. PagedAttention replaces this with the OS virtual-memory trick: each reader gets an **index card** (the block table) listing which physical shelf pages hold their notes, and those pages can be scattered anywhere in the shared pool. No pre-reservation, on-demand allocation, waste bounded to the last partial page.

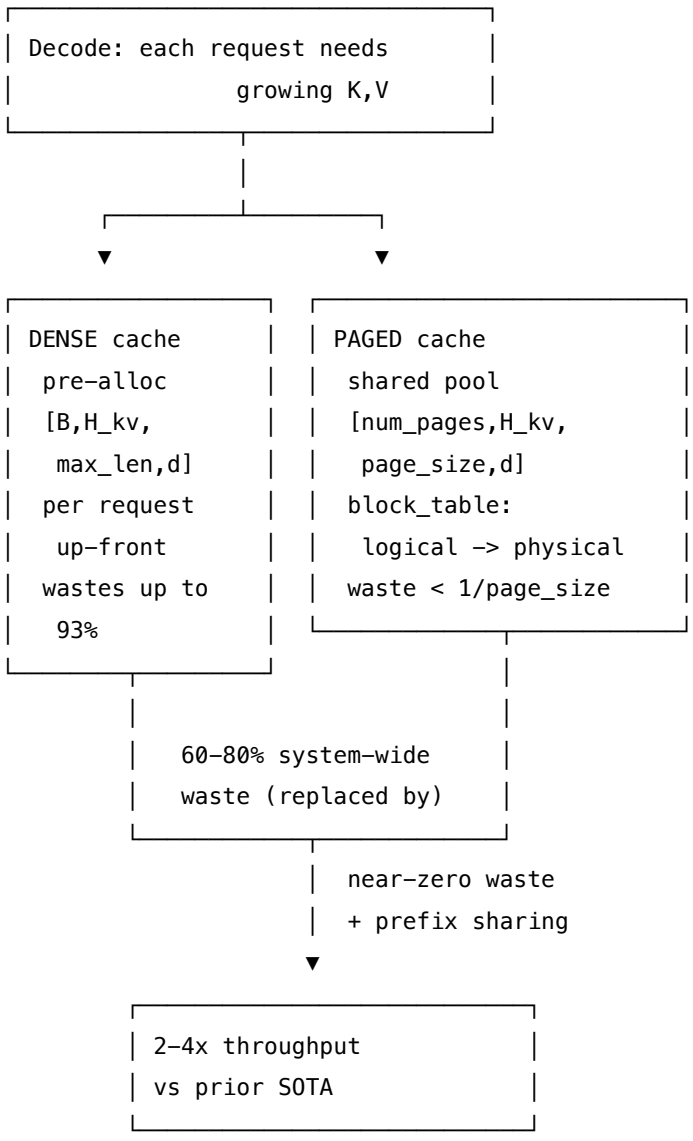
The key insight borrowed from OS virtual memory: **"blocks as pages, tokens as bytes, sequences as processes"** (vLLM paper §3.1). The paged attention kernel reads K,V through an indirect lookup (like

an OS page table walk) instead of a contiguous slab, costing one extra memory indirection per tile but enabling non-contiguous, on-demand allocation.

12.1.1 The Problem It Solves

Dense KV caching pre-allocates `[B, H_kv, max_seq_len, d]` per request up front. With `max_seq_len=8192` but only 512 tokens used, **93.75%** of that slab is dead VRAM. For 100 concurrent requests, this wastes **375 GiB** of 400 GiB reserved. vLLM measured that prior serving systems wasted **60–80%** of KV memory system-wide. Dead VRAM means fewer concurrent requests → lower GPU utilization → lower throughput.

12.1.2 How It Works



12.2 Worked Example

All numbers from `paged_attention_output.txt`.

12.2.1 Section A: The Dense-Waste Problem

```
max_seq_len reserved = 8192, actual tokens used = 512
waste fraction = 1 - 512/8192 = 0.9375 = 93.75%
```

Per-request dense KV bytes (LLaMA-7B: 32 layers, 32 KV heads, $d = 128$):

```
2(\text{KV}) \times 32 \times 32 \times 8192 \times 128 \times 2\text{ B} = 4,294,967,296\text{ bytes} = 4.000\te
```

100 concurrent requests (each using only 512 tokens):

```
reserved = 400.0 GiB
used     = 25.0 GiB
wasted   = 375.0 GiB  (93.8% of reserved!)
```

Paged cache waste bound (only the last page of each request is partial):

page_size	worst-case internal waste
16	6.25% ← vLLM default
128	0.78% ← tiny-llm

vLLM measured: PagedAttention wastes $< 4\%$ in practice vs 60–80% for prior systems.

Two different numbers — keep them distinct: | Number | What it measures | |--------|----------------| | **93.75%** | One request's reserved-vs-used slab $(1 - 512/8192)$ | | **60–80%** | System-wide KV waste measured by vLLM across all concurrent requests | | $< 4\%$ | System-wide waste under PagedAttention |

12.2.2 Section B: Page Pool + Free List

The pool is a shared slab `[num_pages, H_kv, page_size, D]`. The free list is the OS frame allocator.

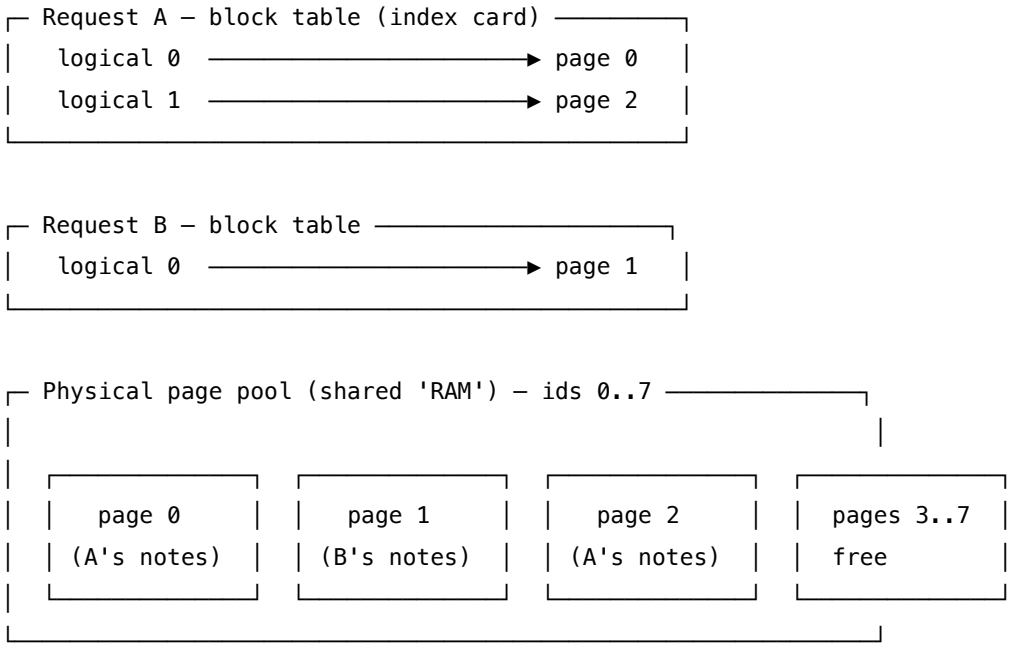
```
page_size=2, H_kv=2, D=8, num_pages=4
key_pages shape: (4, 2, 2, 8)  = [num_pages, H_kv, page_size, D]
```

step	action	free_list after	pool.used
1	allocate() → 0	[1, 2, 3]	[0]
2	allocate() → 1	[2, 3]	[0, 1]
3	allocate() → 2	[3]	[0, 1, 2]
4	free_page(0)	[0, 3]	[1, 2]
5	allocate() → 0	[3]	[0, 1, 2]

Step 5: `free_page(0)` returned page 0, and the next `allocate()` immediately **reused** it. No fragmentation.

12.2.3 Section C: The Block Table (Logical → Physical)

Analogy: each request gets an *index card* listing which physical pages hold their tokens IN ORDER. Pages can be scattered anywhere in the pool.



From the output (interleaved allocation so storage is scattered):

```
A.page_ids  = [0, 2]   (logical 0 -> phys 0, logical 1 -> phys 2)
A.page_lens = [2, 1]   (page 0 full, page 1 partial — the ONLY waste)
B.page_ids  = [1]      (logical 0 -> phys 1)
B.page_lens = [2]
pool.used   = [0, 1, 2]
pool.free   = [3, 4, 5, 6, 7]
```

A's storage is scattered `[0, 2]` — non-contiguous! A interleaved with B in the free list.

Logical → Physical address translation for Request A (`page_size=2`):

logical pos `t`	`page_idx = t//PS`	`slot = t%PS`	`page_id = block_table[t//PS]`
0	0	0	0
1	0	1	0
2	1	0	**2**

GOLD: `A logical pos 2 → page_id 2, slot 0`

12.2.4 Section F: The Paged Attention Kernel (Indirect K/V Gather)

The kernel gathers K,V from **scattered pages** via the block table. This is a FlashAttention variant where the K/V fetch is indirect.

Indirect-lookup formula (the heart of the kernel):

```
For logical token position `col` of request `batch`:
    page_idx = col // page_size                    # logical page
    slot     = col %  page_size                    # offset within page
    page_id  = block_table[batch, page_idx]        # OS page-table lookup
    k_idx    = ((page_id * H_kv + kv_head) * page_size + slot) * D + c
    score   += q[c] * key_pages[k_idx]             # dot-product contribution
```

Contrast with dense sequential addressing:

```
    k_ptr = k_base + (j * Bc + b) * D + c          # one slab, contiguous
```

Worked example (logical pos 2, head 0, dim 0):

```
page_idx = 2 // 2 = 1
slot     = 2 %  2 = 0
page_id  = block_table[1] = 1
k_idx    = ((1*2 + 0)*2 + 0)*8 + 0 = 32
K via formula   = 0.3010
K via tensor[…] = 0.3010  ✓ MATCH
```

Attention output comparison (token 2 as query, head 0, dims 0..3):

```
paged (indirect gather): [0.7067, 0.7077, 0.7087, 0.7097]
dense  (gather+matmul):  [0.7067, 0.7077, 0.7087, 0.7097]
[check] paged-attn == dense-attn at tol 1e-05: OK
```

12.2.5 Section G: Paged == Dense == Raw (Invariant)

Building the same 4-token K cache three ways:

path	K[0,0,0,:4] (token0)	K[0,0,3,:4] (token3)
no-cache (raw)	[0.101, 0.102, 0.103, 0.104]	[0.401, 0.402, 0.403, 0.404]
dense cache	[0.101, 0.102, 0.103, 0.104]	[0.401, 0.402, 0.403, 0.404]
paged cache (PS=2)	[0.101, 0.102, 0.103, 0.104]	[0.401, 0.402, 0.403, 0.404]

```
[check] ALL THREE PATHS MATCH: OK
```

PagedAttention only changes **where** the bytes live (scattered pages vs contiguous slab) — never **what** they are.

12.3 Complexity & Trade-offs

Metric	Dense Cache	Paged Cache	Notes
Memory reserved	`[B,H_kv,max_len,d]` up-front	on-demand pages	Dense wastes 93.75% for short seqs
Worst-case waste	**93.75%** (one req) / **60–80%** (system)	$< 4\%$ (system)	vLLM paper §3.2
K/V addressing	sequential: `base + j*D`	indirect: `((page_id*H+head)*PS+slot)*D+c`	One extra indirection
Prefix sharing	impossible (private slabs)	ref-counted pages	CoW for forked sequences
Rewind (spec decode)	truncate offset	pop pages to free list	Use `ceil`, not `floor`
Throughput gain	baseline	**2–4× vs prior SOTA**	vLLM paper §5
page_size tradeoff	—	smaller = finer sharing, more overhead	vLLM default: 16

12.4 Common Interview Questions & How to Answer

12.4.1 Q1: How does PagedAttention eliminate KV cache fragmentation?

- **Answer**: Dense KV caching pre-allocates `max_seq_len` slots per request at admission, wasting 93.75% for a 512-token sequence on an 8192 slot slab. PagedAttention uses the OS virtual-memory trick: a shared pool of fixed-size pages and a per-request block table mapping logical page indices to physical page ids. Pages are allocated on-demand and returned to the pool when the request finishes. The only waste is the last partial page per request — bounded by `1/page_size` ($< 4\%$ in practice).

12.4.2 Q2: Explain the block table and how the paged attention kernel uses it.

- **Answer**: The block table is the per-request logical → physical page index: `page_ids[logical_page] = physical_page_id`. For logical token position t, the kernel computes `page_idx = t // page_size`, `slot = t % page_size`, then looks up `page_id = block_table[page_idx]`. The K address is then `k_idx = ((page_id*H_kv + kv_head)*page_size + slot)*D + c`. This one extra indirection enables scattered physical storage. Concrete example: for logical pos 2 with `page_size=2`, `page_idx=1, slot=0, page_id=2, k_idx=32` — verified to produce $K = 0.3010$, matching dense.

12.4.3 Q3: What is the `rewind` operation and why must it use `ceil` not `floor`?

- **Answer**: `rewind(n)` undoes the last n appended K,V tokens during speculative decoding rejection. It computes `target_pages = ceil(new_offset / page_size)` and pops pages beyond that target to the free list. Using `floor` would be wrong at exact page boundaries: offset $= 8$, rewind$(1) \rightarrow$ new $= 7$, ceil$(7/4) = 2$ pages kept (correct: $4 + 3$ tokens), but floor$(7/4) = 1$ would drop a still-needed page → silent data loss.

12.4.4 Q4: How does PagedAttention enable prefix sharing across requests?

- **Answer**: Because pages are allocated from a shared pool with a block table, two requests that wrote identical token prefixes can reference the **same physical page** (ref-counted). The `BlockManager` uses content-addressing (chained hash of token ids) to detect when an incoming request's prefix matches an already-allocated page, and assigns it the same physical page id with `ref_count++`. This enables sharing system prompts, few-shot contexts, and chat history without recomputing KV.

12.4.5 Q5: What are the two distinct waste numbers from the vLLM paper and what do each measure?

- **Answer**: (1) **93.75%**: per-request arithmetic $1 - 512/8192$ — illustrates how wasteful dense pre-allocation is for one sequence. (2) **60–80%**: system-wide KV memory waste measured by vLLM across heterogeneous concurrent requests (internal + external fragmentation + over-reservation). PagedAttention brings this to $< 4\%$ system-wide. Don't confuse the two — one is illustrative per-request math, the other is a production measurement.

12.5 Pro-Tip: How to Impress the Interviewer

- **State the OS analogy precisely**: "blocks as pages, tokens as bytes, sequences as processes" — directly from the vLLM paper §3.1. This shows you've read the primary source.
- **Give the indirect-lookup formula**: `k_idx = ((page_id*H_kv + kv_head)*page_size + slot)*D + c`. Know it cold — interviewers love candidates who can trace through the address computation.
- **Mention the kernel equivalence proof**: paged and dense attention produce identical outputs (verified at tol 1e-05). PagedAttention changes *where* bytes live, never *what* they are.
- **Know the ceil vs floor trap**: `rewind` with `floor` at exact page multiples silently drops tokens. Always `ceil(new_offset / page_size)`.
- **Connect to throughput**: the 2–4× throughput gain comes almost entirely from fitting more concurrent sequences in the same GPU VRAM by eliminating wasted reservations.

13 LLM Block Manager (Paged Memory Allocation)

- **Category**: LLM Systems
- **Difficulty**: Hard
- **Target Role**: LLM Inference Engineer / LLM Serving Engineer
- **Source**: vLLM paper (SOSP 2023) / nano-vllm
- **Flashcards**: LLM Systems deck

13.1 Concept Overview

Imagine an operating system's virtual memory manager, but instead of managing virtual-to-physical RAM pages for running processes, it manages token-to-GPU-VRAM blocks for active LLM generation requests. In naive LLM serving engines, a request is pre-allocated a contiguous chunk of memory equal to the

maximum possible generation length (`max_seq_len`). The **LLM Block Manager** solves this waste by breaking the Key-Value (KV) cache into fixed-size physical blocks (or pages) containing a small number of tokens (e.g., 16 tokens). It maps a request's logical sequence of tokens to these physical blocks using a per-sequence block table, allocating new blocks on demand only when page boundaries are crossed.

13.1.1 The Problem It Solves

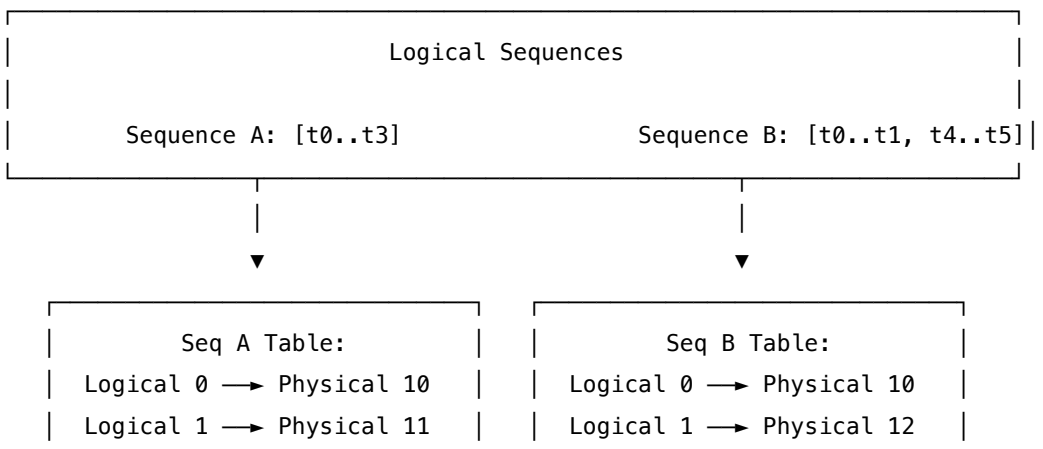
Without a Block Manager, static memory allocation suffers from severe fragmentation and memory waste. Memory is wasted in three ways: 1. **Internal Fragmentation**: Space is reserved for the maximum generation length, but the request finishes early (e.g., reserving 2048 slots but generating only 300). 2. **Reservation Waste**: Space is reserved for future tokens that have not been generated yet. 3. **External Fragmentation**: Memory is allocated contiguously, so varying request lengths leave unusable gaps.

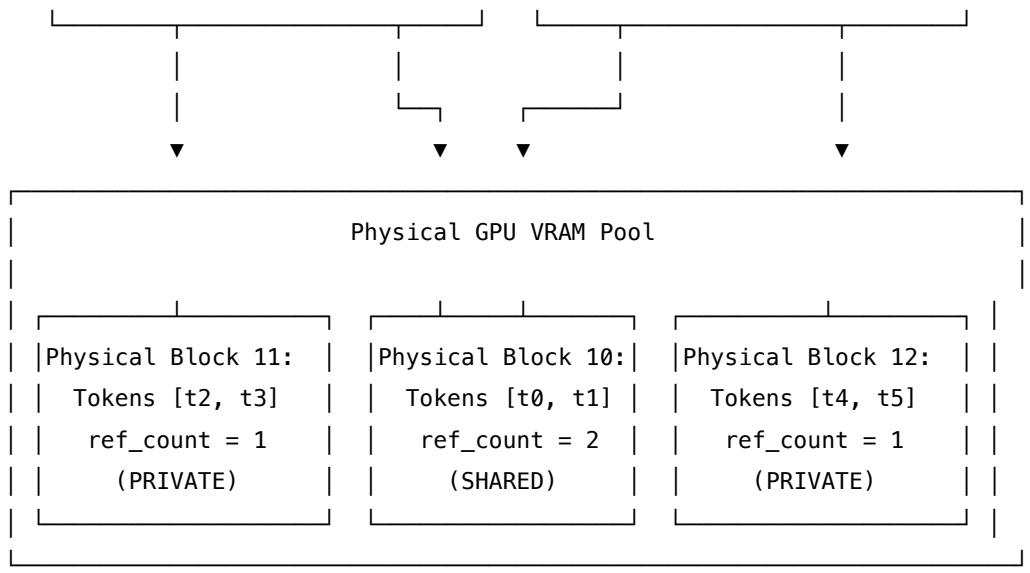
In naive servers, this waste accounts for **60% to 80%** of total GPU memory. This limits the maximum batch size, directly degrading throughput. The Block Manager eliminates external fragmentation and reduces internal fragmentation to less than the size of a single block (under 4% waste for a block size of 16).

13.1.2 How It Works

1. **Physical Block Pool**: The engine pre-allocates a global pool of physical block frames in GPU memory.
2. **Logical Blocks**: The prompt and generated tokens of a sequence are grouped into logical blocks of `block_size` tokens.
3. **Block Table**: A lookup table maps each logical block index `i` of a sequence to a physical block ID in the global pool.
4. **Reference Counting (`ref_count`)**: Each physical block tracks how many active sequences are reading it. When `ref_count > 1`, the block is shared (e.g., a shared system prompt). When `ref_count` drops to `0`, the block is returned to the free list, but its content remains cached (reclaimable) until overwritten.
5. **Chained Hashing**: To identify shared prefixes, each block's content is fingerprinted using a hash function. To guarantee prefix correctness, block k's hash is chained to block $k-1$'s hash:

$$\text{Hash}_k = H(\text{Tokens}_k \parallel \text{Hash}_{k-1})$$





13.2 Worked Example

This example uses verified numbers from the companion code (`block_size = 2` tokens per block, global pool of 6 physical blocks).

13.2.1 1. Prefix Fingerprinting via Chained Hash

Two requests arrive at the serving stack: - **Request A**: `[10, 11, 12, 13]` → Logical Block 0: `[10, 11]`, Logical Block 1: `[12, 13]` - **Request B**: `[10, 11, 14, 15]` → Logical Block 0: `[10, 11]`, Logical Block 1: `[14, 15]`

Both requests share the 2-token prefix `[10, 11]`.

Request	Block Index	Tokens	Prefix (Prev Hash)	Chained Hash	Result
A	0	`[10, 11]`	`-1` (root)	`0xfece5a30e12404d4`	Shared with B
B	0	`[10, 11]`	`-1` (root)	`0xfece5a30e12404d4`	Shared with A
A	1	`[12, 13]`	`0xfece5a30e12404d4`	`0x2cb48a641a5b15d5`	Diverges
B	1	`[14, 15]`	`0xfece5a30e12404d4`	`0x2ab4c06583ce30d5`	Diverges

Gold Fingerprint Value: `hash(A.block0) = hash(B.block0) = 18360711896818255060` (`0xfece5a30e12404d4`). Because the fingerprints for Logical Block 0 match, Request B can reuse physical Block 0 instead of recomputing it.

13.2.2 2. Physical Allocation and Reference Count

When Request A is allocated, the manager pops blocks from the free list. When Request B is allocated, it reuses A's Block 0 (incrementing its reference counter).

GPU Block Pool State (A and B allocated):

Physical Block ID	Reference Count	State	Reader Sequences	Token IDs
0	2	LIVE (Shared)	`['A', 'B']`	`[10, 11]`
1	1	LIVE (Private)	`['A']`	`[12, 13]`
2	1	LIVE (Private)	`['B']`	`[14, 15]`

The shared physical Block 0 has `ref_count = 2`, saving one prefill block computation.

13.2.3 3. Partial Block Registration Pitfall

Suppose Request C is executing: `[10, 11, 12]` with `block_size = 2`. This comprises 2 logical blocks: - Block 0: `[10, 11]` (FULL) - Block 1: `[12]` (PARTIAL — 1 of 2 slots filled)

During post-step processing, the manager maps:

$$\text{start} = \frac{\text{cached\_tokens}}{\text{block\_size}} = \frac{0}{2} = 0$$

$$\text{end} = \frac{\text{cached\_tokens} + \text{scheduled\_tokens}}{\text{block\_size}} = \frac{3}{2} = 1$$

The manager registers blocks in the range `[0..1)`, which means it registers **Block 0 only**. Block 1 is skipped because it is partial. > [!IMPORTANT] > If a partial block was registered, a future request for `[10, 11, 12, 99]` would falsely match the cache on the partial block `[12]`, causing a cache hit on mismatched tokens.

13.2.4 4. Decode Growth Rule

During autoregressive token generation, a request appends tokens one by one. A new physical block is allocated **only** when `len(seq) % block_size == 1`.

Let's trace Request A (`block_size = 2`) starting at length 4 (`block_table = [0, 1]`):

Step	Generated Token	Length	`len % block_size`	New Block Allocated?	`block_table`
1	20	5	$5 \pmod 2 = 1$	**Yes** (opens new page)	`[0, 1, 2]`
2	21	6	$6 \pmod 2 = 0$	No (fills slot 1 of block 2)	`[0, 1, 2]`
3	22	7	$7 \pmod 2 = 1$	**Yes** (opens new page)	`[0, 1, 2, 3]`

13.2.5 5. Preemption and Prefix Reclaim

When the GPU runs out of physical blocks, the scheduler preempts a sequence (e.g., Sequence A) and releases its blocks: - **Block 1** (private to A): `ref_count` drops from $1 \to 0$. It is returned to the free list. - **Block 0** (shared with B): `ref_count` drops from $2 \to 1$. It remains in memory because B is still running.

Crucially, **the hash and KV data of Block 1 remain in memory** even though its `ref_count` is 0. If Request D (`[10, 11, 12, 13, 20, 21]`) arrives, `can_allocate` performs the chained hash walk: - Block 0: hash hit on `0xfece5a30e12404d4` (reuses live Block 0, `ref_count` $1 \rightarrow 2$). - Block 1: hash hit on `0x2cb48a641a5b1565` (reclaims Block 1 from the free list, `ref_count` $0 \rightarrow 1$). Request D skips prefill compute for **4 tokens** (`num_cached_tokens = 4`), saving CPU/GPU execution time.

13.3 Complexity & Trade-offs

Metric	Value / Behavior	Notes
Metadata Overhead	$\mathcal{O}\left(\frac{\text{seq\_len}}{\text{block\_size}}\right)$	Memory needed to store the block tables (very small; e.g., 4 bytes per block mapping).
Lookup Cost	$\mathcal{O}$(num_blocks)	Traversing chained block hashes to find cache hits during admission.
Block Size = 16	Default in vLLM	Balance between memory overhead and sharing granularity.
Block Size = 2	Used in code simulation	Maximizes prefix-sharing opportunities at the cost of larger block tables.

13.3.1 Block Size Trade-offs

- **Smaller Block Size** (e.g., 2 tokens):
  - *Pros*: Captures very short prefix matches. Minimizes internal fragmentation.
  - *Cons*: Explodes the size of the block table. Increases CPU overhead because the attention kernel must fetch many scattered memory pages.
- **Larger Block Size** (e.g., 64 tokens):
  - *Pros*: Reduces block table overhead. Keeps memory layouts more contiguous, improving GPU memory coalescing during reads.
  - *Cons*: Increases internal fragmentation (wasting up to 63 slots on the last block). Mismatches prefix sharing if prompts diverge mid-block.

13.4 Common Interview Questions & How to Answer

13.4.1 Q1: How does a paged memory block manager handle physical memory allocation, and what happens when the GPU runs out of free blocks during generation?

- **Answer**: The block manager allocates fixed-size physical blocks from a pre-allocated memory pool on demand. Sequences do not get contiguous virtual memory; they write to pages scattered across VRAM, mapped via a block table. If a decode step needs a new block and the pool is exhausted (OOM), the block manager returns an OOM signal to the scheduler. The scheduler must then preempt

one or more active sequences, transitioning them back to `WAITING` or `SWAPPED`. The block manager frees the preempted sequence's blocks by decrementing their `ref_count`. Blocks with `ref_count == 0` are placed back on the free list.

13.4.2 Q2: Why must we use chained hashes instead of hashing each block's content individually?

- **Answer**: If we hashed each block's content in isolation (e.g., $H(\text{block\_tokens})$), two completely unrelated requests that happen to contain the same words in a single block would be mapped to the same physical page. This would result in incorrect sharing. Hashing a block chained to the hash of the preceding block ($H(\text{block\_tokens} \parallel \text{prev\_hash})$) ensures that a block's fingerprint represents the **entire sequence of tokens up to that block**. This guarantees cache hits only occur on true prefix matches.

13.5 Pro-Tip: How to Impress the Interviewer

- **Co-Write (CoW) and Beam Search**: Show deep understanding of how parallel sampling/beam search forks are structured. Since forks share the same history blocks, incrementing their `ref_count` lets them read from the same memory. Only when a branch diverges does it trigger a Copy-on-Write to allocate a private physical block, saving massive amounts of GPU memory.
- **Radix Tree Contrast**: Real systems use Radix Trees (like SGLang's RadixAttention) rather than flat hash lists to cache prefixes. This is because flat hash caches are block-aligned; any system prompt that does not align with the block boundary gets cut off, whereas a Radix Tree enables token-granular sharing.

14 Continuous Batching & LLM Scheduler

- **Category**: LLM Systems
- **Difficulty**: Hard
- **Target Role**: LLM Inference Engineer / LLM Serving Engineer
- **Source**: Orca (OSDI 2022) / vLLM paper (SOSP 2023)
- **Flashcards**: LLM Systems deck

14.1 Concept Overview

In standard deep learning serving, requests are batched together at the request level (**static batching**): the server waits for a batch of requests to arrive, runs them all together, and returns the results only when the longest request completes. For LLM generation, which is autoregressive (token-by-token), static batching leads to terrible GPU efficiency because different requests generate different numbers of tokens, leaving GPU cores idle while waiting for the longest request.

The **LLM Scheduler** implements **continuous batching** (also known as iteration-level scheduling), making scheduling decisions after every single model forward pass (one token step). It manages a state machine

that transitions requests from `WAITING` to `RUNNING` and `FINISHED`, dynamically inserting new prompts and evicting sequences on Out-Of-Memory (OOM) conditions.

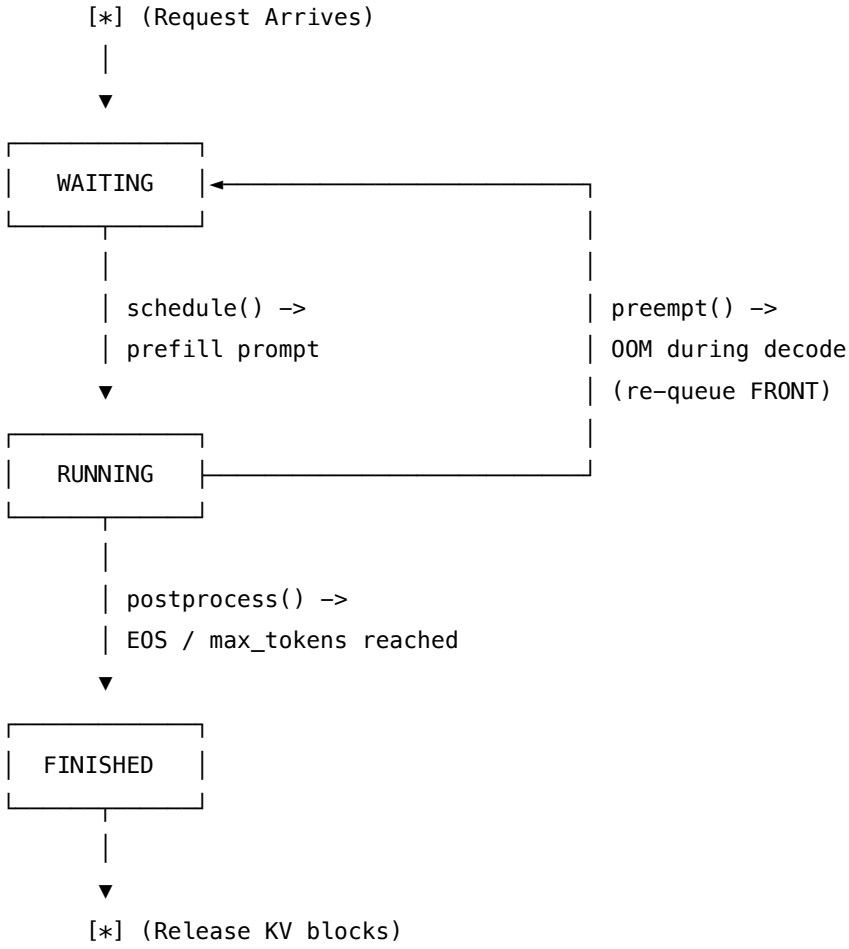
14.1.1 The Problem It Solves

Static batching pads shorter requests in a batch with dummy tokens until the longest request completes. If Request A needs 3 tokens and Request A's neighbor Request B needs 6 tokens, Request A's slot is padded and stays idle for 3 steps, wasting **25% or more** of GPU compute.

Continuous batching solves this by immediately evicting Request A when it finishes, and slotting in a new waiting request (e.g., Request C) at the very next iteration. This raises GPU utilization from ~75% to **92%+** under heterogeneous workloads.

14.1.2 How It Works

The scheduler orchestrates execution by running a cycle: **Schedule** → **Execute (Model Forward)** → **Postprocess**.



- **Prefill Priority**: New prompts in the `WAITING` queue are scheduled before running decodes to minimize Time-to-First-Token (TTFT).
- **Chunked Prefill**: Processing a massive prompt (e.g., 8192 tokens) in a single step would starve all active decodes. The scheduler chunks long prompts over multiple steps using `num_cached_tokens` to

track progress. To prevent multiple prompts from starting and chunking simultaneously, **only the first sequence in the waiting queue is allowed to chunk per step**.

- **Preemption (Recompute Mode)**: If a decode step needs a new block and the VRAM pool is empty, the scheduler evicts the **newest** running sequence back to `WAITING` at the front of the queue, freeing its physical blocks. It will be re-prefilled from its cached token boundary once VRAM is free.

14.2 Worked Example

14.2.1 1. Static vs. Continuous Batching Timeline

Consider two requests: R0 requires 3 tokens, R1 requires 6 tokens. Later, R2 arrives requiring 2 tokens.

Static Batching Timeline (wastes 3 slots; 75% useful): * Steps 1–3: R0 and R1 both execute. * Steps 4–6: R0 is finished. Its slot is padded with idle work. R1 continues executing. * R2 must wait until Step 7 to begin.

Continuous Batching Timeline (wastes 1 slot; 92% useful): * Steps 1–3: R0 and R1 both execute. * Step 4: R0 finishes. **R2 immediately slots into R0's freed seat**. * Steps 5–6: R1 and R2 execute. R2 finishes at Step 5, and R1 finishes at Step 6.

14.2.2 2. Decode Preemption Trace

- **VRAM Pool**: 2 physical blocks.
- **Running Queue**: `[s0, s1]`. Each holds 1 block. VRAM is full (`free = 0`).
- **Decode Step**: Sequence 0 generates a token that crosses a block boundary (length grows $2 \to 3$). It needs a new physical block, but none are free.
- **Preemption Action**: The scheduler pops the newest sequence (`s1`) from the running queue, deallocates its blocks, sets its status to `WAITING` and `is_prefill = True`, and pushes it to the **front** of the waiting queue.
- **Result**: `s1`'s block is freed and immediately allocated to `s0`, allowing `s0` to complete its decode step.

14.2.3 3. Full Multi-Step Schedule Trace

We trace three sequences with prompt lengths `[3, 5, 2]` under a budget of `max_num_batched_tokens = 4`, `block_size = 2`, and `max_tokens = 2`.

Step	Scheduled Sequences (Seq:Tokens)	Total Tokens	Is Prefill?	Notes
1	`seq0: 3`	3	True	`seq0` fits fully in the budget of 4.
2	`seq1: 4`	4	True	`seq1` needs 5 tokens. It is **chunked** (4 scheduled). `seq1` stays WAITING.

Step	Scheduled Sequences (Seq:Tokens)	Total Tokens	Is Prefill?	Notes
3	`seq1: 1`, `seq2: 2`	3	True	`seq1` tail (1 token) + `seq2` full prompt (2 tokens) scheduled.
4	`seq0: 1`, `seq1: 1`, `seq2: 1`	3	False	**Decode Step**: All three generate 1 token, hit `max_tokens=2`, and finish.

Sequence 1 `num_cached_tokens` growth: `0 -> 4 -> 5 -> 6`. It caches 4 prompt tokens in Step 2, completes the last prompt token in Step 3, and decodes in Step 4.

Why Seq 0 Idled in Steps 2–3: Because new prefills (`seq1` and `seq2`) have priority over active decodes to keep TTFT low, `seq0` must wait for the prefill queue to clear before it is allowed to decode.

14.3 Complexity & Trade-offs

Metric	Value / Behavior	Notes
Scheduling Overhead	$\mathcal{O}$(active_sequences)	Executed on the CPU every step. Must be kept < 1 ms to avoid stalling the GPU.
Budget Limit	`max_num_batched_tokens`	Limits the number of tokens processed in a single forward pass to prevent GPU out-of-memory.
Preemption Recovery	**Recomputation** (Default)	Discards KV cache and re-prefills. Wastes GPU compute but requires no PCIe transfer overhead.
Preemption Recovery	**Swapping**	Swaps KV cache to CPU RAM. Saves compute but introduces high PCIe copy latency.

14.3.1 Prefill vs. Decode Co-batching Trade-offs

- **Separate Steps** (nano-vllm default):
  - *Pros*: Simple to batch. Keeps CUDA shapes uniform (either all prefill matrix-multiplication kernels or all decode vector-matrix kernels).
  - *Cons*: Prefilled sequences must idle while waiting for other sequences' prefill chunks to complete.

- **Mixed Co-batching** (production vLLM):
  - *Pros*: High GPU efficiency by overlapping compute-bound prefills with memory-bound decodes in the same forward pass.
  - *Cons*: Requires complex attention kernels (e.g., FlashDecoding) that can handle heterogeneous query lengths and block offsets.

14.4 Common Interview Questions & How to Answer

14.4.1 Q1: What is continuous batching (iteration-level scheduling) and how does it improve serving throughput compared to static batching?

- **Answer**: Static batching treats a batch as a single unit, running all sequences to completion and padding finished slots with dummy work. Continuous batching schedules at the iteration level (one forward pass/token generation). It inspects the batch after every step, allowing completed sequences to be released and new sequences to be added immediately. This eliminates padding waste and significantly increases GPU occupancy under varying sequence lengths.

14.4.2 Q2: How does the scheduler handle preemption when VRAM is exhausted, and how does it determine which sequence to preempt?

- **Answer**: When a decode step requires a new KV block but the memory manager has none free, the scheduler must preempt one or more sequences. It preempts sequences in reverse-chronological order, evicting the **newest running sequence** first. This sequence's physical blocks are deallocated, its state is set back to `WAITING` with `is_prefill = True`, and it is re-queued at the **front** of the waiting queue. The newest sequence is chosen because it has completed the least amount of generation work, minimizing the wasted GPU compute of recomputation. Under First-Come-First-Serve (FCFS) scheduling, evicting the newest sequence approximates a Least Recently Used (LRU) policy.

14.4.3 Q3: Why does chunked prefill restrict chunking to only the first waiting sequence per step?

- **Answer**: If multiple waiting sequences were allowed to chunk simultaneously, each would consume a portion of the step's token budget, causing a ”parade” of partial prefill chunks that would continuously starve active decode sequences. By restricting chunking to only the first sequence in the waiting queue, the scheduler ensures at most one sequence runs in a partial chunked prefill state. Once it completes its prefill and transitions to `RUNNING`, the next sequence in line can begin chunking. This protects active decodes from long-term latency starvation.

14.5 Pro-Tip: How to Impress the Interviewer

- **TTFT vs. ITL Trade-off**: Explain that while prefill priority minimizes Time-To-First-Token (TTFT), it can cause Inter-Token Latency (ITL) spikes for already-running decodes. In production, serving systems use **chunked prefill co-batching** to limit the maximum number of prefill tokens

per step. By mixing small prefill chunks with decodes, they bound the step latency, keeping ITL smooth while maintaining reasonable TTFT.

15 RadixAttention (Prefix Caching with a Radix Tree)

- **Category**: LLM Systems
- **Difficulty**: Expert
- **Target Role**: LLM Inference Engineer / LLM Serving Engineer
- **Source**: SGLang paper (Zheng et al., 2023) / RadixAttention
- **Flashcards**: LLM Systems deck

15.1 Concept Overview

In LLM serving, workflows like multi-turn chat, few-shot prompting, and agent loops reuse large portions of prompts across steps. A **prefix cache** stores the Key-Value (KV) cache of these prompts in GPU VRAM so that subsequent requests sharing the same prefix can skip prefill computation.

While basic engines use a flat hash table that requires block alignment (e.g., vLLM's BlockManager), **RadixAttention** manages VRAM using a **radix tree** (a compressed trie) on the CPU. The edges of the tree store token segments of arbitrary lengths, and the nodes point to their corresponding KV caches. This allows for token-granular, tree-structured sharing that matches the exact branching structure of conversational prompts.

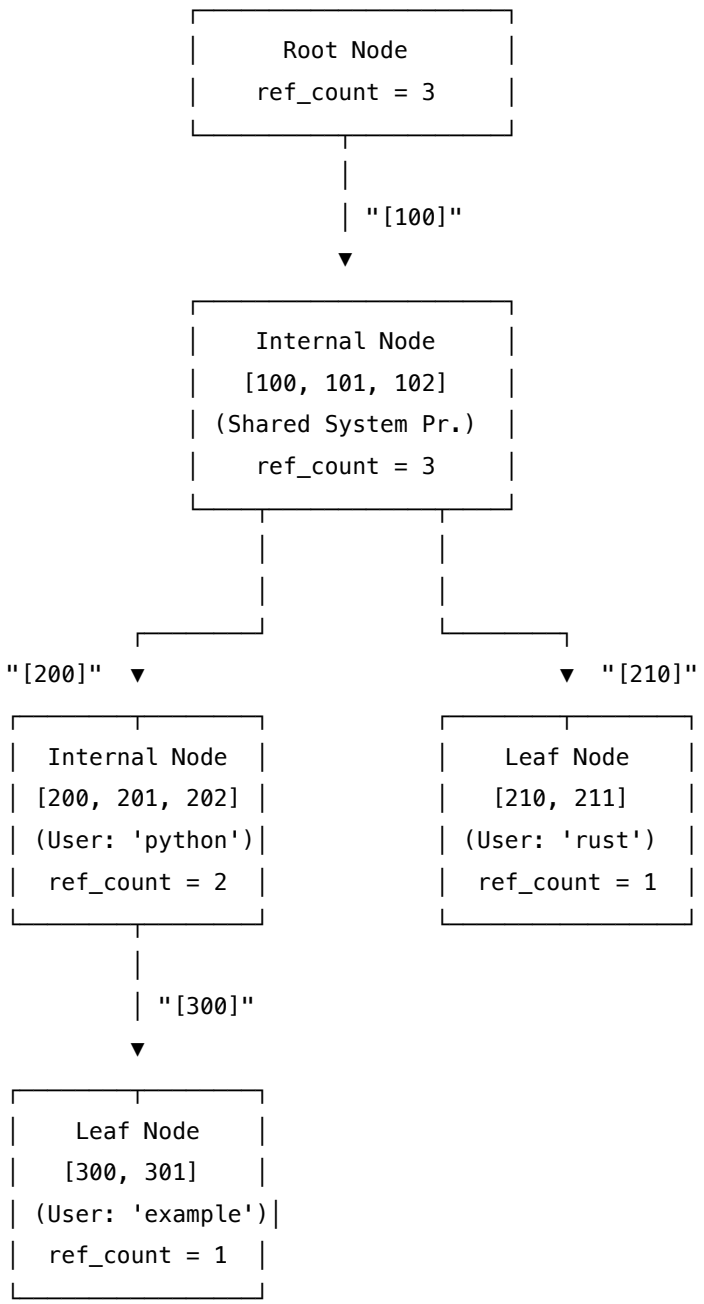
15.1.1 The Problem It Solves

Chained-hash block managers (like vLLM's basic cache) group tokens into fixed-size blocks (e.g., `block_size = 16`). Because the cache is block-aligned, it can only reuse **whole blocks**. If a shared system prompt is 25 tokens long: * The first block (16 tokens) is successfully cached and shared. * The remaining 9 tokens are packed into the second block alongside query-specific tokens. Since the rest of the block differs for every user query, the second block's hash is unique to each request. Consequently, those 9 shared tokens must be recomputed for every request. RadixAttention solves this by allowing token-granular sharing, saving **9 system tokens** (or up to **5× throughput** under multi-turn agent workloads).

15.1.2 How It Works

1. **Radix Tree Index**: The CPU maintains a radix tree where the path from the root to a node represents a sequence of tokens.
2. **Token-Granular Allocation**: The GPU page pool allocates memory at the level of a single token (`page_size = 1`), matching the tree's granularity.
3. **Longest Prefix Match (LPM)**: When a new prompt arrives, the engine walks the tree to find the longest matching path, returning the matched token length and the target node.
4. **Insert & Split**: The prompt is inserted. If the match stops mid-edge, the edge is **split**: the matched prefix becomes a new internal node, and the old tail and new query suffix become child branches.

5. **Reference Counting & LRU Eviction**: Nodes track active generation readers (`ref_count`). When a request finishes, `ref_count` is decremented. When memory is full, the engine evicts the **least-recently-used (LRU) leaf nodes** with `ref_count == 0`, recursively pruning upwards.



15.2 Worked Example

This example uses verified numbers from the companion code (`block_size = 2` tokens per block for the flat-hash comparison).

15.2.1 1. The Flat-Hash Blind Spot

We insert `Q1 = [100, 101, 102, 200, 201, 202]` into a flat hash cache. The cache registers 3 full blocks: * Block 0 `[100, 101]`: hash = `0xecd0403d962ff8f4` * Block 1 `[102, 200]`: hash = `0x75ad5e3ce0e74fe3` * Block 2 `[201, 202]`: hash = `0x1a04813492aee2f7`

Next, we probe `Q2 = [100, 101, 102, 210, 211]`, which shares the 3-token system prefix `[100, 101, 102]`.

- **Block 0** `[100, 101]`: Hash hit! (Reuses Block 0)
- **Block 1** `[102, 210]`: Hash is `0x35e2a5e982cdf239` → **MISS** (because Q1's Block 1 was `[102, 200]`). The flat hash only matches **2 tokens**. The 3rd system token (`102`) is lost to block misalignment.

15.2.2 2. Radix Tree Construction (Q1, Q2, Q3)

- **Insert Q1** (`[100, 101, 102, 200, 201, 202]`): Cold miss. The tree creates a single leaf edge:

```
root {ref=1}
  └─ [100] [100, 101, 102, 200, 201, 202] {ref=1, leaf}
```

- **Insert Q2** (`[100, 101, 102, 210, 211]`): LPM matches 3 tokens (`[100, 101, 102]`). The edge is **split** at token 3.

```
root {ref=2}
  └─ [100] [100, 101, 102] {ref=2}
       ├─ [200] [200, 201, 202] {ref=1, leaf}
       └─ [210] [210, 211] {ref=1, leaf}
```

  Q2 matches 3 tokens, skipping prefill for the entire system prompt.

- **Insert Q3** (`[100, 101, 102, 200, 201, 202, 300, 301]`): LPM matches 6 tokens (`[100, 101, 102, 200, 201, 202]`). The Q1 leaf is promoted to a shared internal node, and Q3's suffix is attached:

```
root {ref=3}
  └─ [100] [100, 101, 102] {ref=3}
       ├─ [200] [200, 201, 202] {ref=2}
       │    └─ [300] [300, 301] {ref=1, leaf}
       └─ [210] [210, 211] {ref=1, leaf}
```

  Q3 matches 6 tokens, skipping prefill for the prompt and user question.

15.2.3 3. Cumulative Savings

For a workload of Q1, Q2, and Q3:

Query	Prompt Tokens	Radix Hit	Flat-Hash Hit	Radix Advantage
Q1	`[100, 101, 102, 200, 201, 202]`	0	0	0
Q2	`[100, 101, 102, 210, 211]`	**3**	2	**+1 token**

Query	Prompt Tokens	Radix Hit	Flat-Hash Hit	Radix Advantage
Q3	`[100, 101, 102, 200, 201, 202, 300, 301]`	6	6	0 (boundary match)
Total	**19 tokens**	**9 saved**	**8 saved**	**+1 token**

15.2.4 4. LRU Leaf Eviction Trace

- **Step 1: Release Q2**: The request finishes. The ref counts along Q2's path are decremented.

```
root {ref=2}
  └─ [100] [100, 101, 102] {ref=2}
       ├─ [200] [200, 201, 202] {ref=2}
       │    └─ [300] [300, 301] {ref=1, leaf}
       └─ [210] [210, 211] {ref=0, leaf}  <-- EVICTABLE
```

- **Step 2: Eviction**: Memory limits trigger `evict_lru_leaves(1)`. The engine identifies leaf `[210, 211]` with `ref_count == 0` and removes it, releasing its GPU memory.
- **Step 3: Upward Pruning**: The walk checks the parent node `[100, 101, 102]`. Since it still has the child branch `[200,...]`, the parent cannot be pruned.
- **Step 4: Future Query Match**: A new query `Q4 = [100, 101, 102, 210, 999]` arrives. Since `[210, 211]` was evicted, `Q4` matches only **3 tokens** (the system prompt) instead of 4.

15.3 Complexity & Trade-offs

Metric	Complexity / Value	Notes
Match Latency	$O(L)$	Walk from the root down to the first divergence; sub-linear in practice by using child dicts.
Insert Latency	$O(L)$	Matches and inserts the remaining suffix, splitting edges in $O(1)$ pointer swaps.
Page Allocator	`page_size = 1`	Eliminates fragmentation but increases block allocation metadata frequency.
Scheduling Policy	**Cache-Aware**	Schedulers sort the waiting queue to group requests sharing the same prefix together.

15.4 Common Interview Questions & How to Answer

15.4.1 Q1: How does RadixAttention eliminate the block-alignment limitation found in flat chained-hash caches?

- **Answer**: In a flat-hash cache, the dedup unit is a fixed-size block (e.g., 16 tokens). If a shared prefix diverges mid-block, the entire block is marked unique to that request, preventing sharing of the matched tokens within that block. RadixAttention represents prefixes as paths in a radix tree. It performs a Longest Prefix Match (LPM) at the token level, matching the exact length of the shared prefix. When a divergence occurs mid-edge, it splits the edge at the exact token index, creating a shared parent node. Combined with a page allocator that supports single-token page sizes, it achieves token-granular sharing.

15.4.2 Q2: Why does RadixAttention evict only leaves, and how does the recursive eviction process work?

- **Answer**: Internal nodes represent shared prefix backbones that other active child sequences still read. Evicting an internal node would corrupt the lookup path for its descendants. Therefore, eviction is restricted to leaf nodes (the tips of the tree) with a `ref_count == 0`. The eviction manager selects the coldest eligible leaf node using a Least Recently Used (LRU) policy and releases its KV cache. It then walks up to the parent node. If the parent has no other children and has `ref_count == 0`, the parent is also pruned. This process continues recursively until it hits a node that is either still shared (`ref_count > 0`) or has other active child branches.

15.5 Pro-Tip: How to Impress the Interviewer

- **Cache-Aware Scheduling**: Explain that prefix caching is only half the battle. To maximize VRAM hits, the scheduler should use **cache-aware scheduling**: sorting incoming requests in the waiting queue by their matched prefix lengths. By scheduling requests that share the same prefix in the same batch, the engine keeps the shared prefix hot in GPU memory, avoiding eviction thrashing.

16 CUDA Graphs for LLM Decode Acceleration

- **Category**: LLM Systems
- **Difficulty**: Expert
- **Target Role**: LLM Inference Engineer / LLM Serving Engineer
- **Source**: NVIDIA CUDA Programming Guide / vLLM paper
- **Flashcards**: LLM Systems deck

16.1 Concept Overview

LLM autoregressive generation (the decode phase) is memory-bandwidth bound. Every step, the GPU runs the **same** sequence of kernels (RMSNorm, projections, attention, MLP) at the **same** shapes, reading and writing to the KV cache one token at a time. In eager PyTorch, every kernel is launched from the

Python interpreter on the host CPU one by one. Each launch costs $\sim$ 5–14 μs of CPU-to-GPU driver overhead.

A **CUDA Graph** addresses this overhead by capturing the entire kernel sequence once as a static Directed Acyclic Graph (DAG). Instead of launching 70+ kernels individually, the engine replays the entire sequence with a **single** host call, collapsing host launch latency and maximizing GPU occupancy.

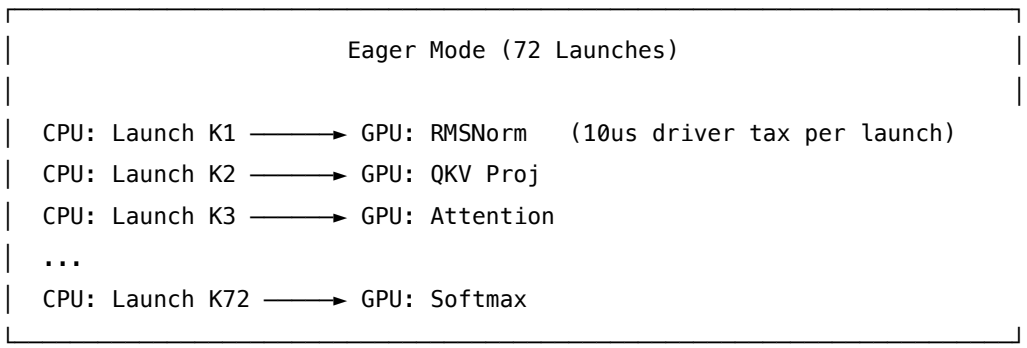
16.1.1 The Problem It Solves

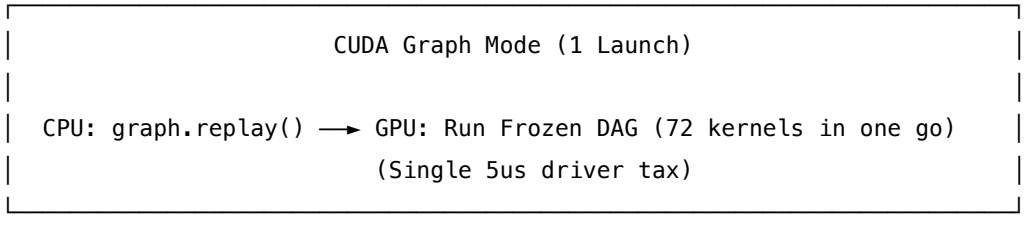
During the decode step, a typical LLM executes $\sim$ 50–100 individual GPU kernels. At an average of 10 μs of launch overhead per kernel, this creates 720 **μs of pure CPU host-side overhead** per step.

For small models or small batch sizes, this host latency accounts for **over 50% of the entire decode step time**. CUDA Graphs eliminate this overhead by packaging all kernels into a single unit, reducing launch tax to a single graph launch of $\sim$ 5 μs.

16.1.2 How It Works

1. **Warmup Run**: The engine runs the model once with dummy data. This primes PyTorch's lazy allocator pools, cuBLAS handles, and cuDNN algorithm selections.
2. **Graph Capture**: The engine runs the model inside a capture context manager (`torch.cuda.graph(g)`). During this run, kernels are appended to the graph DAG rather than executed on the GPU.
3. **Graph Storage**: Graphs are captured for a discrete set of batch sizes (e.g., `graph_bs = [1, 2, 4, 8, 16, 32]`).
4. **Replay**: At runtime, the engine rounds the active batch size up to the nearest captured bucket, writes input data in-place to pre-allocated tensors, and invokes `graph.replay()`.
5. **Sentinel Padding**: Unused slots in the rounded batch size are padded with a `-1` sentinel. The attention kernel reads this sentinel and skips VRAM writes for those slots, preventing memory corruption.





16.2 Worked Example

This example uses verified numbers from the simulation code for a model with 24 layers and 3 kernels per layer ($N_{\text{kernels}} = 72$).

16.2.1 1. Eager vs. Graphed Latency (Batch = 1)

- **Kernel Launch Overhead**: 10 μs per kernel.
- **Kernel Compute Time**: 8 μs per kernel.
- **Graphed Launch Overhead**: 5 μs total.

$$\text{Eager Step Total} = (72 \times 10\ \mu\text{s}) + (72 \times 8\ \mu\text{s}) = 720\ \mu\text{s} + 576\ \mu\text{s} = 1296\ \mu\text{s}$$

$$\text{Graphed Step Total} = 5\ \mu\text{s} + 576\ \mu\text{s} = 581\ \mu\text{s}$$

- **Launch-Overhead Speedup**: $\frac{720\ \mu\text{s}}{5\ \mu\text{s}} = \mathbf{144}\times$
- **Total Step Speedup**: $\frac{1296\ \mu\text{s}}{5\ \mu\text{s}+576\ \mu\text{s}} = \mathbf{2.23}\times$

16.2.2 2. Batch-Size Scaling

Because launch overhead is fixed while compute scales linearly with batch size, CUDA Graphs help most at small batch sizes:

Batch Size	Compute Time (μs)	Eager Launch Tax (μs)	Launch Fraction (Eager)	Total Speedup
1	576	720	55.6%	**2.231×**
2	1152	720	38.5%	1.618×
4	2304	720	23.8%	1.310×
8	4608	720	13.5%	1.155×
16	9216	720	7.2%	1.078×
32	18432	720	3.8%	1.039×
64	36864	720	1.9%	1.019×

*At batch size 64, compute dominates, making the launch tax negligible (*1.9%*).*

16.2.3 3. Replay and Padding Sentinels

Active batch size is 3. The engine selects the smallest captured graph size ≥ 3, which is 4. The graph runs with 4 slots, meaning slot 3 is padding. To prevent writing garbage data into VRAM, the engine pads inputs and uses a sentinel: * **Real KV Cache Slot Mapping**: `[4, 17, 9]` * **Graph Slot Mapping Input**: `[4, 17, 9, -1]`

GPU Kernel Execution Behavior:

Slot Index	`slot_mapping` value	Target Token	GPU Attention Kernel Action
0	4	Token 0	Write KV to Physical Slot 4
1	17	Token 1	Write KV to Physical Slot 17
2	9	Token 2	Write KV to Physical Slot 9
3	**-1**	Padding	**SKIP (sentinel detected)**

[!CAUTION] If slot 3 was not padded with the `-1` sentinel, the GPU would write garbage KV data to a random memory address, silently corrupting the cache of other active requests.

16.3 Complexity & Trade-offs

Metric	Eager	Graphed	Notes
Launch Latency	$\mathcal{O}(N_{\text{kernels}})$	$\mathcal{O}(1)$	Collapses $N \times 10$ μs down to a single ~ 5 μs call.
VRAM Footprint	Dynamic	Static + Pre-allocated	Input, output, and intermediate activation tensors must be pre-allocated for every graph batch size.
Warmup Cost	0	High (seconds)	Compiles kernels and instantiates memory pools during engine initialization.
Control Flow	Dynamic	Static (no CPU branching)	No CPU-side conditional branching is allowed inside the graph.

16.4 Common Interview Questions & How to Answer

16.4.1 Q1: Why are CUDA Graphs used for the decode phase of LLM serving but not the prefill phase?

- **Answer**: CUDA Graphs require strictly fixed tensor shapes, sizes, and memory strides because memory offsets are frozen during capture. In the prefill phase, prompt lengths vary across requests

(e.g., one request has 10 tokens, another has 500). This causes dynamic tensor shapes, preventing graph capture. In contrast, the decode phase processes exactly 1 token per sequence per step. The tensor shapes depend only on the batch size, which is bounded and predictable. We can pre-capture graphs for common batch sizes (T-shirt sizing) and run the decode loop using CUDA Graphs.

16.4.2 Q2: Explain the requirement of a "warmup run" before capturing a CUDA Graph and what happens if you skip it.

- **Answer**: PyTorch, cuBLAS, and cuDNN use lazy initialization: they allocate internal workspace pools, choose algorithms, and JIT-compile PTX kernels on their first execution. If we capture a graph without a warmup run, these initialization operations are recorded inside the graph. On replay, the graph will crash or run into invalid pointer states because these host-side setups cannot be repeated in a device-side replay. A warmup run executes the model once with dummy data to force initialization before capture begins.

16.5 Pro-Tip: How to Impress the Interviewer

- **In-place Copy Contract**: Explain that because the input tensor addresses are frozen during graph capture, you cannot swap input tensor objects at runtime (e.g., `x = fresh_tensor`). Instead, you must copy data **in-place** to pre-allocated tensors mapped to the graph (e.g., `graph_input.copy_(runtime_input)`).
- **Scale Impact in Large Models**: Acknowledge that while CUDA Graphs achieve 144× launch speedups, the total decode step speedup for production models (like Llama-3-70B) is closer to 10–20% because weight-reading compute dominates. However, this is a critical win for interactive latency and TTFT.

17 Parameter-Efficient Fine-Tuning: LoRA & QLoRA

- **Category**: LLM Systems
- **Difficulty**: Hard
- **Target Role**: LLM Systems Engineer / ML Platform Engineer
- **Source**: PEFT_LORA.md + peft_lora_output.txt (Hu et al. 2021, Dettmers et al. 2023, Chen et al. 2023, Sheng et al. 2023)
- **Flashcards**: LLM Systems deck

17.1 Concept Overview

Think of pre-trained LLM weights as a massive marble statue, and fine-tuning as carving modifications into it. Instead of chipping away at the original marble (which would require making a separate copy of the heavy statue for every single task), LoRA bolts a small, adjustable wireframe skeleton onto the outside. This skeleton takes the input signals, processes them through a pair of lightweight, low-rank matrices, and adds the adjustment back to the statue's output.

Specifically, LoRA freezes the pre-trained weight matrix $W_0 \in \mathbb{R}^{d\times k}$ and approximates the task-specific update matrix $\Delta W \in \mathbb{R}^{d\times k}$ as the product of two low-rank matrices $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$, where the rank $r \ll \min(d, k)$. By training only A and B, we reduce the number of trainable parameters by orders of magnitude (e.g., $10,000\times$ fewer parameters for GPT-3). QLoRA goes a step further by compressing the frozen base weights to a specialized 4-bit NormalFloat (NF4) representation to slash the base model VRAM footprint, while maintaining fp16 adapters. In multi-adapter serving, we can run multiple independent task-specific adapters concurrently on a single shared base model, routing different tokens in a batch to different adapters via grouped GEMMs.

17.1.1 The Problem It Solves

Full fine-tuning requires updating and saving all $d \times k$ weights for each task. For a model like GPT-3 175B, each fine-tuned variant requires saving a separate 350 GB checkpoint (in FP16). Serving $N = 100$ independent personalized versions would require storing and hosting ~ 35 TB of weights, which is financially and operationally prohibitive. At a micro-level, for a single layer of Qwen3-0.5B with hidden dimension $d = k = 896$, full fine-tuning must track and store $d \cdot k = 802,816$ trainable parameters for just one layer of one task.

Additionally, training memory footprint is dominated by optimizer states (momentum and variance) in Adam. For each trainable parameter, we store 16 bytes of optimizer states (4 bytes master parameter, 4 bytes grad copy, 4 bytes momentum, 4 bytes variance). Full parameter updates thus consume massive memory. LoRA limits training to only the low-rank path $r(d + k)$, which represents $\sim 0.1\% - 1\%$ of the base model weights, dropping optimizer memory overhead to negligible levels.

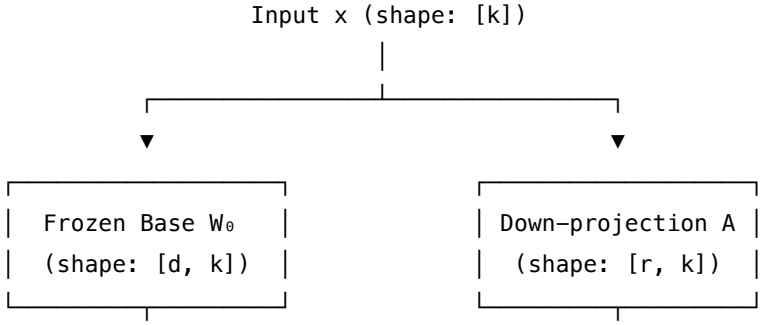
17.1.2 How It Works

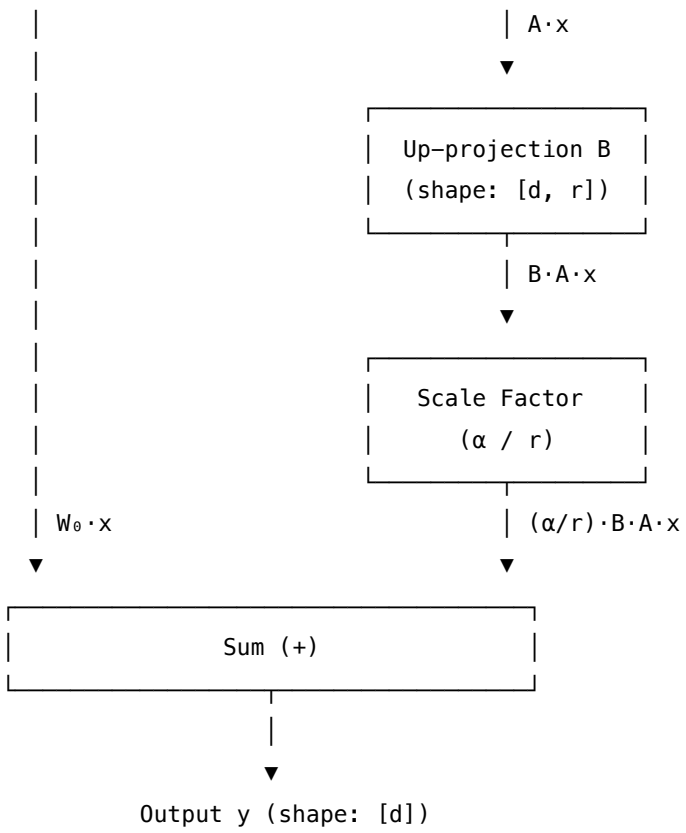
LoRA structures the forward pass of a linear projection layer as:

$$y = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} B(Ax)$$

Where: - $W_0 \in \mathbb{R}^{d\times k}$ is the frozen pre-trained weight matrix. - $x \in \mathbb{R}^k$ is the input vector. - $A \in \mathbb{R}^{r\times k}$ is the down-projection matrix, initialized using a random Gaussian distribution: $A \sim \mathcal{N}(0, \sigma^2)$. - $B \in \mathbb{R}^{d\times r}$ is the up-projection matrix, initialized to all zeros ($B = 0$). - α is a constant scaling hyperparameter. - r is the inner rank.

Since $B = 0$ at the start of training, the update $\Delta W = \frac{\alpha}{r} BA = 0$, meaning the adapter starts as a pure no-op, preserving base model behavior exactly.





During inference, if only a single adapter is deployed, we can pre-compute $W_{\text{merged}} = W_0 + \frac{\alpha}{r}BA$ and run a normal linear layer ($y = W_{\text{merged}}x$), introducing **zero** additional inference latency.

For multi-adapter serving, we avoid merging and instead run: 1. A shared, frozen base forward pass (W_0x) once for all tokens in a batch. 2. A specialized grouped GEMM kernel (such as Punica's SGMV/BGMV) to batch the low-rank paths for different tokens that target different adapters in a single GPU pass. 3. S-LoRA style paged adapter memory management, mapping non-contiguous blocks of adapter weights to GPU memory dynamically (analogous to PagedAttention for KV cache).

17.2 Worked Example

17.2.1 1. Zero at Initialization (Step 0)

Using $d = 8, k = 8, r = 2, \alpha = 16$ (scaling factor $\alpha/r = 8.0$), we initialize A randomly and set $B = 0$. Given input:

$$x = [0.5, -0.3, 0.8, -0.1, 0.4, -0.6, 0.2, 0.7]$$

From `peft_lora_output.txt`:

```
y_init = [0.2, -0.205, 0.96, -0.45, 0.5, 0.045, 0.085, 0.27]
W₀·x   = [0.2, -0.205, 0.96, -0.45, 0.5, 0.045, 0.085, 0.27]
```

Since $B = 0$, $y_{\text{init}} = W_0x$ exactly.

17.2.2 2. After Training (Step 1)

After training, B is updated with learned values. The forward pass outputs: - **Base Path (W_0x):** `[0.2, -0.205, 0.96, -0.45, 0.5, 0.045, 0.085, 0.27]` - **Adapter Path ($\frac{\alpha}{r}B(Ax)$):** `[-0.0192, 0.0446, 0.0456, 0.0643, -0.0221, 0.0765, 0.0045, 0.0379]` - **Total Output ($y = W_0x + \frac{\alpha}{r}B(Ax)$):** `[0.1808, -0.1604, 1.0056, -0.3857, 0.4779, 0.1215, 0.0895, 0.3079]`

The gold validation pin is $y[0] = 0.180800$.

17.2.3 3. Parameter Savings Across Ranks (Layer $d = k = 8$)

Full fine-tuning requires $d \cdot k = 64$ parameters.

Rank (r)	LoRA Params $r(d+k)$	Ratio $r(d+k)/dk$	Savings
1	16	0.2500	4.00×
2	32	0.5000	2.00×
4	64	1.0000	1.00× (Break-even)
8	128	2.0000	0.50× (LoRA is larger)

Note: For a real-world attention layer with $d = k = 12288$ and $r = 4$, full fine-tuning requires $150,994,944$ parameters, whereas LoRA requires only $98,304$ parameters (a $1536\times$ reduction).

17.2.4 4. QLoRA: NF4 Base + FP16 Adapters

For a Qwen3-0.5B layer ($d = k = 896$), the memory footprints are: - **FP16 Base:** $802,816 \times 2 = 1,605,632$ bytes ≈ 1.606 MB - **NF4 4-bit Base:** $802,816/2 = 401,408$ bytes ≈ 0.401 MB ($4.0\times$ smaller)

The NF4 codebook values normalized to $[-1, 1]$:

```
[-1.0, -0.6962, -0.5251, -0.3949, -0.2844, -0.1848, -0.0911, 0.0,
  0.0796,  0.1609,  0.2461,  0.3379,  0.4407,  0.5626,  0.7230,  1.0]
```

Comparing outputs for QLoRA forward pass: - **FP16 Base (Exact):** `[0.1808, -0.1604, 1.0056, -0.3857, 0.4779, 0.1215, 0.0895, 0.3079]` - **QLoRA (NF4 quantized base):** `[0.1572, -0.1323, 1.0179, -0.3589, 0.4801, 0.1080, 0.0769, 0.2974]` - **Max Absolute Difference:** `0.0281` (arising purely from base rounding error; the adapter path is exact).

17.2.5 5. Multi-Adapter Serving Math

To serve $N = 1000$ personalized tasks with $d = k = 12288$ (GPT-3 attention layer) and $r = 64$:

Strategy	Weights in VRAM	Notes
N **Full Replicas (Full FT)**	$1000 \times 150,994,944 = 151.0$ B weights	Unscalable
1 FP16 Base + N **LoRA**	$150,994,944 + 1000 \times 1,572,864 = 1.72$ B weights	$87.8\times$ savings

Strategy	Weights in VRAM	Notes
1 NF4 Base + N QLoRA	$37,748,736$ (NF4) $+ 1000 \times$ $1,572,864 = 1.61$ B weights	$93.8\times$ savings

---

17.3 Complexity & Trade-offs

Metric	Value	Notes
Trainable Parameter Count	$O(r(d+k))$	Reduction by $1000\times$ to $10,000\times$ compared to full fine-tuning.
Memory Footprint (Training)	Base Model + $O(r(d+k))$	QLoRA allows base model to be quantized to 4-bit (NF4), reducing base footprint by $4\times$.
Serving Overhead per Task	$r(d+k)$ weights	$\sim 1.04\%$ of a base layer per task for $r = 64$, $d = k = 12288$.
Inference Latency	0 (if merged) | Grouped GEMM Overhead (if serving multi-adapters)	Merge is only possible if serving a single task. Multi-adapter serving requires running $W_0 x$ once + token-specific $B(Ax)$ paths.
Batching Efficiency	Reduced if naive loops are used	Naive loops process adapters sequentially. Punica grouped GEMM or S-LoRA paged adapter memory restores batched GPU execution.

17.4 Common Interview Questions & How to Answer

17.4.1 Q1: Why do we initialize B to zero and A to a random Gaussian in LoRA? What happens if both are initialized to zero, or if both are initialized randomly?

- **Answer**: We initialize $A \sim \mathcal{N}(0, \sigma^2)$ and $B = 0$ so that the initial update matrix is $\Delta W = \frac{\alpha}{r} BA = 0$. This ensures that at step 0 of training, the model behaves exactly like the pre-trained base model.
  - If both A and B were initialized to zero, the gradients for both matrices would be zero during backpropagation, preventing the model from learning anything (symmetry break failure).
  - If both were initialized to non-zero random values, the initial output would be distorted by the untrained adapter path ($\Delta W \neq 0$), causing optimization instability at the start of training and degrading the base model's zero-shot capabilities.

17.4.2 Q2: How does QLoRA achieve 4-bit training without losing accuracy compared to 16-bit? Explain the role of NF4 and Double Quantization.

- **Answer**: QLoRA achieves this via three core techniques:
  1. **NF4 (NormalFloat4) Data Type**: It is an information-theoretically optimal quantization scheme for normally distributed data. Since neural network weights typically follow a zero-mean normal distribution, NF4 spaces its 16 quantization levels on the quantiles of a normal distribution. This packs levels more densely around zero where most weights reside, reducing quantization error.
  2. **Double Quantization (DQ)**: It quantizes the quantization constants (scales) themselves. For instance, quantizing block-level scales from 32-bit floats to 8-bit floats reduces memory overhead from 32 bits per block of 64 weights to 8 bits, saving roughly 0.37 bits/parameter ($\sim$ 3 GB for a 65B model).
  3. **FP16/BF16 Computation**: During forward and backward passes, the NF4 weights are dequantized on-the-fly to 16-bit floats. Gradients are computed and backpropagated only through the fp16/bf16 adapter weights (A and B). The base model remains frozen in 4-bit storage.

17.4.3 Q3: When serving thousands of adapters on a single GPU in multi-tenant serving, how do Punica and S-LoRA optimize memory and compute?

- **Answer**: Multi-tenant serving faces two primary issues: execution serialization (GPU starvation) and VRAM capacity limitations.
  - **Punica (Grouped GEMM)**: A naive batching approach runs sequential matrix multiplications for different adapters in a batch, which wastes GPU parallelism. Punica utilizes a specialized grouped GEMM kernel (such as SGMV/BGMV) that performs all adapter matmuls for their respective token slices in a single batched GPU kernel execution (modeled as `einsum("tdr,tr->td")`).
  - **S-LoRA (Paged Adapter Memory)**: Since loading all adapters into VRAM causes out-of-memory (OOM) errors, S-LoRA implements a unified page-based memory manager for adapters, mimicking PagedAttention. It shards adapter weights into non-contiguous physical pages in GPU memory. Only the adapters needed for the current active batch are swapped in, allowing thousands of adapters to be served concurrently with sub-millisecond swap times.

17.5 Pro-Tip: How to Impress the Interviewer

- **Demonstrate deep hardware intuition by discussing the "Grouped GEMM" kernel mechanics.** Explain that when batching requests across different adapters, the batch cannot be processed by a single standard GEMM since different rows need different weight matrices (A_i, B_i). Mention **Punica's SGMV** (Segmented Gather Matrix-Vector multiplication) which enables a single kernel to compute $Y_i = B_i(A_i X_i)$ for variable-sized sequences without padding, avoiding CPU-GPU launch overheads and fully saturating GPU tensor cores.

18 Gradient Checkpointing (Activation Recomputation)

- **Category**: LLM Systems
- **Difficulty**: Hard
- **Target Role**: LLM Systems Engineer / ML Platform Engineer
- **Source**: GRADIENT_CHECKPOINTING.md + gradient_checkpointing_output.txt (Chen et al. 2016, Gruslys et al. 2016, Korthikanti et al. 2022)
- **Flashcards**: LLM Systems deck

18.1 Concept Overview

Think of training a deep neural network like embarking on a long hike where you must carry snacks (intermediate activations) to eat on the way back (backward pass) to compute gradients. In vanilla backpropagation, you carry all L snacks at once, which requires a heavy, linearly growing backpack. In full recomputation, you carry no snacks at all, but every time you need one, you have to hike all the way back to the trailhead (network input) to fetch it, resulting in a slow, quadratic $O(L^2)$ time penalty. Gradient checkpointing offers the optimal middle path: you carry a snack only at every $\sqrt{L}$-th layer, keeping only $\sqrt{L}$ checkpoints. On the way back, when you need a missing snack, you only detour at most $\sqrt{L}$ steps from the nearest checkpoint to recompute it. This drops activation memory from $O(L)$ to $O(\sqrt{L})$ at the cost of running a single extra forward pass (roughly 33% compute overhead).

18.1.1 The Problem It Solves

Training a modern LLM stores three primary pools of memory: model weights + optimizer states (which scale as $20N$ bytes per parameter for DDP), gradients, and intermediate activations. The activation memory scales linearly with depth, batch size, sequence length, and hidden dimension:

$$\text{Activation Memory} = L \times B \times S \times D \times \text{dtype\_bytes}$$

At a standard LLaMA-3 8B configuration (depth $L = 32$, batch size $B = 32$, sequence length $S = 4096$, hidden dimension $D = 4096$), a single layer's intermediate activation is:

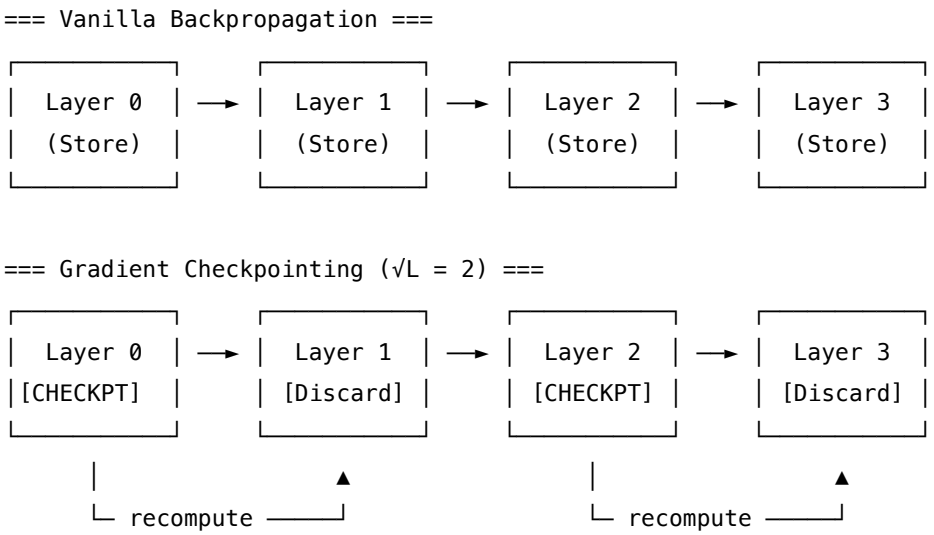
$$1 \times 32 \times 4096 \times 4096 \times 2 \text{ bytes (in fp16/bf16)} = 1073.74 \text{ MB} \approx 1.07 \text{ GB}$$

Across all 32 layers, the total activation memory peaks at 34.36 **GB**, which is more than double the memory needed to store the model weights themselves (16 GB in FP16). As sequence lengths or batch sizes grow, activations quickly bottleneck the system and cause Out-of-Memory (OOM) errors.

18.1.2 How It Works

Gradient checkpointing splits the L layers of a network into $\approx \sqrt{L}$ segments of length $\approx \sqrt{L}$. 1. **Forward Pass**: It computes the full forward pass, but saves (checkpoints) only the boundary inputs at every $\sqrt{L}$-th layer. All intermediate activations within the segments are immediately thrown away. 2. **Backward Pass**: When backpropagation enters a segment of length $\sqrt{L}$, it takes the saved segment boundary input, runs a local forward pass across the $\sqrt{L}$ layers in that segment to recompute the missing intermediate activations,

and then performs the backward pass for that segment. Once the segment's gradients are computed, the recomputed activations are discarded.



During training, this recomputation runs exactly one extra forward pass for each non-checkpointed layer, introducing a predictable $\sim 33\%$ compute overhead, because a typical training step already costs roughly 3 forward-pass equivalents (1 forward + 1 backward, where the backward pass costs $\approx 2\times$ the forward pass due to calculating gradients for both weights and activations). Adding 1 extra forward pass represents $\frac{1}{3} \approx 33.3\%$ overhead.

18.2 Worked Example

18.2.1 1. Verification of Activation Memory Math

For a batch size $B = 32$, sequence length $S = 4096$, hidden dimension $D = 4096$, and dtype size of 2 bytes (FP16/BF16), the activation size of a single layer A is:

$$A = 32 \times 4096 \times 4096 \times 2 = 1,073,741,824 \text{ bytes} = 1.0737 \text{ GB}$$

If we train a 32-layer model:

$$\text{Vanilla Activation Memory} = 32 \times 1.0737 \text{ GB} = 34.36 \text{ GB}$$

18.2.2 2. Comparison of the Three Strategies ($L = 32$)

From the verified execution statistics in `gradient_checkpointing_output.txt`:

Strategy	Stored Activations	Memory Multiplier	Recomputed Layers	Total Compute Units (Fwd+Recompute+Bwd)	Compute vs. Vanilla
Store-All (Vanilla)	32	32× (34.36 GB)	0	32+0+64 = 96	1.00×

Strategy	Stored Activations	Memory Multiplier	Recomputed Layers	Total Compute Units (Fwd+Recompute+Bwd)	Compute vs. Vanilla
Full Recompute	1 (input only)	1× (1.07 GB)	$\frac{L(L+1)}{2} = 528$	$32 + 528 + 64 = 624$	6.50×
Selective $\sqrt{L}$	$\lceil\sqrt{32}\rceil = 6$	6× **(6.44 GB)**	32	$32 + 32 + 64 = 128$	1.33× (+33.3%)

The checkpoints for $L = 32$ are located at indices: `[0, 6, 12, 19, 25, 31]`.

18.2.3 3. Scaling Checkpoints with Depth

The benefit of $\sqrt{L}$ scaling increases with model depth:

Depth (L)	Store-All Memory (L)	Selective $\sqrt{L}$ Memory ($\lceil\sqrt{L}\rceil$)	Savings Factor
4	4×	2×	2.0×
16	16×	4×	4.0×
32	32×	6×	5.3×
64	64×	8×	8.0×
128	128×	12×	10.7×
1000	1000×	32×	31.2×

18.2.4 4. Gradient Correctness Proof

Gradient checkpointing does not approximate gradients. As verified in `gradient_checkpointing_output.txt` on a real 4-layer MLP network (with layers 1-2 checkpointed, `use_reentrant=False`): - `forward output match? max|h_ref - h_ckp| = 0.00e+00` - `loss match? |loss_ref - loss_ckp| = 0.00e+00` - `input-grad match? max|dx_ref - dx_ckp| = 0.00e+00`

18.3 Complexity & Trade-offs

Metric	Value	Notes
Activation Memory	$O(\sqrt{L})$	Reduces activation footprint from $O(L)$ to $O(\sqrt{L})$. For $L = 1000$, drops from 1000× to 32×.
Compute Overhead	+33.3%	Requires exactly one extra forward pass per step.

Metric	Value	Notes
Gradient Accuracy	Bit-identical (0.00 difference)	The backward pass operates on recomputed activations that match the original forward outputs exactly.
Optimizer Memory	Unaffected	Checkpointing only reduces activation memory. Model parameters and optimizer states remain sharded by DDP/ZeRO.
RNG State Tracking	Requires re-saving seeds	Operations like dropout rely on random states. PyTorch's checkpoint API saves and restores RNG states during recomputation to preserve determinism.

18.4 Common Interview Questions & How to Answer

18.4.1 Q1: Why is the compute overhead of gradient checkpointing $\approx 33\%$ and not 100%, even though we run the forward pass twice?

- **Answer**: A standard training iteration consists of one forward pass and one backward pass. Calculating the gradients during the backward pass requires twice the amount of floating-point operations (FLOPs) of a forward pass (since we compute gradients with respect to both the weights and the inputs of each layer). Therefore:

$$\text{Vanilla Iteration Cost} = 1 \text{ Fwd} + 1 \text{ Bwd} \approx 3 \text{ Fwd Units}$$

When gradient checkpointing is enabled, we run the initial forward pass (1 Fwd), discard intermediate activations, and then during the backward pass we recompute the forward pass layer-by-layer (1 Fwd) in segments before executing the backward step (1 Bwd):

$$\text{Checkpointed Iteration Cost} = 1 \text{ Fwd} + 1 \text{ Recompute Fwd} + 1 \text{ Bwd} \approx 4 \text{ Fwd Units}$$

The additional cost is $\frac{4-3}{3} = \frac{1}{3} \approx 33.3\%$.

18.4.2 Q2: What is the difference between PyTorch's `use_reentrant=True` and `use_reentrant=False` in the checkpoint API? Why does this matter for DDP/FSDP?

- **Answer**:
  - **Reentrant Checkpointing (`use_reentrant=True`)**: This is the legacy implementation. It runs the forward pass under a `no_grad` block and manually constructs a nested autograd graph using custom `Function` blocks. It does not support standard autograd hooks properly and fails when

mixing checkpointed blocks with DDP/FSDP (causing incorrect gradient sync hooks, hanging, or crashes during backward passes).
  - **Non-reentrant Checkpointing (`use_reentrant=False`)**: This is the modern implementation (introduced in PyTorch 2.0). It uses PyTorch's standard autograd engine to trace the backward pass, utilizing hook mechanisms. It fully supports double backward passes, works seamlessly with DDP/FSDP gradient synchronization hooks, and handles modules with unused inputs. It is the recommended setting and must be passed explicitly.

18.4.3 Q3: Explain Megatron-LM's "selective activation recomputation" and how it differs from Chen et al.'s standard $\sqrt{L}$ checkpointing.

- **Answer**:
  - Standard $\sqrt{L}$ checkpointing (Chen et al. 2016) checkpoints the boundary inputs of every $\sqrt{L}$-th *entire layer* (attention + MLP blocks) and recomputes the entire layer's forward pass.
  - Megatron-LM's selective activation recomputation (Korthikanti et al. 2022) focuses on checkpointing only the parts of a layer that consume massive memory but are cheap to compute. Specifically, the attention head activations (e.g., attention scores $S = \text{Softmax}(\frac{QK^T}{\sqrt{d_k}})$ of shape $[B, H, S, S]$) and dropout masks are highly memory-intensive but computationally cheap. By checkpointing only these specific intermediates and storing the rest of the layer normally, it recovers most of the memory savings with significantly lower compute overhead (often $< 10\%$).

18.5 Pro-Tip: How to Impress the Interviewer

- **Address the RNG state preservation problem under checkpointing.** Explain that since operations like Dropout or stochastic depth rely on random number generators (RNG), simply re-running the forward pass during backward would yield different random masks, resulting in incorrect gradients. Explain that `torch.utils.checkpoint` handles this by caching the current GPU RNG state (`torch.cuda.get_rng_state()`) at the checkpoint boundaries during the forward pass and restoring it before running the recomputation, ensuring mathematical determinism and gradient correctness.

19 NCCL Collective Communication Operations

- **Category**: LLM Systems
- **Difficulty**: Expert
- **Target Role**: LLM Systems Engineer / ML Platform Engineer
- **Source**: NCCL_COLLECTIVES.md + nccl_collectives_output.txt (Patarasuk & Yuan 2009, Gibiansky 2017)
- **Flashcards**: LLM Systems deck

19.1 Concept Overview

Think of collective communication as a bucket brigade running along a circle of workers (a logical ring of GPUs). If you need to average gradients across K GPUs to keep them in sync, a naive centralized approach

appoints one ”foreman” GPU to collect everyone's data, sum it up, and mail copies back. This creates a massive bandwidth bottleneck at the foreman, scaling as $O(K^2 \cdot N)$ total fabric traffic where N is the message size. NCCL solves this by organizing GPUs in a logical ring where each GPU only communicates with its left and right neighbors. By sharding the message into K chunks and passing them sequentially around the ring, every GPU computes the final sum after $2(K-1)$ steps. Crucially, the total data sent per GPU is limited to $2\frac{K-1}{K}N \approx 2N$ bytes, which is completely independent of the world size K. This plateauing communication overhead is what enables training models across thousands of GPUs.

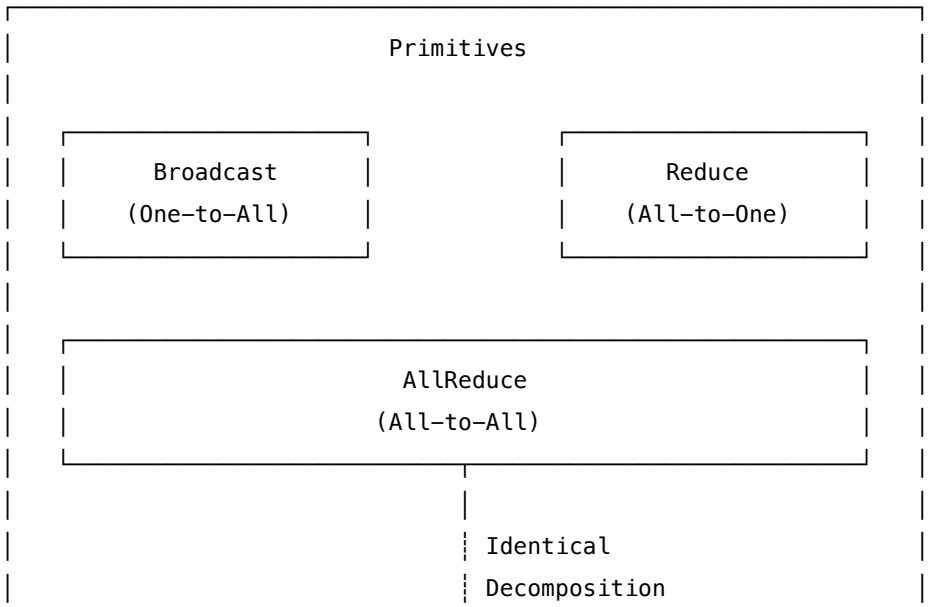
19.1.1 The Problem It Solves

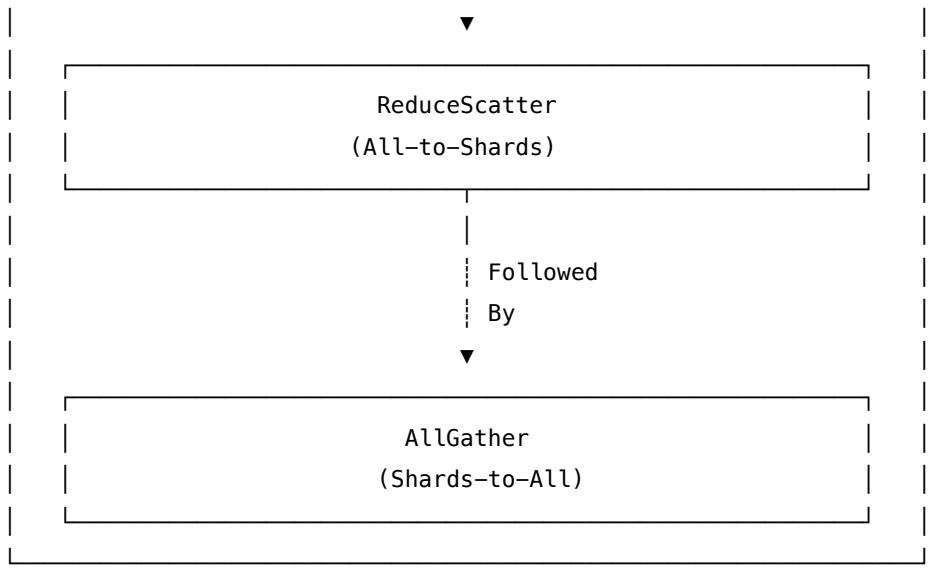
Naively coordinating communications across multiple GPUs leads to severe bandwidth bottlenecks. If we use a centralized root rank (naive reduce-to-root then broadcast) to run AllReduce on a vector of size N elements across K ranks: - The root rank must receive $(K-1) \cdot N$ elements. - The root rank must send $(K-1) \cdot N$ elements back. - The total bottleneck load on the root rank is $2(K-1) \cdot N$ elements.

For $K = 4$ ranks and $N = 16$ elements: - NAIVE Root Bottleneck $= 2 \times (4-1) \times 16 = 96$ elements transferred at the root. - RING-AllReduce per-GPU load $= 2 \times \frac{4-1}{4} \times 16 = 24$ elements transferred. Thus, the naive method incurs a 4.0× higher load on the root node compared to any node in the ring. As K scales to hundreds or thousands of GPUs, this bottleneck scales linearly with K, rendering distributed training impossible without ring collectives.

19.1.2 How It Works

NCCL implements 5 core collective primitives that define the vocabulary of distributed training: 1. **Broadcast**: Copies a buffer from a designated `root` rank to all other ranks. 2. **Reduce**: Performs an element-wise reduction (usually SUM) across all ranks, storing the final result on the `root` rank only. 3. **AllReduce**: Performs an element-wise reduction across all ranks and distributes the final result to all ranks. 4. **ReduceScatter**: Performs an element-wise reduction across all ranks, splits the result into K chunks, and distributes chunk r to rank r. 5. **AllGather**: Gathers a shard from each rank, concatenates them in rank order, and distributes the full concatenated buffer to all ranks.





19.1.2.1 Ring-AllReduce Mechanics The ring-AllReduce algorithm splits the buffer of N elements on each rank into K equal chunks. It then runs in two phases, each consisting of $K-1$ steps: - **Phase 1: Scatter-Reduce ($K-1$ steps)**: Every rank i sends its chunk c to rank $i+1$ (modulo K) while receiving a chunk from rank $i-1$. The received chunk is accumulated (added) to the local chunk. After $K-1$ steps, each rank holds the complete element-wise sum of one specific chunk. - **Phase 2: AllGather ($K-1$ steps)**: Every rank sends its fully summed chunk to rank $i+1$ to overwrite (not accumulate) the stale data. After another $K-1$ steps, every rank holds the fully summed buffer of N elements.

19.2 Worked Example

19.2.1 1. Verification of the Collective Primitives ($K = 4$, $N = 4$)

Inputs on each rank r are initialized to distinct values `[10r+1, 10r+2, 10r+3, 10r+4]`: - Rank 0: `[1, 2, 3, 4]` - Rank 1: `[11, 12, 13, 14]` - Rank 2: `[21, 22, 23, 24]` - Rank 3: `[31, 32, 33, 34]`

From `nccl_collectives_output.txt`:

- **Broadcast (root=0)**: All ranks receive Rank 0's buffer:
  - Rank 0, 1, 2, 3: `[1, 2, 3, 4]`
- **Reduce (root=0, op=sum)**: Sum stored on Rank 0 only:
  - Rank 0: `[64, 68, 72, 76]` (derived from $1+11+21+31=64$, etc.)
  - Rank 1, 2, 3: unchanged.
- **AllReduce (op=sum)**: Sum distributed to all ranks:
  - Rank 0, 1, 2, 3: `[64, 68, 72, 76]`
- **ReduceScatter (op=sum)**: Summed chunks of size $N/K = 1$ scattered to respective ranks:
  - Rank 0: `[64]` (summed chunk 0)
  - Rank 1: `[68]` (summed chunk 1)
  - Rank 2: `[72]` (summed chunk 2)
  - Rank 3: `[76]` (summed chunk 3)
- **AllGather**: Gathers the scattered shards back:

– Rank 0, 1, 2, 3: `[64, 68, 72, 76]`

19.2.2 2. The Identity: AllReduce ≡ ReduceScatter + AllGather

Running `ReduceScatter` followed by `AllGather` on the above input yields: - Output: `[64, 68, 72, 76]` - Diff: `max|AllReduce - (ReduceScatter+AllGather)| = 0.000e+00` This identity holds bit-for-bit, which forms the theoretical foundation of the ZeRO optimization framework.

19.2.3 3. Ring-AllReduce Execution ($K = 4, N = 16$)

Every rank starts with an identical vector $v = [1, 2, 3, \dots, 16]$. - **Expected Sum**: $4 \cdot v = [4, 8, 12, \dots, 64]$ - **Elements Sent per GPU**:

$$\text{Data Sent} = 2\frac{K-1}{K}N = 2 \times \frac{3}{4} \times 16 = 24 \text{ elements}$$

- **Validation**: - Ring output on all ranks matches expectations exactly (Gold Scalar `result[0] = 4`). - Total measured elements sent per GPU = `24`.

19.2.4 4. Per-GPU Bandwidth and Scalability (Message Size $N = 16$)

As $K \to \infty$, the factor $2\frac{K-1}{K} \to 2$, capping communication volume per GPU at $2N$ elements.

K (Ranks)	Elements Sent per GPU ($2\frac{K-1}{K}N$)	Ratio to N	Ratio to $2N$ (Ceiling)
2	16.00	1.000	0.500
4	**24.00**	1.500	0.750
8	28.00	1.750	0.875
16	30.00	1.875	0.938
64	31.50	1.969	0.984
256	31.88	1.992	0.996

19.2.5 5. Wall-Clock Timing Analysis (1 GB Gradient, $K = 8$ GPUs)

Using different fabrics with per-link directional bandwidth B: - **NVLink 4.0 (A100)**: $B \approx 600$ GB/s aggregate (300 GB/s per direction) - **InfiniBand NDR**: $B \approx 50$ GB/s - **PCIe Gen4**: $B \approx 64$ GB/s - **100GbE Ethernet**: $B \approx 12.5$ GB/s

For 1 GB gradient on $K = 8$ GPUs over NVLink (600 GB/s): - **Ring-AllReduce Per-GPU Data**: $2 \times \frac{7}{8} \times 1$ GB $= 1.75$ GB - **Ring-AllReduce Execution Time**: $\frac{1.75 \text{ GB}}{600 \text{ GB/s}} = 2.92$ ms (Exact) - **Naive Root Execution Time**: $\frac{K \cdot N}{B} = \frac{8 \text{ GB}}{600 \text{ GB/s}} = 13.33$ ms - **Speedup**: Ring is 4.56× faster (requires only 0.22× the naive execution time).

19.3 Complexity & Trade-offs

Collective	Elements Sent per GPU	Total Fabric Traffic	Use Case
Broadcast	N (Root sends, others receive)	$O(K \cdot N)$	Weight initialization sync
Reduce	$\frac{K-1}{K}N$ (Ring)	$O(K \cdot N)$	Loss evaluation
AllReduce	$2\frac{K-1}{K}N \approx 2N$	$O(K \cdot N)$	DDP gradient averaging, TP O-projections
ReduceScatter	$\frac{K-1}{K}N \approx N$	$O(K \cdot N)$	ZeRO gradient partitioning
AllGather	$\frac{K-1}{K}N \approx N$	$O(K \cdot N)$	ZeRO parameter reconstruction

---

19.4 Common Interview Questions & How to Answer

19.4.1 Q1: Prove that the communications volume per GPU in Ring-AllReduce is bounded by $2N$ elements and is independent of the number of ranks K.

- **Answer**: In Ring-AllReduce, we split the message of N elements into K chunks of size N/K. The algorithm consists of two phases: Scatter-Reduce and AllGather.
  - **Scatter-Reduce**: Runs for $K-1$ steps. In each step, each GPU sends exactly one chunk of size N/K to its right neighbor.

$$\text{Data sent per GPU in Phase 1} = (K-1) \cdot \frac{N}{K} \text{ elements}$$

  - **AllGather**: Also runs for $K-1$ steps. In each step, each GPU sends one fully accumulated chunk of size N/K to its right neighbor.

$$\text{Data sent per GPU in Phase 2} = (K-1) \cdot \frac{N}{K} \text{ elements}$$

  - **Total Data Sent**:

$$\text{Total Data Sent per GPU} = 2 \cdot (K-1) \cdot \frac{N}{K} = 2\frac{K-1}{K}N \text{ elements}$$

    As the world size $K \to \infty$, the factor $\frac{K-1}{K} \to 1$, making the limit:

$$\lim_{K \to \infty} 2\frac{K-1}{K}N = 2N \text{ elements}$$

    This bound is independent of K, meaning the communication volume per GPU plateaus at $2N$ elements.

19.4.2 Q2: Why does Tensor Parallelism (TP) require high-bandwidth NVLink within a node, while Pipeline Parallelism (PP) can run over InfiniBand across nodes?

- **Answer**:

- **Tensor Parallelism (TP)** shards model layers (e.g., column-parallel and row-parallel matmuls). It requires executing an **AllReduce operation twice per transformer layer** (one after the self-attention block, and one after the MLP block) to synchronize activations. Because this occurs within the forward and backward passes of *every single layer*, the communication latency is on the critical path of execution. Slow links (like InfiniBand or PCIe) would stall the GPU. Hence, TP is confined to NVLink-connected GPUs within a single node.
- **Pipeline Parallelism (PP)** splits the model across layer boundaries. It only requires point-to-point **Send/Recv** operations to pass intermediate activations and gradients at the boundaries between pipeline stages (typically once per micro-batch). This point-to-point communication is much less frequent and has lower bandwidth requirements, allowing PP to scale across nodes via InfiniBand.

19.4.3 Q3: Explain the role of the collective identity AllReduce ≡ ReduceScatter+AllGather in the ZeRO memory optimization framework.

- **Answer**: In standard Data Parallelism (DDP), gradients are averaged across all ranks using a single `AllReduce` operation, and every rank maintains an identical copy of the weights, gradients, and optimizer states. By decomposing `AllReduce` into `ReduceScatter` followed by `AllGather`, ZeRO partitions these redundant states:
  - **ZeRO-1 (Optimizer State Partitioning)**: Ranks run a `ReduceScatter` on gradients, so each rank only gets $1/K$ of the averaged gradients. Ranks update only their $1/K$ slice of optimizer states and master weights. Ranks then run an `AllGather` to distribute updated weights to all ranks.
  - **ZeRO-2 (Gradient Partitioning)**: Gradients are reduced directly into $1/K$ partitions via `ReduceScatter` during the backward pass, discarding the rest of the gradients immediately to save VRAM.
  - **ZeRO-3 (Parameter Partitioning)**: Weights themselves are sharded. Ranks run `AllGather` on-demand to fetch weights for a layer during the forward pass, and discard them immediately after. The total volume of communications remains $2N$ bytes (equivalent to AllReduce), but partitioning the states in between the collectives reduces VRAM overhead significantly.

19.5 Pro-Tip: How to Impress the Interviewer

- **Contrast the Ring algorithm with the Tree-based collective algorithm.** Explain that while the Ring algorithm is bandwidth-optimal for large messages (like LLM gradients), it has a latency cost that scales linearly with K (i.e., $2(K-1)$ startup latencies). For small messages or large world sizes, modern NCCL uses **Double Binary Tree** topologies, which reduce the latency scaling factor to $O(\log K)$ while maintaining high bandwidth utilization, automatically switching between Ring and Tree protocols based on message size and topology.

20 Distributed Data Parallel (DDP)

- **Category**: LLM Systems

- **Difficulty**: Hard
- **Target Role**: LLM Systems Engineer / ML Platform Engineer
- **Source**: DDP.md + ddp_output.txt (Goyal et al. 2017, Micikevicius et al. 2017)
- **Flashcards**: LLM Systems deck

20.1 Concept Overview

Think of Distributed Data Parallel (DDP) like a chorus line of identical dancers. Each dancer watches a different part of the stage (a different slice of the mini-batch) and makes moves. After every step, the dancers whisper their corrections to everyone else and compute the average adjustment (AllReduce). Because they start with identical moves and always apply the exact same averaged adjustment, they can never drift out of sync, no matter how long the show runs. In system terms, DDP replicates the entire model and optimizer states on every GPU. Each GPU executes a forward and backward pass on its local data slice to compute local gradients. An AllReduce operation then averages these gradients across all ranks, ensuring that all GPUs take the exact same gradient step and their parameters remain bit-identical forever.

20.1.1 The Problem It Solves

A single GPU hits hard limits in throughput and memory capacity. Modern training requires large effective batch sizes to stabilize optimization. DDP implements data-parallel execution to scale training throughput linearly with the number of GPUs.

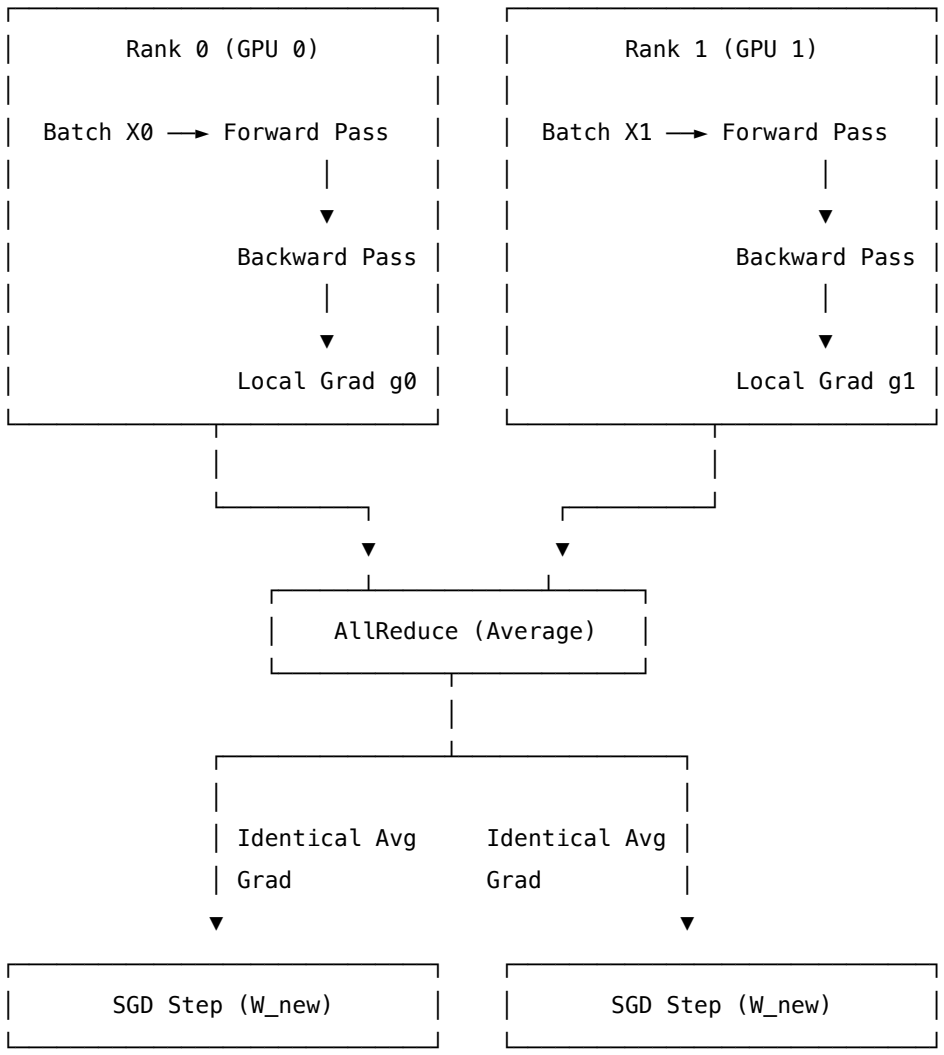
However, replicas will drift if gradients differ. If we do not synchronize, rank-specific data batches ($X_0 \neq X_1$) will generate differing local gradients ($g_0 \neq g_1$). Taking independent SGD steps would cause the parameter sets to diverge immediately. Additionally, training consumes massive memory: the model and optimizer states require a redundant 20 **bytes per parameter** on every GPU (including master weights and Adam states in mixed-precision training). DDP manages this redundancy by ensuring perfect synchronization of gradients across the communication fabric.

20.1.2 How It Works

1. **Initialization**: The weights are initialized on a coordinator GPU (Rank 0) and broadcast to all other GPUs, guaranteeing identical starting parameters. Ranks set unique seed offsets based on their rank (e.g., `seed = 1337 + rank`) to ensure they load non-overlapping mini-batches.
2. **Forward and Backward Passes**: Each GPU runs the forward pass on its local batch X_k and executes the backward pass to compute a local gradient g_k.
3. **Autograd Hook Synchronization**: PyTorch DDP installs backward hooks on all parameters. As soon as the autograd engine finishes computing the gradient for a parameter, the hook triggers an immediate `AllReduce-average` operation. This overlaps communication time with the computation of remaining gradients in the backward pass.
4. **Optimizer Step**: Once backward finishes and all gradients are averaged across all ranks, every GPU performs the identical optimizer update:

$$W \leftarrow W - \eta \cdot g_{\text{averaged}}$$

Because the starting weights W and the gradients g_{averaged} are identical, the updated weights remain bit-identical on all nodes.



20.2 Worked Example

20.2.1 1. Simulated 2-Rank Step

We simulate $K = 2$ ranks with a single linear layer $y = xW$ (W shape $[4, 3] = 12$ parameters) and MSE loss. Ranks start with identical initialized weights W:

```
W = [+0.154100, -0.029343, -0.217879, +0.056843, -0.108452, -0.139860,
     +0.040335, +0.083803, -0.071926, -0.040334, -0.059664, +0.018204]
```

Ranks process different batch rows (e.g., $X_0 \neq X_1$): - $X_0[0, :] = [+0.336690, +0.128809, +0.234462, +0.230333]$ - $X_1[0, :] = [-0.988960, +0.957972, +1.322135, +0.817190]$

Local gradients computed in backward: - $g_0[0, :] = [+0.283542, +0.280142, -0.741705]$ - $g_1[0, :] = [-0.033977, +0.354733, -0.016214]$ *(These differ because the training inputs differ).*

AllReduce computes the average gradient across both ranks: - $g_{\text{avg}} = (g_0 + g_1)/2$: `avg[0, :] = [+0.124783, +0.317437, -0.378959]` - Both ranks end up with this exact average gradient. (Gold `avg_grad[0,0] = +0.124783`, `avg_grad[1,2] = -0.167272`).

Taking an SGD step with learning rate $\eta = 0.1$: - $W_{\text{new}} = W - \eta \cdot g_{\text{avg}}$: `W_new[0, :] = [+0.154100, -0.029343, -0.217879] - 0.1 * [+0.124783, +0.317437, -0.378959] = [+0.141621, -0.061087, -0.179983]` - Weights are bit-identical on both ranks (Gold `W_after_step[0,0] = +0.141621`, `W_after_step[1,2] = -0.123132`). - A single GPU running on the concatenated batch $X = [X_0; X_1]$ produces the exact same gradient (`max|diff| = 5.96e-08`), proving that DDP on K GPUs is mathematically identical to single-GPU training with a $K\times$ larger batch.

20.2.2 2. Gradient Accumulation

Gradient accumulation runs multiple micro-batches sequentially on each GPU before updating weights. In a 2-GPU setup ($K = 2$) with batch size $B = 2$ and gradient accumulation steps accum $= 2$: - Effective Batch Size $= B \times \text{accum} \times K = 2 \times 2 \times 2 = 8$ samples - To avoid redundant communications, we disable AllReduce on early micro-steps using PyTorch's `require_backward_grad_sync = False`. - On the last micro-step, we enable synchronization (`require_backward_grad_sync = True`). - Micro-losses are scaled down: $\text{Loss}_{\text{micro}} = \text{Loss}/\text{accum}$. - The accumulated and synced gradient matches the single-GPU gradient on all 8 samples exactly:

$$g_{\text{accum}} = [+0.399230, -0.563779, -0.283541, \dots]$$

20.2.3 3. Mixed Precision & Underflow

In FP16 mixed precision training, small gradients underflow to zero due to FP16's narrow range $[6 \cdot 10^{-5}, 65504]$. - Underflow example: 10^{-8} rounds to `0.0` in FP16, whereas it survives in BF16 (which matches FP32's exponent range). - **GradScaler Solution**: 1. Multiply the loss by a scaling factor $S = 2^{16} = 65536$:

$$\text{scaled\_grad} = 10^{-8} \times 65536 = 0.00065536 \text{ (survives in FP16)}$$

2. Perform backward pass to propagate scaled gradients in FP16. 3. Divide gradients by S (unscale) back to FP32 before the optimizer step:

$$\text{unscaled\_grad} = 0.00065536/65536 \approx 9.997 \cdot 10^{-9} \text{ (recovered!)}$$

20.2.4 4. Memory Bill (Mixed Precision + Adam)

Under mixed precision training, the memory overhead per parameter is 20 **bytes**:

Component	Bytes / Parameter	Rationale
FP16 parameters	2	Weights used in forward pass computation
FP16 gradients	2	Computed during backward pass
FP32 master parameters	4	High-precision copy updated by optimizer

Component	Bytes / Parameter	Rationale
FP32 gradients	4	Upcast gradients for optimizer calculations
FP32 Adam momentum	4	First moment running average
FP32 Adam variance	4	Second moment running average
Total per param	**20**	Redundantly replicated on every GPU

For LLaMA-3 8B, DDP requires 160 **GB** of memory per GPU just for the parameters, gradients, and optimizer states, which is why ZeRO sharding is necessary at scale.

20.2.5 5. Cosine LR Schedule with Warmup

To train stable large batches (Goyal et al. 2017), the learning rate is scaled linearly with batch size, warmed up from 0 to peak over the first W steps, and then decayed to min_lr using a cosine function over T steps. With Peak LR = 1.0, min_lr = 0.1, warmup = 10 steps, decay = 50 steps: - Iter 0: 0.0909 (Warmup) - Iter 6: 0.6364 (Warmup) - Iter 10: 1.0000 (Peak) - Iter 25: 0.7222 (Cosine Decay) - Iter 50: 0.1000 (min_lr) - Iter 55: 0.1000 (Hold)

20.3 Complexity & Trade-offs

Metric	Value / Cost	Notes
Memory Footprint	$20N$ bytes per param	Redundant replication of all weights, gradients, and optimizer states on every node.
Communication Volume	$2\frac{K-1}{K}N \approx 2N$ bytes per step	Independent of K when using Ring-AllReduce.
Communication Overlap	High	Gradients are synced layer-by-layer during backward, hiding communication behind computation.
Mathematical Equivalence	1.00 (Exact)	DDP on K nodes is mathematically identical to training on a single GPU with batch size $K \times B$.
Scaling Efficiency	High	Scales linearly with GPU count, assuming communication is hidden by compute.

20.4 Common Interview Questions & How to Answer

20.4.1 Q1: Why must the loss function be a batch mean (not a sum) for DDP's AllReduce gradient averaging to be mathematically equivalent to single-GPU training?

- **Answer**: The single-GPU training loss on a concatenated batch $X = [X_0; X_1]$ is defined as the mean over all samples in the combined batch:

$$\mathcal{L}(X) = \frac{1}{2B}\sum_{i=1}^{2B}\ell_i = \frac{1}{2}\left[\frac{1}{B}\sum_{i=1}^{B}\ell_i + \frac{1}{B}\sum_{i=B+1}^{2B}\ell_i\right] = \frac{\mathcal{L}(X_0)+\mathcal{L}(X_1)}{2}$$

  Taking the gradient of the loss with respect to parameters W:

$$\nabla_W\mathcal{L}(X) = \frac{\nabla_W\mathcal{L}(X_0)+\nabla_W\mathcal{L}(X_1)}{2}$$

  Because the combined loss is the average of the individual losses, the combined gradient is the **average** of the individual local gradients (g_0 and g_1). Thus, running an `AllReduce-average` on local gradients perfectly reconstructs the global gradient. If we used a sum loss, the combined gradient would be the sum of local gradients ($\nabla_W\mathcal{L}_{\text{sum}} = g_0 + g_1$), and AllReduce-averaging would make the gradients $2\times$ too small.

20.4.2 Q2: How does DDP overlap communication with computation, and why is this critical for scaling?

- **Answer**: DDP does not wait for the backward pass to finish entirely before communicating gradients. PyTorch DDP registers an autograd hook on each parameter.
  - During the backward pass, gradients are computed in reverse order (from the output layer back to the input layer).
  - As soon as the gradient for a parameter is calculated, its autograd hook triggers.
  - DDP organizes parameters into "buckets" (typically 25 MB in size). When all parameters in a bucket have finished computing their gradients, DDP initiates an asynchronous `AllReduce` for that bucket.
  - This allows the GPU to communicate gradient buckets for the late layers over the network (using NCCL) while simultaneously calculating gradients for the early layers. This overlap hides communication latency behind backward pass computation.

20.4.3 Q3: Why is loss scaling necessary for FP16 training, but not for BF16 training?

- **Answer**:
  - **FP16 (Half Precision)** allocating 5 bits for the exponent and 10 bits for the mantissa. Its minimum positive representable value (underflow limit) is $\approx 6\cdot10^{-5}$ (or $6\cdot10^{-8}$ for subnormals). Since gradients in deep networks are often smaller than 10^{-5}, they underflow to `0.0` in FP16, halting training. A `GradScaler` scales the loss up by 2^{16} to shift the gradient values into the representable range of FP16, and divides them back to FP32 before updating.
  - **BF16 (Brain Float 16)** allocates 8 bits for the exponent (matching FP32) and 7 bits for the mantissa. Its minimum positive representable value is $\approx 1.2\cdot10^{-38}$. This massive range ensures that gradients never underflow in BF16, rendering loss scaling unnecessary.

20.5 Pro-Tip: How to Impress the Interviewer

- **Discuss the DDP "Bucket Tuning" strategy.** Mention that DDP's default bucket size (25 MB) is tuned for standard workloads, but can bottleneck training. For small models or slow fabrics, larger buckets reduce the number of small NCCL calls, saturating bandwidth. For giant models, smaller buckets allow synchronization to start earlier in the backward pass, maximizing the overlap between communication and computation. Explain that bucket sizes can be tuned in PyTorch via the `bucket_cap_mb` parameter in the `DistributedDataParallel` constructor.

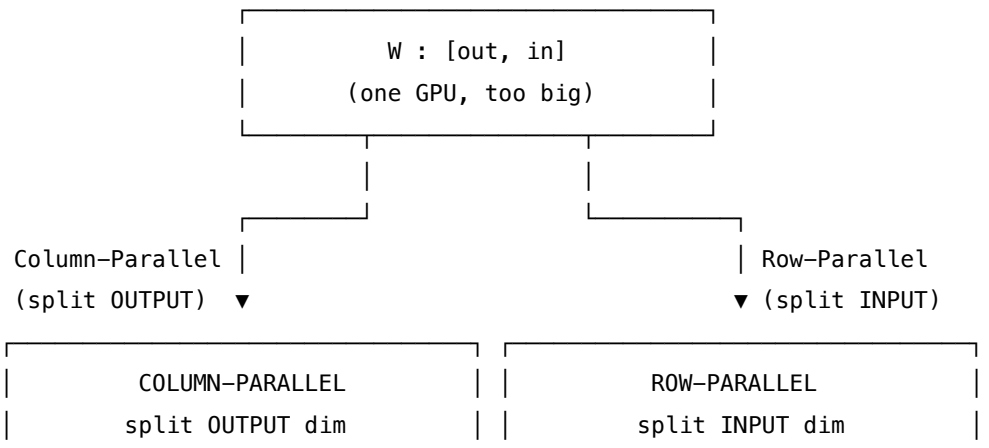
21 Tensor Parallelism (TP)

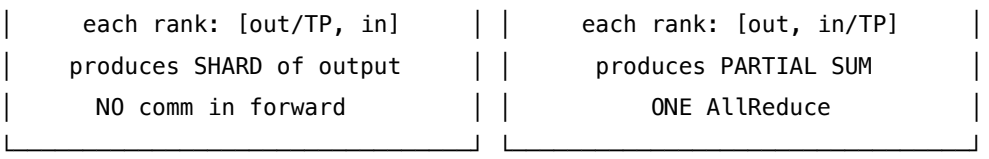
- **Category**: LLM Systems
- **Difficulty**: Expert
- **Target Role**: LLM Systems Engineer / Distributed Systems Engineer
- **Source**: Megatron-LM (Shoeybi et al., 2019)
- **Flashcards**: LLM Systems deck

21.1 Concept Overview

Imagine you have a warehouse with a pallet of goods that is too wide and heavy to fit on a single delivery truck. If you try to fit the whole pallet on one truck, it breaks down. Instead of loading whole pallets onto different trucks, you cut each pallet into **horizontal or vertical strips** and load one strip onto each truck. During the journey, the trucks drive in parallel, carrying out calculations on their specific strips of data. To deliver the final intact goods, the trucks briefly sync up at the destination (via a single network synchronization) to stitch their strips back together.

In LLM systems, **Tensor Parallelism (TP)** applies this sharding strategy to individual weight matrices inside a single Transformer layer. Instead of placing entire layers on different GPUs (which is Pipeline Parallelism), TP cuts each weight matrix along either its output dimension (column-parallel) or its input dimension (row-parallel). GPUs compute their local products in parallel and execute a single collective communication operation (`AllReduce`) to combine their partial results into the correct output.





21.1.1 The Problem It Solves

As LLMs scale, a single weight matrix exceeds the High Bandwidth Memory (HBM) capacity of a single GPU. For example, in a Llama-2-70B-style MLP, the model dimension (E) is $8,192$ and the intermediate dimension (FFN) is $28,672$.

A single FP16 weight matrix (`gate_proj`) of size $[28,672, 8,192]$ contains $234,881,024$ **elements**, which requires 0.44 **GiB** (0.88 GiB in FP32). The MLP block alone contains three such matrices (`gate_proj`, `up_proj`, `down_proj`), consuming 1.3 **GiB** of static weights *per layer* in FP16. Across 80 layers plus embedding and output projection layers, the static weights alone exceed 100 GiB, which cannot fit on a single 80 GB GPU (like the A100 or H100) before even accounting for KV cache, optimizer states, and activation memory.

By sharding every weight matrix by a factor of TP, memory scale savings are linear:

TP	Shard Shape	Elements per Rank	Weight Memory per Rank (FP16)	Savings
1	$[28,672, 8,192]$	$234,881,024$	0.438 GiB	Baseline
2	$[14,336, 8,192]$	$117,440,512$	0.219 GiB	2× smaller
4	$[7,168, 8,192]$	$58,720,256$	0.109 GiB	4× smaller
8	$[3,584, 8,192]$	$29,360,128$	0.055 GiB	8× smaller

21.1.2 How It Works

Megatron-LM introduced a layout strategy that minimizes collective communication by alternating column-parallel and row-parallel layers.

21.1.2.1 1. Column-Parallel Linear (e.g., QKV Projections, MLP Gate/Up) Slices the weight matrix W along its output dimension (dimension 0 in PyTorch `[out, in]`). - **Weight Shards**: Rank r stores W_r of shape $[out/TP, in]$. - **Forward Operation**: Rank r computes $Y_r = XW_r^T$ locally. The input X is replicated in full across all ranks. - **Communication**: Zero communication is required during the forward pass. The output Y_r has shape $[B, out/TP]$. To get the full output, an `AllGather` is needed, *unless* the next layer is row-parallel and can consume the sharded output directly.

21.1.2.2 2. Row-Parallel Linear (e.g., Attention Out Projection, MLP Down) Slices the weight matrix W along its input dimension (dimension 1 in PyTorch `[out, in]`). - **Weight Shards**: Rank r stores W_r of shape $[out, in/TP]$. - **Forward Operation**: Rank r computes $Y_{r,\text{partial}} = X_r W_r^T$ locally, where X_r is the local shard of the input of shape $[B, in/TP]$. - **Communication**: Each rank produces a "partial sum" tensor of shape $[B, out]$. An `AllReduce(sum)` operation must be executed across all TP ranks to sum these partial tensors: $Y = \sum_{r=0}^{TP-1} Y_{r,\text{partial}}$.

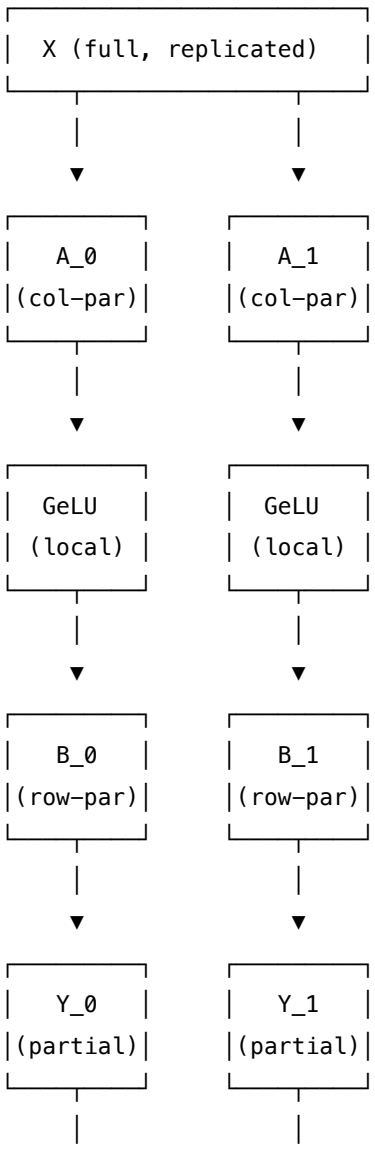
21.1.2.3 3. The Megatron MLP Optimization For a standard MLP layer: $Y = \text{GeLU}(X@A^T)@B^T$. If we sharded both matrices independently, we would need communication collectives between A and B. Megatron-LM eliminates this intermediate communication by making the first layer (A) **column-parallel** and the second layer (B) **row-parallel**:

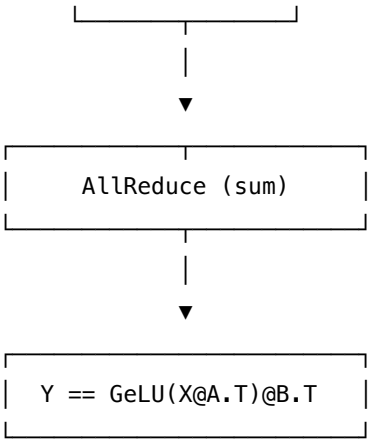
```
GPU 0: Z_0 = GeLU(X @ A_0.T)   -> Shape: [B, L, FFN/TP]
       Y_0 = Z_0 @ B_0.T       -> Shape: [B, L, E] (Partial Sum)

GPU 1: Z_1 = GeLU(X @ A_1.T)   -> Shape: [B, L, FFN/TP]
       Y_1 = Z_1 @ B_1.T       -> Shape: [B, L, E] (Partial Sum)

AllReduce: Y = Y_0 + Y_1       -> Shape: [B, L, E] (Identical to unsharded)
```

Because `GeLU` is an element-wise activation, it can be applied to local shards Z_r independently without needing global information. By feeding the local output of A directly into the row-parallel B, the intermediate synchronization is cancelled. **You pay exactly one `AllReduce` for the entire MLP block.**





21.1.2.4 4. Attention TP The same logic applies to Multi-Head Attention: - Q, K, and V projections are packed into a single **column-parallel** layer. The query/key/value heads are divided evenly: each rank runs self-attention on its local heads (H/TP). - The attention heads do not communicate with each other during QK multiplication or softmax. - The output projection (O) is **row-parallel**. It takes the concatenated local head outputs and projects them back to the hidden size, producing partial sums that are combined via a single `AllReduce` at the end of the block.

21.2 Worked Example

Below are exact values extracted from single-process simulations with $TP = 2$:

21.2.1 1. Matrix Projection (Column-Parallel vs. Row-Parallel)

For a model input X of shape $[1, 8]$ and weight matrix W of shape $[8, 8]$: - **Full Reference Forward ($Y = XW^T$)**:

$$Y = [-0.8029, +0.4008, -1.2338, +0.4680, -0.0224, -0.8494, +0.7460, +0.3059]$$

- **Column-Parallel sharding (W split along dim 0 into two $[4, 8]$ shards)**:
  - Rank 0 computes $Y_0 = XW_0^T = [-0.8029, +0.4008, -1.2338, +0.4680]$
  - Rank 1 computes $Y_1 = XW_1^T = [-0.0224, -0.8494, +0.7460, +0.3059]$
  - Concatenation matches the full Y exactly with **zero** forward communication.
- **Row-Parallel sharding (W split along dim 1 into two $[8, 4]$ shards, input X split into two $[1, 4]$ shards)**:
  - Rank 0 computes partial sum $Y_{0,\text{partial}} = [-0.6191, +0.5776, -1.0092, +0.5250, +0.1207, -1.0074, -0.0579, +$
  - Rank 1 computes partial sum $Y_{1,\text{partial}} = [-0.1837, -0.1768, -0.2245, -0.0570, -0.1431, +0.1580, +0.8039, -$
  - Summing them: $Y = Y_{0,\text{partial}} + Y_{1,\text{partial}} = [-0.8029, +0.4008, -1.2338, +0.4680, -0.0224, -0.8494, +0.7460,$ (exactly identical to the unsharded run, requiring **one** `AllReduce`).

21.2.2 2. The Megatron MLP Trick in Action

Using $TP = 2$, input $X[1, 8]$, column-parallel gate matrix $A[4, 8]$, and row-parallel down matrix $B[8, 4]$:

Tensor	Values	Shape
$Y_{0,\textbf{partial}}$ **Rank 0**	$[+0.0182, +0.0190, +0.0269, -0.0209, +0.0697, +0.0000, -0.0059, -0.0326]$	[1, 8]
$Y_{1,\textbf{partial}}$ **Rank 1**	$[-0.1966, +0.0195, -0.0286, -0.0892, +0.1581, +0.1559, +0.6273, +0.3876]$	[1, 8]
$Y_{\textbf{sum}}$ **(AllReduce)**	$[-0.1784, +0.0385, -0.0017, -0.1101, +0.2278, +0.1560, +0.6214, +0.3550]$	[1, 8]
Full Reference MLP	$[-0.1784, +0.0385, -0.0017, -0.1101, +0.2278, +0.1560, +0.6214, +0.3550]$	[1, 8]

Gold Verification scalar: The first element $Y_{full}[0,0] = \mathbf{-0.178412}$ matches to numerical precision (max difference $< 7.5 \times 10^{-9}$).

21.2.3 3. Packed QKV Projections for Grouped-Query Attention (GQA)

When sharding packed Q, K, and V projections, Grouped-Query Attention ($H_{kv} < H_q$) introduces asymmetry. Take $H_q = 4$, $H_{kv} = 2$, head dimension $D = 2$, and $TP = 2$. - The unsharded packed weight has shape $[(H_q + 2 \cdot H_{kv}) \cdot D, E] = [16, 8]$. - Slicing along the output dimension for $TP = 2$ shards:

Rank	Q Heads	K Heads	V Heads	Shard Rows per Rank
Rank 0	0..1	0..0	0..0	8 rows
Rank 1	2..3	1..1	1..1	8 rows

Divisibility Constraint: To map heads cleanly to ranks, H_q and H_{kv} must both be divisible by TP. If $H_{kv} \pmod{TP} \neq 0$ (e.g. $H_{kv} = 2$ with $TP = 4$), the loader throws an assertion error, which is why production GQA models choose H_{kv} compatible with expected TP groups.

21.3 Complexity & Trade-offs

Metric	Value	Notes
GPU Memory Reduction	$1/TP$ of parameter weights	Reduces memory footprint of weight matrices linearly.
Collective Comm Operations	2 `AllReduce`s per Transformer layer	1 in the Attention block, 1 in the MLP block.
Per-Rank Comm Volume	$2 \cdot \frac{TP-1}{TP} \cdot B \cdot L \cdot E \cdot 2$ bytes	For a ring-AllReduce. Scales with sequence length L and batch size B.
Interconnect Requirement	≥ 300 GB/s bi-directional (NVLink)	High frequency makes it highly sensitive to network latency.

21.3.1 Comm Volume Example

For a large training run with batch size $B = 32$, sequence length $L = 4,096$, hidden size $E = 8,192$ in FP16: - $TP = 2$: Per-rank traffic per AllReduce is 2.00 GiB. - $TP = 4$: Per-rank traffic per AllReduce is 3.00 GiB. - $TP = 8$: Per-rank traffic per AllReduce is 3.50 GiB.

Running this AllReduce over a slow connection is disastrous: - Over **NVLink (300 GB/s)**: One AllReduce takes 223.7 μ**s**. - Over **InfiniBand (25 GB/s)**: One AllReduce takes $2,684.4$ μ**s** (12× slower!). As a result, **TP is strictly confined to intra-node NVLink domains**.

21.4 Common Interview Questions & How to Answer

21.4.1 Q1: Why can we not place a Tensor Parallel group across multiple nodes connected by InfiniBand?

- **Answer**: TP executes communication collectives (`AllReduce`) twice per Transformer layer (once per attention block, once per MLP block) during both the forward and backward passes. At large batch sizes and sequence lengths, the volume of data that needs to be synchronized is massive (e.g., multiple gigabytes per step). An NVLink connection provides up to 300 GB/s per direction (900 GB/s aggregate for H100 SXM5), keeping the communication latency down to hundreds of microseconds. Standard InfiniBand is at least 10× to 20× slower (25–50 GB/s per direction). Moving TP across nodes causes the GPUs to spend more than 80% of their execution cycles waiting on network packets, stalling the entire pipeline.

21.4.2 Q2: How does the backward pass of Column-Parallel and Row-Parallel linear layers handle gradients?

- **Answer**: The communication primitives in the backward pass are the mathematical duals of the forward pass:
  1. **Column-Parallel Linear**: During the forward pass, the input X is replicated across all ranks, and each rank computes a shard of the output. In the backward pass, each rank computes a local gradient w.r.t. the input ∇X_r. To reconstruct the full input gradient ∇X, an **`AllReduce(sum)`** collective must be called.
  2. **Row-Parallel Linear**: During the forward pass, the input is sharded and an `AllReduce` is executed to sum partial outputs. In the backward pass, because the output was replicated after the forward `AllReduce`, the gradient w.r.t. the output ∇Y is replicated across all ranks. Each rank computes its local input gradient ∇X_r without communication. The only communication needed is a local **`AllGather`** or slice if the preceding layer requires sharded activations.

21.4.3 Q3: How do we handle biases in Column-Parallel and Row-Parallel layers without corrupting the math?

- **Answer**: In a Column-Parallel layer ($Y_r = X@W_r^T + bias_r$), the bias can be sharded alongside the weight columns ($bias_r = bias[r \cdot out/TP : (r+1) \cdot out/TP]$). Each rank adds its bias shard locally. In a Row-Parallel layer, the forward pass computes $Y_r = X_r@W_r^T$ on each rank and then sums them: $Y = \sum Y_r + bias$. If we add the entire bias to each rank's local calculation, the final `AllReduce(sum)` will sum the bias TP times, resulting in $TP \cdot bias$. To prevent this, the bias is either **only added on rank 0** (e.g., `bias if tp_rank == 0 else None`) or the bias addition is deferred to after the `AllReduce` operation has completed.

21.5 Pro-Tip: How to Impress the Interviewer

- **Sequence Parallelism Integration**: Explain how Megatron-LM extends TP using **Sequence Parallelism**. While standard TP shards linear projections, non-tensor layers (such as LayerNorm and Dropout) are typically duplicated. Sequence Parallelism shards these layers along the sequence dimension (L/TP). This saves activation memory and is combined with the TP communication loops by replacing the standard `AllGather` and `ReduceScatter` in the TP boundaries with sequence-dimension splits.
- **Explain GQA Head Mapping**: Detail exactly how Grouped-Query Attention head mapping restricts TP scaling. If a model has $H_q = 32$ and $H_{kv} = 8$ heads, and you scale to $TP = 8$, each rank gets $32/8 = 4$ Q-heads and $8/8 = 1$ KV-head. If you try to run $TP = 16$, the KV-heads cannot divide evenly ($8/16 = 0.5$ heads per rank). This causes an assertion failure in standard engines. To avoid this, you must either replicate KV heads across TP ranks or adjust your TP group size.

22 Pipeline Parallelism (PP)

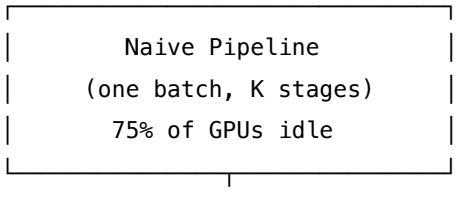
- **Category**: LLM Systems
- **Difficulty**: Expert
- **Target Role**: LLM Systems Engineer / Distributed Systems Engineer
- **Source**: GPipe (Huang et al., 2019) / Megatron-LM Pipeline Parallelism (Narayanan et al., 2021)
- **Flashcards**: LLM Systems deck

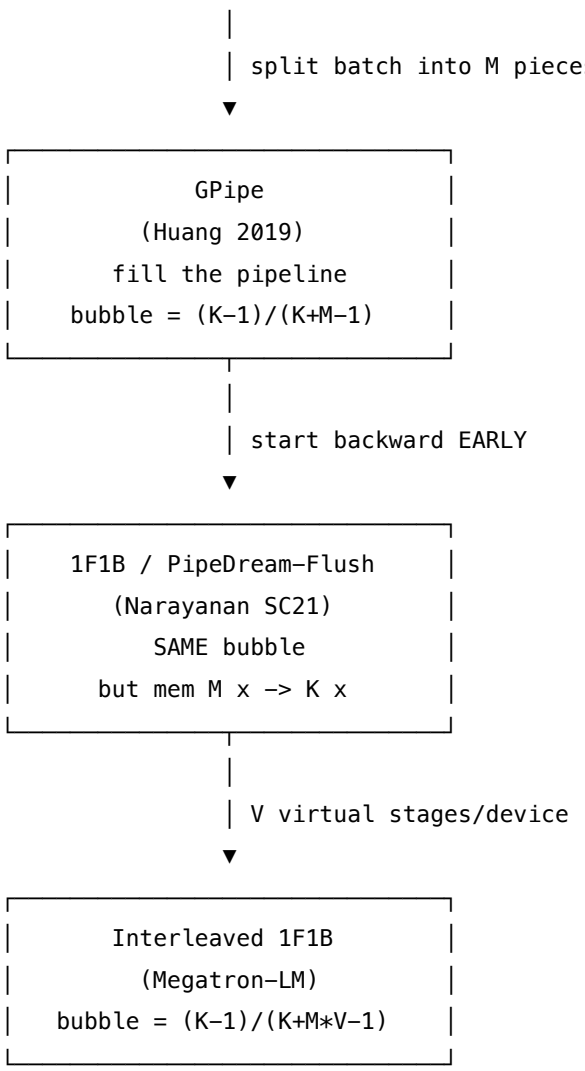
22.1 Concept Overview

Imagine you are managing an automobile assembly line divided into four factories (stages). In a naive setup, Factory 1 builds the chassis, then drives the car to Factory 2 to mount the engine, then to Factory 3 for the body, and Factory 4 for the paint. If you only process **one car at a time**, Factory 2, 3, and 4 sit completely idle while Factory 1 is building the chassis. When the car moves to Factory 2, the other three factories are empty. This massive amount of idle time is called the **pipeline bubble**.

To fix this, you split your shipment into many smaller batches (**micro-batches**). As soon as Factory 1 finishes the chassis for Car 1 and sends it to Factory 2, it immediately starts working on the chassis for Car 2. In steady state, all four factories are working on different cars simultaneously.

In LLM systems, **Pipeline Parallelism (PP)** partitions a model's layers sequentially across K stages (nodes). Rather than executing a whole batch of inputs through the entire model layer-by-layer, PP splits a mini-batch into M micro-batches and streams them through the partitioned stages. The communication is point-to-point: Stage k sends its output activations directly to Stage $k+1$, and sends its gradients back to Stage $k-1$ in reverse.





22.1.1 The Problem It Solves

Tensor Parallelism (TP) is constrained to a single node because its frequent `AllReduce` collectives demand NVLink speeds ($\geq$ 300 GB/s). This limits TP to ≤ 8 GPUs. A 175B model (which requires 350 GB of memory in FP16) or a 1T-class model cannot fit on a single node's total memory, even with $TP = 8$.

To train models that span multiple nodes, we need a parallelization strategy that is tolerant of the lower bandwidth of **InfiniBand / Ethernet (RoCE) (**25–50 **GB/s)**. PP achieves this because it only communicates activation tensors at stage boundaries, which is a tiny fraction of the data transferred by TP's per-layer AllReduces.

For a Llama-2-70B model with $L = 80$ layers and model dimension $E = 8,192$, one layer consumes 1.312 **GiB** of memory in FP16. Splitting layers across K PP stages reduces the model footprint per node:

PP Stages (K)	Layers per Stage	Model Memory per Stage (FP16)	Savings
1	80	105.000 GiB	Baseline
2	40	52.500 GiB	2× smaller

PP Stages (K)	Layers per Stage	Model Memory per Stage (FP16)	Savings
4	20	26.250 GiB	4× smaller
8	10	13.125 GiB	8× smaller
16	5	6.562 GiB	16× smaller

Combined with $TP = 8$ inside each node, a $K = 8$ PP setup can scale a 70B model across 64 GPUs with plenty of room left for the KV cache and activation memory.

22.2 How It Works

22.2.1 1. GPipe Scheduling (Huang et al., 2019)

GPipe splits the mini-batch into M micro-batches. The schedule has three phases: - **Warmup (Fill)**: Micro-batches are fed into the pipeline sequentially. As they flow forward, stages start processing them. - **Steady State**: All stages are busy. - **Cooldown (Drain)**: The pipeline drains as the final forward passes finish and the backward passes flow in reverse.

```
GPipe schedule  (K=4 stages, M=8 micro-batches):
          0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20  21
GPU0:   F0  F1  F2  F3  F4  F5  F6  F7   .   .   .   .   .   .  B7  B6  B5  B4  B3  B2  B1  B0
GPU1:    .  F0  F1  F2  F3  F4  F5  F6  F7   .   .   .   .  B7  B6  B5  B4  B3  B2  B1  B0   .
GPU2:    .   .  F0  F1  F2  F3  F4  F5  F6  F7   .   .  B7  B6  B5  B4  B3  B2  B1  B0   .   .
GPU3:    .   .   .  F0  F1  F2  F3  F4  F5  F6  F7  B7  B6  B5  B4  B3  B2  B1  B0   .   .   .
```

- **Bubble Size**: The idle slots per stage are $K-1$ during fill and $K-1$ during drain, totaling $2(K-1)$ idle slots out of $2(K+M-1)$ total slots:

$$\text{Bubble Fraction} = \frac{K-1}{K+M-1}$$

- **The Catch**: All M micro-batch forwards must complete before the backward pass starts. Ranks must stash the activation tensors for **all** M **micro-batches** in HBM, which causes activation memory to scale linearly with M.

22.2.2 2. 1F1B Scheduling (Narayanan et al., 2021)

To prevent the activation memory bottleneck, **1F1B (One Forward, One Backward)** starts the backward pass as soon as the first micro-batch reaches the final stage. Ranks alternate between executing one forward and one backward pass.

```
1F1B schedule  (K=4 stages, M=8 micro-batches):
          0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20  21
GPU0:   F0  F1  F2  F3   .   .   .  B0  F4  B1  F5  B2  F6  B3  F7  B4   .  B5   .  B6   .  B7
GPU1:    .  F0  F1  F2   .   .  B0  F3  B1  F4  B2  F5  B3  F6  B4  F7  B5   .  B6   .  B7   .
GPU2:    .   .  F0  F1   .  B0  F2  B1  F3  B2  F4  B3  F5  B4  F6  B5  F7  B6   .  B7   .   .
```

```
GPU3:   .   .   .  F0  B0  F1  B1  F2  B2  F3  B3  F4  B4  F5  B5  F6  B6  F7  B7   .   .   .
```

- **Memory Bounding**: Stage k executes $K - k - 1$ warmup forward steps. Once the steady state is reached, every new forward step is paired with a backward step that releases stashed activations. The peak activation memory is bounded to K micro-batches (the number of stages), regardless of how large M gets.
- **Bubble Size**: The total execution time and bubble fraction are identical to GPipe. It is a pure memory optimization.

22.2.3 3. Interleaved 1F1B (Megatron-LM SC21)

To shrink the bubble, each node is assigned V **virtual stages**. For example, with $K = 4$ GPUs, $L = 16$ layers, and $V = 2$: - **Standard PP ($V = 1$)**: GPU0 gets layers 0-3, GPU1 gets 4-7, GPU2 gets 8-11, GPU3 gets 12-15. - **Interleaved PP ($V = 2$)**: GPU0 gets layers 0-1 and 8-9; GPU1 gets 2-3 and 10-11; GPU2 gets 4-5 and 12-13; GPU3 gets 6-7 and 14-15.

A micro-batch visits each device V times. Because each virtual stage contains $1/V$ of the layers, the wall-clock time of each forward/backward step shrinks by V.

$$\text{Interleaved Bubble Fraction} = \frac{K - 1}{K + M \cdot V - 1}$$

At $V = 2$, the bubble size is cut almost in half, but it requires $V - 1$ additional point-to-point sends per micro-batch per rank.

22.3 Worked Example

Below are exact values comparing schedules for a pipeline with $K = 4$ stages, $M = 8$ micro-batches, and varying virtual stages V:

22.3.1 1. Bubble & Memory Performance Table

K (Stages)	M (Micro-batches)	V (Virtual)	Bubble Fraction	Bubble %	GPipe Memory (Multiplier)	1F1B Memory (Multiplier)
4	1	1	0.7500	75.00%	1	4
4	4	1	0.4286	42.86%	4	4
4	8	1	0.2727	27.27%	8	4 *(GOLD Row)*
4	16	1	0.1579	15.79%	16	4
4	32	1	0.0857	8.57%	32	4
4	8	2	0.1579	15.79%	8	4
4	8	4	0.0857	8.57%	8	4
8	32	1	0.1795	17.95%	32	8
8	32	2	0.0986	9.86%	32	8

Verification Pins: - **GPipe vs. 1F1B Memory**: For $M = 8$, GPipe stores 8 activation sets per rank while 1F1B stores at most $K = 4$ sets (2.0× memory reduction). At $M = 32$, the memory reduction is 8.0×. - **Interleaved Bubble reduction**: For $K = 4, M = 8$, going from $V = 1$ to $V = 2$ shrinks the bubble fraction from 0.2727 (3/11) to 0.1579 (3/19) — a 0.58× **reduction**.

22.3.2 2. Point-to-Point (P2P) Communication Arithmetic

Unlike standard data parallelism, which uses `AllReduce`, PP relies on point-to-point `send`/`recv` operations. - Number of active stage boundaries: $K-1$. - For M micro-batches, the forward pass requires $(K-1) \cdot M$ transfers. - The backward pass requires another $(K-1) \cdot M$ transfers. - **Total transfers per step**: $2(K-1) \cdot M$.

For $K = 4$ stages, $M = 2$ micro-batches: - **Total P2P Transfers**: $2(4-1) \cdot 2 = 12$. - **Tensor Shape**: $[B_{\text{micro}}, L_{\text{slice}}, E]$ where micro-batch size $B_{\text{micro}} = 1$, sequence length $L_{\text{slice}} = 1,024$, model dimension $E = 8,192$, in FP16 (2 bytes). - **Single Transfer Volume**: $1 \cdot 1,024 \cdot 8,192 \cdot 2$ bytes $= 16,777,216$ bytes $=$ **16.00** MiB. - **Total Comm Volume per Mini-batch**: $12 \cdot 16.00$ MiB $=$ **0.188** GiB. - **Interconnect Tolerances**: Since this communication volume is small and occurs only at stage boundaries, it runs efficiently even over 25 GB/s InfiniBand links.

22.4 Complexity & Trade-offs

Metric	Value	Notes
Bubble Time Waste	$\frac{K-1}{K+M-1}$ (for $V = 1$)	Decreases as M increases. Target rule of thumb: $M \geq 4K$.
Peak Activation Memory	$K \times$ activations per micro-batch	Bounded by K under 1F1B, saving massive memory vs GPipe's M.
Communication Type	Point-to-Point `send`/`recv`	Low volume, latency-tolerant. Runs efficiently over InfiniBand.
Load Balancing Sensitivity	High	Unequal layers per stage or embeddings/heads can cause pipeline bottlenecks.

22.4.1 The Parallelism Trade-off Table

Axis	Collective	Frequency	Link Requirement
Tensor Parallelism (TP)	`AllReduce`	2× per **layer**	NVLink (≈ 300 GB/s)
Pipeline Parallelism (PP)	`P2P Send/Recv`	$2(K-1)$ per **mini-batch**	InfiniBand (≈ 25–50 GB/s)
Data Parallelism (DP)	`AllReduce`	1× per **optimizer step**	InfiniBand (≈ 25–50 GB/s)

22.5 Common Interview Questions & How to Answer

22.5.1 Q1: Why does 1F1B reduce activation memory compared to GPipe, and does it reduce total training time?

- **Answer**: 1F1B reduces activation memory by changing the execution schedule. In GPipe, all M micro-batch forward passes are executed before any backward passes start, requiring the GPU to store M sets of activations. 1F1B starts backward passes as soon as the first micro-batch reaches the final stage. After a brief warmup phase of $K-1$ steps, each rank alternates between running one forward and one backward pass. Because each backward pass releases the activations stashed by an earlier forward pass, the number of live activation sets is capped at K. However, **1F1B does not reduce total training time**. The total number of forward and backward passes is identical, and the pipeline bubble size remains $\frac{K-1}{K+M-1}$. It is purely a memory layout optimization.

22.5.2 Q2: What is the downside of using a very high virtual stage factor V in Interleaved 1F1B?

- **Answer**: While increasing V reduces the bubble fraction to $\frac{K-1}{K+M\cdot V-1}$, it has two primary downsides:
  1. **Increased Communication Frequency**: Each micro-batch must cross node boundaries $K \cdot V$ times instead of K times. This increases the total number of P2P network transfers by a factor of V. If the interconnect bandwidth is low or network latency is high, the communication overhead can exceed the time saved by shrinking the bubble.
  2. **Inter-Stage Communication Overhead**: Interleaved PP requires communicating both activations and gradients across virtual stages. This consumes extra GPU memory buffers to store incoming/outgoing tensors and complicates the execution scheduler.

22.5.3 Q3: How do we handle weight sharing between the embedding layer (stage 0) and the language model head (stage K-1)?

- **Answer**: In many GPT-style architectures, the input embedding weights and the output token projection weights are shared. Under Pipeline Parallelism, these layers are hosted on completely different physical nodes (Stage 0 and Stage $K-1$). We can handle this in two ways:
  1. **Explicit Synchronization**: The weights are duplicated on both stages. During the backward pass, Stage 0 and Stage $K-1$ compute gradients w.r.t. their local copy. Before the optimizer step, these gradients must be sent across the network and summed via a point-to-point communication hook.
  2. **Model Re-architecture**: Avoid weight sharing. Modern models (like Llama) do not share embedding and LM head weights, which avoids this cross-node synchronization bottleneck.

22.6 Pro-Tip: How to Impress the Interviewer

- **Dynamic Pipeline Balancing**: Explain how to address load imbalances caused by non-uniform layers (e.g., embeddings at Stage 0, output projection head at Stage $K-1$, or activation checkpointing

applied selectively). In production, you don't split layers evenly (e.g., 80/8 = 10 layers per stage). Instead, you run a profiling step and allocate fewer layers to the stages that host the embeddings, the output head, or run without activation checkpointing. This balances the compute time across all stages and minimizes the actual bubble size.

- **Explain PyTorch Pipelining API Integration**: Discuss how `torch.distributed.pipelining` (available in PyTorch $\geq$ 2.4) handles this under the hood. Show that you understand how to partition models on `meta` devices (avoiding loading the whole model into a single GPU's memory during initialization), wrapping the partitioned model in `PipelineStage`, and running it using predefined schedules like `ScheduleGPipe` or `Schedule1F1B`.

23 Zero Redundancy Optimizer (ZeRO)

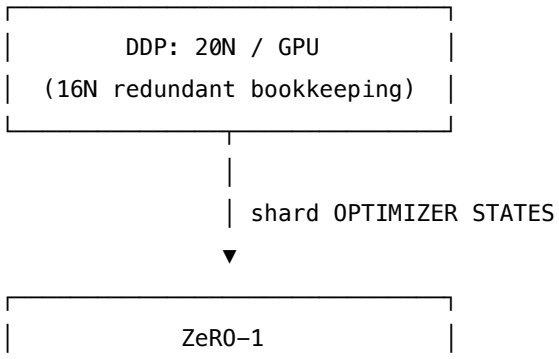
- **Category**: LLM Systems
- **Difficulty**: Expert
- **Target Role**: LLM Systems Engineer / Distributed Systems Engineer
- **Source**: ZeRO (Rajbhandari et al., 2019) / DeepSpeed
- **Flashcards**: LLM Systems deck

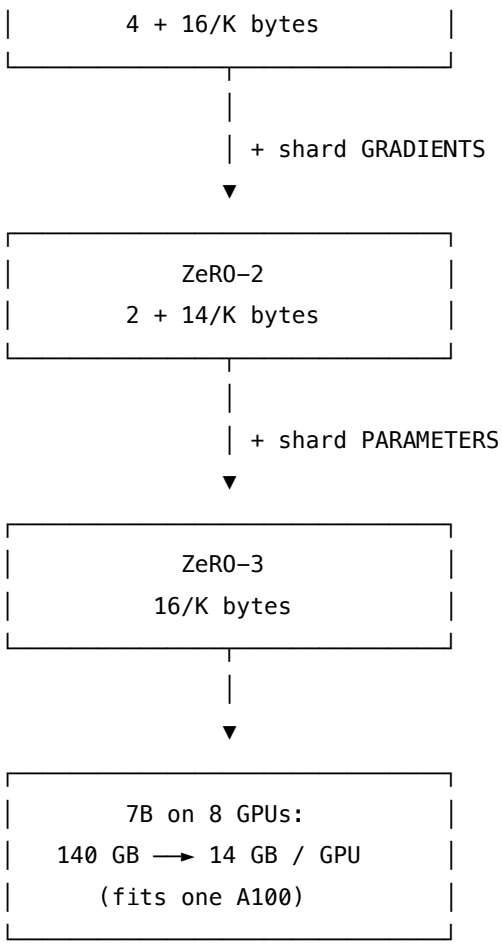
23.1 Concept Overview

Imagine you are a team of eight bookkeepers (GPUs) audit-logging a company's financial ledgers. In a standard setup (like DDP), every bookkeeper is handed an identical, full copy of the entire ledger. Each person has a copy of the master accounts, the daily logs, and the calculation sheets. Photocopying this massive ledger eight times is pure redundancy and wastes a huge amount of desk space (GPU HBM).

Instead, you decide to **shard the ledger**. You split the ledger into eight folders, giving one folder to each bookkeeper. Bookkeeper 0 handles accounts 0–9, Bookkeeper 1 handles accounts 10–19, and so on. During the audit, when a calculation needs the full ledger, you briefly pool the folders together on a table (an `AllGather` collective), perform the calculation, and then immediately return the folders to their respective owners. You do the exact same work, but your desk footprint is reduced to a fraction of the original size.

In LLM systems, the **Zero Redundancy Optimizer (ZeRO)** keeps the data-parallel training flow but shards the states that are normally duplicated across ranks: 1. **Stage 1**: Shards the optimizer states (Adam's master weights, momentum, and variance). 2. **Stage 2**: Also shards the gradients. 3. **Stage 3**: Also shards the model parameters (weights) themselves.





23.1.1 The Problem It Solves

Standard Data Parallelism (DDP) replicates the entire model, gradients, and optimizer states on every single GPU. In mixed-precision training (FP16 weights/gradients with FP32 Adam optimizer states), this bookkeeping creates a massive memory footprint:

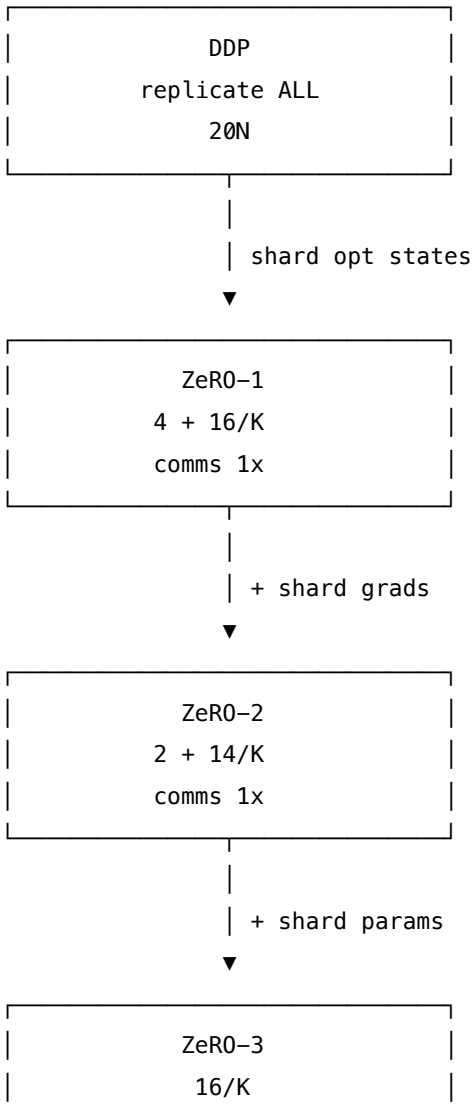
Component	Precision	Bytes per Parameter	Why
Parameters	FP16	2	Used in the forward and backward passes.
Gradients	FP16	2	Computed during the backward pass.
Master Parameters	FP32	4	Master copy updated by the optimizer to avoid underflow.
Grad Copy	FP32	4	Upcast FP16 gradients for optimizer calculations.
Adam Momentum	FP32	4	First moment (beta1 running average).

Component	Precision	Bytes per Parameter	Why
Adam Variance	FP32	4	Second moment (beta2 running average).
TOTAL	**Mixed**	20 **bytes**	Replicated on **every** data-parallel GPU.

Of these $20N$ bytes, $16N$ **bytes is identical bookkeeping** duplicated across all K ranks. For a 7B parameter model, standard DDP requires 140 **GB of memory per GPU** just to hold the static weights and optimizer states, which is too large to fit on a single 80 GB A100 or H100. ZeRO eliminates this redundancy.

23.2 How It Works

ZeRO shards the memory footprint incrementally across the K data-parallel ranks:



```
|          comms 1.5x           |
└──────────────────────────────┘
```

23.2.1 1. ZeRO-1: Optimizer State Partitioning (P_{os})

Ranks retain full copies of the FP16 parameters ($2N$) and gradients ($2N$). However, the four FP32 optimizer columns (master parameters, grad copy, momentum, and variance = $16N$ bytes) are split evenly. Each rank stores and updates only $1/K$ of the optimizer states. - **Footprint**: $2 + 2 + \frac{16}{K} = \mathbf{4 + \frac{16}{K}}$ bytes/parameter. - **Forward & Backward**: Identical to standard DDP. - **Optimizer Step**: Each rank updates its own $1/K$ parameter shard. An `AllGather` collective is then executed to broadcast the updated parameters to all other ranks.

23.2.2 2. ZeRO-2: Gradient Partitioning (P_g)

Along with the optimizer states, the FP16 gradients are sharded. During the backward pass, as soon as a layer's gradient is computed, it is sent to its owner rank using a `ReduceScatter` collective and immediately deleted on all other ranks. - **Footprint**: $2 + \frac{2+12}{K} = \mathbf{2 + \frac{14}{K}}$ bytes/parameter (the $4N$ FP32 grad copy column is eliminated since the optimizer upcasts the local FP16 gradient shard transiently). - **Optimizer Step**: Each rank updates its local shard of parameters using its local gradient shard. The updated weights are distributed via `AllGather`.

23.2.3 3. ZeRO-3: Parameter Partitioning (P_p)

Even the persistent FP16 parameters are sharded. Each GPU only stores a $1/K$ slice of the model weights. - **Footprint**: $\frac{2+2+12}{K} = \mathbf{\frac{16}{K}}$ bytes/parameter (at large K, the memory requirement drops to near zero). - **Forward Pass**: Before a layer runs, ranks execute an `AllGather` to reconstruct the full weights for that layer. The forward pass is run, and the non-owned weights are discarded immediately. - **Backward Pass**: The weights are gathered again via `AllGather` to compute the gradients. After compute, they are discarded, and the gradients are `ReduceScattered` to their owners.

23.3 Worked Example

Below are exact values comparing standard DDP and the three ZeRO stages:

23.3.1 1. Memory Reduction Scaling (FP16/FP32 Adam)

23.3.1.1 Tiny Gold Benchmark ($N = 1,000,000$ parameters, $K = 4$ GPUs)

- **DDP**: 20 bytes/param $\cdot\ 10^6 = \mathbf{20.00}$ MB
- **ZeRO-1**: $(4 + 16/4)$ bytes/param $\cdot\ 10^6 = \mathbf{8.00}$ MB
- **ZeRO-2**: $(2 + 14/4)$ bytes/param $\cdot\ 10^6 = \mathbf{5.50}$ MB
- **ZeRO-3**: $(16/4)$ bytes/param $\cdot\ 10^6 = \mathbf{4.00}$ MB

23.3.1.2 Production Scale ($N = 7$B parameters, $K = 8$ GPUs)

- **DDP**: 20 bytes/param $\cdot\ 7 \times 10^9 = \mathbf{140.00}$ GB (exceeds an 80 GB GPU)

- **ZeRO-1**: $(4 + 16/8)$ bytes/param $\cdot\, 7 \times 10^9 = \mathbf{42.00}$ GB
- **ZeRO-2**: $(2 + 14/8)$ bytes/param $\cdot\, 7 \times 10^9 = \mathbf{26.25}$ GB
- **ZeRO-3**: $(16/8)$ bytes/param $\cdot\, 7 \times 10^9 = \mathbf{14.00}$ GB (fits comfortably on an 80 GB A100/H100)

23.3.2 2. A Step Simulation of ZeRO-1

Using $N = 8$ parameters, $K = 4$ ranks (each rank owns a chunk of 2 parameters), learning rate $\eta = 0.1$: - **Initial parameters (replicated on all ranks)**:

$$P_{init} = [+0.7705, -0.1467, -1.0894, +0.2842, -0.5423, -0.6993, +0.2017, +0.4190]$$

- **Local gradients computed by each rank during backward**: - Rank 0: `[-0.2158, -0.1210, -0.1790, +0.0546, -0.2570, +0.3302, -0.3214, +0.0368]` - Rank 1: `[-0.1699, +0.1119, -0.2676, -0.4527, +0.1111, +0.4370, +0.2819, +0.2325]` - Rank 2: `[+0.0576, +0.3791, -0.3871, -0.2373, -0.0063, -0.2155, +0.1556, -0.3938]` - Rank 3: `[+0.0576, +0.1628, -0.6656, +0.0777, -0.3089, -0.1502, +0.0820, -0.2754]`

- **Step 1: `ReduceScatter(grads)`** Ranks sum and average the gradients for their assigned parameter index:
  - Rank 0 owns $[0 : 2]$: avg_grad $= [-0.0676, +0.1332]$
  - Rank 1 owns $[2 : 4]$: avg_grad $= [-0.3748, -0.1394]$
  - Rank 2 owns $[4 : 6]$: avg_grad $= [-0.1153, +0.1003]$
  - Rank 3 owns $[6 : 8]$: avg_grad $= [+0.0495, -0.1000]$
- **Step 2: Local Optimizer Update** Each rank updates its assigned parameter shard using its local FP32 optimizer state:
  - Rank 0 updates $[0 : 2]$: $P_{new}[0 : 2] = [+0.7705, -0.1467] - 0.1 \cdot [-0.0676, +0.1332] = [+0.7773, -0.1600]$
  - Rank 1 updates $[2 : 4]$: $P_{new}[2 : 4] = [-1.0894, +0.2842] - 0.1 \cdot [-0.3748, -0.1394] = [-1.0519, +0.2982]$
  - Rank 2 updates $[4 : 6]$: $P_{new}[4 : 6] = [-0.5423, -0.6993] - 0.1 \cdot [-0.1153, +0.1003] = [-0.5307, -0.7093]$
  - Rank 3 updates $[6 : 8]$: $P_{new}[6 : 8] = [+0.2017, +0.4190] - 0.1 \cdot [+0.0495, -0.1000] = [+0.1967, +0.4290]$
- **Step 3: `AllGather(params)`** Ranks broadcast their updated shards to reconstruct the full parameter vector:

$$P_{final} = [+0.7773, -0.1600, -1.0519, +0.2982, -0.5307, -0.7093, +0.1967, +0.4290]$$

All ranks hold identical updated parameters, matching the mathematical output of standard DDP.

23.4 Complexity & Trade-offs

Strategy	Collective Per Step	Per-Device Comm Volume	Comm Overhead	Memory Footprint (7B, K=8)
DDP	`AllReduce(grad)`	$2.0 \cdot \Psi$	1.00×	140 GB (redundant)
ZeRO-1	`ReduceScatter(grad)` + `AllGather(param)`	$2.0 \cdot \Psi$	1.00×	42 GB
ZeRO-2	`ReduceScatter(grad)` + `AllGather(param)`	$2.0 \cdot \Psi$	1.00×	26.25 GB
ZeRO-3	`AG(param fwd)` + `AG(param bwd)` + `RS(grad)`	$3.0 \cdot \Psi$	1.50×	14 GB

(Note: Ψ represents the number of model parameters.)

23.4.1 The Communication Volume Identity

Why do ZeRO-1 and ZeRO-2 have the same communication volume as DDP? It comes down to the NCCL collective identity:

$$\text{AllReduce(tensor)} \equiv \text{ReduceScatter(tensor)} \rightarrow \text{AllGather(tensor)}$$

DDP sums gradients using a single `AllReduce(grad)` of size Ψ. ZeRO-1 and ZeRO-2 split this into `ReduceScatter(grad)` (size Ψ) and `AllGather(param)` (size Ψ). The volume is identical ($2 \cdot \Psi$ bytes transferred per rank). **ZeRO-1 and ZeRO-2 are free memory optimizations that incur no extra network traffic.**

ZeRO-3 adds one extra `AllGather` step. Because parameters are sharded, they must be gathered once for the forward pass and gathered again for the backward pass. This raises the total volume to $3 \cdot \Psi$, representing a 1.5× **increase (a 50% communication tax)**.

23.5 Common Interview Questions & How to Answer

23.5.1 Q1: Does ZeRO-3 increase the communication volume by 3× compared to standard DDP?

- **Answer**: No. While ZeRO-3 uses **three collective operations** per step (AllGather for weights in forward, AllGather for weights in backward, and ReduceScatter for gradients) compared to DDP's **one collective operation** (`AllReduce` for gradients), the actual **communication volume is only** 1.5× **DDP**. A standard ring-AllReduce transfers $2 \cdot \Psi$ bytes of data per rank. In ZeRO-3, the three collectives (`ReduceScatter` and two `AllGather`s) each transfer $1 \cdot \Psi$ bytes per rank. The total data transferred is $3 \cdot \Psi$ bytes, which is a 50% increase over DDP's $2 \cdot \Psi$ bytes, not a 3× increase.

23.5.2 Q2: How does ZeRO-3 hide the latency of its extra AllGather collectives?

- **Answer**: ZeRO-3 hides the communication overhead using **parameter prefetching** and **overlap communication**. While the GPU is computing the forward pass of layer l, the system issues an asynchronous `AllGather` command to fetch the weights for layer $l+1$ in a separate CUDA stream. By the time layer l completes, the weights for $l+1$ are already present in HBM. During the backward pass, the system prefetches the weights for $l-1$ while computing the gradients for layer l, and streams the `ReduceScatter` of layer l's gradients in the background.

23.5.3 Q3: What is the difference between ZeRO-Offload and ZeRO-Infinity?

- **Answer**: Both are extensions of ZeRO that offload states to host CPU memory or NVMe drives, trading PCIe bandwidth for GPU HBM:
  1. **ZeRO-Offload (ZeRO-2 base)**: Offloads the $16N$ bytes of optimizer states and gradients to host CPU RAM. The optimizer update is executed on the CPU, and the updated weights are copied back to the GPU. This is optimal for single-node systems.
  2. **ZeRO-Infinity (ZeRO-3 base)**: Offloads sharded parameters, gradients, and optimizer states to host CPU RAM or NVMe storage. It uses NVMe-to-CPU and CPU-to-GPU pipelining to support training models with trillions of parameters that exceed the cluster's aggregate GPU memory.

23.6 Pro-Tip: How to Impress the Interviewer

- **FSDP (Fully Sharded Data Parallel) vs. DeepSpeed ZeRO-3**: Show you know the ecosystem by explaining that PyTorch's native **FSDP** is a production implementation of the ZeRO-3 protocol. Contrast their approaches: FSDP wraps modules into units (`FullyShardedDataParallel` blocks) and treats a set of parameters as a single flat tensor (`FlatParameter`), which allows it to run a single, contiguous `AllGather` per block. This is cleaner and often faster than DeepSpeed's hook-based sharding of individual parameter tensors.
- **Explain Checkpoint Consolidation**: Highlight the operational challenges of ZeRO-3. Because weights are sharded across K ranks, saving a checkpoint directly results in K sharded files. If you change the number of GPUs (K) in the next run, the checkpoint cannot be loaded. To resolve this, you must run a consolidation script (e.g., DeepSpeed's `zero_to_fp32.py`) to merge the K shards back into a single unified PyTorch `state_dict`.

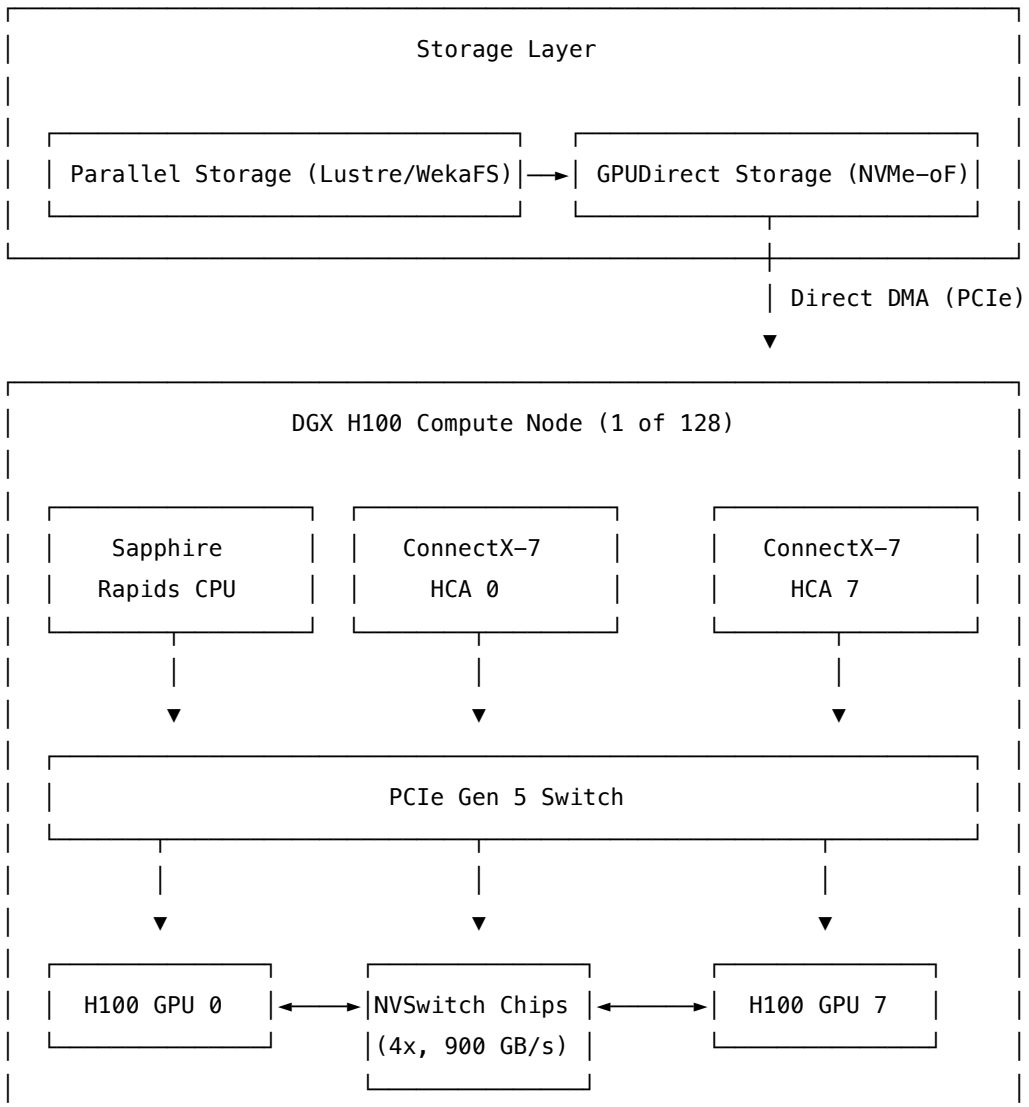
24 Distributed GPU Training

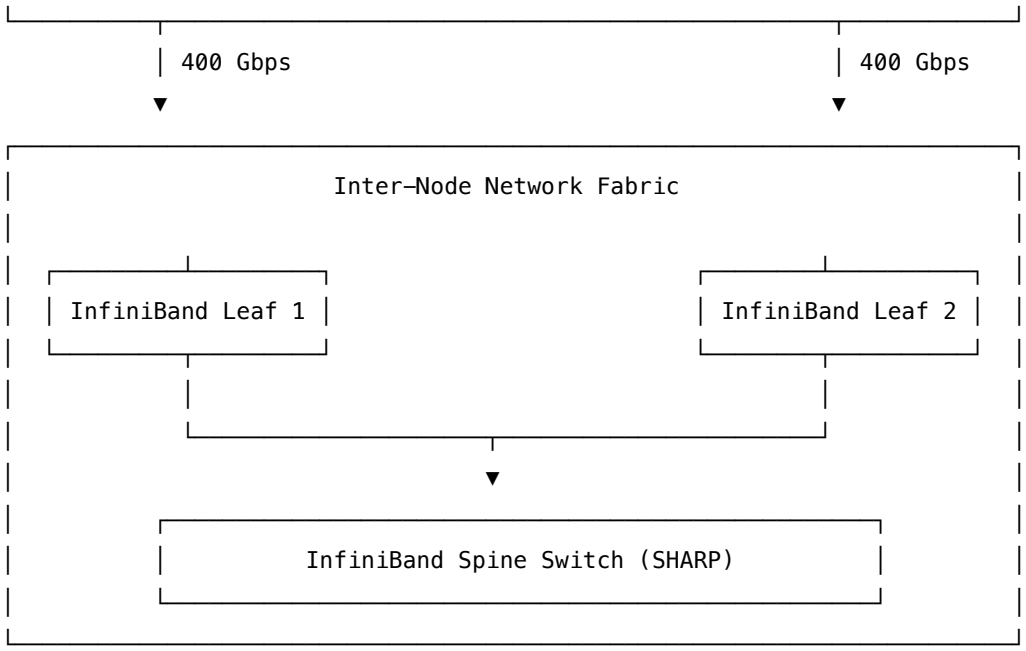
- **Category**: LLM Systems
- **Difficulty**: Hard
- **Target Role**: LLM Systems Engineer / Distributed Systems Engineer / Platform Architect
- **Source**: Megatron-LM / DeepSpeed / PyTorch FSDP (NVIDIA H100 Architecture)
- **Flashcards**: LLM Systems deck

24.1 Concept Overview

Think of training a trillion-parameter Large Language Model (LLM) as coordinating a massive construction project to build a skyscraper using a crew of 1,024 crane operators (GPUs). You cannot fit the entire blueprint (model parameters and optimizer states) onto a single desk (GPU memory). Instead, you split the blueprint into sections and assign them to teams of workers. To succeed, the crane operators must be connected by high-speed walkie-talkies (the network fabric) that let them communicate without delays. Furthermore, you need a conveyor belt (GPUDirect Storage) that streams concrete (datasets) directly to the site without intermediate unloading, and a system to replace any crane operator who falls ill (fault tolerance) within minutes to prevent the whole structure from collapsing.

In ML systems, **Distributed GPU Training** refers to the infrastructure and system design required to scale LLM training across multi-node, multi-GPU clusters. This involves three planes of operation: 1. **Control & Orchestration Plane**: Allocates compute resources and schedules tasks based on physical network topology. 2. **Data Ingestion Plane**: Streams training datasets directly from storage to GPU memory. 3. **Compute & Communication Plane**: Executes forward/backward math and synchronizes parameters using high-performance network collectives.





24.1.1 The Problem It Solves

When scaling training to $1,024\times$ NVIDIA H100 GPUs (128× DGX H100 nodes), conventional training setups break down: - **Memory Footprint**: A 1-Trillion Parameter model requires 16 **TB** of static memory in mixed-precision training (weights: 2 TB, gradients: 2 TB, Adam states: 12 TB). This is far beyond the 80 GB capacity of a single GPU. - **Communication Bottlenecks**: Standard Ethernet or PCIe communication speeds are too slow to sync weights at high frequencies, leaving GPUs starved for work. - **Storage Bottlenecks**: Standard IO pipelines copy data from NVMe storage to the host OS Page Cache, then to host CPU memory, and finally to GPU memory. This saturates host CPU cores and limits throughput. - **Hardware Failures**: In a cluster with 1,024 GPUs, the Mean Time Between Failures (MTBF) is measured in days. A single GPU, PCIe link, or network switch failure will crash the entire synchronous training job if not mitigated.

24.2 Worked Example

Let's design a distributed training platform to train a **1-Trillion Parameter Model ($N = 10^{12}$)** across **1,024 NVIDIA H100 SXM5 GPUs** (128× DGX H100 nodes).

24.2.1 1. Static Memory Budget Calculation

- **Model Parameters (FP16)**: $2 \cdot N = \mathbf{2}$ TB
- **Gradients (FP16)**: $2 \cdot N = \mathbf{2}$ TB
- **Optimizer States (FP32 Adam)**: $12 \cdot N = \mathbf{12}$ TB (master weights: 4 TB, momentum: 4 TB, variance: 4 TB)
- **Total Static Model Memory**: **16** TB (excludes dynamic activation memory)

24.2.2 2. 3D Parallelism + ZeRO-1 Configuration

To distribute the 16 TB static memory, we implement a hybrid parallelism strategy: 1. **Tensor Parallelism (TP)**: Set $TP = 8$. All 8 GPUs inside a single DGX H100 node form a TP group, sharding matrices over the high-speed NVLink fabric. 2. **Pipeline Parallelism (PP)**: Set $PP = 16$. The model's layers are split sequentially across 16 nodes. 3. **Data Parallelism (DP) with ZeRO-1**: Set $DP = 8$ (calculated as $\frac{1{,}024 \text{ total GPUs}}{TP \cdot PP} = 8$). ZeRO-1 shards the 12 TB of optimizer states across the 8 data-parallel replicas.

24.2.2.1 Sharded Memory Math per GPU:

- **Layers per Node**: $L_{total}/PP = L/16$.
- **Parameters per Node**: $N/PP = 10^{12}/16 = \mathbf{62.5}$ Billion parameters per node.
- **Parameters per GPU (after $TP = 8$)**: 62.5 B/8 = **7.8125** Billion parameters.
- **Weight Memory per GPU (FP16)**: 7.8125 B $\cdot$ 2 bytes = **15.625** GB.
- **Gradient Memory per GPU (FP16)**: 7.8125 B $\cdot$ 2 bytes = **15.625** GB.
- **Optimizer States per GPU (sharded by $DP = 8$ via ZeRO-1)**:

$$\text{Optimizer State per GPU} = \frac{7.8125 \text{ B} \cdot 12 \text{ bytes}}{8 \text{ DP ranks}} = 11.719 \text{ GB}$$

- **Total Static Memory per GPU**: 15.625 GB + 15.625 GB + 11.719 GB = **42.969** GB.
- This easily fits inside the 80 GB capacity of an H100 GPU, leaving 37 GB for dynamic activation memory and the KV cache.

24.2.3 3. Ingestion & Storage Bandwidth

- **GPUDirect Storage (GDS)**: We bypass CPU memory and the host Page Cache by linking Mellanox ConnectX-7 network cards directly to the GPU HBM3 over the PCIe Gen5 switch.
- This achieves a read bandwidth of 215 **GB/s** per DGX node, ensuring that data ingestion never bottlenecks the GPU compute.

24.3 Complexity & Trade-offs

Metric	Target Value	System Trade-offs & Notes
Intra-Node Bandwidth	900 GB/s (bi-directional)	Delivered by 18 NVLinks per H100 SXM5. Optimal for low-latency TP.
Inter-Node Bandwidth	400 Gbps (ConnectX-7)	Fat-tree topology (1 HCA per GPU). Backed by InfiniBand Quantum-2 switches.
Ingestion Read Bandwidth	215 GB/s	Enabled by GPUDirect Storage (GDS). Avoids host CPU bottlenecks.

Metric	Target Value	System Trade-offs & Notes
Model FLOPs Utilization (MFU)	$> 50\%$	MFU of $\sim 52\%$ is targeted using FlashAttention-3 and FP8 precision.
Fault Recovery Downtime	< 5 minutes	Automated health checks, job rescheduling, and NVMe-backed checkpointing.

24.4 Failure Mode and Effects Analysis (FMEA)

Failure Mode	Root Cause	Impact	Mitigation Strategy
Straggler Node	GPU thermal throttling or PCIe lane degradation (e.g. falling from Gen5 to Gen1).	The entire cluster slows down to match the speed of the slowest node during synchronous collectives.	Host agents monitor node health (e.g. via `dcgmPMGetMetrics`). If clock speeds drop or PCIe bandwidth falls below 50 GB/s, the scheduler marks the node as unhealthy, drains it, and replaces it.
Network Link Degradation	Loose transceivers or packet drop on inter-node fabric.	The NCCL communication ring collapses, causing training to timeout and crash.	Enable adaptive routing on the Quantum-2 switches. If error rates exceed a threshold, NCCL dynamically routes around the degraded link via alternative paths.
Silent Data Corruption (SDC)	Hardware faults in Tensor Cores or cosmic ray bit flips.	Loss values diverge to `NaN` or model weights degrade silently, wasting compute time.	Run periodic deterministic test steps. Implement real-time gradient checks: if $\|\|\nabla w\|\|_2 >$ threshold, pause training and run hardware diagnostics.

Failure Mode	Root Cause	Impact	Mitigation Strategy
Checkpoint Storage Congestion	1,024 GPUs writing 16 TB of data to Lustre simultaneously.	Storage network becomes saturated, halting training for 30–45 minutes.	**Double-Buffered Host Memory Staging**: Write checkpoints to local node NVMe SSDs (≈ 2.5 GB/s per drive, taking < 15 seconds). Resume training immediately while a background service streams the data to Lustre.

24.5 Common Interview Questions & How to Answer

24.5.1 Q1: Explain NCCL Ring vs. Tree collective algorithms and how the library chooses between them.

- **Answer**:
  - **Ring Algorithm**: Tensors are split into chunks and circulated around a logical ring of GPUs. The data volume transferred per rank for an `AllReduce` is $2 \cdot \frac{K-1}{K} \cdot S$, where S is the tensor size and K is the number of GPUs. This algorithm is bandwidth-efficient, making it optimal for **large tensors**, but it has higher latency because it scales linearly with K.
  - **Tree Algorithm**: Tensors are reduced up a double-binary tree and then broadcast back down. The latency scales logarithmically ($\log K$), making it optimal for **small tensors** or when the node count K is very large.
  - NCCL measures cluster latencies during startup and dynamically switches from Tree to Ring once the tensor size exceeds a specific threshold (typically around 16 MB).

24.5.2 Q2: What happens when packet loss occurs on a RoCE v2 cluster vs. an InfiniBand cluster?

- **Answer**:
  - **InfiniBand**: Uses credit-based flow control at the link layer. A sender only transmits packets when the receiver has buffer space. This prevents packet loss from buffer overflows.
  - **RoCE v2**: Relies on Ethernet, which is lossy. To make it lossless, you must configure **PFC (Priority Flow Control)** to send PAUSE frames when switch buffers fill, and **ECN (Explicit Congestion Notification)** to mark packets when congestion is detected, allowing senders to throttle their transmission rates. If PFC is misconfigured, packet drops will trigger Go-Back-N retransmissions at the transport layer, causing NCCL timeouts and slashing training performance to near-zero.

24.5.3 Q3: How do you optimize memory consumption during LLM training without offloading to CPU?

- **Answer**:
  1. **Activation Checkpointing (Recomputation)**: Discard activations computed during the forward pass and recalculate them during the backward pass. This reduces activation memory from $\mathcal{O}(L)$ (where L is the number of layers) to $\mathcal{O}(\sqrt{L})$ at the cost of 33% extra compute.
  2. **FlashAttention-3**: Computes attention on-chip in GPU SRAM without materializing the intermediate $S \times S$ attention matrix to global HBM3, saving up to 80% of attention activation memory.
  3. **Sequence Parallelism**: Extends Tensor Parallelism by splitting activations along the sequence length dimension for layers that are not split by standard TP (such as LayerNorm and Dropout).

24.6 Pro-Tip: How to Impress the Interviewer

- **Maximize MFU with FlashAttention-3 & FP8 precision**: Show you understand hardware optimizations. Explain how you would combine **FlashAttention-3** (which overlaps softmax and matmuls inside the GPU's Tensor Cores) and **FP8 Precision Scaling** (which uses 8-bit floats for forwards and backwards while maintaining FP16 master copies) to push Model FLOPs Utilization (MFU) past the typical 35% threshold up to 52%.
- **Deep-Dive into NCCL Network Tuning**: Demonstrate familiarity with network configuration by citing specific tuning parameters. Detail how setting `NCCL_CROSS_NIC=1` enables multi-HCA routing to balance traffic across cards, setting `NCCL_BUFFSIZE=4194304` (sets the ring buffer size to 4 MB) avoids bottlenecks in high-bandwidth channels, and setting `NCCL_COLLNET_ENABLE=1` offloads collective aggregation to Quantum-2 switches using SHARP.

25 Mixture-of-Experts (MoE) Routing

- **Category**: LLM Systems
- **Difficulty**: Expert
- **Target Role**: LLM Inference Architect / ML Platform Engineer
- **Source**: Sparsely-Gated MoE (Shazeer et al., 2017) / GShard (Lepikhin et al., 2020) / Switch Transformer (Fedus et al., 2022) / Mixtral (Jiang et al., 2024) / DeepSeek-V3 (DeepSeek-AI, 2024)
- **Flashcards**: LLM Systems deck

25.1 Concept Overview

Mixture-of-Experts (MoE) is like a consulting firm consisting of a receptionist (the **router**) and a wall of specialists (the **experts**). Instead of forcing every employee to read and work on every single incoming document, the receptionist scans the document and routes it to only the top-k most relevant specialists.

In deep learning architectures, a standard dense FFN activates all its parameters for every token, meaning that scaling knowledge capacity requires a proportional scale-up in per-token compute. MoE breaks this

lock: it replaces the FFN block with a router and E expert MLPs. By routing each token to only the top-k experts ($k \ll E$), total model capacity (knowledge) scales with E, but the active per-token compute (FLOPs) remains bounded by the small k experts. For example, a model can have 671 billion parameters of total capacity but run at the latency and compute cost of a 37 billion parameter dense model.

25.1.1 The Problem It Solves

In a standard Transformer, parameters reside heavily in the FFN layer. Making this FFN wider to increase the model's knowledge capacity directly scales the per-token computational cost (FLOPs) and latency. - **Without MoE**: A dense SwiGLU FFN with hidden size $D = 8$ and intermediate size $F = 16$ requires **384 parameters** per token, translating to **768 FLOPs/token** (excluding attention). - **With MoE ($E = 4, k = 2$)**: The model stores **1536 parameters** (4× the capacity), but computes only **768 active parameters** and **1536 FLOPs/token**.

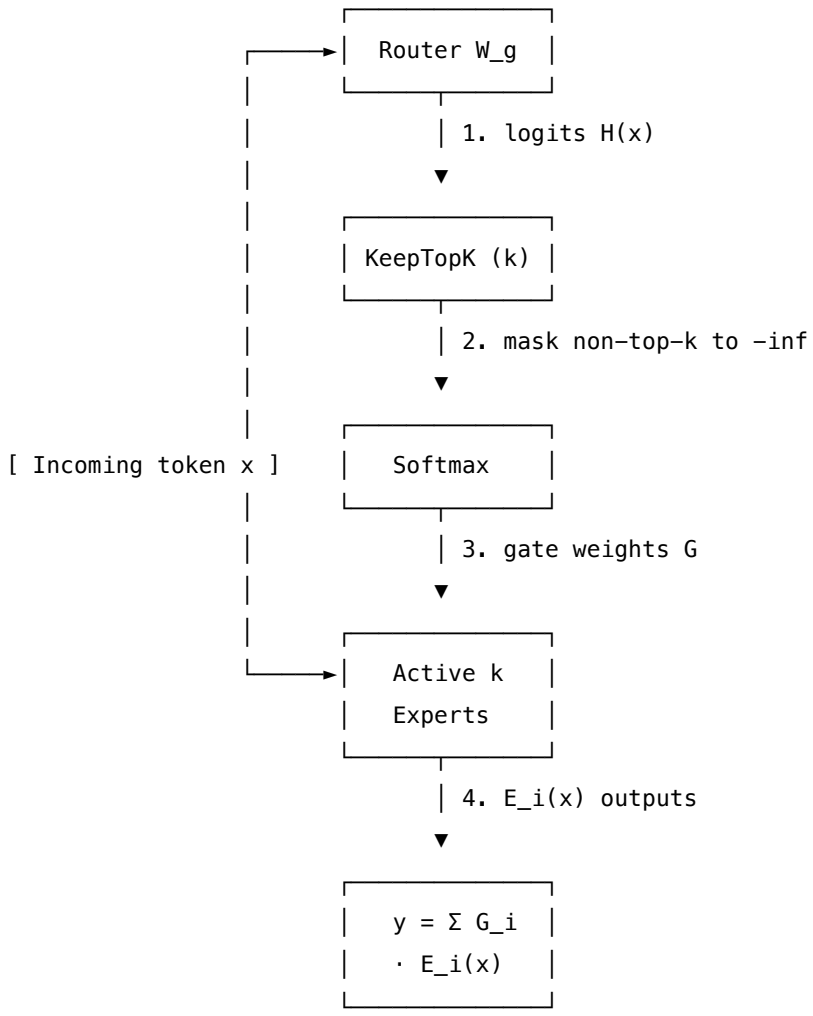
On a production scale, this parameter-to-FLOPs decoupling is massive: - **Mixtral 8x7B**: Holds **46.7B total parameters**, but only **12.9B parameters are active** per token. - **DeepSeek-V3**: Holds **671B total parameters**, but only **37B parameters are active** per token.

25.1.2 How It Works

1. **Logit Computation**: The router (a linear layer $W_g \in \mathbb{R}^{E \times D}$) scores each expert for an input token $x \in \mathbb{R}^D$:
$$H(x) = x \cdot W_g^T \in \mathbb{R}^E$$
2. **KeepTopK Sparsification**: All logits except the top-k largest values are set to $-\infty$.
3. **Renormalized Softmax**: A softmax is applied over the sparsified logits to produce gating weights $G(x)_i$:
$$G(x) = \text{softmax}(\text{KeepTopK}(H(x), k))$$
This ensures that inactive experts receive a gating weight of exactly 0, and the k active experts' weights sum to 1.
4. **Expert Computation & Output Synthesis**: The token is forwarded through only the chosen k experts (each is a SwiGLU MLP: $E_i(x) = \text{down}(\text{silu}(\text{gate}(x)) \odot \text{up}(x))$). The output is the weighted sum:
$$y = \sum_{i \in \text{active}} G(x)_i \cdot E_i(x)$$
5. **Load Balancing**: Without a penalty, routers suffer from **router collapse**, where they send all tokens to 1–2 favorite experts, leaving the others untrained. To prevent this, two auxiliary losses are optimized during training:
   - **Load-balance loss (L_{bal})**: Penalizes uneven distribution of hard routing counts (f_i) and soft gate probabilities (P_i):
$$L_{bal} = E \cdot \sum_{i=1}^{E} f_i \cdot P_i \quad (\text{minimum } = k \text{ when balanced})$$
   - **Router z-loss (L_z)**: Penalizes large router logits to prevent softmax saturation and stabilize

FP8/FP16 training:

$$L_z = \frac{1}{N}\sum_{j=1}^{N}(\log\sum_{i=1}^{E} e^{H(x_j)_i})^2$$



25.2 Worked Example

This example walks through routing for a model with hidden dimension $D = 8$, expert intermediate dimension $F = 16$, $E = 4$ total experts, and $k = 2$ active experts per token.

25.2.1 1. Top-k Routing Step-by-Step (Token $m = 0$)

- **Input Hidden State**:

$$x[0,0] = [+0.9635, +0.7436, +0.4504, -1.0528, +0.3392, -0.6173, -0.0215, -0.8023]$$

- **Raw Router Logits** ($H[0,0] = x[0,0] \cdot W_g^T$):

$$H[0,0] = [+0.0952, +0.3465, -0.0658, -0.0547]$$

- **Top-2 Selection**:

$$\text{Values} = [+0.3465, +0.0952] \quad \rightarrow \quad \text{Experts} = [1, 0]$$

- **KeepTopK Masking** (set non-top-2 logits to $-\infty$):

$$\text{KeepTopK}[0, 0] = [+0.0952, +0.3465, -\infty, -\infty]$$

- **Renormalized Softmax Gating Weights** ($G[0, 0]$):

$$G[0, 0] = [+0.4375, +0.5625, 0.0000, 0.0000] \quad \text{(sums to 1.0000)}$$

25.2.2 2. Router Decisions Across 4 Tokens ($N = 4$)

Token (m)	Logits ($H[0, m]$)	Chosen Experts	Renormalized Gate Weights ($G[0, m]$)
0	`+0.095, +0.347, -0.066, -0.055`	**1, 0**	`+0.4375, +0.5625, +0.0000, +0.0000`
1	`+0.141, +0.232, +0.376, +0.061`	**2, 1**	`+0.0000, +0.4640, +0.5360, +0.0000`
2	`+0.011, -0.042, -0.371, +0.222`	**3, 0**	`+0.4474, +0.0000, +0.0000, +0.5526`
3	`+0.130, +0.052, -0.142, +0.132`	**3, 0**	`+0.5004, +0.0000, +0.0000, +0.4996`

25.2.3 3. Output Combination (Token $m = 0$)

The token is routed only to Experts 0 and 1: - $E_0(x[0, 0]) = [+0.010, -0.012, -0.004, +0.006, \dots]$ - $E_1(x[0, 0]) = [+0.007, +0.006, -0.008, -0.012, \dots]$ - Outputs of Expert 2 and 3 are ignored since their gate weights are 0. - **Combined Output**:

$$y[0, 0] = 0.5625 \cdot E_1(x) + 0.4375 \cdot E_0(x) = [+0.008519, -0.001960, -0.006698, -0.004337, -0.006180, -0.000598, -0.0014$$

25.2.4 4. Load Balance Loss Calculation

From the routing decisions above: - **Routing Fraction (f_i)**: Fraction of total selections ($N \cdot k = 8$) routed to expert i: - $f_0 = 3/4 = 0.7500$ - $f_1 = 2/4 = 0.5000$ - $f_2 = 1/4 = 0.2500$ - $f_3 = 2/4 = 0.5000$ - Check: $\sum_i f_i = 2.0 = k$. - **Average Gate Probability (P_i)**: - $P_0 = 0.3461$, $P_1 = 0.2566$, $P_2 = 0.1340$, $P_3 = 0.2632$. (Check: $\sum_i P_i = 1.0$) - **Load-balance Loss (L_{bal})**:

$$L_{\text{bal}} = 4 \cdot (0.75 \cdot 0.3461 + 0.5 \cdot 0.2566 + 0.25 \cdot 0.1340 + 0.5 \cdot 0.2632) = 4 \cdot 0.553034 = 2.212137$$

Since $L_{\text{bal}} = 2.212 > 2.0$, the router is slightly imbalanced, providing gradients to redirect training. - **Router z-loss ($L_{\mathbf{z}}$)**:

$$L_{\text{z}} = 2.164589$$

25.2.5 5. Token Dropping under Expert Capacity C

When running distributed training using **Expert Parallelism (EP)**, experts reside on different GPUs. GPUs allocate static buffers using an **Expert Capacity** factor to cap memory and transfer size:

$$C = \text{capacity\_factor} \cdot \frac{N \cdot k}{E}$$

For capacity_factor $= 1.0$, $N = 4$, $k = 2$, $E = 4$, we get $C = 2.0$. - Expert 0 receives 3 tokens (t_0, t_2, t_3) $\rightarrow$ **drops 1 token** (only 2 run; dropped token passes through residual only). - Expert 1 receives 2 tokens $(t_0, t_1) \rightarrow$ exact fit. - Expert 2 receives 1 token $(t_1) \rightarrow$ **padded with 1 zero**. - Expert 3 receives 2 tokens $(t_2, t_3) \rightarrow$ exact fit.

Note: Dropping tokens degrades performance. Production serving engines (like vLLM) run with dynamic capacity / no-drop policies.

25.3 Complexity & Trade-offs

Metric	Complexity / Value	Notes
Router Time Complexity	$\mathcal{O}(B \cdot L \cdot E \cdot D)$	Linear projection of hidden states to E logits.
Sparsification Complexity	$\mathcal{O}(B \cdot L \cdot E \log k)$	Finding the top-k elements using a min-heap.
All-to-All Communication	$\mathcal{O}(\text{EP size})$	Bottleneck in distributed systems; mitigated by overlapping communication with compute.
Grouped GEMM	$\mathcal{O}(\sum M_i \cdot D \cdot F)$	Avoids padding execution buffers to $\max(M_i)$, speeding up training/inference.

25.4 Common Interview Questions & How to Answer

25.4.1 Q1: What is "router collapse," and how do load-balancing loss and DeepSeek-V3's auxiliary-loss-free bias prevent it?

- **Answer**: **Router collapse** is a common MoE failure mode where the routing network converges to selecting a small subset of experts (often just 1 or 2) for all tokens. As a result, only those few experts receive gradients and train, while the other experts remain idle and unoptimized.
    1. **Auxiliary Load-Balancing Loss (L_{bal})**: It computes the product of the routing fraction f_i (hard routing count) and the average gate probability P_i (soft probability). This product is differentiable through P_i. Minimizing L_{bal} pushes the router toward routing tokens uniformly across all E experts.

2. **DeepSeek-V3's Aux-Loss-Free Bias**: A high L_{bal} coefficient can compete with the main language modeling loss, degrading model quality. DeepSeek-V3 solves this by keeping the model weights' gradients focused entirely on the main loss. Instead, it adds a learnable bias term b_i directly to the logits: $G(x)_i = \text{softmax}(H(x)_i + b_i)$. If an expert becomes overloaded during a step, its bias b_i is decremented; if it is starved, its bias is incremented. This dynamically balances the load with **zero gradient interference**.

25.4.2 Q2: Why is Grouped GEMM essential for Mixture-of-Experts inference, and what problem does it solve?

- **Answer**: During MoE execution, different tokens are routed to different experts, leading to uneven workload distributions where expert i receives M_i tokens. Standard batch matrix multiplication (`torch.bmm`) requires uniform dimensions across all batches. Without Grouped GEMM, the engine would have to pad the inputs of all experts to the maximum number of tokens routed to any single expert: $\max_i(M_i)$. This padding wastes massive amounts of compute and memory bandwidth on dead rows containing zero-padding. **Grouped GEMM** (implemented in Triton or CUTLASS) executes all E independent matrix multiplications of shape $(M_i \times N \times K)$ in a single GPU kernel without padding, processing the ragged buffers natively and saving significant compute.

25.5 Pro-Tip: How to Impress the Interviewer

- **Shared Expert Design**: Demonstrate knowledge of modern architectural optimizations. Explain how modern architectures (like DeepSeek-V3) combine routed experts with a **shared, always-on expert**. In DeepSeek-V3, the output is calculated as $y = y_{shared} + y_{routed}$. The shared expert captures generic, task-agnostic features across all tokens. This offloads general knowledge representation from the routed experts, allowing them to specialize in fine-grained features, substantially improving parameter efficiency.
- **Dequantization Placement**: Explain that in high-performance MoE serving, expert weights are stored in 4-bit or 8-bit precision. To avoid saturating memory bandwidth, weights are dequantized **on the fly** in SRAM right before matrix multiplication rather than in VRAM.

26 Speculative Decoding

- **Category**: LLM Systems
- **Difficulty**: Expert
- **Target Role**: LLM Inference Architect / ML Platform Engineer
- **Source**: Fast Inference from Transformers via Speculative Decoding (Leviathan et al., 2023) / Speculative Sampling (Chen et al., 2023)
- **Flashcards**: LLM Systems deck

26.1 Concept Overview

Autoregressive decoding in Large Language Models is severely memory-bound: generating one token requires loading the entire model's weights (often tens of gigabytes) from High-Bandwidth Memory (HBM) to the GPU's registers, performing only 2 FLOPs per parameter.

Speculative decoding shifts the workload from memory-bound to compute-bound. It employs a fast, cheap **draft model** (e.g., 0.5B parameters) to autoregressively sketch K candidate tokens ahead. A large, expensive **target model** (e.g., 70B parameters) then verifies all K candidates in a *single parallel forward pass*. A **rejection sampling** step decides which candidates to keep. When a candidate is rejected, the target model's KV cache is rolled back, and the correct token is resampled. Crucially, the mathematical formulation of rejection sampling guarantees that the final output distribution is *exactly identical* to what the target model would have produced alone, resulting in zero quality degradation.

26.1.1 The Problem It Solves

During standard autoregressive decode, the arithmetic intensity is extremely low:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes Loaded}} = \frac{2 \cdot P \cdot 1}{P \cdot 2} = 1 \text{ FLOP/byte}$$

On an NVIDIA A100 GPU (312 TFLOPS FP16 peak / 2.0 TB/s HBM bandwidth), the system's execution crossover is **156 FLOP/byte**. An intensity of 1 FLOP/byte means the GPU math units run at only **0.6% of peak**, stalled 99.4% of the time waiting for weights.

By verifying K tokens in a single target pass, speculative decoding loads the target model weights once but performs $2 \cdot P \cdot K$ FLOPs, scaling the intensity:

K	Target FLOPs	Weights Loaded (ONCE)	Arithmetic Intensity	GPU Peak Capacity %
1 (Baseline)	1.40×10^{11}	1.40×10^{11} bytes	**1 FLOP/B**	0.6%
2	2.80×10^{11}	1.40×10^{11} bytes	**2 FLOP/B**	1.3%
4	5.60×10^{11}	1.40×10^{11} bytes	**4 FLOP/B**	2.6%
8	1.12×10^{12}	1.40×10^{11} bytes	**8 FLOP/B**	5.1%
16	2.24×10^{12}	1.40×10^{11} bytes	**16 FLOP/B**	10.3%

By increasing K, speculative decoding amortizes weight loading, boosting GPU utilization and resulting in **2×–3× latency speedups** in practice.

26.1.2 How It Works

1. **Drafting Step**: The draft model runs K sequential forward steps to produce draft tokens $t_0, t_1, \ldots, t_{K-1}$ and their corresponding draft probabilities $q_0, q_1, \ldots, q_{K-1}$.
2. **Parallel Verification**: The target model runs a single forward pass over the block $[t_0, t_1, \ldots, t_{K-1}]$

using a standard lower-triangular causal mask:

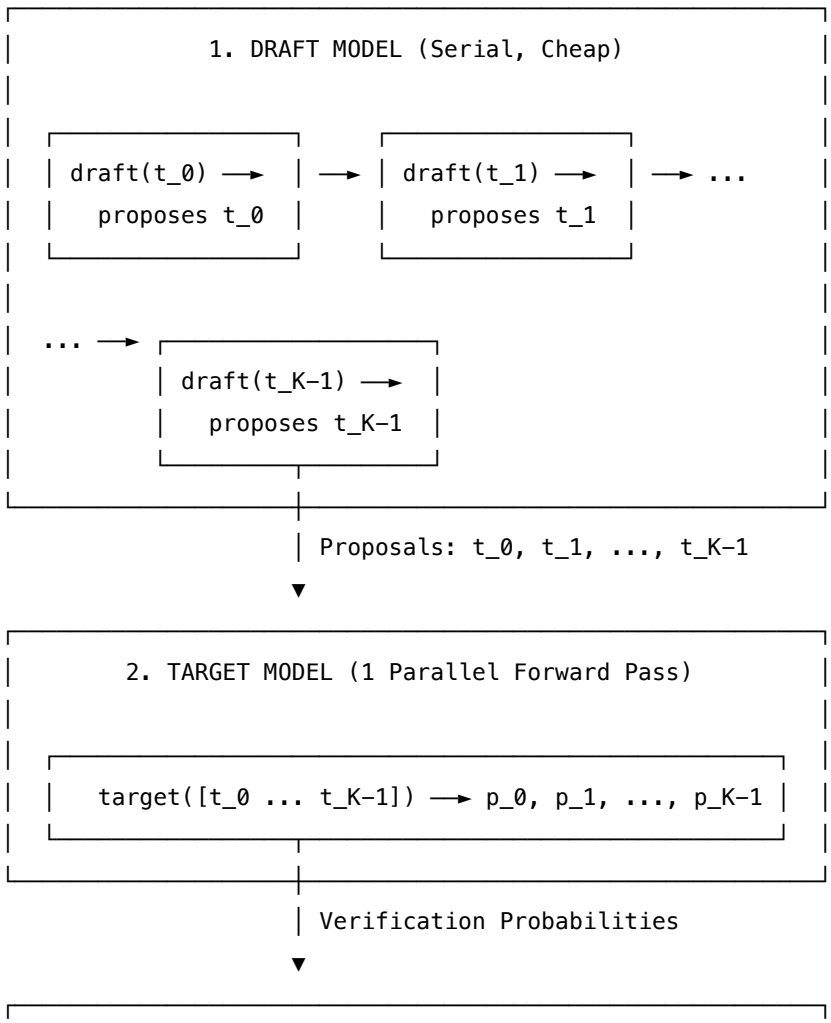
$$\text{Causal Mask} = \begin{pmatrix} \text{att} & . & . & . \\ \text{att} & \text{att} & . & . \\ \text{att} & \text{att} & \text{att} & . \\ \text{att} & \text{att} & \text{att} & \text{att} \end{pmatrix}$$

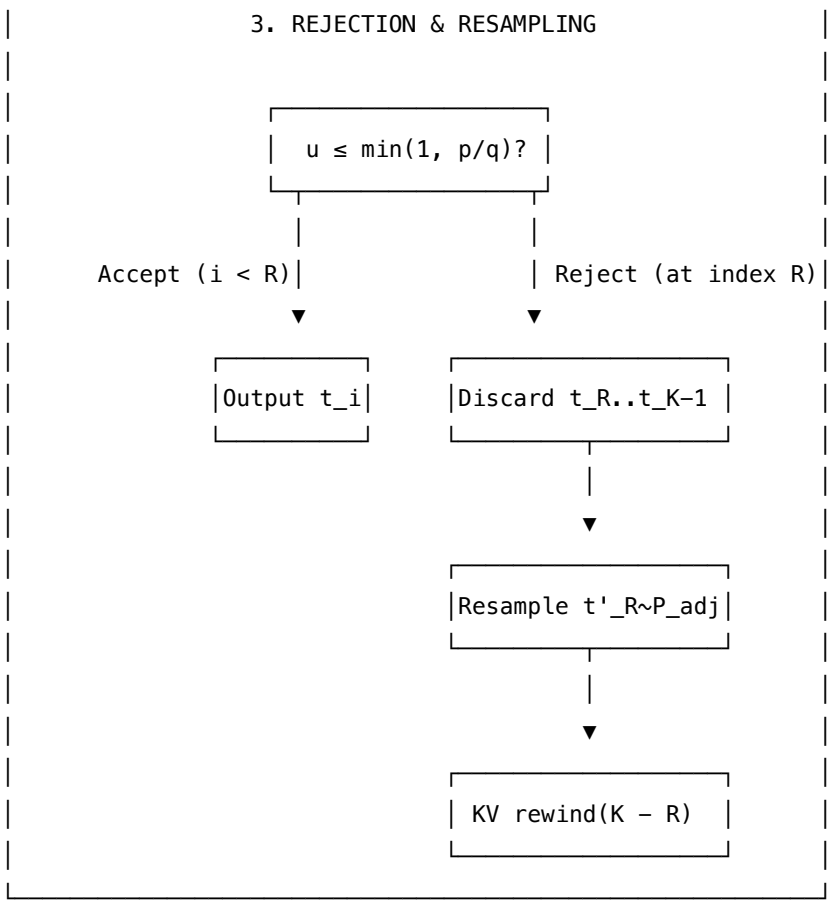
This computes the true target probabilities $p_i(x \mid t_0 \ldots t_{i-1})$ for each token position i in parallel.

3. **Rejection Sampling**: For each token position i from 0 to $K-1$:
   - Draw a uniform random variable $u \sim U(0,1)$.
   - **Accept** t_i if $u \leq \min\left(1, \frac{p_i(t_i)}{q_i(t_i)}\right)$.
   - If **rejected** at index R: discard $t_R \ldots t_{K-1}$. Resample the corrected token t'_R from the adjusted distribution:

$$P_{adj}(x) = \frac{\max(0, p_R(x) - q_R(x))}{Z} \quad \text{where } Z = \sum_y \max(0, p_R(y) - q_R(y))$$

     Terminate the verification round.
4. **KV Cache Rollback**: On rejection at index R, the last $K-R$ tokens appended to the target model's KV cache are invalid and must be removed using a `rewind(K - R)` operation.





26.2 Worked Example

This example uses verified numbers from a simulated speculative decoding run with vocabulary size $V = 8$ and draft length $K = 4$.

26.2.1 1. Probability Distributions

Let the draft probabilities (q) and target probabilities (p) be: - $q = [0.0938, 0.2550, 0.0569, 0.1547, 0.0466, 0.2088, 0.0695$
- $p = [0.2377, 0.1761, 0.0356, 0.1068, 0.0263, 0.2903, 0.0480, 0.0791]$

The acceptance ratios $\min(1, \frac{p_i}{q_i})$ for all tokens in the vocabulary are:

Token Idx	Word	Draft $q(x)$	Target $p(x)$	p/q	$\min(1, \frac{p}{q})$	Always Accept?
0	the	0.0938	0.2377	2.534	**1.0000**	**YES** ($p \geq q$)
1	cat	0.2550	0.1761	0.690	0.6905	No (accept w.p. 0.69)
2	xyz	0.0569	0.0356	0.625	0.6248	No (accept w.p. 0.62)

Token Idx	Word	Draft $q(x)$	Target $p(x)$	p/q	$\min(1, \frac{p}{q})$	Always Accept?
3	sat	0.1547	0.1068	0.690	0.6905	No (accept w.p. 0.69)
4	qqq	0.0466	0.0263	0.565	0.5653	No (accept w.p. 0.56)
5	on	0.2088	0.2903	1.390	**1.0000**	**YES** ($p \geq q$)
6	a	0.0695	0.0480	0.690	0.6905	No (accept w.p. 0.69)
7	mat	0.1146	0.0791	0.690	0.6905	No (accept w.p. 0.69)

26.2.2 2. The Verification Trace

The draft model proposes tokens: `['a', 'sat', 'cat', 'on']` (token indices `[6, 3, 1, 5]`).

- **Step 0**: Verify $t_0 = 6$ ("a").
  - $q(t_0) = 0.0695$, $p(t_0) = 0.0480$.
  - Acceptance threshold: $\min(1, \frac{p}{q}) = 0.6905$.
  - Draw $u = 0.4963$.
  - **Decision**: **ACCEPT** (since $0.4963 \leq 0.6905$).
- **Step 1**: Verify $t_1 = 3$ ("sat").
  - $q(t_1) = 0.1547$, $p(t_1) = 0.1068$.
  - Acceptance threshold: $\min(1, \frac{p}{q}) = 0.6905$.
  - Draw $u = 0.7682$.
  - **Decision**: **REJECT** (since $0.7682 > 0.6905$).

Because index $R = 1$ is rejected, draft tokens t_1, t_2, t_3 are discarded.

26.2.3 3. Resampling at Rejection Index $R = 1$

We construct P_{adj} using the normalizer $Z = \sum \max(0, p - q) = 0.2254$:

$$P_{adj} = [0.6383, 0.0, 0.0, 0.0, 0.0, 0.3617, 0.0, 0.0]$$

Only tokens `0` ("the") and `5` ("on") have mass (where $p > q$). Drawing from P_{adj} yields token **5** ("on"). - **Final Output**: `['a', 'on']` (indices `[6, 5]`). - **Committed Tokens**: **2** (1 accepted from draft + 1 resampled). - **KV Cache Rollback**: During verification, $K = 4$ tokens were appended to the KV cache, moving the cache length from 3 (prefill) to 7. Upon rejection at $R = 1$, the engine performs `rewind(K - R) = rewind(3)` to restore the cache length to 4 (representing 3 prefill + 1 accepted token).

26.3 Complexity & Trade-offs

Metric	Complexity / Value	Notes
Draft Latency	$K \cdot \text{Cost}_{draft}$	K serial forward steps on the cheap draft model.
Verify Latency	$1 \cdot \text{Cost}_{target}$	1 parallel forward step on the large target model.
KV Cache Truncation	$\mathcal{O}(1)$ (dense) / $\mathcal{O}(\text{pages})$ (paged)	Tearing out rejected tokens. Paged cache blocks must be freed.
Expected Speedup S	$\frac{1-\alpha^{K+1}}{(1-\alpha)(1+K\gamma)}$	Where α is acceptance rate, $\gamma = \frac{\text{Cost}_{draft}}{\text{Cost}_{target}}$.

26.4 Common Interview Questions & How to Answer

26.4.1 Q1: Prove that the output distribution of speculative decoding exactly matches that of the target model alone.

- **Answer**: The probability of outputting a token x at step i is the sum of two mutually exclusive events: accepting x if proposed by the draft model, or rejecting the draft proposal and sampling x from the adjusted distribution P_{adj}:

$$P(\text{output} = x) = P(\text{draft proposes } x \text{ and is accepted}) + P(\text{reject}) \cdot P_{adj}(x)$$

$$P(\text{output} = x) = q(x) \cdot \min\left(1, \frac{p(x)}{q(x)}\right) + Z \cdot \frac{\max(0, p(x) - q(x))}{Z}$$

$$P(\text{output} = x) = \min(q(x), p(x)) + \max(0, p(x) - q(x))$$

  - **Case 1**: If $p(x) \geq q(x)$, then $\min(q(x), p(x)) = q(x)$ and $\max(0, p(x) - q(x)) = p(x) - q(x)$. The sum is:

$$q(x) + p(x) - q(x) = p(x)$$

  - **Case 2**: If $p(x) < q(x)$, then $\min(q(x), p(x)) = p(x)$ and $\max(0, p(x) - q(x)) = 0$. The sum is:

$$p(x) + 0 = p(x)$$

  Thus, in both cases, the probability of generating x is exactly $p(x)$, matching the target model's output distribution.

26.4.2 Q2: Under what conditions does speculative decoding result in a slowdown ($S < 1$), and how do we optimize the draft length K?

- **Answer**: Speculative decoding slows down inference when the cost of drafting plus the cost of target verification exceeds the cost of standard autoregressive decoding. This happens when:
  1. The draft model is too large, meaning the cost ratio $\gamma = \text{Cost}_{draft}/\text{Cost}_{target}$ is too high.
  2. The draft model is inaccurate, leading to a low acceptance rate α, which causes most draft

tokens to be rejected. For instance, with $\alpha = 0.2, K = 4, \gamma = 0.5$, the speedup S falls to **0.42×** (a major slowdown). To optimize K, we analyze the derivative of the speedup equation:

$$S = \frac{1 - \alpha^{K+1}}{(1 - \alpha)(1 + K\gamma)}$$

- When α is high (e.g., 0.9), increasing K maximizes speedup (e.g., optimal $K \approx 10$ at $\gamma = 0.1$ yielding 3.40×).
- When α is low (e.g., 0.3), the optimal K is 1 (yielding 1.02×), and any larger K degrades performance due to wasted target computations.

26.5 Pro-Tip: How to Impress the Interviewer

- **Tree Attention Verification (Medusa/EAGLE)**: Standard speculative decoding proposes a single linear chain of tokens. Impress the interviewer by suggesting **Tree Attention**. Instead of a single sequence, the draft mechanism (like Medusa's heads or EAGLE) proposes a branching tree of multiple candidate sequences. The target model verifies all branches of the tree in parallel using a tree-structured causal mask. This raises the effective acceptance rate α by checking multiple high-probability paths simultaneously, achieving speedups of **2.7×–3.5×** without needing a separate draft model.
- **Off-by-One Paged KV Cache Trapping**: In PagedAttention serving engines, rolling back the KV cache during rejection requires freeing page blocks. Warn the interviewer of the **ceil-division page boundary trap**: if cache indices are zero-indexed and page size is 16, rolling back from cache length 33 to 32 must release the page block hosting indices 32-47. If the index subtraction is not block-aligned, memory leaks or pointer corruption will occur.

27 LMCache (Hierarchical, Global KV Cache Pooling)

- **Category**: LLM Systems
- **Difficulty**: Hard
- **Target Role**: LLM Inference Architect / ML Platform Engineer
- **Source**: LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference (Liu et al., 2025) / Mooncake: A KVCache-centric Disaggregated Architecture (Qin et al., 2024)
- **Flashcards**: LLM Systems deck

27.1 Concept Overview

Prefix caching mechanisms (such as SGLang's RadixAttention or vLLM's BlockManager) store the Key-Value (KV) cache of prompt prefixes in GPU memory to avoid recomputing prefill steps. However, these systems are node-local: if a repeat prompt is routed to a different node in a cluster, the cache is missed, forcing a slow, redundant prefill.

LMCache extends local prefix caching to a **global, hierarchical KV cache pool**. It spans the entire

cluster's memory hierarchy: GPU VRAM → CPU DRAM → Local NVMe → Remote S3/Redis. Using content-addressed hashing, it creates unique signatures for fixed-size token chunks. When a local cache miss occurs, the node queries a cluster-wide lookup index. If the chunk is cached anywhere in the cluster, the node streams the KV cache pages over high-speed networks (RDMA or optimized TCP/IP) directly into its local VRAM block table. Prefill computation is skipped, replacing a heavy compute bottleneck with a fast network transfer.

27.1.1 The Problem It Solves

In a multi-node serving cluster, load balancing requests randomly distributes matching prompts. - **Without LMCache**: A request landing on a cold node triggers a full prefill, loading all model weights (e.g., 2.08 GB for a 1.04B parameter GQA model) to perform attention math. - **With LMCache**: The cold node pulls the precomputed KV cache instead.

For a GQA model (layers = 24, $n_{\text{q\_heads}} = 16$, $n_{\text{kv\_heads}} = 2$, head_dim = 128, prompt length $S = 512$ tokens), the KV cache size is:

$$\text{Size}_{\text{KV}} = 2 \cdot \text{layers} \cdot n_{\text{kv\_heads}} \cdot \text{head\_dim} \cdot S \cdot \text{bytes} = 2 \cdot 24 \cdot 2 \cdot 128 \cdot 512 \cdot 2 = 12,582,912 \text{ bytes} \approx 12.00 \text{ MiB}$$

We compare the transfer latency of this 12.00 MiB KV cache against the target model's prefill roofline floor at batch size 1:

Operation / Path	Bandwidth	Latency	vs. Prefill Floor
Recompute Prefill (A100 Floor)	—	**3.41 ms**	Baseline
Local DRAM → VRAM (PCIe Gen5)	64.0 GB/s	**0.197 ms**	**17.3× faster**
Remote DRAM → VRAM (RoCE 400G)	50.0 GB/s	**0.252 ms**	**13.5× faster**
Remote DRAM → VRAM (RoCE 100G)	12.5 GB/s	**1.007 ms**	**3.4× faster**
Local NVMe → VRAM (SSD)	7.0 GB/s	**1.798 ms**	**1.9× faster**

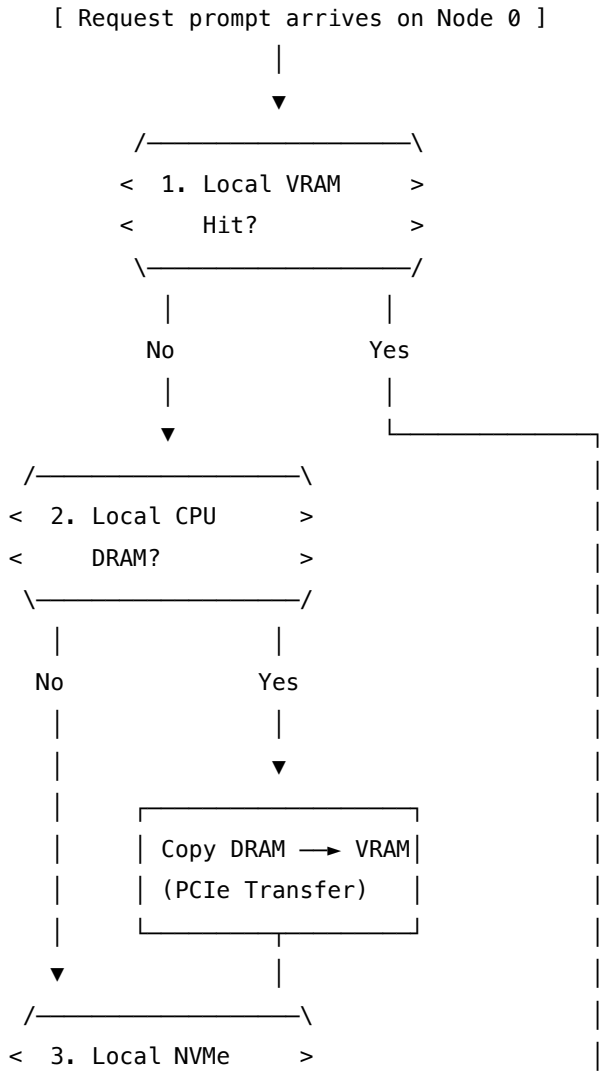
Moving the KV cache over RoCE 400G takes just **0.252 ms**, which is 13.5× faster than the absolute fastest theoretical prefill (3.41 ms). On real hardware, where prefill execution suffers from low Model FLOPs Utilization (MFU) at batch=1, the win is even larger.

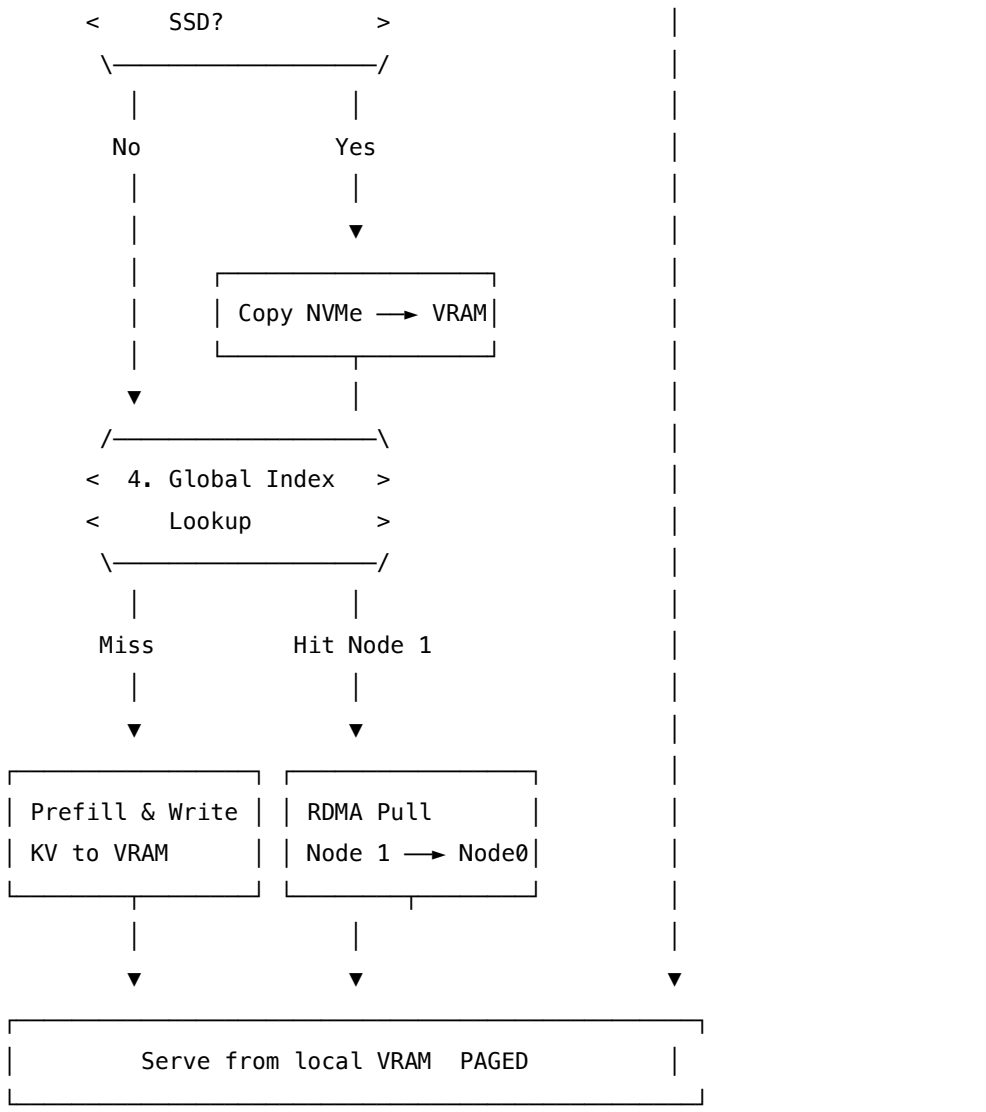
27.1.2 How It Works

1. **KV Chunking**: Input prompts are parsed into chunks of a fixed token length (e.g., `chunk_size = 2` tokens).
2. **Content-Addressable Signatures**: Each chunk is hashed by its token values using a hashing algorithm (like FNV-1a):
   - Tokens `[100, 101]` → Signature `0xecd0403d962ff8f4`

- Tokens `[102, 103]` → Signature `0xec2574297233efd4`
- Tokens `[104, 105]` → Signature `0xde27887129c0e434`

3. **Hierarchical Lookup**:
   - Check local node GPU VRAM.
   - Check local node CPU DRAM.
   - Check local node NVMe.
   - Check LMCache Global Index (e.g., a distributed hash table mapping signature → node IP + memory tier).
4. **RDMA/TCP Stream**: If the chunk lives in a remote node's CPU DRAM, the local engine requests an RDMA pull. The remote DRAM streams the KV tensor over the network straight into the local GPU VRAM.
5. **Block Table Mapping**: The pulled bytes are mapped into the local PagedAttention block table (logical page → physical page). Once mapped, the attention kernel reads the pages seamlessly; the transfer is fully transparent.
6. **VRAM Eviction Offloading**: When a node runs low on VRAM, rather than evicting and losing the KV cache, LMCache offloads it down the tier ladder (to DRAM, NVMe, or S3) to be retrieved later.





27.2 Worked Example

This example demonstrates how local-only prefix caching fails under load balancing, and how LMCache resolves it in a 4-node cluster ($N = 4$) processing 4 identical prompt queries.

27.2.1 1. The Multi-Node Miss (Local-Only Prefix Cache)

Four requests for the same prompt arrive. The load balancer distributes them across the 4 nodes: - **Request 0** routed to **Node 0**: Cold miss. Prefills and stores KV chunks (c_0, c_1, c_2) in Node 0's VRAM. - **Request 1** routed to **Node 2**: Cold miss. Node 2 check its local VRAM $\rightarrow$ Miss. Node 2 recomputes prefill. - **Request 2** routed to **Node 3**: Cold miss. Node 3 check its local VRAM $\rightarrow$ Miss. Node 3 recomputes prefill. - **Request 3** routed to **Node 1**: Cold miss. Node 1 check its local VRAM $\rightarrow$ Miss. Node 1 recomputes prefill.

Local Hit Rate: $1/4 = \mathbf{25\%}$ (only the first node's cache was utilized, despite the KV cache existing in the cluster).

27.2.2 2. The LMCache Global Index & Transfer

Under LMCache, when Node 1 processes Request 3: 1. **Local Check**: Miss. 2. **Global Query**: Node 1 queries the global index for the three chunk signatures: - `0xecd0403d962ff8f4` → Found: `Node 0 VRAM` - `0xec2574297233efd4` → Found: `Node 0 VRAM` - `0xde27887129c0e434` → Found: `Node 0 VRAM` 3. **RDMA Pull**: Node 1 initiates a 400G RoCE pull. - Chunks (c_0, c_1, c_2) totaling 192 bytes (in this small example) are copied from Node 0 VRAM → Node 1 VRAM. - Node 1 maps these pages into its block table: - `logical 0` → `physical page 0` - `logical 1` → `physical page 1` - `logical 2` → `physical page 2` 4. **Result**: Prefill is skipped. Gathered KV is byte-equal to the original.

Prompt	Cached on Node	Request Routed to	Local Hit?	LMCache Global Hit?	Action
0	Node 0	Node 0	**YES**	**YES**	Local VRAM Read
1	Node 1	Node 2	No	**YES**	RoCE Pull from Node 1 DRAM
2	Node 2	Node 3	No	**YES**	RoCE Pull from Node 2 DRAM
3	Node 3	Node 1	No	**YES**	RoCE Pull from Node 3 DRAM

Global Hit Rate: $4/4 = \mathbf{100\%}$. LMCache amplifies effective cache hit rate by $\mathbf{4\times}$ in this workload, scaling with the number of nodes N.

27.3 Complexity & Trade-offs

Metric	Complexity / Value	Notes
Local Index Lookup	$\mathcal{O}(L)$	Walk local radix tree or probe hash table.
Global Index Lookup	$\mathcal{O}(L \cdot \text{RTT}_{Redis})$	Network round-trip to the distributed index.
KV Copy Bandwidth	Up to 50 GB/s (RoCEv2)	Transfer latency is linear with the chunk size and prompt length.
Metadata Overhead	$\mathcal{O}(S/\text{chunk\_size})$	Small chunk sizes allow fine-grained matches but explode index metadata size.

27.4 Common Interview Questions & How to Answer

27.4.1 Q1: Why does LMCache use content-addressed chunk signatures rather than chained signatures for the global index?

- **Answer**: Local prefix caches (such as SGLang's RadixAttention) typically use **chained hashing** (e.g., $Hash(block_i) = F(Hash(block_{i-1}), tokens_i)$) to represent the prefix history. This works well for walking a local radix tree from the root down to find the longest matching prefix. However, chained hashing is history-dependent: if two prompts share a middle segment but have different starting tokens, their chained hashes for that shared segment will differ. LMCache uses **content-addressed, unchained hashing** at the chunk level (e.g., FNV-1a over only the tokens inside the chunk). Because the signature depends only on the chunk's own token values, identical chunks generate the identical signature regardless of their prefix histories. This allows different prompts with shared inner segments (e.g., a shared document in a RAG pipeline) to match and pull KV caches globally, enabling much higher cache reuse across diverse workloads.

27.4.2 Q2: Under what conditions does LMCache fail to beat the prefill cost, and how does the memory tier ladder mitigate this?

- **Answer**: LMCache beats prefill when the network transfer latency is lower than the time required to compute the prefill:

$$\text{Latency}_{\text{transfer}} = \frac{\text{Size}_{\text{KV}}}{\text{Bandwidth}} < \text{Latency}_{\text{prefill}}$$

  This inequality can fail in two scenarios:

  1. **De-cached/Deep Tiers**: If the KV cache has been evicted to slow storage tiers like local NVMe (7 GB/s) or WAN S3, the bandwidth drops. For a very long prompt, the transfer time from NVMe may approach or exceed the GPU prefill computation time.
  2. **Short Prompts**: For very short prompts (e.g., <64 tokens), the prefill cost is extremely small (~1 ms). In this regime, the latency of querying the global index (a network round-trip to Redis) plus the transfer overhead exceeds the cost of just doing the prefill on-device. LMCache's hierarchical memory tier ladder mitigates this by applying an LRU eviction policy. Hot, frequently queried prefixes are kept high in the hierarchy (GPU VRAM or CPU DRAM) to ensure high transfer bandwidth, while cold, historical prefixes are spilled to NVMe/S3. If a global lookup points to a slow tier where transfer exceeds prefill cost, the engine can fall back to local recomputation.

27.5 Pro-Tip: How to Impress the Interviewer

- **Context Truncation Vulnerability**: Demonstrate real-world systems awareness. Point out that in many multi-turn chat applications, engines apply **context truncation** (discarding oldest system messages or early chat history) to fit within context length limits. Context truncation changes the prefix sequence, which breaks prefix cache matching. Point out that according to the LMCache paper, context truncation can **roughly halve the prefix-cache hit ratio**. Suggest mitigating this by using sliding window attention or designing custom chunking boundaries that align with the truncated context offsets.

- **Unified Engine Integration**: Explain that LMCache is designed to be **engine-agnostic**. It uses a modular KV Cache Connector that intercepts the physical page pool allocations of SGLang and vLLM. This allows cross-engine cache sharing, where a prompt prefilled by SGLang can be pulled and decoded by a vLLM instance in the same cluster.

28 Disaggregated Serving (DistServe / Mooncake)

- **Category**: LLM Systems
- **Difficulty**: Expert
- **Target Role**: LLM Inference Architect / ML Platform Engineer
- **Source**: DistServe (Zhong et al., 2024) / Mooncake (Qin et al., 2025) / Splitwise (Microsoft, 2023)
- **Flashcards**: LLM Systems deck

28.1 Concept Overview

An LLM request consists of two phases with opposite computational profiles: the **prefill** phase (which processes the prompt and is compute-bound) and the **decode** phase (which generates tokens one by one and is memory-bandwidth-bound). In standard co-located serving engines (like vLLM or Orca), these two phases run on the same GPU.

Disaggregated serving separates the prefill and decode phases onto **distinct GPU pools** connected by high-speed networks. The prefill pool processes the prompt, generates the Key-Value (KV) cache, and transmits the KV cache bytes over RDMA to the decode pool, which handles token generation.

```
Co-located (1 GPU): prefill blocks decode → ITL spikes
[ Decode Batch (20ms) ] → [ Prefill (80ms) ] → [ Decode Batch (stalled!) ] → ITL = 100ms (5x spike)


Disaggregated (2 pools): parallel execution → flat ITL
Prefill Pool: [ Prefill (80ms) ] —(KV Transfer: 1.3ms)—> Decode Pool: [ Decode Batch (20ms) ] (flat ITL)
```

28.1.1 The Problem It Solves

Co-located serving causes severe **phase interference**. Eager schedulers prioritize the prefill phase to keep the Time-To-First-Token (TTFT) low. However, if a long prefill request (e.g., 512 tokens, taking **80 ms**) enters the execution queue, the active decode batch must stall. This causes the Inter-Token Latency (ITL) to spike from a baseline of **20 ms** to **100 ms** (a **5.0× spike**), violating user-facing latency SLAs.

Disaggregated serving resolves this by separating the compute resources. The prefill pool is configured with Tensor Parallelism (TP) for fast matrix multiplication to minimize TTFT, while the decode pool uses Data Parallelism (DP) and large batching to maximize throughput and keep ITL low and flat. The only overhead is the network transfer of the KV cache.

For a Llama-3-8B model ($layers = 32, n\_q\_heads = 32, n\_kv\_heads = 8, head\_dim = 128$, prompt length $S = 512$ tokens), the KV cache size is:

$$\text{Size}_{\text{KV}} = 2 \cdot \text{layers} \cdot \text{n}_{\text{kv\_heads}} \cdot \text{head\_dim} \cdot S \cdot \text{bytes} = 2 \cdot 32 \cdot 8 \cdot 128 \cdot 512 \cdot 2 = 67,108,864 \text{ bytes } (\approx 64.0 \text{ MiB})$$

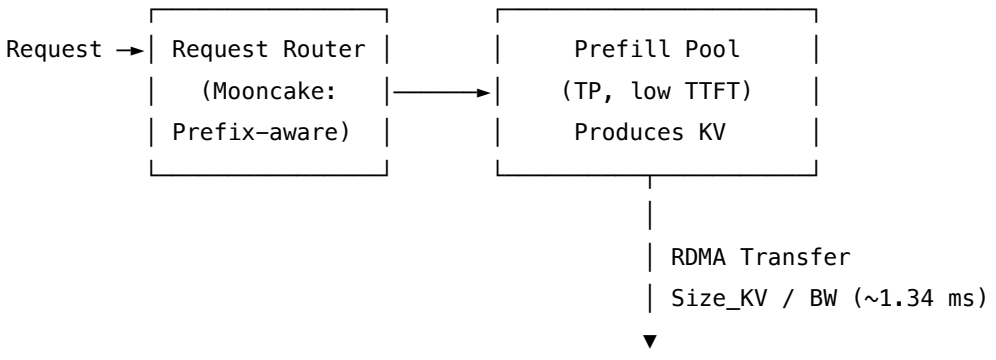
We compare the network transfer latency of this 64.0 MiB KV cache against the prefill roofline floor on an NVIDIA A100 GPU (peak FP16 TFLOPS = 312, crossover = 156 FLOP/B):

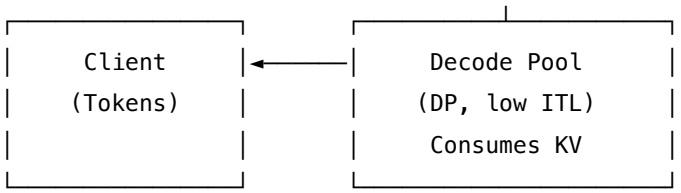
Transfer Path / Operation	Bandwidth	Transfer Latency	vs. Prefill Floor
Prefill Recomputation (A100 Floor)	—	**22.91 ms**	Baseline
Intra-node NVLink (GPU ↔ GPU)	600.0 GB/s	**0.112 ms**	**204.8× faster**
Remote RoCEv2 / IB (400 Gbps network)	50.0 GB/s	**1.342 ms**	**17.1× faster**
Remote RoCEv2 / IB (200 Gbps network)	25.0 GB/s	**2.684 ms**	**8.5× faster**
100 GbE network	12.5 GB/s	**5.369 ms**	**4.3× faster**

At 400 Gbps RoCEv2, the transfer takes only **1.34 ms**. This is negligible compared to the prefill roofline floor of **22.91 ms** (and empirical prefill times of ~**80 ms**). Disaggregation adds only a **1.68%** overhead to TTFT (80 ms → 81.34 ms) but completely eliminates the 5.0× ITL spikes, allowing engines to serve **7.4× more requests** under strict latency SLAs.

28.1.2 How It Works

1. **Request Routing**: An incoming query goes to the request router. In advanced systems (like Mooncake), the router uses **KV-centric prefix-aware routing** to route requests to the prefill node holding the longest matching prefix of the prompt.
2. **Prefill Execution**: The chosen prefill node computes the KV cache for the prompt (or only the suffix on a prefix hit) and generates the first output token.
3. **KV Cache Transfer**: The prefill GPU streams the KV cache over RDMA (RoCEv2 or NVLink) into the target decode GPU's VRAM.
4. **Decode Execution**: The decode GPU appends the pages to its local block table (under PagedAttention). It continues generating subsequent tokens autoregressively at low, flat ITL.
5. **Dynamic Co-optimization**: The platform searches for the optimal ratio of prefill-to-decode GPUs (e.g., 2P:1D) based on traffic profiles to maximize overall system goodput.





28.2 Worked Example

This example demonstrates how Mooncake's KV-centric routing and disaggregated serving collaborate to minimize compute and eliminate latency spikes.

28.2.1 1. Unified KV Cache Catalog Setup

Suppose we have 3 prefill nodes caching the following prefixes: - **System Prompt**: 8 tokens `[100..107]` - **Node 0**: Caches `system_prompt` + `query_a` (`[200, 201, 202]`, total 11 tokens) - **Node 1**: Caches `system_prompt` only (total 8 tokens) - **Node 2**: Caches `system_prompt` + `query_b` (`[203, 204, 205]`, total 11 tokens)

28.2.2 2. KV-Centric Routing & Prefill

A new request arrives: `system_prompt` + `query_a` + `new_question` (13 tokens total). - The router queries the global catalog:

Node	Longest Cached Prefix Match	Match Length	Remaining Suffix to Prefill
Node 0	`system_prompt` + `query_a`	**11 tokens**	**2 tokens**
Node 1	`system_prompt`	8 tokens	5 tokens
Node 2	`system_prompt`	8 tokens	5 tokens

The router selects **Node 0**. - Node 0 only computes the KV cache for the **2 suffix tokens** (skipping 11 tokens, an **85% prefix compute saving**). - Node 0 generates the first output token.

28.2.3 3. KV Transfer & Decode Execution

Node 0 transfers the KV cache to the decode pool: - The decode pool node maps the received KV pages into its block table: - `logical 0` → `physical page 0` - `logical 1` → `physical page 1` - `logical 2` → `physical page 2` - The transfer is byte-equal to the original precomputed tensors. The decode pool begins generating output tokens.

28.2.4 4. End-to-End Latency Comparison

Metric	Co-Located (1 GPU)	Disaggregated (2 Pools)	Advantage of Disaggregation
TTFT (No Contention)	80 ms	81.34 ms	Slight 1.68% network overhead
TTFT (Under Load)	80+ ms (stalled by active decodes)	~81.34 ms	Predictable first-token latency
ITL Baseline	20 ms	20 ms	Identical speed
ITL (Under Prefill Load)	**100 ms (SPIKE)**	**20 ms (FLAT)**	**Eliminates 5× tail latency spikes**
Goodput (SLO-bound)	Lower (broken by ITL spikes)	**7.4× higher**	Predictable throughput under load

---

28.3 Complexity & Trade-offs

Metric	Complexity / Value	Notes
Routing Overhead	$\mathcal{O}(L \cdot \text{RTT}_{index})$	Router checks prefix catalog before dispatching.
KV Transfer Complexity	$\mathcal{O}$(Size_KV)	Linear with prompt length and layer dimension.
SLO Goodput Ratio	Up to 7.4× improvement	Eliminating ITL variance increases overall compliant throughput.
GPU Hardware Overhead	High (decoupled pool)	Requires setting aside separate clusters. If prefill:decode ratio is misconfigured, one pool idles.

28.4 Common Interview Questions & How to Answer

28.4.1 Q1: Why do prefill and decode phases benefit from different parallelization strategies, and how does disaggregated serving exploit this?

- **Answer**: The prefill and decode phases have different hardware bottlenecks:
  1. **Prefill**: Processes all prompt tokens in parallel, resulting in large matrix multiplications with high arithmetic intensity. It is highly **compute-bound**. To minimize user-perceived TTFT, we want to split the computation of this single prompt across multiple GPUs. Thus, the prefill pool is configured with **Tensor Parallelism (TP)**, which splits weights within a layer across GPUs, reducing matmul latency at the cost of intra-layer communication.
  2. **Decode**: Generates one token at a time, performing small matrix-vector multiplications that read all model weights. It is highly **memory-bandwidth-bound**. Tensor Parallelism is inefficient here because it introduces frequent synchronization barriers for tiny computations. Instead,

the decode pool is configured with **Data Parallelism (DP)** or **Pipeline Parallelism (PP)**. This allows the system to process thousands of concurrent, independent generation streams in large batches, maximizing weight reuse and throughput without synchronization overheads. Disaggregated serving allows us to tune these parallelization strategies **independently** on their respective pools. In a co-located engine, we are forced to use the same parallelism strategy (usually TP) for both phases, compromising the efficiency of the decode step.

28.4.2 Q2: What is Mooncake's "prediction-based early rejection policy," and why is it needed in disaggregated serving?

- **Answer**: In a disaggregated serving cluster, the prefill and decode pools have finite capacities. Under high traffic, the decode pool can become saturated with active generation streams. If we continue to accept new requests, the prefill pool will compute them, transfer the KV cache, and flood the decode pool. This would force the decode pool to increase its batch sizes beyond its hardware threshold, leading to queuing delays and causing active streams to miss their ITL/TPOT SLOs. Mooncake implements a **prediction-based early rejection policy**. When a new request arrives, the scheduler estimates its generation length and calculates the future resource load on the decode pool. If the scheduler predicts that the decode pool cannot serve the new request without causing existing streams to violate their ITL SLOs, it rejects the request *before* performing prefill. This prevents wasting prefill compute and protects active users from latency degradation, maintaining high **goodput** under overload conditions.

28.5 Pro-Tip: How to Impress the Interviewer

- **Disaggregated Cache Tiering**: Suggest caching KV pages on the **CPU DRAM of the decode nodes** rather than on the prefill nodes. When the prefill pool finishes, it transfers the KV cache straight to the destination decode node's CPU DRAM. When the decode node's scheduler allocates a slot, the KV cache is copied locally from CPU DRAM $\rightarrow$ GPU VRAM over PCIe Gen5. This offloads VRAM capacity pressure from both GPU pools and hides transfer latency behind execution.
- **Prefix-Aware Routing Overlap**: Point out that the router should route requests to prefill nodes based not only on prefix match length but also on **network topology**. Routing to a prefill node on the same rack as the destination decode node allows the KV cache transfer to use intra-rack RoCEv2, avoiding inter-rack network congestion.

29 KTransformers (Heterogeneous CPU/GPU Expert Offloading)

- **Category**: LLM Systems
- **Difficulty**: Expert
- **Target Role**: LLM Inference Architect / ML Platform Engineer
- **Source**: KTransformers SOSP Paper (Chen et al., 2025) / llama.cpp
- **Flashcards**: LLM Systems deck

29.1 Concept Overview

Running ultra-large Mixture-of-Experts (MoE) models like DeepSeek-V3/R1 (671B parameters) on consumer hardware is constrained by GPU memory limits. A standard 24 GB GPU cannot fit the model, even when compressed. Naive offloading schemes (like llama.cpp's layer-by-layer offload) stream the entire model's weights across the slow PCIe bus for every single token, causing generation to stall.

KTransformers introduces **expert offloading**. Instead of moving weights, it partitions the execution: the embeddings, attention layers, router networks, always-on shared experts, and KV caches reside permanently on the GPU, while the routed expert weights are parked in CPU DRAM. For every token step, only the tiny **activation hidden states** (~14 KB) cross the PCIe bus to the CPU. The CPU executes the active experts using hardware vector extensions (Intel AMX) at high speed and returns the activation outputs to the GPU. This eliminates the PCIe bandwidth bottleneck, enabling local consumer boxes to run trillion-parameter MoE models at interactive speeds.

```
Layer Offload (llama.cpp) — PCIe Bottleneck:
[ GPU VRAM ] <=== (Stream 350 GB Weights / Token) === [ CPU DRAM ] (0.18 tok/s)

Expert Offload (KTransformers) — Activation-only:
[ GPU VRAM ] ── (Stream 14 KB Hidden State) ──> [ CPU DRAM (AMX Compute) ]
[ GPU VRAM ] <── (Stream 14 KB Out Activation) ── [ CPU DRAM ] (Fast execution)
```

29.1.1 The Problem It Solves

DeepSeek-V3 holds **671B parameters**, requiring: - **FP16 / BF16**: **1342.00 GB** of storage (14.6× over a 24 GB GPU). - **W4A16 (4-bit)**: **335.50 GB** (14.0× over). - **W4A16 + Scales/Bias**: **350.00 GB**.

If we offload this model layer-by-layer and stream weights over PCIe:

$$\text{Latency}_{layer} = \frac{\text{Model Size}}{\text{PCIe Bandwidth}}$$

- **PCIe Gen4 x16** (64 GB/s): $\frac{350 \text{ GB}}{64 \text{ GB/s}} = \mathbf{5.47}$ s/token $\quad(\approx 0.18$ tok/s)
- **PCIe Gen5 x16** (128 GB/s): $\frac{350 \text{ GB}}{128 \text{ GB/s}} = \mathbf{2.73}$ s/token $\quad(\approx 0.37$ tok/s)

Under layer offload, generating a standard 1000-token answer takes **1.5 hours**. KTransformers replaces weight streaming with activation streaming. For DeepSeek-V3 ($hidden = 7168$), a single expert-call activation (hidden state) size is:

$$\text{Size}_{activation} = \text{batch} \cdot \text{hidden} \cdot \text{bytes} = 1 \cdot 7168 \cdot 2 = 14,336 \text{ bytes } (\approx 14.3 \text{ KB})$$

Moving this activation across the PCIe bus takes: - **PCIe Gen4 x16** (64 GB/s): $\frac{14{,}336 \text{ bytes}}{64 \times 10^9 \text{ bytes/s}} = \mathbf{0.22}$ s - **PCIe Gen5 x16** (128 GB/s): $\frac{14{,}336 \text{ bytes}}{128 \times 10^9 \text{ bytes/s}} = \mathbf{0.11}$ s

Even when accounting for the worst-case total round-trips ($k = 8$ experts, 61 layers, both directions):

$$\text{Worst-case Volume} = 8 \cdot 61 \cdot 14,336 \text{ bytes} \cdot 2 = 13,991,936 \text{ bytes} \approx \mathbf{13.99} \text{ MB}$$

This full-token transfer takes only **218.6 µs** over PCIe Gen4, which is **25,014× fewer bytes** than layer-by-layer offloading.

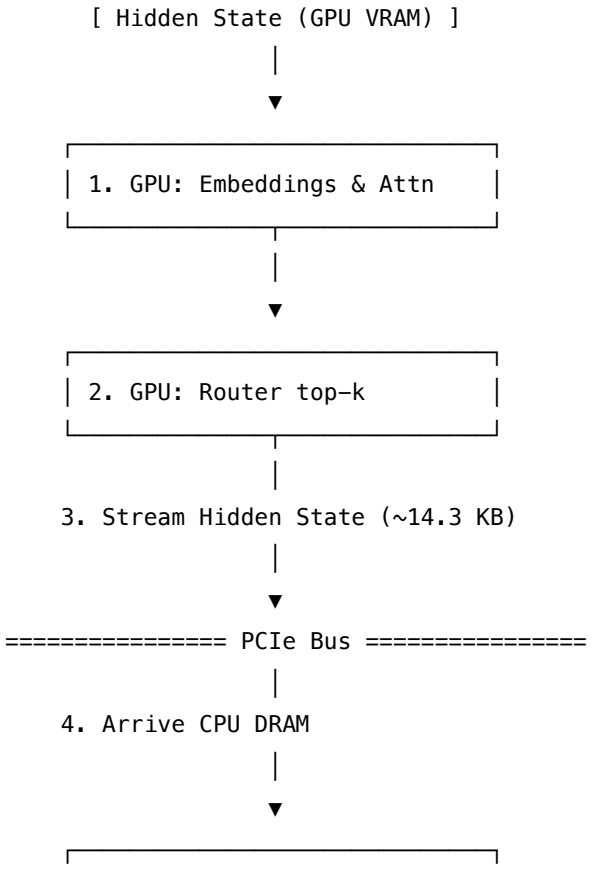
29.1.2 How It Works

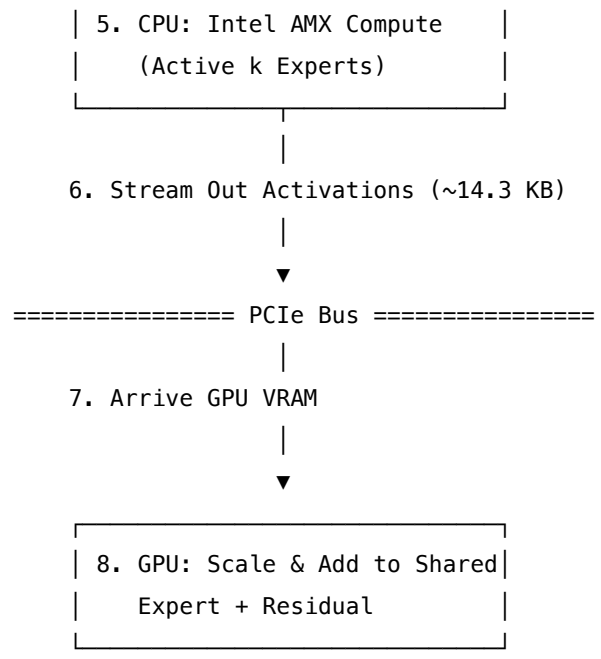
1. **Static Memory Allocation**:
   - **GPU VRAM**: embedding, attention, router, shared expert, KV cache (using PagedAttention).
   - **CPU DRAM**: 256 routed expert weights quantized to 4-bit (330 GB).
2. **Attention & Routing on GPU**: The GPU runs the attention block and passes the hidden state to the router, which calculates the top-k expert assignments ($k = 8$).
3. **Activation Streaming**: The GPU copies the 14.3 KB hidden state to CPU DRAM over PCIe.
4. **CPU Systolic Compute**: The CPU identifies the k active experts. It loads their 4-bit weights and performs the SwiGLU matrix math:

   $$E_i(x) = \text{down}(\text{silu}(\text{gate}(x)) \odot \text{up}(x))$$

   This computation is accelerated using **Intel AMX (Advanced Matrix Extensions)**, which processes tiled matrix multiplications in hardware, bypassing CPU vector ALU limits.
5. **Activation Return & Synthesis**: The CPU transfers the expert outputs back to the GPU. The GPU scales the outputs by the gate weights and sums them with the always-on shared expert output to update the residual stream:

   $$y = y_{\text{shared}} + \sum_{i \in \text{active}} G(x)_i \cdot E_i(x)$$





29.2 Worked Example

This example demonstrates the data volume comparison between naive layer offloading and expert offloading on a quantized DeepSeek-V3 model.

29.2.1 1. Model Partitioning Footprint

Sub-layer / Block	Precision	Parameter Count	Location	Memory Footprint
Embeddings & Attention	FP16	~37B	GPU VRAM	**74.00 GB**
Shared Expert	FP16	~1B	GPU VRAM	**2.00 GB**
Routed Experts (256)	W4A16	~633B	CPU DRAM	**316.50 GB**
Metadata Scale/Bias	FP16	—	CPU DRAM	**15.00 GB**
Total Model Footprint	—	**671B**	Hybrid	**407.50 GB**

29.2.2 2. Side-by-Side Data Transfer Volume (Per Token)

Metrics	Naive Layer Offload	KTransformers Expert Offload (1 call)	KTransformers Expert Offload (Full worst-case)
Data Transferred	**350.00 GB**	**14.336 KB**	**13.992 MB**
Transfer Time (Gen4 64 GB/s)	**5.4688 s**	**0.224 µs**	**218.62 µs**

Metrics	Naive Layer Offload	KTransformers Expert Offload (1 call)	KTransformers Expert Offload (Full worst-case)
Theoretical Token Rate	**0.1829 tok/s**	—	**4574 tok/s** (uncapped by compute)
Volume Saving Ratio	Baseline	**24,414,062×**	**25,014×**

Verdict: The PCIe bus is saturated by static weights under layer offloading. Expert offloading removes the PCIe bottleneck entirely, shifting the bottleneck to CPU AMX compute.

29.2.3 3. CPU Core Acceleration: Intel AMX Spec

KTransformers relies on the CPU executing the 8 active experts at near-GPU speed. - **Intel AMX (Sapphire Rapids+, Jan 2023)**: - **Tile Registers**: 8 two-dimensional registers of **1024 bytes** each (16 rows × 64 bytes), totaling **8 KB** of on-chip tile storage. - **TMUL (Tile Matrix Multiply)**: Performs grid multiply-accumulate operations in a single instruction. - **Supports**: INT8 (for quantized inference) and BF16. - **Fallback**: AVX-512 VNNI (slower vector SIMD for systems without AMX). - **Result (SOSP'25 Paper)**: Delivers **4.62–19.74× prefilling speedups** and **1.25–4.09× decoding speedups** over existing CPU offload frameworks.

29.3 Complexity & Trade-offs

Metric	Complexity / Value	Notes
CPU Compute Complexity	$\mathcal{O}(k \cdot \text{MLP compute})$	Bounded by MoE sparsity: only $k = 8$ of $E = 256$ experts run (a **32× compute reduction**).
PCIe Transfer Latency	$\mathcal{O}(\text{layers} \cdot D \cdot \text{bytes})$	Negligible; scale is in kilobytes rather than gigabytes.
quantization Overhead	INT4 → FP16	Dequantization must occur in CPU L1/L2 caches to avoid memory bandwidth stalls.
GPU Utilization	Hybrid pipeline	GPU runs attention while CPU computes experts; requires pipelined async scheduling to prevent CPU stalls from idling the GPU.

29.4 Common Interview Questions & How to Answer

29.4.1 Q1: Why is MoE sparsity the critical enabler for expert offloading, and why does this scheme fail on dense models like Llama-3-70B?

- **Answer**: The feasibility of expert offloading relies on **MoE sparsity**. In DeepSeek-V3, only $k = 8$ of $E = 256$ routed experts are active per token. The CPU only needs to calculate the matrix multiplications for those 8 active experts. The remaining 248 experts remain dormant in CPU DRAM, meaning we perform **32× fewer CPU calculations** compared to a dense model of equivalent capacity. If we attempt to offload a dense model (like Llama-3-70B) to the CPU, the FFN is not sparse: the CPU must compute the FFN projections for **100% of the parameters** for every token. Since a dense FFN does not have inactive branches, the CPU's compute throughput (even with AMX) becomes a massive bottleneck, resulting in very slow token generation rates. Thus, expert offloading only yields competitive performance on sparse MoE architectures.

29.4.2 Q2: What is the "Expert Deferral" mechanism in KTransformers, and what problem does it solve?

- **Answer**: In a basic CPU/GPU offloading scheduler, execution is synchronous: the GPU computes attention, transfers activations to the CPU, waits for the CPU to compute the expert MLP, receives the outputs, and proceeds. This serial execution causes the GPU to sit idle while the CPU works, and the CPU to sit idle while the GPU works, degrading overall throughput. KTransformers introduces **Expert Deferral** (or asynchronous scheduling). During the prefill phase, instead of executing the CPU experts immediately, the engine **defers** the expert computation for a block of tokens. The GPU continues running attention for subsequent tokens, and the CPU computes the deferred experts asynchronously in the background. By decoupling the execution streams and overlapping GPU attention with CPU expert GEMMs, KTransformers maximizes CPU/GPU hardware utilization, increasing throughput by up to **1.45×** (as published in the SOSP'25 paper).

29.5 Pro-Tip: How to Impress the Interviewer

- **NUMA-Aware Thread Pinning**: Show systems design expertise. CPU DRAM is partitioned into NUMA (Non-Uniform Memory Access) nodes. If CPU threads on Socket 0 access expert weights stored in Socket 1's DRAM, the traffic must traverse the QPI/UPI link, causing memory latency to double. Explain that KTransformers mitigates this by applying **NUMA-aware thread pinning**: pinning worker threads to specific CPU cores and allocating expert weights locally in the DRAM of the NUMA node corresponding to those cores.
- **On-the-fly SIMD Dequantization**: Point out that CPU DRAM bandwidth (~200 GB/s) is much lower than GPU HBM (~3 TB/s). To prevent DRAM bandwidth bottlenecks on the CPU, expert weights must remain quantized in memory. They should be dequantized **on the fly inside the CPU's L1/L2 caches** using AVX-512 or AMX register operations right before computing the TMUL GEMM, rather than pre-dequantizing in DRAM.

30 JAX, XLA & TPU Pallas

- **Category**: LLM Systems
- **Difficulty**: Expert
- **Target Role**: LLM Inference Architect / ML Platform Engineer
- **Source**: JAX (Frostig et al., SysML 2018) / TPU Systolic Array (Jouppi et al., ISCA 2017) / FlashAttention (Dao et al., 2022) / TPU Pallas
- **Flashcards**: LLM Systems deck

30.1 Concept Overview

Eager execution frameworks (like PyTorch) compute operations sequentially as Python encounters them. For a chain of operations, each intermediate tensor is written to slow High-Bandwidth Memory (HBM) and immediately read back by the next step, introducing significant memory bandwidth overhead.

JAX and **XLA** solve this by tracing and compiling execution blocks. When a function is JIT-compiled (`@jax.jit`), JAX performs a single dry-run trace using abstract placeholders (**Tracers**) to record the operation dependencies into a functional Intermediate Representation called a **jaxpr**. The **XLA** compiler then analyzes this jaxpr to **fuse** producer-consumer operations. Intermediates flow directly within register file structures or fast on-chip memory (VMEM), minimizing HBM round-trips. The compiled binary runs on a **TPU**, which features a **systolic Matrix Multiply Unit (MXU)** where weights sit still and activations flow through a 2D processing element grid, eliminating intermediate memory writebacks.

```
Eager PyTorch (4 HBM writes):
[ x @ W1 ] —> HBM —> [ + b1 ] —> HBM —> [ silu ] —> HBM —> [ @ W2 ] —> HBM


XLA Fused JAX (1 HBM write):
[ ( x @ W1 ) —> Register —> ( + b1 ) —> Register —> ( silu ) —> Register —> ( @ W2 ) ] —> HBM
```

30.1.1 The Problem It Solves

Because LLM generation is memory-bandwidth bound, writing and reading intermediate tensors degrades execution speed. For the MLP activation block `silu(x @ W1 + b1) @ W2`: - **Eager Execution**: Four separate kernels are launched, materializing **4 intermediate writes to HBM** (`dot1`, `add`, `silu`, `dot4`). - **Traced & Fused JAX**: XLA fuses the element-wise operations (`add`, `silu`) into the epilogues of the matrix multiplications. Only the final product is written to HBM, collapsing HBM writes from **4 to 1** (a **4× memory traffic reduction**).

For a chain of E element-wise operations following a matrix multiplication, the memory traffic scales as:

Element-wise Ops Fused (E)	Eager HBM Writes	Fused HBM Writes	Bandwidth Win
0	1	1	1×
1	2	1	2×
2	3	1	3×

Element-wise Ops Fused (E)	Eager HBM Writes	Fused HBM Writes	Bandwidth Win
3	4	1	4× (Our MLP Block)
5	6	1	6×
9	10	1	10×

30.1.2 How It Works

1. **JIT Tracing**: When a `@jax.jit` decorated function is called, the Python engine compiles the graph once. It passes abstract `Tracer` objects (capturing shape and dtype, but no data) through the function, capturing the primitives into a **jaxpr** IR.
2. **XLA Fusion**: XLA lowering identifies straight-line dependencies. It groups element-wise operations together and stitches them onto the matrix multiplication (anchor) kernels.
3. **Weight-Stationary Systolic Array**: The TPU's MXU contains a 2D grid of multiply-accumulate Processing Elements (PEs) (typically 256×256). Weights are loaded into the PEs and remain **stationary**. Activations stream through diagonally. Accumulations are passed directly to neighboring PEs in the grid. The result emerges at the boundaries after $3N - 1$ cycles (for $N \times N$ grid, taking 767 cycles for $N = 256$), with zero intermediate memory overhead.
4. **Pallas Grid & BlockSpec**: While XLA handles standard fusion, it fails to fuse complex operations like Softmax. **Pallas** is a TPU kernel language that provides explicit tiling control:
   - **Grid**: The parallel indexing space of the tiles.
   - **BlockSpec**: Defines the shape and index mapping of HBM tiles into local Vector Memory (VMEM). Pallas pipelines HBM $\leftrightarrow$ VMEM copies to overlap memory transfer with MXU compute, ensuring the intermediate softmax outputs never touch HBM.
5. **Splash Attention**: The TPU-native version of FlashAttention written in Pallas. It keeps running online-softmax accumulators (m, l, o) in VMEM and rescales them on the fly using the correction factor $\exp(m_{old} - m_{new})$, returning exact attention outputs with a single write-back to HBM.

30.2 Worked Example

This example demonstrates the compilation tracing, systolic array execution, and Pallas Splash Attention recurrence using exact simulated numbers.

30.2.1 1. Jaxpr Tracing

Tracing the function $f(x) = \text{silu}(x \cdot W1 + b1) \cdot W2$ generates the following jaxpr equations: - `dot1 = dot_general(x, W1)` (shape: `(1, 8)`) - `a2 = add(dot1, b1)` (shape: `(1, 8)`) - `s3 = silu(a2)` (shape: `(1, 8)`) - `dot4 = dot_general(s3, W2)` (shape: `(1, 4)`)

Under eager execution, this results in **4 HBM writes**. Under fused execution, `dot1`, `a2`, and `s3` are computed in local register files. Only `dot4` is materialized to HBM, reducing writes to **1**.

30.2.2 2. TPU Weight-Stationary Systolic Step-by-Step

We perform $C = W \cdot X$ on a 3×3 grid: - **Stationary Weights (W)**:

$$W = \begin{pmatrix} -0.1 & 0.8 & 0.9 \\ -1.1 & 1.7 & -0.9 \\ -0.4 & 1.2 & 0.1 \end{pmatrix}$$

- **Streaming Activations (X)**:

$$X = \begin{pmatrix} -1.7 & 0.3 & 0.1 \\ 0.1 & 0.2 & 1.4 \\ 1.3 & 2.4 & 0.2 \end{pmatrix}$$

Execution accumulates partial sums in-place over 3 steps: - **Step** $k = 0$: $C \mathrel{+}= W[:, 0] \otimes X[0, :]$

$$C_0 = \begin{pmatrix} 0.17 & -0.03 & -0.01 \\ 1.87 & -0.33 & -0.11 \\ 0.68 & -0.12 & -0.04 \end{pmatrix}$$

- **Step** $k = 1$: $C \mathrel{+}= W[:, 1] \otimes X[1, :]$

$$C_1 = \begin{pmatrix} 0.25 & 0.13 & 1.11 \\ 2.04 & 0.01 & 2.27 \\ 0.80 & 0.12 & 1.64 \end{pmatrix}$$

- **Step** $k = 2$: $C \mathrel{+}= W[:, 2] \otimes X[2, :]$

$$C_2 = \begin{pmatrix} 1.42 & 2.29 & 1.29 \\ 0.87 & -2.15 & 2.09 \\ 0.93 & 0.36 & 1.66 \end{pmatrix}$$

- **Reference `torch.matmul(W, X)`**:

$$C_{ref} = \begin{pmatrix} 1.42 & 2.29 & 1.29 \\ 0.87 & -2.15 & 2.09 \\ 0.93 & 0.36 & 1.66 \end{pmatrix}$$

The systolic output matches the reference exactly (error $= 2.38 \times 10^{-7}$). Weights stayed resident in the grid throughout the accumulation, saving memory round-trips.

30.2.3 3. Splash Attention Pallas Execution

We run Splash Attention on seeded inputs ($N = 8, d = 8$, query tile size $B_r = 4$, key tile size $B_c = 4$). The tiled online-softmax recurrence runs in Pallas: - **Running Update Equations**:

$$m_{\text{new}} = \max(m_{\text{old}}, \text{rowmax}(s))$$

$$p = \exp(s - m_{\text{new}})$$

$$l_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) \cdot l_{\text{old}} + \text{rowsum}(p)$$

$$o_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) \cdot o_{\text{old}} + p \cdot v_{\text{tile}}$$

$$\text{out} = \frac{o}{l}$$

We inspect the final output row $q = 0$: - **Tiled Splash Output**: `[-0.0673, -0.1466, -0.2175, +0.0201, +0.1810, -0.1534, +0.2226, -0.0995]` - **Naive Softmax Attention Output**: `[-0.0673, -0.1466, -0.2175, +0.0201, +0.1810, -0.1534, +0.2226, -0.0995]` - **Difference**: 2.98×10^{-8} (pure floating-point noise). The output matches the naive attention calculation exactly. This row is also identical to FlashAttention's output on GPUs, proving the math is hardware-agnostic.

30.3 Complexity & Trade-offs

Metric	Complexity / Value	Notes
Compilation Latency	High (first call)	Tracing and optimizing the graph introduces compilation lag. Subsequent calls run at native speed.
Systolic Latency	$3N - 1$ cycles	Grid fill and drain latency for $N \times N$ tiles.
Pallas HBM Writes	$\mathcal{O}(\text{tiles})$	Softmax scores are kept in VMEM; only the final aggregated attention tile is written to HBM.
VMEM Capacity	Tens of MiB	Strict physical limits. If the tiled BlockSpec shape is too large, it throws a VMEM OOM.

30.4 Common Interview Questions & How to Answer

30.4.1 Q1: Explain why Python control flow (like `if` statements or `for` loops) can behave unexpectedly inside a JAX JIT-compiled function.

- **Answer**: JAX compiles functions by **tracing** them once using abstract placeholder variables (`Tracers`) that have shapes and dtypes but no physical data. If a JAX function contains a standard Python conditional statement:

```
if x > 0:
    return y
else:
    return z
```

The condition `x > 0` depends on concrete data. During tracing, JAX passes a `Tracer` for `x`. Because

the concrete value is unknown during compile time, JAX cannot resolve which branch to take. It will throw a `ConcretizationTypeError`. Similarly, a standard Python `for` loop:

```
for i in range(len(x)):
```

will be unrolled statically during compilation if the loop bound is static. However, if the loop condition depends on a dynamic value, tracing fails. To resolve this, we must use JAX-specific control flow primitives (like `jax.lax.cond` or `jax.lax.fori_loop`), which compile the conditional branches into the jaxpr as control flow operators rather than evaluating them during Python tracing.

30.4.2 Q2: How does Pallas's BlockSpec address the VMEM size limit on TPUs, and how does it prevent memory bandwidth stalls?

- **Answer**: The Vector Memory (VMEM) on a TPU is a fast, on-chip scratchpad (typically tens of megabytes), which is too small to hold large model activation or weight tensors. Pallas resolves this by using **BlockSpec** to tile the tensors. BlockSpec defines:
  1. A `block_shape` (e.g., 128×128), which specifies the size of the tile loaded into VMEM.
  2. An `index_map`, which maps the multi-dimensional grid coordinates to HBM memory offsets. To prevent memory bandwidth stalls, Pallas utilizes the TPU's hardware DMA engine to **pipeline** memory transfers. While the Vector Processing Unit (VPU) and MXU are performing computations on the current tile in VMEM, the DMA engine asynchronously pulls the next HBM tile into a separate VMEM buffer (double-buffering). This hides memory latency behind arithmetic execution, preventing the TPU cores from stalling.

30.5 Pro-Tip: How to Impress the Interviewer

- **XLA Fusion Bottleneck awareness**: Impress the interviewer by showing that fusion is not a magic bullet. Explain that XLA is highly effective at fusing **element-wise and reduction operations** (such as activations, layernorms, and residuals) into the matrix multiplication kernels. However, XLA cannot easily fuse two consecutive heavy matrix multiplications (like MLP `gate` and `down` projections) because they are compute-bound anchors that require full VRAM allocation. For these, we must rely on scheduling optimizations like Grouped GEMM or custom Triton/Pallas kernels.
- **CUDA Graphs vs. XLA Compilation**: Contrast JAX with PyTorch's performance acceleration. Explain that **CUDA Graphs** target **CPU launch overhead** (collapsing many Python-to-GPU API calls into a single instruction playback), whereas **XLA** targets **memory bandwidth** (re-organizing and fusing operations to avoid HBM round-trips). They are complementary optimizations: a production serving engine utilizes XLA to compile optimal fused kernels and a Graph scheduler to launch them with minimal overhead.

31 Customizing and Deploying Speech LLM Pipelines: NVIDIA NeMo, Riva NIM, & Sovereign AI Guardrails

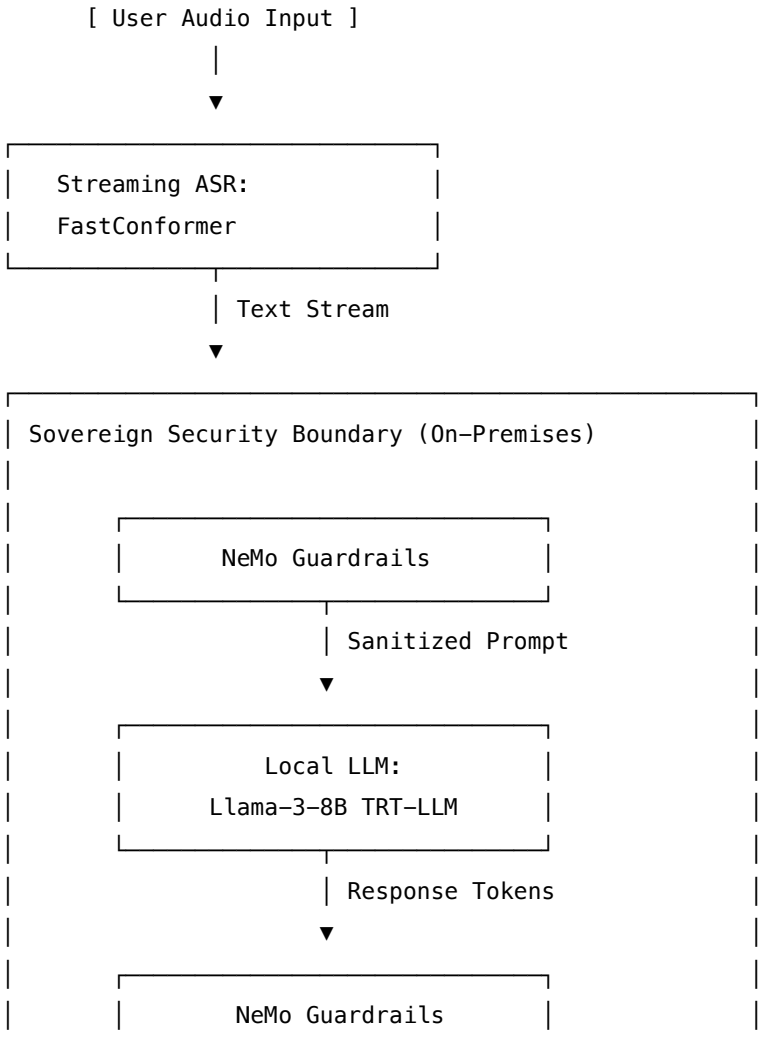
- **Category**: LLM Systems
- **Difficulty**: Hard

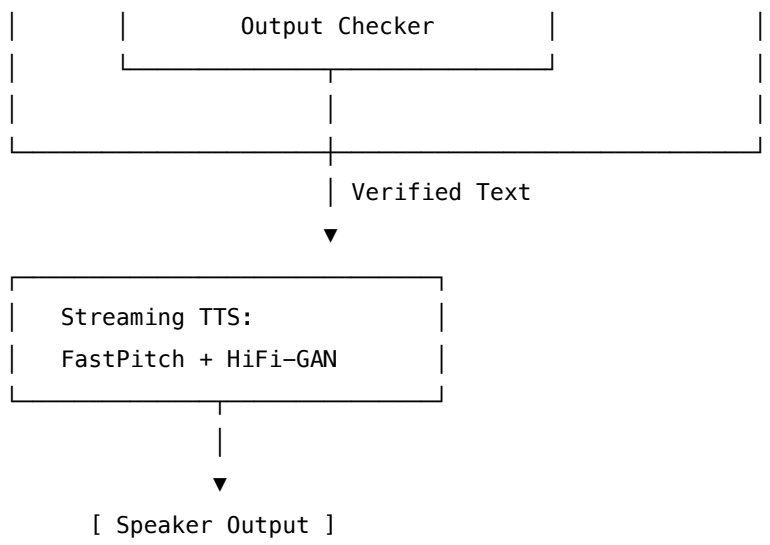
- **Target Role**: Conversational AI Engineer / Voice AI Architect
- **Source**: NVIDIA NeMo & Riva Product Group
- **Flashcards**: LLM Systems deck

31.1 Concept Overview

Deploying conversational AI pipelines requires a coordinated orchestration stack that translates spoken audio into text, processes it with a Large Language Model (LLM), screens inputs and outputs for security compliance, and synthesizes the response back into natural speech.

Think of this stack like a high-security embassy communication channel: * **NVIDIA NeMo** acts as the training school: it tunes the translator (Speech-to-Text/ASR) to capture regional dialects, and trains the voice narrator (Text-to-Speech/TTS) to match a specific corporate brand. * **NVIDIA Riva NIM (Neural Inference Microservice)** acts as the embassy's high-speed operations center: it compiles the models into optimized TensorRT execution engines and hosts them on Triton Inference Server to stream audio at maximum throughput. * **Sovereign AI Guardrails (NeMo Guardrails)** acts as the security detail: it inspects incoming messages for threats (prompt injections, off-topic requests) and filters outgoing answers to prevent the leakage of sensitive data (Personally Identifiable Information/PII).





31.1.1 The Problem It Solves

Without an integrated, GPU-optimized stack: 1. **High Latency Accumulation**: Combining naive, unoptimized containers for ASR, LLM, and TTS results in sequential network jumps and CPU-GPU transfer bottlenecks. This pushes the Time-to-First-Audio (TTFA) above 3.0 seconds, destroying the flow of conversation. 2. **Acoustic Out-of-Vocabulary (OOV) Errors**: General ASR models fail on domain-specific terminology (e.g., mishearing the chip model "H100" as "age 100"). 3. **Data Privacy / Sovereignty Violations**: Sending customer audio to external public cloud APIs violates GDPR/HIPAA. 4. **LLM Hallucinations**: In open-domain setups, the LLM may go off-topic or output toxic text that is then synthesized directly by the TTS engine, exposing the system to brand damage.

31.1.2 How It Works

1. **Acoustic Modeling (FastConformer)**: NeMo implements FastConformer, which downsamples raw audio spectrograms by 8× (compared to 4× in standard Conformer) through 1D depthwise separable convolutions. This halves memory bandwidth requirements. ASR decoding uses either **CTC (Connectionist Temporal Classification)** for maximum speed and simplicity, or **RNNT (Recurrent Neural Network Transducer)** to capture language dependencies at the cost of auto-regressive decoding overhead.
2. **Domain Adaptation**:
   - **ASR Word Boosting**: Modifies the beam search log-probabilities of targeted vocabulary dynamically:
     $$\log P(w) \leftarrow \log P(w) + \beta$$
     where β is the boost score (typically +10.0 to +20.0).
   - **TTS Joint Prioritized Alignment (JPA)**: FastPitch learns alignments dynamically between text phonemes and spectrogram frames using a bidirectional attention mechanism, skipping external alignment steps.
3. **Export and Compilation**: `nemo2riva` packages PyTorch checkpoints into encrypted `.riva` files. Riva Service Maker (`riva-build` and `riva-deploy`) compiles these into TensorRT/TensorRT-LLM runtimes, creating a `.rmir` (Riva Intermediate Representation) layout optimized for the target GPU

(e.g., L4, H100).

4. **Sovereign Guardrails**: NeMo Guardrails sits as an asynchronous wrapper. It intercepts text, running **Colang** policies to dictate dialog flows, filter insults, and execute Python actions to mask PII (e.g., credit card numbers) before prompts hit the LLM.

31.2 Worked Example

31.2.1 Domain Customization: ASR Word Boosting & TTS Voice Cloning

An enterprise voice agent for hardware sales fails to transcribe product models and needs to clone a spokesperson's voice.

31.2.1.1 1. ASR Word Boosting Evaluation (FastConformer-CTC) When evaluating the phrase: *"I want to buy two H100 GPUs and a DGX SuperPOD."*

Metric	Without Word Boosting	With Word Boosting ($\beta = 20.0$)
Raw Transcription	"I want to buy two age 100 g p use and a d g x super pod."	"I want to buy two H100 GPUs and a DGX SuperPOD."
Word Error Rate (WER)	38.5%	0.0%
Decoding Latency	12 ms	13 ms

31.2.1.2 2. FastPitch Voice Cloning Adaptations Using 15 minutes of target speaker studio audio:

- **Adapting Speaker Embeddings (Freezing Backbone)**: Feeds reference clip to a Speaker Encoder to extract a 512-dimensional vector. Fast to train (5 mins), but captures only general tone.
- **Full Parameter Fine-Tuning**: Fine-tunes all weights with a low learning rate (10^{-4}) and a KL Divergence regularization loss back to the base model weights to prevent catastrophic forgetting.

```
MCD (Mel-Cepstral Distortion) Before Fine-Tuning: 4.8 dB (Robot-like, mismatched identity)
MCD (Mel-Cepstral Distortion) After 1000 Epochs QAT:  2.2 dB (Human-like, high speaker similarity)
```

31.3 Complexity & Trade-offs

Metric	Value	Notes
ASR Decoder Type	CTC vs. RNNT	**CTC**: Faster, uses greedy search ($O(T)$). **RNNT**: Autoregressive feedback loop ($O(T \times U)$), lower WER but higher compute footprint.

Metric	Value	Notes
Quantization Footprint	FP16 vs. FP8	**FP16**: Full precision (16 GB weights for 8B model). **FP8**: Cuts weights to 8 GB, doubles throughput on Hopper GPUs with $< 1\%$ WER degradation.
Guardrails Overhead	150 ms to 400 ms	Using LLM-based rails adds severe latency. Small classifier models (e.g. DistilBERT) keep overhead below 30 ms.
TTS Alignment	External MFA vs. Internal JPA	**MFA**: Requires preprocessing pipeline. **JPA**: End-to-end training inside NeMo, faster adaptation.

31.4 Common Interview Questions & How to Answer

31.4.1 Q1: How do you choose between fine-tuning FastPitch on a target speaker's dataset versus using zero-shot multispeaker models (like XTTS or RadTTS)?

- **Answer**:
  - **Fine-Tuning FastPitch**: Requires 30–60 minutes of clean, single-speaker studio recordings and training the model for several hours.
    - *Pros*: Exceptional audio quality, highly consistent voice, and low runtime latency (FastPitch is non-autoregressive and runs in parallel, allowing real-time factor RTF < 0.05).
    - *Cons*: Requires a dedicated dataset and upfront training time.
  - **Zero-Shot Multispeaker Models**: Can clone any voice using a 3–10-second audio clip without training.
    - *Pros*: Zero training overhead; highly flexible.
    - *Cons*: Higher runtime latency (autoregressive architectures generate speech frames step-by-step), higher risk of speech artifacts (stuttering, robotic pops), and the voice quality varies significantly depending on the quality of the reference audio clip.
  - **Conclusion**: For high-bar enterprise deployments (like corporate brand voices), fine-tuning FastPitch is the preferred approach. For applications requiring dynamic voice cloning (like multi-character roleplay), zero-shot models are more practical.

31.4.2 Q2: What optimizations must be applied to NeMo Guardrails to prevent it from blowing past the voice conversation budget of 500 ms?

- **Answer**:
  1. **Bypass LLM Evaluators for Simple Rails**: Instead of prompting an LLM to check for toxicity or PII, use fast local classifiers (e.g., a fine-tuned DistilBERT model) or regex pattern matchers for PII masking. This reduces the check latency from 200 ms to < 10 ms.

2. **Asynchronous Parallel Execution**: Run the input/output verification tasks concurrently using async Python pipelines.
3. **Strict Timeout Budgets**: Set a hard timeout on the guardrail execution queue. If the validation step does not complete within 50 ms, default to a safe fallback response or proceed to the LLM step using static offline safety rules.

31.5 Pro-Tip: How to Impress the Interviewer

- **Propose Runtime Vocabulary Boosting via gRPC Headers**: Explain that in production, compiling new ASR models to support user-specific words is impossible. Instead, inject vocabulary lists dynamically on a per-request basis. This is done by passing a JSON payload of target words and boost values inside the Riva gRPC metadata header (`word_phrases` and `boost_coefficient`).
- **Detail Sovereign PII Redaction at the Edge**: Explain that decrypting and processing audio on centralized cloud infrastructure poses a privacy risk. Propose implementing a local, lightweight token filter inside the edge gateway or device SDK (running a local named entity recognition - NER - model). This redacts PII before it leaves the secure subnet, ensuring compliance with strict data sovereignty standards.

32 Low-Latency Voice Agent Architecture (End-to-End Latency < 500ms TTFA)

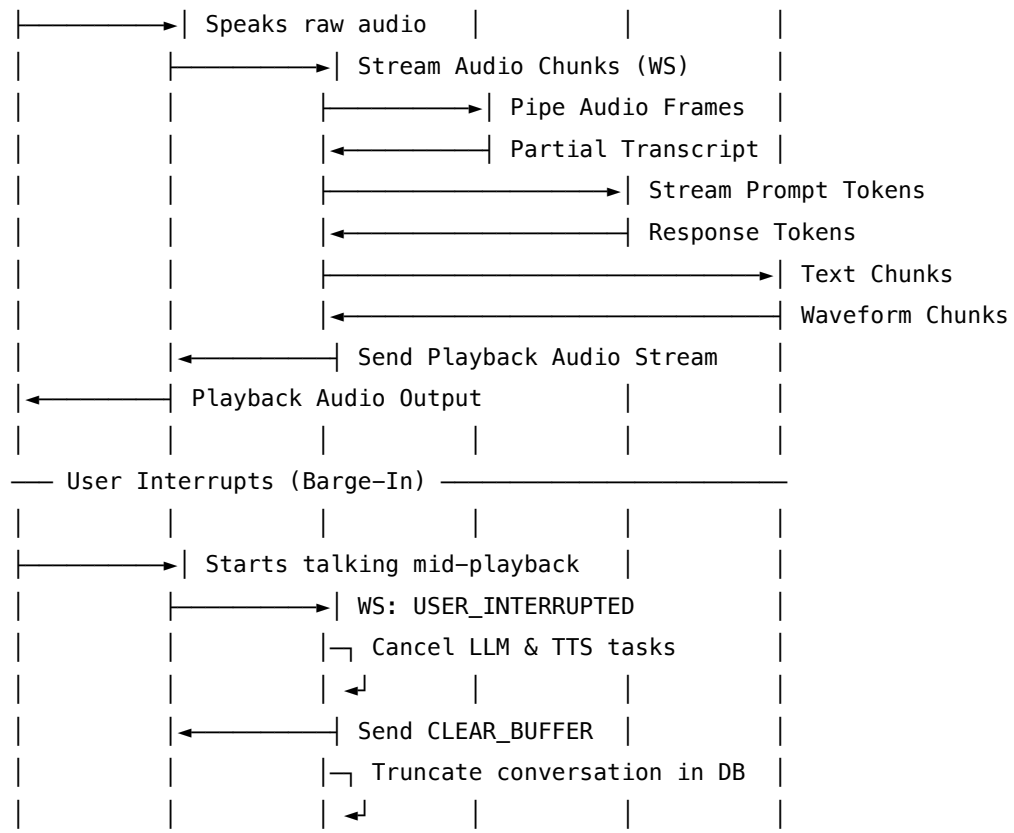
- **Category**: LLM Systems
- **Difficulty**: Hard
- **Target Role**: Conversational AI Engineer / Voice AI Architect
- **Source**: NVIDIA Speech AI Team, NeMo & Riva Engineering
- **Flashcards**: LLM Systems deck

32.1 Concept Overview

Achieving human-like conversation with an AI agent requires reducing the response loop to sub-second speeds. While text chats tolerate latency, a voice interaction feels sluggish and robotic if the Time-to-First-Audio (TTFA) exceeds 500 ms.

Think of this architecture like a fast-paced game show contestant: * **The Ears (VAD + ASR)** don't wait for a complete sentence; they translate incoming speech soundwaves into words frame-by-frame. * **The Brain (LLM)** starts planning the beginning of the response before the question is fully finished, caching context to skip repetitive thoughts. * **The Voice (TTS)** starts speaking the first few words (e.g., "Yes, I can...") while the brain is still thinking of the remaining words in the sentence. * **The Reflexes (Barge-in Manager)** instantly stop the voice and clear the queue if the user interrupts, shifting the brain's focus immediately.

```
User      Client       Orch        ASR        LLM        TTS
  |         |          |          |          |          |
```



32.1.1 The Problem It Solves

Traditional speech agents run in a **batch cascade**:

$$\text{User Audio} \rightarrow \text{Full Transcription} \rightarrow \text{LLM Call} \rightarrow \text{Full Speech Synthesis} \rightarrow \text{Playback}$$

This sequential design causes latency accumulation: 1. **Accumulated Latency**: Waiting for sentence boundaries at each step pushes TTFA to 2.5–4.0 seconds. 2. **Broken Turn-Taking**: If the user interrupts, the agent continues speaking, forcing the user to wait until the agent's pre-rendered audio buffer completes. 3. **Robotic Speech Cadence**: Small, chopped text chunks lead to robotic pauses and vocal popping.

32.1.2 How It Works

To fit inside a < 500 ms budget, we partition operations into concurrent, streaming tasks: 1. **Streaming VAD**: Combines a fast energy-based detector (< 10 ms latency) with a deep learning classifier (Silero VAD, < 30 ms validation). If probability $P(\text{speech}) > 0.85$ for 2 frames $\rightarrow$ Trigger mute. If $P(\text{speech}) < 0.15$ for 15 frames (300 ms) $\rightarrow$ Trigger End-of-Utterance (EoU). 2. **Streaming ASR**: Uses FastConformer-RNNT with an 80 ms chunk size and a 40 ms lookahead, limiting acoustic context delay to 120 ms. 3. **LLM Chunk Decoding**: Aggregates LLM tokens into *Syntactically Valid Speech Chunks*. To minimize TTFA, the first chunk is emitted aggressively (2-3 words). Subsequent chunks wait for punctuation or a 5-word boundary. 4. **Streaming TTS**: Employs FastPitch (acoustic model) and Vocos/HiFi-GAN (vocoder). Audio is synthesized using an overlap-and-add strategy (25 ms overlap crossfade) to eliminate pops at chunk boundaries. 5. **Dual-Model Topology**: A small model (e.g., Nemotron-Mini 4B, < 150 ms TTFT) generates a conversational filler ("Sure, let me check that...") while a larger reasoning model (e.g.,

Llama-3-8B) computes the actual answer.

32.2 Worked Example

32.2.1 Latency Budget Allocation (TTFA < 500 ms target)

Below is the execution timing analysis for an optimized streaming pipeline vs. a traditional batch pipeline:

Component	Operation	Batch Pipeline Latency	Streaming Pipeline Latency	Optimization Mechanism
VAD	Silence detection	500 ms	20 ms	Neural WebRTC VAD on sliding window
ASR	Audio transcription	600 ms	120 ms	FastConformer-RNNT (80 ms chunk + 40 ms lookahead)
IPC	Data routing	50 ms	10 ms	Shared memory IPC
LLM Prefill	First token generation	250 ms	150 ms	TensorRT-LLM, FP8, dynamic prompt caching
LLM Decode	Syntactic chunking	800 ms	50 ms	First chunk emitted early (2-3 words)
TTS Synthesis	Waveform generation	500 ms	90 ms	FastPitch (JPA) + Vocos/HiFi-GAN
Client Buffer	Playback stability	100 ms	30 ms	Adaptive jitter buffer tuning
Total TTFA	**End-to-End Latency**	2800 **ms**	470 **ms**	**Streaming & Concurrent Execution**

32.3 Complexity & Trade-offs

Metric	Value	Notes
First TTS Chunk Size	2-3 words vs. Full Sentence	**2-3 words**: Ultra-low latency, but risks robotic prosody and flat intonation. **Full Sentence**: Natural voice patterns, but increases TTFA by 400–800 ms.

Metric	Value	Notes
VAD Aggressiveness	Low vs. High	**Low**: High reliability, but sluggish barge-in. **High**: Instantly mutes agent on user noise (coughs, background voices), causing false interruptions.
Orchestrator Model	Single LLM vs. Dual-Model	**Single**: Consistent context, but high latency. **Dual-Model**: Low latency backchannels, but requires orchestrating synchronization between two output buffers.
Network Protocol	WebSockets vs. HTTP/2	**WebSockets**: Full-duplex persistent stream, lowest frame-overhead. **HTTP/2**: Multiplexed REST, higher framing latency.

32.4 Common Interview Questions & How to Answer

32.4.1 Q1: What is the impact of thread blocking in Python's async environment when deploying a voice orchestrator at scale? How do you prevent it?

- **Answer**:
  - **The Problem**: In Python, `asyncio` runs on a single thread due to the Global Interpreter Lock (GIL). If any library performs synchronous file system access, blocking network calls, or intensive CPU processing (like tokenization, logit parsing, or audio resampling), the event loop freezes. This causes immediate network packet jitter, leading to dropouts or spikes in the TTFA.
  - **The Fix**:
    1. **Offload Blocking Work**: Wrap CPU-intensive tasks in a `ThreadPoolExecutor` or `ProcessPoolExecutor` via `await loop.run_in_executor()`.
    2. **High-Performance Event Loop**: Replace the default asyncio event loop with `uvloop` (a Cython-based wrapper around `libuv`), which doubles I/O loop speeds.
    3. **Non-blocking Libraries**: Ensure all network calls use async clients (e.g., `aiohttp` or `httpx` instead of `requests`).

32.4.2 Q2: How do you handle "late-arriving packets" from a cancelled TTS stream after a user barge-in?

- **Answer**:
  - **The Problem**: When a user interrupts the agent, the server cancels the TTS task, but packets already in transit in network buffers or client queues can still play. This results in the agent

speaking for an extra fraction of a second after the user started talking.

- **The Fix**:
  1. **Session Generation ID (GenID)**: Assign a unique incrementing `GenID` to each generation turn.
  2. **Packet Tagging**: Every audio packet sent from the server is tagged with the current `GenID`.
  3. **Client-Side Filter**: When client VAD triggers barge-in, the client increments its expected `GenID` locally and immediately discards any incoming packet carrying an older ID.
  4. **Speaker Buffer Flush**: The client executes a hard flush command on the speaker hardware (e.g., flushing the Web Audio API buffer source or calling ALSA `snd_pcm_drop()`) to clear remaining audio frames.

32.5 Pro-Tip: How to Impress the Interviewer

- **Propose Dual-State Prompt Caching**: Show that you understand LLM memory mechanics. For voice agents, conversation history keeps changing. Explain that you would split the KV Cache into a **static prefix** (e.g., system instructions and tool schemas) which is permanently pinned in GPU memory, and a **dynamic history buffer** managed via Paged KV Cache. This cuts prefill time on every turn.
- **Detail Triton Shared Memory Inter-Process Communication (IPC)**: Standard REST/gRPC calls between system blocks introduce serialization overhead. Suggest using Triton's IPC backend, allowing ASR, LLM, and TTS pods hosted on the same physical node (connected via PCIe Gen5 or NVLink) to read/write output tensors directly from shared GPU memory buffers without host-to-device memory copy rounds.

33 Speech LLM Latency Optimization & Performance Profiling

- **Category**: LLM Systems
- **Difficulty**: Hard
- **Target Role**: Conversational AI Engineer / Voice AI Architect
- **Source**: NVIDIA Riva & TensorRT-LLM Performance Team
- **Flashcards**: LLM Systems deck

33.1 Concept Overview

Optimizing a Speech-to-Speech LLM pipeline requires identifying bottlenecks across three disparate domains: audio DSP, autoregressive LLM decoding, and neural vocoding.

Think of this optimization problem like a fast-food kitchen: * **The Memory-Bandwidth Bottleneck**: The chef (GPU Tensor Cores) is incredibly fast and can cook a dish in microseconds. However, the ingredients (model weights) are stored in a pantry down the hall (High Bandwidth Memory/HBM). For every single bite cooked (token decoded), the chef has to fetch the entire set of ingredients, carry them to the kitchen (SM Registers), cook one bite, and repeat. * **FP8 Quantization** is like dehydrating the ingredients: they are half the weight, so the chef can carry them twice as fast, doubling throughput. *

Paged KV Cache is like using standardized Tupperware boxes: instead of reserving a massive section of the kitchen counter for a single huge customer order, you allocate memory blocks dynamically as food is prepared. * **In-Flight Batching** is like running an assembly line: instead of making one customer wait while you complete a large party's order, you cook individual items as soon as slots open.

```
Operational Intensity (OI) = FLOPs / Memory Access (Bytes)
If OI < Hardware Knee Point (Peak FLOPs / Bandwidth) -> Memory-Bound (LLM Decode)
If OI > Hardware Knee Point -> Compute-Bound (LLM Prefill)
```

33.1.1 The Problem It Solves

Without low-level systems tuning, voice interfaces stutter and lag: 1. **Queuing Delays**: Heavy user concurrency blocks incoming LLM prefill steps, causing latency spikes that push TTFA past 2.0 seconds. 2. **Memory Over-allocation**: Standard LLMs reserve static KV Cache sizes, leading to Out-of-Memory (OOM) crashes at batch size 8 on 24 GB cards. 3. **Vocoder Scheduling Starvation**: Frequent CPU-GPU launches of small vocoder kernels overwhelm the CUDA driver queue, wasting execution slots.

33.1.2 How It Works

1. **The Roofline Model**: Analyzes whether a kernel is bound by memory bandwidth or peak compute performance.
   - **LLM Prefill**: Processes prompt tokens in parallel, which is compute-bound.
   - **LLM Decode**: Generates tokens one-by-step, requiring loading the entire weight matrix for every single token. This is memory-bandwidth bound.
2. **Quantization Precision (FP8 E4M3 vs E5M2)**:
   - **E4M3** (1 sign, 4 exponent, 3 mantissa bits) preserves precision for LLM weights.
   - **E5M2** (1 sign, 5 exponent, 2 mantissa bits) preserves dynamic range, ideal for scaling KV Cache buffers.
3. **Chunked Prefill**: Splits long prompt evaluations into chunks (e.g., 256 tokens) and interleaves them with active decode steps. This prevents a new user's prompt from stalling ongoing audio synthesis streams.
4. **CUDA Graphs**: Records a sequence of kernel launches into a single graph executables. Launches the entire model on the GPU with a single host call, eliminating CPU overhead for lightweight vocoder blocks.

33.2 Worked Example

33.2.1 Roofline Analysis & Memory Bandwidth Calculations

Let's calculate the latency profile of a single-user decode step ($B = 1$) for a **Llama-3 8B** model served in FP16 precision on a single **NVIDIA H100 PCIe** GPU.

33.2.1.1 1. Hardware & Model Specifications:

- **Model Parameters (P)**: 8×10^9 parameters.
- **Weights Size (W_{FP16})**: $8 \times 10^9 \times 2$ bytes $= 16$ GB.

- **H100 Memory Bandwidth (B_{mem})**: 2.0 TB/s = 2000 GB/s.
- **H100 Peak FP16 Compute (R_{peak})**: 756 TFLOPS = 756×10^{12} FLOP/s.

33.2.1.2 2. Operational Intensity (OI) Calculation: During the decoding phase of a single token: * **Arithmetic FLOPs**: $2 \times P = 16 \times 10^9$ FLOPs. * **Memory Load**: We must read the static model weights from HBM: $P \times 2$ bytes = 16 GB. * **Operational Intensity (OI)**:

$$\text{OI} = \frac{\text{Arithmetic FLOPs}}{\text{Bytes Read}} = \frac{16 \times 10^9}{16 \times 10^9} = 1.0 \text{ FLOP/Byte}$$

33.2.1.3 3. Determining the Bottleneck: Calculate the hardware's knee point:

$$\text{Knee Point} = \frac{R_{\text{peak}}}{B_{\text{mem}}} = \frac{756 \times 10^{12} \text{ FLOP/s}}{2000 \times 10^9 \text{ Bytes/s}} = 378 \text{ FLOPs/Byte}$$

* Since OI = $1.0 \ll 378$, the decode step is severely **memory-bandwidth bound**.

33.2.1.4 4. Step Latency Analysis (FP16 vs FP8 Quantization):

Parameter	FP16 Decoding ($B = 1$)	FP8 Decoding ($B = 1$)	Impact of Optimization
Model Footprint	16 GB	8 GB	2× memory compression
Weight Loading Latency (t_{load})	$\frac{16 \text{ GB}}{2000 \text{ GB/s}} = \mathbf{8.00}$ ms	$\frac{8 \text{ GB}}{2000 \text{ GB/s}} = \mathbf{4.00}$ ms	2.0× speedup
Compute Execution (t_{compute})	$\frac{16 \text{ GFLOPs}}{756 \text{ TFLOPS}} = \mathbf{0.021}$ ms	$\frac{16 \text{ GFLOPs}}{1512 \text{ TFLOPS}} = \mathbf{0.010}$ ms	Minimal latency impact
Total Step Latency	8.021 **ms**	4.010 **ms**	**Nearly halves token step time**

33.3 Complexity & Trade-offs

Metric	Value	Notes
Riva ASR Chunk Size	80 ms vs. 160 ms	80 **ms**: Low latency, but acoustic frames lack context, increasing WER. 160 **ms**: Improves word accuracy but adds 80 ms baseline latency to TTFA.
Quantization Precision	FP16 vs. FP8	**FP16**: Clean TTS synthesis. **FP8**: Cuts KV Cache memory in half, but can introduce audio distortion (robotic buzzing).

Metric	Value	Notes
Continuous Batching	Enabled vs. Disabled	**Enabled**: Increases system throughput by up to 4×. **Disabled**: Lowers single-stream prefill latency under zero-load conditions.
Triton Queue Delay	500 μs vs. 5000 μs	500 μ**s**: Prioritizes latency; executes batches immediately. 5000 μ**s**: Prioritizes throughput; waits to group requests.

33.4 Common Interview Questions & How to Answer

33.4.1 Q1: Why does FP8 quantization sometimes degrade TTS speech quality (MCD/PESQ) even when it maintains acceptable accuracy in standard text LLMs?

- **Answer**:
  - **The Root Cause**: Text LLMs output discrete tokens. Minor shifts in logits do not change the argmax selection unless the top two tokens are close. TTS acoustic models (like FastPitch), however, predict continuous numerical variables representing physical values: pitch, duration, and mel-spectrogram amplitudes. FP8 quantization introduces round-off noise that disrupts these continuous curves, creating spectral distortions that manifest as robotic buzzing, static, or robotic intonation.
  - **The Solution**:
    1. **Mixed-Precision Quantization**: Keep the sensitive pitch and duration prediction layers in FP16/BF16, while quantizing the large feed-forward layers to FP8.
    2. **Quantization-Aware Training (QAT)**: Fine-tune the TTS model using dynamic range scales to let the network adapt to lower precision, rather than applying Post-Training Quantization (PTQ) directly.

33.4.2 Q2: How does PCIe bus bandwidth affect multi-GPU scale-out for real-time streaming voice agents?

- **Answer**:
  - **The Problem**: When running Tensor Parallelism (TP) to split an LLM across multiple GPUs, the GPUs must exchange partial layer activation values during every self-attention step using collective operations like `AllReduce`.
  - **The Impact**: On PCIe Gen4 x16 (31.5 GB/s bidirectional bandwidth), the transmission delay of layer activations between cards can exceed the actual compute time on the GPU. For voice agents with strict latency targets, this communication bottleneck can add 10 to 30 ms to the step time.

- **The Solution**: Deploy systems on high-bandwidth NVLink nodes (900 GB/s bidirectional bandwidth), which reduces communication overhead to negligible fractions of a millisecond.

33.5 Pro-Tip: How to Impress the Interviewer

- **Propose GPUDirect RDMA for Voice RTP Streams**: Explain that in high-concurrency systems, streaming audio packets through the CPU host network stack introduces interrupts and kernel-to-user-space memory copies. Suggest deploying **GPUDirect RDMA** to copy incoming voice RTP packets directly from the Mellanox ConnectX NIC memory to Triton's GPU HBM buffers over PCIe, bypassing the CPU entirely and cutting packet transit latency from 15 ms to < 1 **ms**.
- **Detail CUDA Graph Execution for Vocoders**: Acoustic vocoders and ASR feature extractors run small kernels frequently. Under high concurrency, CPU host kernel launch overhead (5–10 μs per kernel) can exceed the actual execution time on the GPU. Explain how capturing these models as **CUDA Graphs** compiles the operations into a single execution plan on the GPU, eliminating CPU driver overhead.

34 Speech LLMs: Multimodal Audio-Language Models (ALMs)

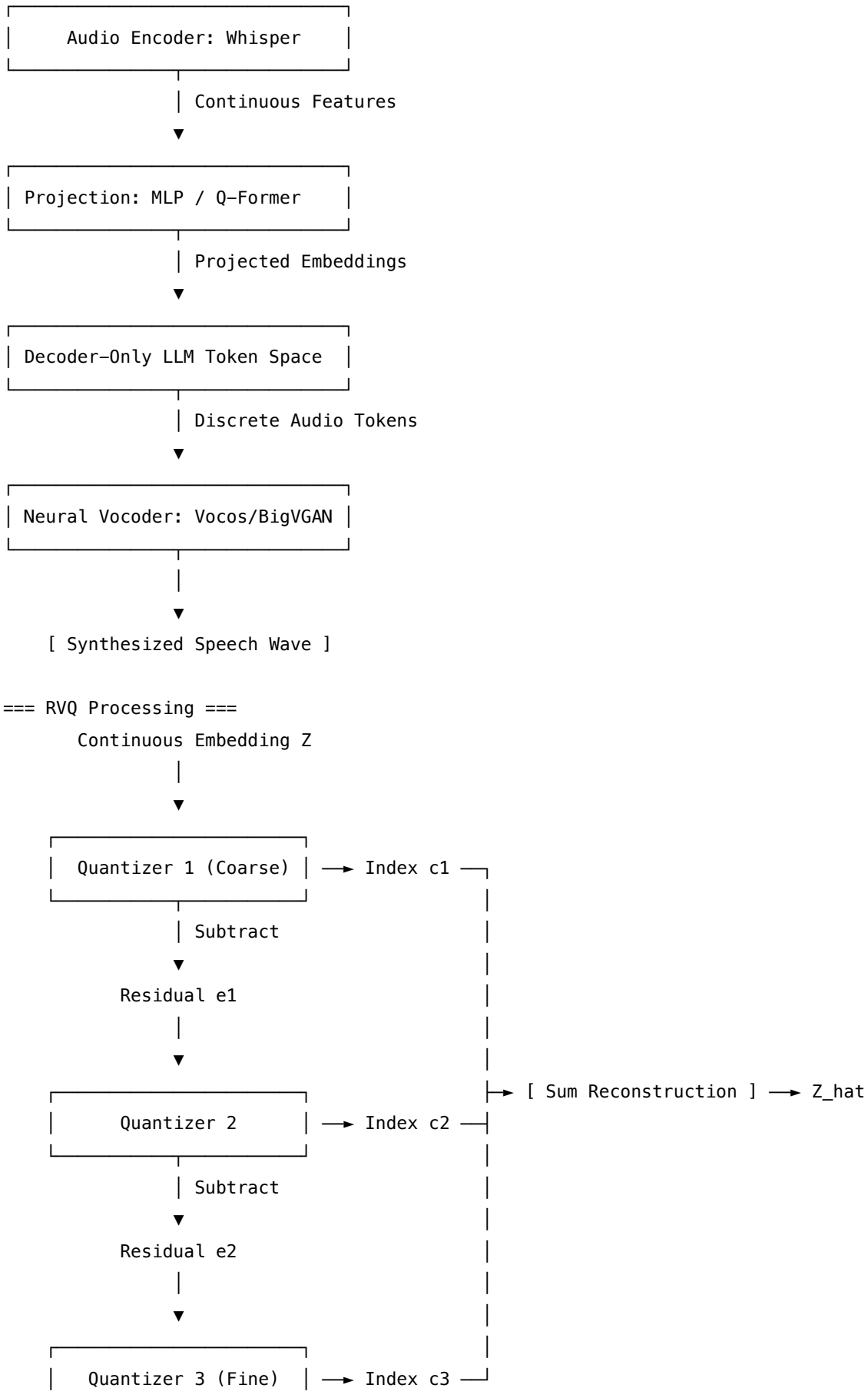
- **Category**: LLM Systems
- **Difficulty**: Hard
- **Target Role**: Conversational AI Engineer / Voice AI Architect
- **Source**: AudioPaLM / GPT-4o / SeamlessM4T research papers
- **Flashcards**: LLM Systems deck

34.1 Concept Overview

Traditional voice assistants rely on a cascaded chain of three distinct models: speech-to-text transcription, a text LLM, and text-to-speech synthesis. Native Audio-Language Models (ALMs) replace this cascade with a single, end-to-end neural network that ingests and generates audio and text tokens natively.

Think of this transition like translating a live performance: * **The Cascaded Pipeline**: You write down the spoken words on paper (ASR), translate the paper text (LLM), and have a separate speaker read the translation aloud (TTS). The pitch, emotion, sarcasm, and background music are completely lost. * **The Native Multimodal ALM**: A bilingual voice actor listens to the performance and immediately translates it live, preserving the exact tone, laughter, emotion, and pace. * **Residual Vector Quantization (RVQ)** is like painting with hierarchical brushes: instead of saving the exact color of every pixel (raw audio), the first brush paints the coarse shapes (Layer 1), the second adds shadows (Layer 2), and the third adds fine details (Layer 3). The final sound is the sum of these layers.

```
=== Native Multimodal ALM ===
     [ User Speech Waveform ]
                 |
                 ▼
```


Audio Encoder: Whisper
Continuous Features
Projection: MLP / Q-Former
Projected Embeddings
Decoder-Only LLM Token Space
Discrete Audio Tokens
Neural Vocoder: Vocos/BigVGAN
[Synthesized Speech Wave]
=== RVQ Processing ===
Continuous Embedding Z
Quantizer 1 (Coarse)
Index c1
Subtract
Residual e1
Quantizer 2
Index c2
Subtract
Residual e2
Quantizer 3 (Fine)
Index c3
[Sum Reconstruction]
Z_hat

34.1.1 The Problem It Solves

Cascaded voice systems suffer from fundamental architectural flaws: 1. **Loss of Paralinguistics**: Discarding acoustic cues (sarcasm, anger, hesitation, gender, or background noise) results in dry, context-blind LLM responses. 2. **Error Propagation**: If ASR mishears a word, the LLM cannot correct it, and spelling/punctuation errors lead to robotic TTS pronunciation stutters. 3. **Framing Latency**: Compounding serialization and execution overhead across three independent models makes sub-300 ms conversational loops mathematically impossible.

34.1.2 How It Works

1. **Neural Audio Codecs**: Raw audio waveforms (16–48 kHz) are compressed into discrete sequences using CNN encoder-decoders. To represent the continuous space without infinite codebook memory, **Residual Vector Quantization (RVQ)** applies R cascaded codebooks. Each codebook quantizes the residual error of the previous step:
$$\hat{\mathbf{z}} = \sum_{r=1}^{R} \mathbf{q}_r$$
This represents each audio frame as an array of R discrete indices (typically $K = 1024$ vocabulary size per codebook).
2. **Projection Modules**: Continuous features from ASR encoders (like Whisper) are mapped to the LLM token dimension using an MLP or a **Q-Former**. Frame-stacking or convolutional downsampling is applied to reduce the sequence length.
3. **Token Interleaving & Serialization**: To generate audio from the LLM, the multi-codebook RVQ tokens must be fed into the autoregressive loop. Models use either:
   - **Flat Serialization**: Linearizes codebooks sequentially (e.g., $A_{1,1}, A_{1,2}, \dots, A_{1,R}, A_{2,1}, \dots$).
   - **Delayed Pattern**: Generates the R codebook streams in parallel but shifts each stream by 1 token step to model cross-codebook dependency.
   - **Hierarchical Generation**: The LLM predicts the first (coarse) codebook stream, and a small feedforward net predicts the remaining $R - 1$ streams.
4. **Vocoding**: The output tokens are mapped back to audio waveforms using generative models: **HiFi-GAN** (using Multi-Period and Multi-Scale discriminators), **Vocos** (predicts Fourier-domain magnitude/phase directly), or **BigVGAN** (integrates Snake activation functions to avoid aliasing).

34.2 Worked Example

34.2.1 Neural Audio Codec Comparison for LLM Ingestion

Below is the trade-off matrix for the leading neural codecs used to represent audio inside autoregressive Transformer loops:

Codec	Downsampling Factor	Frame Rate	Codebooks (R)	Bitrate Range	Reconstruction Quality (ViSQOL)	Key Innovation
SoundStream (Google)	320×	50 Hz (20 ms frames)	up to 16	3–18 kbps	≈ 3.5 (Good)	WaveGAN-style adversarial training
EnCodec (Meta)	320×	75 Hz (13.3 ms frames)	up to 32	1.5–24 kbps	≈ 3.7 (Very Good)	Multiscale spectrogram discriminators + LSTM
Descript Audio Codec (DAC)	512×	47 Hz (21 ms frames)	up to 9	1.5–9 kbps	≈ 4.2 (Excellent)	Periodic Snake activations to prevent aliasing

34.2.1.1 Snake Activation Function formula:

$$f(x) = x + \frac{1}{\alpha}\sin^2(\alpha x)$$

The periodic term $\sin^2$ *introduces an inductive bias for periodic waveforms, preserving high-frequency details (e.g., vocal fricatives) at lower bitrates.*

34.3 Complexity & Trade-offs

Metric	Value	Notes
Audio Token Serialization	Flat vs. Delayed vs. Hierarchical	**Flat**: Increases context length by $R\times$, leading to $O((T \cdot R)^2)$ self-attention cost. **Delayed**: Parallel streams, but requires complex attention masking. **Hierarchical**: Keeps LLM context tiny ($O(T^2)$), but requires training a separate acoustic vocoder.

Metric	Value	Notes
Audio Frame Compression	320× vs. 512×	**320x**: Higher fidelity (75 Hz frame rate), but results in 75 tokens per second of audio. **512x**: Fewer tokens (47 per second), reducing LLM attention overhead by 37% at the cost of transient clarity.
Vocoder Architecture	Time Domain (HiFi-GAN) vs. Fourier Domain (Vocos)	**HiFi-GAN**: Captures rich, detailed audio phase, but slow due to sequential transposed convolutions. **Vocos**: Instantaneous Fourier magnitude/phase prediction, running at 5× speedup.
RVQ Codebooks (R)	$R = 1$ vs. $R = 8$	**R=1**: Tiny vocabulary overhead, but voice sounds highly compressed (robotic). **R=8**: Natural timbre and pitch, but multiplies token generation steps by 8×.

34.4 Common Interview Questions & How to Answer

34.4.1 Q1: How do you address the issue of "codebook collapse" in RVQ training, where only a fraction of the available codebook vectors are actively used?

- **Answer**:
  - **The Problem**: During training, a few codebook entries receive the majority of updates, while the remaining entries are ignored. Once an entry is neglected, its probability of selection falls to zero, reducing the effective bandwidth of the codec.
  - **The Solutions**:
    1. **Random Restarts**: Monitor the usage frequency of all codebook vectors. If a vector's assignment rate falls below a threshold (e.g., 10^{-5}), replace its weights with a random training sample from the current batch plus minor Gaussian noise.
    2. **K-Means Initialization**: Initialize the codebook vectors by running K-means clustering on the first batch of encoder features, ensuring vectors are distributed evenly across the data manifold.
    3. **L2 Normalization**: Project both encoder features and codebook weights onto a unit sphere (L2 normalization) and perform quantization using Cosine Similarity instead of Euclidean distance. This prevents scale variations from clustering updates around a few large vectors.

34.4.2 Q2: Why is the rate mismatch between speech tokenization and text tokenization a major problem for multimodal LLMs, and how do you resolve it?

- **Answer**:
  - **The Problem**: Text tokenizers process roughly 3–4 tokens per second of spoken speech. In contrast, neural audio codecs (like EnCodec) at 75 Hz with 8 codebooks generate $75 \times 8 = 600$ tokens per second of audio. This 150× mismatch causes the audio tokens to completely dominate the self-attention context window, slowing down generation speed and limiting conversational history.
  - **The Solutions**:
    1. **Temporal Convolutions / Stacking**: Apply a 1D convolution layer with a stride of 2 or 4 over the audio encoder embeddings before feeding them to the LLM, reducing the input sequence length.
    2. **Q-Former Compression**: Use a Querying Transformer (Q-Former) with a fixed set of learnable query tokens (e.g., 64 queries per audio segment) to extract semantic vectors via cross-attention, decoupling token length from audio duration.
    3. **Hierarchical Modeling**: Only output the first RVQ codebook from the LLM, offloading the remaining codebook generation to a non-autoregressive parallel prediction model.

34.5 Pro-Tip: How to Impress the Interviewer

- **Detail RVQ Loss Formulations and Stop-Gradients**: Show you know the mathematics of quantizers. Write the codebook loss and commitment loss equations, highlighting the use of the stop-gradient operator (`sg`) to copy gradients from the decoder back to the encoder:

$$\mathcal{L}_{\text{RVQ}} = \|\text{sg}[\mathbf{z}] - \hat{\mathbf{z}}\|_2^2 + \beta\|\mathbf{z} - \text{sg}[\hat{\mathbf{z}}]\|_2^2$$

  Explain that the first term updates the codebook vectors towards the encoder outputs, while the second term (commitment loss with weighting factor $\beta \approx 0.25$) prevents the encoder outputs from fluctuating too wildly between codebook entries.
- **Propose Multi-Period Discriminators (MPD) for Vocoding**: To explain how vocoders synthesize high-fidelity voices, detail how **Multi-Period Discriminators** reshape a 1D audio waveform into 2D matrices using prime number periods (e.g., 2, 3, 5, 7, 11). This allows the discriminator to inspect periodic patterns and harmonics in disjoint bands, which is why neural vocoders can reconstruct realistic pitch structures without sounding robotic.

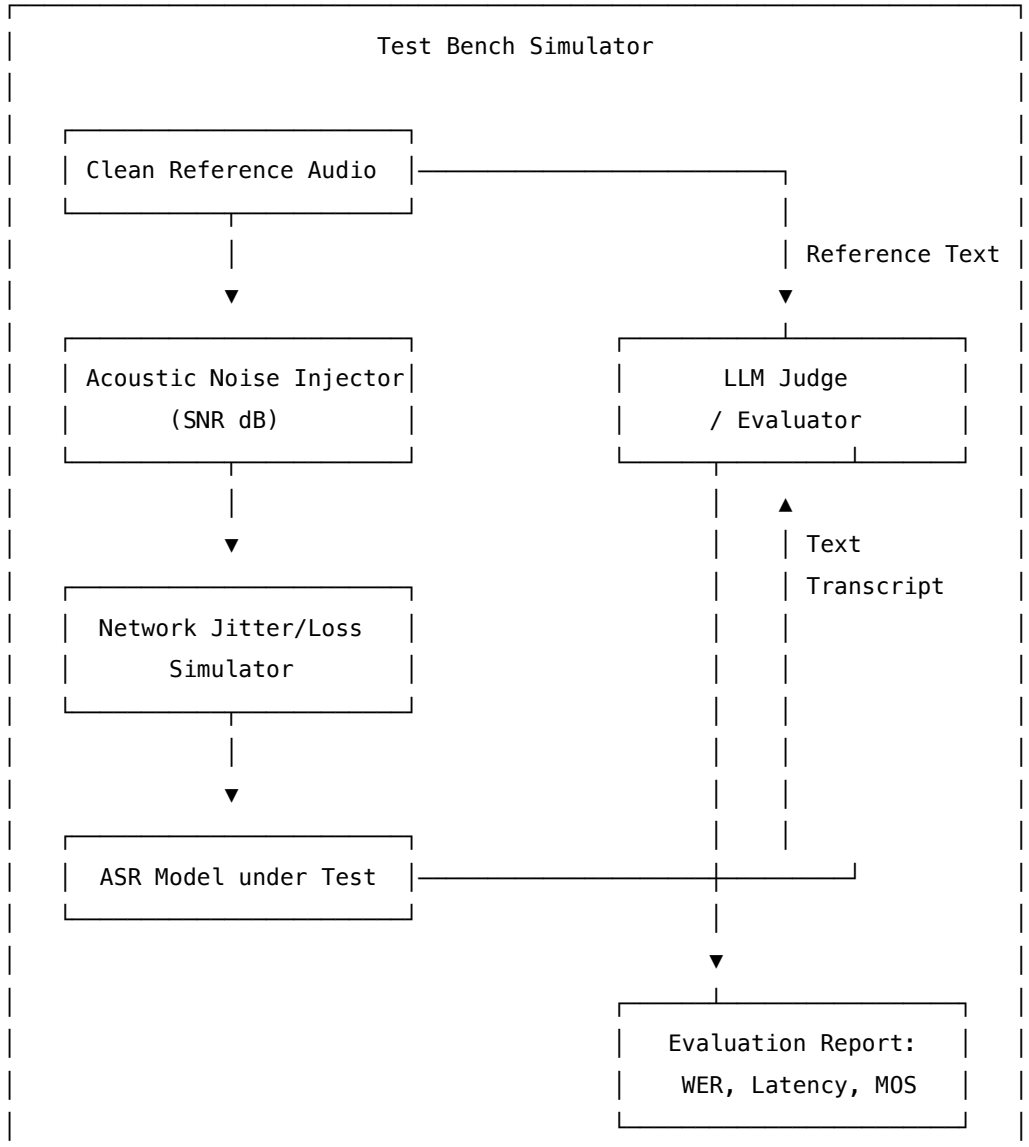
35 Speech LLMs: Agent Benchmarking & Evaluation Frameworks

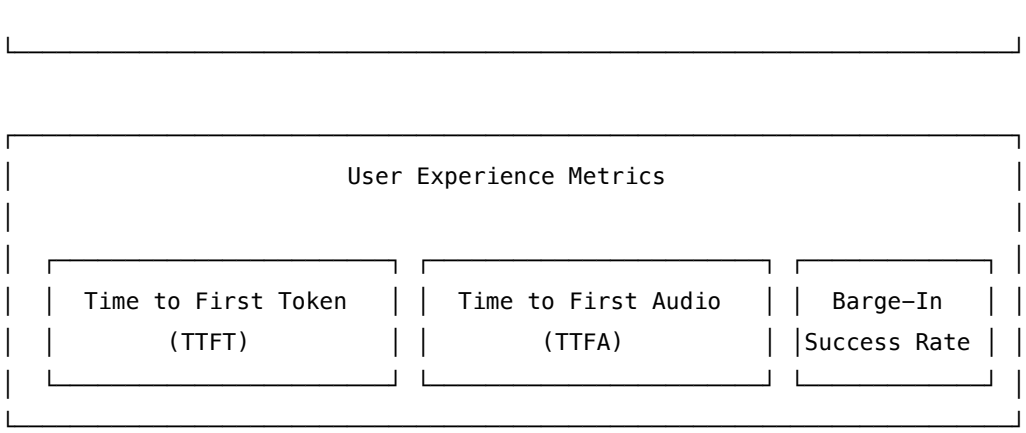
- **Category**: LLM Systems
- **Difficulty**: Hard
- **Target Role**: Conversational AI Engineer / Voice AI Architect
- **Source**: Google & NVIDIA Evaluation Frameworks
- **Flashcards**: LLM Systems deck

35.1 Concept Overview

Deploying interactive voice agents into production requires robust benchmarking frameworks. Unlike text-based chatbots, voice agents must operate under real-world degradation: acoustic noise, room reverberation, microphone clipping, network jitter, and packet loss.

Think of evaluating a voice agent like testing a drive-thru intercom system at a busy restaurant: * **Signal-to-Noise Ratio (SNR)** is like testing the intercom during a heavy rainstorm with loud car engines idling nearby. Can the system still understand the order? * **Speech Quality (POLQA/MCD)** evaluates the cashier's voice: does it sound like a friendly, clear human, or a crackly, robotic walkie-talkie? * **TTFT vs. TTFA** is the latency gap: TTFT is how fast the cashier registers the answer in their brain (first text token), whereas TTFA is how long it takes for their voice to actually travel out of the intercom speaker (first audio wave). * **Barge-In Success** is when you interrupt the cashier mid-sentence. Does the cashier immediately stop talking to listen, or do they keep reading their script over you?





35.1.1 The Problem It Solves

Without automated voice benchmarking frameworks: 1. **Unnoticed Performance Drift**: Upgrades to LLM engines can inadvertently introduce token delays that break streaming ASR-TTS coordination. 2. **Brittle Turn-Taking**: VAD models calibrated in quiet environments fail in production, leading to constant false-interruptions or dead air. 3. **Audio Timbre Degradation**: Fast-pitch or vocoder optimizations that speed up inference might degrade audio quality, introducing robotic clicks that repel users.

35.1.2 How It Works

1. **Physical & Perceptual Quality Metrics**:
   - **PESQ (ITU-T P.862)**: Compares reference and degraded signals after time-alignment. Limited to wideband 16 kHz telephony.
   - **POLQA (ITU-T P.863)**: Successor to PESQ, supporting full-band 48 kHz high-fidelity audio, modeling modern packet loss concealment (PLC) and time-stretching.
   - **Mel-Cepstral Distance (MCD)**: A physical distance metric measuring spectral divergence between Mel-Frequency Cepstral Coefficients (MFCCs). Lower is better.
   - **ViSQOL**: Google's structural spectrogram similarity index (SSIM) metric, scoring audio from 1.0 to 5.0.
2. **Transcription Metrics**: ASR output is compared to ground-truth scripts via Levenshtein distance:

$$\text{WER} = \frac{S + D + I}{N}$$

   where S = substitutions, D = deletions, I = insertions, and N = reference word count.
3. **Latency Benchmarking**: TTFT measures server token generation onset. TTFA measures the full loop: client silence onset (T_{EOS}) to client speaker playback activation ($T_{\text{playback\_start}}$).
4. **Conversation Flow Metrics**:
   - **Barge-In Success Rate**: Percentage of times the agent halts playback and flushes downstream network buffers within 150 ms of user speech detection.
   - **Turn-Taking Accuracy**: Measures false-interruptions (agent cut off user mid-thought) and sluggish turns (agent waited > 1.0 second to respond).

35.2 Worked Example

35.2.1 Automated Test Simulator Metrics Report

We run a simulated test bench evaluating a voice agent under varying room noise levels (SNR) and network packet loss rates.

35.2.1.1 1. Test Conditions & Inputs:

- **Audio Source**: 2-second clean mono WAV file of the phrase: *"Hello, I would like to book a flight to Seattle."*
- **Acoustic Noise**: Gaussian white noise injected at target SNR levels.
- **Network Profile**: 80 ms mean delay, 25 ms jitter, variable packet loss.

35.2.1.2 2. Before / After Performance Matrix:

Test Scenario	Room Noise (SNR)	Network Packet Loss	Word Error Rate (WER)	ViSQOL Score (MOS)	Mean TTFA (ms)	Task Success (LLM-as-a-Judge)
1. Ideal Baseline	Clean (> 40 dB)	0%	0.0%	4.5 (Excellent)	420 ms	**SUCCESS** (5.0/5.0)
2. Moderate Noise	Noisy Room (15 dB)	2%	10.0% (Seattle misheard)	3.8 (Good)	450 ms	**SUCCESS** (4.0/5.0)
3. High Degradation	Heavy Static (5 dB)	10%	45.0% (phrase fragmented)	2.1 (Poor)	780 ms (jitter buffer delay)	**FAILED** (1.2/5.0)

35.2.1.3 3. Mel-Cepstral Distortion (MCD) Calculation:

$$\text{MCD} = \frac{10\sqrt{2}}{\ln 10} \frac{1}{T} \sum_{t=1}^{T} \sqrt{\sum_{k=1}^{D} (mc_{t,k} - mc'_{t,k})^2}$$

For high-fidelity TTS vocoding, production-grade output should aim for a MCD < 2.5 dB relative to clean speaker recordings.

35.3 Complexity & Trade-offs

Metric	Value	Notes
Audio Quality Evaluation	POLQA vs. ViSQOL	**POLQA**: High accuracy, models telephony networks, but requires expensive commercial licenses. **ViSQOL**: Open-source (Google), excellent for neural codecs, but sensitive to time-alignment offsets.
ASR Metric	WER vs. CER	**WER**: Standard for English. **CER**: Best for phonetically dense languages (Mandarin) or morphology-rich languages (Finnish) where sub-word changes modify meaning.
VAD Silence Window	250 ms vs. 500 ms	**250ms**: Lowers TTFA, but causes frequent false-interruptions if the user pauses to think. **500ms**: Natural conversations, but adds 250 ms of dead air to every turn.
Jitter Buffer Depth	Static (30 ms) vs. Adaptive	**Static**: Predictable latency, but causes audio drops during network spikes. **Adaptive**: Adjusts buffer depth dynamically, preventing drops but increasing mean latency.

35.4 Common Interview Questions & How to Answer

35.4.1 Q1: How do you measure Time-to-First-Audio (TTFA) accurately in a streaming WebSocket-based system?

- **Answer**:
  - We cannot rely on server logs because they omit network transit and client-side playback buffering. To measure TTFA:
    1. **Inject Timestamp Markers**: When the client-side VAD detects the end of the user's speech, it records a local microsecond timestamp (T_{EOS}) and sends a metadata frame containing this timestamp to the orchestrator.
    2. **Propagate Markers**: The orchestrator carries this start timestamp through the pipeline. When the first synthesized audio chunk is generated, the server tags the binary WebSocket frame with the original T_{EOS}.
    3. **Client Playback Logging**: When the client-side audio hardware player callback processes

the very first PCM sample of this tagged frame, the client records the current local clock time ($T_{\text{playback\_start}}$).

4. **Calculate Loop Latency**:

$$\text{TTFA} = T_{\text{playback\_start}} - T_{\text{EOS}}$$

This measures the actual end-to-end user experience, including all networking and hardware overhead.

35.4.2 Q2: How do you evaluate the robustness of a voice agent's VAD and turn-taking behavior before deploying it to production?

- **Answer**:
  - We use **Conversational Audio Mixes** for automated stress testing:
    1. **Synthetic Audio Generation**: Mix clean speech recordings with varying types of background noise at target SNR levels (5 dB to 20 dB).
    2. **Background Distractors**: Inject non-speech sounds (breathing, sighing, keyboard typing, page turning) and "babble noise" (overlapping background chatter) to test if the VAD triggers false speech starts.
    3. **Pause Injection**: Insert silence windows of varying lengths (100–800 ms) inside the input audio. We measure if the VAD triggers an End-of-Utterance (EoU) during a mid-sentence thinking pause, helping us tune the silence threshold window.

35.5 Pro-Tip: How to Impress the Interviewer

- **Propose Automated Dual-Audio Loopback Benchmarking**: Describe setting up an automated physical test bench using two docker containers on a network namespace. One container runs a **User Simulator** (playing raw user audio files and listening for responses), while the second hosts the **Voice Agent under Test**. By establishing a virtual audio loopback (e.g., using Pulseaudio virtual sinks or ALSA loopback devices), the user simulator can play audio, detect agent playback, measure barge-in speeds by playing interrupting audio mid-stream, and log millisecond-accurate TTFA directly from virtual hardware clocks.
- **Detail Hardware Contention Profiling under Load**: Explain that when scaling voice agents, CPU/GPU context-switching overhead causes latency spikes. Propose benchmarking under simulated load (using tools like Locust to simulate 1000 concurrent streaming gRPC calls) while profiling kernel execution queues with NVIDIA Nsight Systems. This helps identify if Triton's dynamic batcher is starving TTS vocoder kernels during heavy LLM prefill spikes.